

SPCA102N

POSTGRADUATE COURSE

Master of Computer Applications

FIRST YEAR

FIRST SEMESTER

CORE PAPER - II

DIGITAL LOGIC FUNDAMENTALS



**INSTITUTE OF DISTANCE EDUCATION
UNIVERSITY OF MADRAS**

WELCOME

Warm Greetings.

It is with a great pleasure to welcome you as a student of Institute of Distance Education, University of Madras. It is a proud moment for the Institute of Distance education as you are entering into a cafeteria system of learning process as envisaged by the University Grants Commission. Yes, we have framed and introduced as per AICTE Choice Based Credit System(CBCS) in Semester pattern from the academic year 2020-21. You are free to choose courses, as per the Regulations, to attain the target of total number of credits set for each course and also each degree programme. What is a credit? To earn one credit in a semester you have to spend 30 hours of learning process. Each course has a weightage in terms of credits. Credits are assigned by taking into account of its level of subject content. For instance, if one particular course or paper has 4 credits then you have to spend 120 hours of self-learning in a semester. You are advised to plan the strategy to devote hours of self-study in the learning process. You will be assessed periodically by means of tests, assignments and quizzes either in class room or laboratory or field work. In the case of PG (UG), Continuous Internal Assessment for 20(25) percentage and End Semester University Examination for 80 (75) percentage of the maximum score for a course / paper. The theory paper in the end semester examination will bring out your various skills: namely basic knowledge about subject, memory recall, application, analysis, comprehension and descriptive writing. We will always have in mind while training you in conducting experiments, analyzing the performance during laboratory work, and observing the outcomes to bring out the truth from the experiment, and we measure these skills in the end semester examination. You will be guided by well experienced faculty..

I invite you to join the CBCS in Semester System to gain rich knowledge leisurely at your will and wish. Choose the right courses at right times so as to erect your flag of success. We always encourage and enlighten to excel and empower. We are the cross bearers to make you a torch bearer to have a bright future.

With best wishes from mind and heart,

DIRECTOR

COURSE WRITER

Dr. S. SASIKALA, M.C.A., M.Phil., Ph.D.,
Asst. Prof. in Computer Science
Institute of Distance Education
University of Madras
Chennai - 600 005.

EDITING

Dr. S. SASIKALA, M.C.A., M.Phil., Ph.D.,
Asst. Prof. in Computer Science
Institute of Distance Education
University of Madras
Chennai - 600 005.

MCA DEGREE COURSE
FIRST YEAR
FIRST SEMESTER
Core Paper - II
DIGITAL LOGIC FUNDAMENTALS
SYLLABUS

Objectives of the course: To acquire fundamental principles of two-valued logic and various devices used to implement logical operations on variables. To lay the foundation for further studies in areas such as communication, VLSI, computer, microprocessor.

Course Outcomes: After successful completion of this course, the students should be able to Use the basic logic gates and various reduction techniques of digital logic circuit in detail. Design combinational and sequential circuits. Design and implement hardware circuit to test performance and application. Understand the architecture and use of microcontrollers for the basic operations and simulate using simulation software.

Unit-I : Number System – Converting numbers from one base to another – Complements – Binary Codes – Integrated Circuits – Boolean algebra – Properties of Boolean algebra – Boolean functions – Canonical and Standard forms – Logical Operations – Logic gates – Karnaugh Map up to 6 variables – Don't Care Condition – Sum of Products and Products of Sum simplification – Tabulation Method.

Unit-II : Adder – Subtractor – Code Converter – Analyzing a combinational Circuit – Multilevel NAND and NOR circuits – Properties of XOR and equivalence functions – Binary Parallel Adder – Decimal Adder – Magnitude Comparator – Decoders – Multiplexers – ROM – PLA.

Unit-III : Flip Flops – Triggering of flip-flops – Analyzing a sequential circuit – State reduction – excitation tables – Design of sequential circuits – Counters – Design with state equation – Registers – Shift Registers – Ripple and synchronous Counters.

Unit-IV : Memory Unit – Processor Organization - Bus Organization – Scratch Pad memory – ALU – Design of ALU – Status Register – Effects of Output carry – Design of Shifter – Processor Unit – Microprogramming – Design of specific Arithmetic Circuits

Unit-V : Accumulator – Design of Accumulator – Computer Design – System of Configuration – Instruction and Data formats – Instruction sets – Timing and Control – Execution of Instruction – Design of Computer – Hardwired control – PLA Control and Microprogram control

Recommended Texts:

- a. M. Morris Mano, 2011, Digital Logic and Computer Design, Thirteenth Impression, Pearson Education, Delhi

Reference Books:

- b. M. M. Mano and C.R.Kime, 2001, Logic and Computer Design Fundamentals, 2nd Edition, Pearson Education, Delhi.
- c. Givone, 2002, Digital Principles Design, Tata McGraw Hill, New Delhi.
- d. C. H. Roth , Jr, 2005, Fundamentals of Logic Design ,5th Edition, Thomson Learning, Singapore.

MCA DEGREE COURSE
FIRST YEAR
FIRST SEMESTER
Core Paper - II
DIGITAL LOGIC FUNDAMENTALS
SCHEME OF LESSONS

Sl.No.	Title	Page
1.	Number Systems	1
2.	Arithmetic in Number Systems	13
3.	Complements	21
4.	Codes	33
5.	Logic Gates	45
6.	Boolean Algebra	60
7.	Laws and Theorems of Boolean Algebra	83
8.	Arithmetic Building Blocks	92
9.	Simplifying Logic Circuits	112
10.	Karnaugh Map Simplification - I	127
11.	Karnaugh Map Simplification - II	167
12.	Tabulation Method	175
13.	Flip-Flops	187
14.	Registers	205
15.	Counters	209
16.	Design of sequential circuits with state equations	224
17.	Memory Organisation	236
18.	Instruction Formats and Addressing Modes	249
19.	Instruction Set	261
20.	Central Processing Unit (CPU)	281
21.	Control Logic Design	285
22.	Arithmetic Logic Unit (ALU)	294
23.	Input / Output Organisation	304

LESSON - 1

NUMBER SYSTEMS

Structure

- 1.0 Objectives
- 1.1 Introduction
- 1.2 Counting in Binary
- 1.3 Counting in Octal
- 1.4 Usefulness of Octal System
- 1.5 Counting in Hexa Decimal
- 1.6 Conversions
- 1.7 Summary
- 1.8 Model Questions

1.0 Objectives

After studying this lesson you should be able to Understand:

- To introduce binary, octal, hexadecimal number systems.
- To convert decimal to binary / binary to decimal.
- To convert decimal to octal / octal to decimal.
- To convert decimal to hexadecimal / hexadecimal to decimal.
- To convert octal / hexa to binary and vice versa.

1.1 Introduction

We are familiar with the decimal number system which is used in our day-to-day work. In the decimal number system there are ten digits which are used to form decimal numbers. Ten separate symbols 0,1,2,3,4,5,6,7,8 and 9 are used to represent ten decimal digits. A digital computer stores, understand and manipulates information composed of only zeros and ones. A programmer who works on a computer is allowed to use decimal digits; letter A,B,C, ..., Z, a,b,c ..., z, usual special symbols +, -, etc. These are converted to binary codes in the forms of 0's

and 1's within the computer. To understand the operation of a computer a knowledge of binary, octal and hexa decimal number system is essential.

Base or Radix: The base or radix of number system is defined as the number of symbols used by that number system.

The Decimal number system is said to be of base or radix 10.

The Binary number system is said to be of base 2.

The Octal number system is said to be of base 8.

The Hexa decimal number system is said to be of base 16.

1.2 Counting in Binary

The Binary symbols are 0 and 1, so that counting in binary number system, a digit is incremented from 0 to 1. Once it reaches 1, recycles to 0 on the next count.

Example : 0, 1, 10, 11, 100, 101, 110 ,....
 1111, 10001, 10010, 10011, 10100 ,....

1.3 Counting in Octal

The octal symbols are 0, 1, 2, 3, 4, 5, 6, 7, so that counting in Octal, is a digit incremented from 0 to 7. Once it reaches 7, recycles to 0 on the next count.

Example : 0, 1, 2, 3, 4, 5, 6, 7, 10, 11, ... 17, 20, ...
 65, 66, 67, 70, 71, 77, 80, 81
 255, 256, 257, 260, 261 ... 267, 270, 271 ...

1.4 Usefulness of Octal System

The ease with which conversion can be made between octal and binary makes the octal system attractive as a “shorthand” means of expressing large binary numbers. In computer work, binary number with upto 64 bits are not uncommon. When dealing with large quantity of binary numbers of many bits, it is convenient and more efficient for us to write the numbers in

Octal rather than Binary. Keeping in mind, however, that the digital circuits and systems work strictly in Binary; we are using Octal system only as a convenience for the operators of the system.

1.5 Counting in Hexa Decimal

Hexa decimal symbols are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, & F. When counting in Hexa, each digit position can be incremented (increased by 1) from 0 to F. Once a digit position reaches the value F, it is reset to 0 and the next digit position is incremented.

Example : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 10, 11,
 38, 39, 3A, 3B, 3E, 3F, 40, 41
 6F8, 6F9, 6FA, 6FB 6FE, 6FF, 700, 701;

1.6 Conversions

1.6.1 Decimal To Binary

Method I : Express the given decimal number as a sum of powers of 2 and then write it in the appropriate bit position.

$$\begin{aligned}
 45_{10} &= 32 + 8 + 4 + 1 \\
 &= 2^5 + 0 + 2^3 + 2^2 + 0 + 2^0 \\
 &= 1 \ 0 \ 1 \ 1 \ 0 \ 1_2 \\
 &= (101101)_2
 \end{aligned}$$

Method II : Repeated Division : Divide the given decimal number repeatedly by 2 and write down the remainder after each division until a quotient of 0 is obtained. The Binary result is obtained by writing the first remainder as the LSB and the last remainder as the MSB.

Example : Convert the decimal number (45_{10}) to a binary number.

Quotient	Remainder	
$45 \% 2 = 22$	1	LSB

22 % 2 = 11	0	
11 % 2 = 5	1	
5 % 2 = 2	1	
2 % 2 = 1	0	
1 % 2 = 0	1	MSB

Hence $(45)_{10} = (101101)_2$

To Convert Decimal Fraction : Multiply the given decimal fraction by 2 to give an integer and fraction. The new fraction is again multiplied by 2 to give a new integer and new fraction. This process is continued until the fraction becomes 0 or the number gets sufficient accuracy.

Fraction	Co-efficients
$0.6875 \times 2 = 1 + 0.3750$	1
$0.3750 \times 2 = 0 + 0.7500$	0
$0.7500 \times 2 = 1 + 0.5000$	1
$0.5000 \times 2 = 1 + 0.0000$	1

Hence, $(0.6875)_{10} = (1011)_2$

1.6.2 Decimal To Octal

Repeated Division : Divide the given decimal number repeatedly by 8 and write down the remainders after each division until a quotient of 0 is obtained. The Octal result is obtained by writing the first remainder as the LSB and the last remainder as the MSB.

Example : $266_{10} = 412_8$

QUOTIENT	REMAINDER
$266 \% 8 = 33$	2 LSB
$33 \% 8 = 4$	1
$4 \% 8 = 0$	4 MSB

Hence, $266_{10} = 412_8$

To Convert Decimal Fraction : Multiply the given decimal fraction by 8 to give a new integer and new fraction. Continue this process until the fraction becomes 0 or until the number gets sufficient accuracy.

Example : 0.03125_{10}

	Integer	Fraction	C
$0.03125 \times 8 = 0.25000$	0	+ 0.25000	0
$0.25 \times 8 = 2.0000$	2	+ 0.00000	2

Hence, $0.03125_{10} = 0.02_8$

1.6.3 Decimal To Hexa Decimal

Repeated Division : Divide the given decimal number repeatedly by 16 and write down the remainders after each division until a quotient of 0 is obtained. The Hexa Decimal result is obtained by writing the first remainder as the LSB and the last remainder as the MSB.

Example : $423_{10} = (1A7)_{16}$

Quotient	Remainder	
$423 \% 16 = 26$	7	LSB
$26 \% 16 = 1$	A	
$1 \% 16 = 0$	1	MSB

Hence, $423_{10} = (1A7)_{16}$

To convert decimal fraction : Multiply the given decimal fraction by 16 to give an integer and fraction. Multiply again the new fraction by 16 to give a new integer and new fraction. Continue this process until the fraction becomes 0 or until the number gets sufficient accuracy.

Example : 0.1045_{10}

$$\begin{aligned}
 0.1045 \times 16 &= 1.672 \quad 1 + 0.672 \quad 1 \\
 0.6720 \times 16 &= 10.752 \quad 10 + 0.752 \quad 10 = A \\
 0.752 \times 16 &= 12.032 \quad 12 + 0.032 \quad 12 = C \\
 0.032 \times 16 &= 0.512 \quad 0 + 0.512 \quad 0 \\
 0.512 \times 16 &= 8.192 \quad 8 + 0.192 \quad 8 \\
 0.192 \times 16 &= 3.072 \quad 3 + 0.072 \quad 3
 \end{aligned}$$

Hence, $0.1045_{10} = 0.1AC083_{16}$

1.6.4 Binary to Decimal

Any binary number can be converted to decimal by forming the sum of product of each coefficient with the corresponding positional weights (powers of 2).

Example : 1010.011_2

$$\begin{aligned}
 &= 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} \\
 &= 2^3 + 2^1 + 2^{-1} + 2^{-2} + 2^{-3} \\
 &= 8 + 2 + 0.5 + 0.25 + 0.125 \\
 &= 10.875_{10}
 \end{aligned}$$

1.6.5 Octal to Decimal

Any octal number can be converted to decimal by forming the sum of the product of each coefficient with the corresponding positional weights (powers of 8).

$$\begin{aligned}
 \text{Example : } (630.4)_8 &= 6 \times 8^2 + 3 \times 8^1 + 0 \times 8^0 + 4 \times 8^{-1} \\
 &= 384 + 24 + 0.5 \\
 &= 408.5_{10}
 \end{aligned}$$

1.6.6 Hexa Decimal to Decimal

Any Hexa Decimal number can be converted to decimal by forming the sum of the product of each coefficient with the corresponding power of 16 (positional weights).

Example : $BAD.C_{16}$

$$\begin{aligned}
 &= B \times 16^2 + A \times 16^1 + D \times 16^0 + C \times 16^{-1} \\
 &= 11 \times 256 + 10 \times 16 + 13 \times 1 + 12/16 \\
 &= 2816 + 160 + 13 + 0.75 \\
 &= 2989.75_{10}
 \end{aligned}$$

1.6.7 Octal to Binary

To convert Octal to Binary, convert each octal digit to its 3-Bit Binary equivalent. Binary equivalent is given in the following table.

Octal	Binary
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Example: $472_8 = 100111010_2$

4	7	2
100	111	010

Example: $356.17_8 = 11101110.001111_2$

3	5	6	.1	7
011	101	110	.001	111

1.6.8. Hexa Decimal to Binary

To convert Hexa Decimal to Binary convert each Hexa Decimal Digit to it's 4-Bit Binary equivalent. See the table for Hexa Decimal Binary equivalent.

Table : Hexa-Decimal & Binary Equivalent

Hexa Decimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Example : $BAD.DAD_{16}$

B	A	D	.	D	A	D
1011	1010	1101	.	1101	1010	1101

$BAD.DAD_{16} = 101110101101.110110101101$

1.6.9 Binary to Octal

To convert Binary to Octal group, the bits of the given Binary Number are to be divided into groups of 3 bits starting at the Binary Point. Then convert each group into its Octal Equivalent. If needed, add leading zeros at the end to get the octal number (group of 3 bits).

Example : 100111010.011010110_2

100	111	010	.	011	010	110
4	7	2	.	3	2	6

$100111010.011010110_2 = 472.326_8$

1.6.10 Binary to Hexa Decimal

To convert Binary to Hexa Decimal number, group it into 4 bits starting at the Binary Point. Then convert each group into its Hexa Decimal equivalent.

Example : 1110100110.1001110_2

0011	1010	0110	.	1001	1100
3	A	6	.	9	C

$1110100110.1001110_2 = 3A6.9C_{16}$

1.6.11 Octal to Hexa Decimal

To Convert Octal to Hexa Decimal, convert Octal to Binary, and then convert Binary to Hexa decimal.

Example : Convert $(732)_8$ to Hexa decimal

First convert $(732)_8$ to binary. Each digit is represented by a group of three bits as shown

$$(7 \quad 3 \quad 2)_8$$

$$(111 \quad 011 \quad 010)_2$$

This binary combination is arranged in a group of four bits as

$$0001 \quad 1101 \quad 1010$$

This is now represented in hexa decimal as

$$I \quad D \quad A$$

Thus

$$(732)_8 = (IDA)_{16}$$

1.6.12 Hexa Decimal To Octal

To convert Hexa Decimal to Octal, convert Hexa Decimal to Binary and then convert Binary to Octal.

Example : Convert $AB.92_{16}$ to Octal

First convert $(AB.92)_{16}$ to Binary

$$A \quad B \quad . \quad 9 \quad 2$$

$$1010 \quad 1011 \quad . \quad 1001 \quad 0010$$

This Binary combination is arranged in a group of three bits as

$$010 \quad 101 \quad 011 \quad . \quad 100 \quad 100 \quad 100$$

This combination is now converted into octal

$$2 \quad 5 \quad 3 \quad . \quad 4 \quad 4 \quad 4$$

$$\text{Thus, } (AB.92)_{16} = (253.444)_8$$

Note :

The word bit is used as an abbreviation for binary digit. A binary number with four bits is called a nibble. A binary digit with eight bits is called a byte.

Two binary digits can represent four numbers (2²) namely 00, 01, 10 and 11.

No. of Bits	Total No. of combinations
2	4
3	8
4	16
-	-
-	-
8	256
-	-
10	1024
-	-
12	4096
-	-
16	65536

1.7 Summary

A computer can understand the positional number system where there are only a few symbols called digits and these symbols represent different values depending on the position they occupy in the number.

The number system that we use in our day-to-day life is the decimal number system. Decimal number system has base 10 as it uses 10 digits from 0 to 9.

The binary number system is the simplest number system.

- % The binary number system contains two digits, 0 and 1.
- % The binary number system is used to represent data for digital and computer systems.

Octal numbers allow reading large binary numbers by breaking the binary number into groups of three.

- % The octal number system is referred to as base 8.

The hexadecimal number system breaks binary numbers into groups of four to reduce error when entering data.

- % The hexadecimal number system is a base-16 system.

1.8 Model Questions

- (1) What is the binary number that follows 101111.
- (2) $(125)_{10} = (x)_8 = (y)_{16} = (z)_2$, find x, y, z.
- (3) $(29A.8)_{16} = (x)_{10} = (y)_2 = (z)_8$, find x, y, z.
- (4) $(1100\ 1001\ .\ 101)_2 = (x)_{10} = (y)_8 = (z)_{16}$, find x, y, z.
- (5) Represent $(2^{11} - 1)$ in decimal, binary and hexadecimal.

Solutions :

- (1) 11 0000
- (2) 178; 7D; 111 1101
- (3) 666.5; 10 1001 1010.1; 1232.4
- (4) 201.625; 311.5; C9.A
- (5) 2047; 111 1111 1111; 7FF

LESSON - 2

ARITHMETIC IN NUMBER SYSTEMS

Structure

- 2.0 Objectives**
- 2.1 Introduction**
- 2.2 Binary Addition**
- 2.3 Binary Subtraction**
- 2.4 Binary Multiplication**
- 2.5 Binary Division**
- 2.6 Addition of octal numbers**
- 2.7 Addition of Hexa Decimal numbers**
- 2.8 Summary**
- 2.9 Model Questions**

2.0 Objectives

After studying this lesson you should be able to understand:

- Addition, subtraction, multiplication and division in binary number system.
- Simple exercises in addition of octal and hexa decimal numbers.

2.1 Introduction

Binary arithmetic is essential part of all the digital computers and many other digital system. It is a key for binary subtraction, multiplication, division. There are four rules of binary addition. Subtraction and Borrow, these two words will be used very frequently for the binary subtraction.

2.2 Binary Addition

The addition of 2 Binary numbers is performed in exactly the same manner as the addition of decimal numbers. The same general steps are followed in Binary Addition. The following 4 rules are used in adding the 2 Binary numbers. They are :

Rules:

$$0 + 0 = 0$$

$$0 + 1 = 1 + 0 = 1$$

$$1 + 1 = 10 = 0 \text{ with a carry of } 1$$

$$1 + 1 + 1 = 11 = 1 \text{ with a carry of } 1.$$

The last case outputs when the 2 Bits in a certain positions are 1 and there is a carry from the previous position.

Example : Add $6_{10} + 3_{10}$

$$\begin{array}{r} 11 \quad (3) \\ 110 \quad (6) \\ \hline 1001 \quad (9) \end{array}$$

Example : Add $9_{10} + 15_{10}$

$$\begin{array}{r} 1001 \quad (9) \\ 1111 \quad (15) \\ \hline 11000 \quad (24) \end{array}$$

Example : Add $3.375_{10} + 2.750_{10}$

$$\begin{array}{r} 11.011 \quad (3.375) \\ 10.110 \quad (2.750) \\ \hline 110.001 \quad (6.125) \end{array}$$

It is not necessary to consider the addition of more than 2 Binary numbers at a time, because in all digital system the circuit that actually performs the addition can handle only 2 numbers at a time. When more than 2 numbers are to be added, the first 2 are added together and then the sum is added to the third number and so on. Addition is the most important arithmetic operation in digital system. The operation of subtraction, multiplication and division are performed in most of the modern digital computers in terms of addition only.

2.3 Binary Subtraction

The subtraction of Binary Numbers is performed in exactly the same manner as the subtraction of decimal numbers. The same general steps are followed in Binary Addition. The following 4 rules are used in subtracting 2 Binary numbers.

Rules:

$$0 - 0 = 0$$

$$1 - 0 = 1; 0 - 1 = 1 \text{ with borrow } 1$$

$$1 - 1 = 0$$

Example : $7_{10} - 5_{10}$

$$\begin{array}{r} 111 \text{ (7)} \\ -101 \text{ (-5)} \\ \hline 010 \text{ (2)} \\ \hline \end{array}$$

Example : $13_{10} - 10_{10}$

$$\begin{array}{r} 1101 \text{ (13)} \\ -1010 \text{ (-10)} \\ \hline 0011 \text{ (3)} \\ \hline \end{array}$$

Example : $4_{10} - 7_{10}$

$$\begin{array}{r} 100 \text{ (4)} \\ -111 \text{ (-7)} \\ \hline -011 \text{ (-3)} \\ \hline \end{array}$$

In the last example we have subtracted the smaller magnitude from the larger magnitude and put the sign of the larger magnitude.

Example : 10111 (23)

$$\begin{array}{r} 10111 \\ -11011 \\ \hline -00100 \\ \hline \end{array}$$

Some digital computers subtract in this way (i.e.) by using column by column subtraction method.

This method involves complex circuitry. However there are other methods that require less circuitry. These methods use the concept of complements.

2.4 Binary Multiplication

The following 4 rules are used to multiply 2 Binary Numbers.

Rules :

$$0 \times 0 = 0$$

$$0 \times 1 = 0$$

$$1 \times 0 = 0$$

$$1 \times 1 = 1$$

To multiply larger Binary Numbers (i.e.) numbers with several bits, form the partial products and then add.

Example : Multiply 7 by 5

$$\begin{array}{r} 111 \\ 101 \\ 111 \\ 000 \\ \hline 111 \\ \hline 10011 \end{array}$$

Example : Multiple 22 by 6.

$$\begin{array}{r}
 10110 \text{ (22)} \\
 \times 110 \text{ (6)} \\
 \hline
 00000 \\
 10110 \\
 10110 \\
 \hline
 10000100 \text{ (132)}
 \end{array}$$

2.5 Binary Division

The process for dividing one Binary number (the dividend) by another (the divisor) is the same as that which is followed for decimal numbers. The actual process is simpler in Binary because when we are checking to see how many times the divisor “goes into” the dividend, there are only 2 possibilities, 0 or 1.

Example : $1001_2 \div 11_2$

$$\begin{array}{r}
 11 \\
 11 \overline{) 1001} \\
 \underline{11} \\
 11 \\
 \underline{11} \\
 0
 \end{array}$$

$1001_2 \div 11_2 = 11_2$

Example : $1010.01_2 \cdot 10_2$

$$\begin{array}{r}
 101.001 \\
 10 \overline{) 1010.01} \\
 \underline{10} \\
 010 \\
 \underline{10} \\
 0.010 \\
 10 \\
 \underline{00} \\
 1010.01_2 \cdot 10_2 = 101.001_2
 \end{array}$$

2.6 Addition of Octal Numbers

When adding two octal numbers if the sum equals $(8)_{10}$ it is taken as 0 to carry 1. Similarly, if the sum equals $(9)_{10}$, it is taken as 1 carry 1, etc.

Examples :

$$\begin{array}{r}
 (3)_8 \\
 + (4)_8 \\
 \hline
 (7)_8
 \end{array}$$

$$\begin{array}{r}
 (4)_8 \\
 + (4)_8 \\
 \hline
 (10)_8
 \end{array}$$

$$\begin{array}{r}
 (14)_8 \\
 + (35)_8 \\
 \hline
 (51)_8
 \end{array}$$

$$\begin{array}{r}
 (375)_8 \\
 + (560)_8 \\
 \hline
 (1155)_8
 \end{array}$$

2.7 Addition of Two Hexa Decimal Numbers

When adding two hexadecimal numbers if the sum equals $(16)_{10}$ it is taken as 0 to carry 1. Similarly, if the sum equals $(17)_{10}$, it is taken as 1 carry 1, etc.

Examples :

$$\begin{array}{r} (8)_{16} \\ + (7)_{16} \\ \hline (0F)_{16} \\ \hline \end{array}$$

$$\begin{array}{r} (8)_{16} \\ + (8)_{16} \\ \hline (10)_{16} \\ \hline \end{array}$$

$$\begin{array}{r} (8)_{16} \\ + (9)_{16} \\ \hline (11)_{16} \\ \hline \end{array}$$

$$\begin{array}{r} (18)_{16} \\ + (2A)_{16} \\ \hline (42)_{16} \\ \hline \end{array}$$

2.8 Summary

Binary arithmetic is essential part of all the digital computers and many other digital system.

It is a key for binary subtraction, multiplication, division. There are four rules of binary addition.

Subtraction and Borrow, these two words will be used very frequently for the binary subtraction.

Binary multiplication is similar to decimal multiplication. It is simpler than decimal multiplication because only 0s and 1s are involved.

Binary division is similar to decimal division. It is called as the long division procedure.

2.9 Model Questions

- (1) $(1101)_2 + (1011)_2$
- (2) $(10011.11)_2 + (11001.101)_2$
- (3) $(1101)_2 - (1011)_2$
- (4) $(11001.101)_2 - (10011.11)_2$
- (5) $(1101)_2 \times (1011)_2$
- (6) $(110110)_2 \div (101)_2$
- (7) $(1234)_8 + (7654)_8$
- (8) $(ABCD)_{16} + (4444)_{16}$

Solutions

- (1) $(11000)_2$
- (2) $(101101.011)_2$
- (3) $(0010)_2$
- (4) $(00101.111)_2$
- (5) $(10001111)_2$
- (6) Q : $(1010)_2$; R : $(100)_2$
- (7) $(11110)_8$
- (8) $(F011)_{16}$

LESSON - 3

COMPLEMENTS

Structure

- 3.0 Objectives**
- 3.1 Introduction**
- 3.2 Definition**
- 3.3 Subtraction with R's Complement**
- 3.4 Subtraction with (r-1)'s Complement**
- 3.5 Comparison Between 1's and 2's Complement**
- 3.6 Representing Signed Numbers**
- 3.7 Fixed Point Representation**
- 3.8 Addition with sign-magnitude Representation**
- 3.9 Addition with sign-1's Complement Representation**
- 3.10 Addition with sign-2's Complement Representation**
- 3.11 Overflow**
- 3.12 Summary**
- 3.13 Model Questions**

3.0 Objectives

To introduce and learn

- 9's complement and 10's complement in decimal number system.
- 1's complement and 2's complement in binary number system.
- Representation of negative numbers.
- Addition / Subtraction using complements.

3.1 Introduction

In this chapter we focus Complements. Complements are used in the digital computers in order to simplify the subtraction operation and for the logical manipulations. The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement. The 2's complement of binary number is obtained by adding 1 to the Least Significant Bit (LSB) of 1's complement of the number.

3.2 Definition

Complements are used in computers for simplifying the subtraction operations and for logical manipulations. There are two types of complements for each base, r (radix) system:

- (1) the r 's complement and
- (2) $(r-1)$'s complement

When the base value is substituted, the two types receive the names as 2's and 1's complement for Binary numbers or 10's and 9's complement for Decimal Numbers.

The r 's Complement : Given a positive Number N is base r with an integer part of n digits, the r 's complement of N is defined as

$$r^n - N \quad \text{for } N \neq 0 \text{ and}$$

$$0 \text{ for } N = 0$$

Example :

$$10\text{'s complement of } 5252010 = 10^5 - 52520 = 47480$$

$$10\text{'s complement of } 0.326710 = 1 - 0.3267 = 0.6733$$

$$10\text{'s complement of } 25.63910 = 100 - 25.639 = 74.361.$$

Example :

$$\begin{aligned} 2\text{'s complement of } 1011002 &= (2^6) - 101100 \\ &= 1000000 - 101100 \\ &= 010100. \end{aligned}$$

$$\begin{aligned}
 \text{2's complement of } 101.1001 &= (23) - 101.1001 \\
 &= 1000 - 101.1001 \\
 &= \begin{array}{r} 0010.0111 \\ 1000.0000 \\ 101.1001 \\ \hline 0010.0111 \end{array}
 \end{aligned}$$

The 10's complement of a Decimal Number can be formed by leaving all least significant zeros unchanged, subtracting the first non zero least significant digit from 10 and then subtracting all other higher significant digits from 9.

The 2's complement of a Binary Number can be formed by leaving all least significant zeros and the first Non zero digit unchanged and then replacing 1's by 0's by 1's in all other higher significant digits.

The (r-1)'s complement : Given a positive number N in base r with an integer part of n-digits and a fraction part of m digits, the (r-1)'s complement of N is defined as

$$r^n - r^m - N$$

$$\begin{aligned}
 \text{Example : 9's complement of } 52520 &= 10^5 - 10^0 - 52520 \\
 &= 105 - 1 - 52520 \\
 &= 99999 - 52520 \\
 &= 47479
 \end{aligned}$$

$$\begin{aligned}
 \text{9's complement of } 0.3267 &= 1 - 10^{-4} - 0.3267 \\
 &= 0.9999 - 0.3267 \\
 &= 0.6732
 \end{aligned}$$

$$\begin{aligned}
 \text{9's complement of } 25.639 &= 102 - 10^{-3} - 25.639 \\
 &= 100 - 0.001 - 25.639 \\
 &= 74.360
 \end{aligned}$$

$$\begin{aligned}
 \text{1's complement of } 101100 &= (2^6 - 1)_{10} - 101100 \\
 &= 111111 - 101100 \\
 &= 010011
 \end{aligned}$$

$$\begin{aligned}
 \text{1's complement of } 0.0110 &= (1 - 2^{-4})_{10} - 0.0110 \\
 &= 1 - 0.0001 - 0.0110 \\
 &= 0.1111 - 0.0110 \\
 &= 0.1001
 \end{aligned}$$

1.0000

0.0001

0.1111

The 9's complement of a Decimal Number is formed simply by subtracting every digit from 9. The 1's complement of a Binary number is formed simply by changing 0's to 1's and 1's to 0's.

From Definitions

$$\begin{aligned}
 r\text{'s complement} &= r^n - N \\
 (r-1)\text{'s complement} &= r^n - r^m - N \\
 (r-1)\text{'s complement} &= (r^n - N) - r^m \\
 &= r\text{'s comp} - r^m \\
 r\text{'s complement} &= (r-1)\text{'s comp} + r^m
 \end{aligned}$$

3.3 Subtraction with R's Complement

The subtraction of 2 positive number (M-N), both of base r may be done as follows:

- (a) Add the Minuend M to the r's complement of the subtra hend N.
- (b) Inspect the result obtained in Step (a) for an End-carry:
- (c) If an End-carry occurs discard it,

$$(a) \quad 72532 + 9\text{'s comp } (03250)$$

$$= 72532 + 96749$$

$$= + 69282$$

$$72532$$

$$96749$$

$$169281$$

$$+ 1$$

$$+ 69282$$

$$\text{Example : } 3250_{10} - 72532_{10}$$

$$M = 03250$$

$$N = 72532$$

Answer : 69282

$$(a) \quad 03250 + 9\text{'s comp } (72532)$$

$$= 03250 + 27467$$

$$= 30717 \text{ No. end. Carry}$$

Answer :- (9's comp (30717)

$$- 69282$$

3.5 Comparison between 1's and 2's Complements

The 1's complement has the advantage of being easier to implement by digital components since the only thing that must be done is to change 0's into 1's and 1's into 0's. The implementation of 2's complement may be obtained in 2 ways (i) by adding 1 to LSB of the 1's complement, (ii) by leaving all leading 0's in the LSB positions and the first 1 unchanged and then changing all 1's into 0's and all 0's into 1's.

During subtraction of 2 numbers by complements, the 2's complement is advantageous in that only one arithmetic addition operation is required. The 1's complement requires 2 Arithmetic additions when an end-around carry occurs.

The 1's complement has the additional disadvantage of possessing 2 arithmetic zeros : one with all 0's and one with all 1's.

Example : $1100 - 1100$

Using 1's complement $1100 + 0011 = 1111$ No carry

Answer :- (1's comp (1111))

= - 0000

Using 2's complement $1100 + 0100 = 0000$

Answer : + 0000

While the 2's complement has only one arithmetic zero, the 1's complement zero can be + ve or -ve which may complicate matters.

3.6 Representing signed Numbers

In digital computers, the binary number are represented by a set of binary storage devices usually Flip-Flops. Each device represents one bit. For example, 6 FF register could store binary numbers ranging from 000000 to 111111 (0 to 63 in Decimal). This represents the magnitude of the number, but does not give information about it's sign or the position of the Binary point. The sign is needed for arithmetic operations, as it shows whether the number is +ve or -ve. The position of the binary point is needed to represent integers, fractions or mixed integer fraction numbers.

The sign of a number in a discrete quantity of information having 2 values: Plus and minus. These 2 values can be represented by a code of One bit. The convention is to represent a plus with a 0 and a minus with a 1. To represent a signed Binary Number in a register we need $n = K+1$ Flip-flops. K flipflops, for the magnitude and one for storing the sign of the number.

The representation of the Binary point is complicated by the fact that it is characterised by a position between 2 Flipflops in the Register. There are 2 possible ways of specifying the position of the Binary point in a Register : (i) by giving it a fixed point position or (ii) by employing a floating-point representation. The 2 positions most widely used are (1) a binary point in the extreme left of the register to make the stored number integer a fraction and (2) a binary point in the extreme right of the registers to make the stored number an integer.

In both cases, the binary point is not physically visible but is assumed from the fact that the number stored in the register is treated as a fraction or an integer. The floating point representation uses a second register to store a number that designates the position of the binary point in the first register.

3.7 Fixed Point Representation

When a fixed-point binary number is positive the sign is represented by 0 and the magnitude by positive binary number. When the number is negative, the sign is represented by 1 and the rest of the number may be represented in any one of three different ways. These are :

1. Sign-magnitude
2. Sign-1's complement and
3. Sign-2's complement.

In the Sign-magnitude representation, the magnitude is represented by a +ve binary number. In the other 2 representations, the number is in the either 1's or 2's complement. If the number is +ve, then the three representations are the same.

As an example, assume that a 7-bit register is available to store the sign and the magnitude of the number. The number 9 in binary number system can be written as follows in the three different representations.

<u>Representation</u>	<u>+9</u>	<u>-9</u>
Sign Magnitude	0 001001	1 001001
Sign – 1's Complement	0 001001	1 110110
Sign – 2's Complement	0 001001	1 110111

3.8 Addition with Sign-Magnitude Representation

The process of adding 2 signed numbers, when the numbers are represented in sign-mag form requires that the comparison of their signs. If the 2 signs are the same, we add the 2 magnitudes. If the signs are not the same, we compare the relative magnitudes of the numbers and then subtract the smaller from the larger. It is necessary also to determine the sign of the result.

3.9 Addition with Sign-1's Complement Representation

The addition of 2 signed binary numbers represented by their 1's complement is obtained from addition of the 2 numbers, including their sign bits. If there is a carry out of the most significant bit, the result is incremented by 1 and its carry is discarded.

Example :	+6	0 000110	-6	1	111001
	+9	0 001001	+9	0	001001
	+15	0 001111	+3	1 0	000010

1

0
000011 = 3

Example :	+6	0	000110	-9	1	110111
	-9	1	110111	-9	1	110111
	-3	1	111101	-18	1 1	101100

1

1
101101

3.10 Addition with Sign 2's Complement Representation

The addition of 2 signed binary numbers with -ve numbers represented by their 2's complement is obtained from the addition of the 2 numbers including their sign bits. A carry in the MSB is discarded.

The advantage of sign-2's complement representation over the sign-1's comp form and sign-magnitude form is that it contains only one type of zero. The other 2 representations have both a +ve zero and a -ve zero.

Example :	+6	0	000110	-6	1	111010
	+9	0	001001	+9	0	001001
	<u>+15</u>	<u>0</u>	<u>001111</u>	<u>+3</u>	<u>1</u>	<u>000011</u>
Example :	+6	0	000110	-9	1	110111
	-9	1	110111	-9	1	110111
	<u>-3</u>	<u>1</u>	<u>111101</u>	<u>-18</u>	<u>1</u>	<u>000010</u>

Representation	+0	-0
Sign-magnitude	0 000000	1 000000
Sign-1's complement	0 000000	1 111111
Sign-2's complement	0 000000	-none-

3.11 Overflow

When 2 number of n-digits each are added and the sum occupies (n+1) digits, it is said that an overflow occurs. This is true for Binary numbers or Decimal numbers whether signed or unsigned. An overflow is a problem in a digital computer because the length of all registers, including memory register are of finite length. A result of n+1 bit cannot be accommodated in a register of standard length n.

An overflow cannot occur after an addition if one number is positive and the other is negative, since adding a +ve number to a -ve number produces a result (+ve or -ve) which is smaller than the larger of the 2 original numbers. An overflow may occur if the 2 numbers are added and both are +ve or both are -ve.

When 2 numbers in sign-magnitude representation are added, an overflow can be easily detected from the carry out of the number bits. When 2 numbers in sign-2's complement

representation are added, the sign bit is treated as part of the number and the end-carry does not necessarily indicate an overflow.

Consider the following examples :

Example 1 : Two signed binary numbers 35 and 40 are stored in two, 7-bit registers. The maximum capacity of the register is $(2^6 - 1) = 63$ and the minimum capacity is $-2^6 = -64$. Since the sum of the numbers is 75, it exceeds the capacity of the register. This is true, if the numbers are both positive or both negative.

	0	1		1	0
+ 35	0	100011	-35	1	011101
+ 40	0	101000	-40	1	011000
+ 75	1	001011	-75	1	0110101
		-53			+ 53

In either case the 7 bit result that should have been +ve is -ve and vice-versa. Obviously, the binary answer is incorrect and the algorithm for adding binary numbers represented in Sign-2's complement as followed previously fails to give correct results when an overflow occurs.

An overflow condition can be detected by observing the carry in to the sign-bit position and the carry out of the sign-bit position. If these 2 carries are not equal an overflow condition is produced. If the 2 carries are applied to an Ex-OR gate an overflow would be detected when the output of the gate is 1.

Example 2 : 2 signed binary numbers 10 and 9 are stored in 5-Bit register. The maximum capacity of the register is $(+2^4 - 1) = +15$ and the minimum capacity is $-2^4 = -16$.

	0	1		1	0
+ 10	0	1010	-10	1	0110
+ 9	0	10010	-9	1	0111
+ 19	1	0011	-19	0	1101
	1	0011 => -13		0	1101 => +13

3.12 Summary

Complements are used in the digital computers in order to simplify the subtraction operation and for the logical manipulations.

The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement.

The 2's complement of binary number is obtained by adding 1 to the Least Significant Bit (LSB) of 1's complement of the number.

3.13 Model Questions

- (1) Represent $(-37)_{10}$ in (a) sign magnitude (b) signed and 1's complement (c) signed and 2's complement.
- (2) Add $(+22)_{10}$ with $(-13)_{10}$ using 9's complement and 10's complement.
- (3) Add $(+34)_{10}$ with $(-56)_{10}$ representing them binary and using 1's complement and 2's complement.

Solutions

- (1) 1010 0101 ; 1101 1010 ; 1101 1011
- (2) 09
- (3) 1110 1001 ; 1110 1010

LESSON - 4

CODES

Strucuture

- 4.0 Objectives
- 4.1 Introduction
- 4.2 Binary Coded Decimal
- 4.3 8421 Code
- 4.4 BCD Addition
- 4.5 Gray Code
- 4.6 Binary to Gray Conversion
- 4.7 Gray to Binary Conversion
- 4.8 Excess-3 Code
- 4.9 Excess-3 Addition
- 4.10 Self Complementing Codes
- 4.11 Alpha Numeric Codes
- 4.12 ASCII Codes
- 4.13 EBCDIC
- 4.14 Parity Method for ErrorDetection
- 4.15 Summary
- 4.16 Model Questions

4.0 Objectives

After studying this lesson you should be able to Understand

- Binary Coded Decimal (BCD)
- Gray Code
- Excess-3 Code
- ASCII Code

4.1 Introduction

In this Chapter We Focus Binary Code. The digital data is represented, stored and transmitted as group of binary bits. This group is also called as binary code. BCD is a way to express each of the decimal digits with a binary code. In the BCD, with four bits we can represent sixteen numbers (0000 to 1111).

4.2 Binary Coded Decimal (BCD)

Decimal, Octal and Hexa decimal numbers can be represented by Binary Digits. Not only numbers, but letters and other symbols can be represented by 1's and 0's. In fact, any entity expressible as numbers, letters or other symbols can be represented by Binary Digits, and therefore can be processed by digital logic circuit. Combinations of binary digits that represent numbers, letters or symbols are digital codes.

4.3 8421-Code

8421 code is a type BCD code and is composed of 4-bits representing the decimal digits 0 thro' 9. The 8421 indicates the binary weights of the 4 bits (2^3 , 2^2 , 2^1 , 2^0).

The ease of conversion between 8421 code numbers and the decimal numbers is the main advantage of the code. The 8421 code is the predominant BCD, and it is referred to BCD, it always mean the 8421 code unless otherwise stated.

Binary coded decimal means the each decimal digit is represented by a Binary code of 4-bits. All we have to remember is that the Ten Binary combinations out of 16-possible combinations of 4-bits, that represents the 10 decimal digits. The other six combinations are not used invalid in 8421 (BCD) codes

Decimal	8421	
0	0000	
1	0001	1010
2	0010	1011
3	0011	1100
4	0100	1101
5	0101	1110

6	0110	1111
7	0111	
8	1000	
9	1001	

- * To express any decimal number in BCD, simply replace each decimal digit by the approximate 4-Bit code.

Example :	3	9.2				34.8	
	0011	1001.0010				0011 0100.1000	
1	5	0	1	4	7	2	
0001	0101	0000	0001	0100	0111	0010	

- * To express any 8421-BCD number in Decimal, start at Binary point, and break the code into groups of 4-Bits, then write the Decimal Digit represented by each 4-Bit Group.

Example :	1000	0110	1001	0111	0100
	<u>8</u>	<u>6</u>	<u>9</u>	<u>7</u>	<u>4</u>
	0001	1000	0110	0000	0111
	<u>1</u>	<u>8</u>	<u>6</u>	<u>0</u>	<u>7</u>

4.4 BCD-Addition

1. Add the two numbers, using the rules of Binary Addition.
2. If a 4-Bit sum is ≤ 9 , it is valid BCD-number
3. If a 4-Bits sum is > 9 or if a carry-out of the group is generated, it is an invalid result. Add 6(=0110) to the 4-bit sum in order to skip the six invalid states and return the code to 8421. If a carry result when 6 is added, simply add the carry to the next 4-bit group.

Ex :

6	0110	23	0010	0011
+2	0010	+45	0100	0101
+8	1000	68	0110	1000

Example :

9	1001		9	1001
+4	0100		+9	1001
13	1101 Invalid BCD		18	10010 Invalid BCD
	(>9)			(>9)
	+0110 Adds 6(0110)		+0110 Add 6 (0110)	
0001	0011 Valid BCD		0001	1000 Valid BCD
1	3		1	8

Example :

67	0110	0111	
53	0101	0011	
120	1011	1010	(Both Groups are invalid (>9))
	0110	0110	Add 6 to Both groups
0001	0010	0000	Valid BCD
1	2	0	

Example :

16	0001	0110	
15	0001	0101	
31	0010	1011	Right Group is invalid (>9)
		0110	Left Group is valid. Add 6 to Right
0011	0001		Group Valid BCD
3	1		

4.5 Gray Code

The gray code is an unweighted code, which means that there are no specific weights assigned to the bit position. The Gray code exhibits only a single-Bit change from one code number to next. This property is important in many applications, where error susceptibility increases with the number of bit changes between adjacent numbers in a sequence. Like binary the gray code can have any number of bits.

Decimal	Binary	Gray
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101

Ex : Numbers going from 3 to 4, the gray code changes from 0010 to 0110, while the binary code changes from 0011 to 0100, a change of 3-bits. The only bit change in this is third bit from the right in the gray code.

4.6 Binary to Gray Conversion

1. The most significant digit (left-most) in the gray code is the same as the corresponding digit in the binary number.
2. Digits Going from left to right in a binary number, add each adjacent pair of Binary digits to get the next Gray code digit. Discard carries.

Ex : 10110_2 to Gray Code 1 0 1 1 0

1	
1 1	$1+0=1$
1 1 1	$0+1=1$
1 1 1 0	$1+1=0$
Gray → 1 1 1 0 1	$1+0=1$

Ans : 1 1 1 0 1

4.7 Gray to Binary Conversion

1. The most significant digit in the Binary is the same as the corresponding digit in the gray code.
2. Add each Binary digit generated to the Gray Digit in the next position. Discard carries.

Example : Gray 1 1 0 1 1 to Binary

1 1 0 1 1	Gray
1 0 0 1 0	Binary

4.8 Excess-3 Code

The Excess-3 code is a Digit code that is derived by adding 3 to each 8421 BCD Code, its also a 4-bit binary Code. Since no definite weights can be assigned to the 4-Bit digit positions, Excess-3 is an unweighted code.

Example : Excess code for Decimal 2 is

$$2 + 3 = 5 \rightarrow 0101$$

Excess code for Decimal 9 is

$$9 + 3 = 12 \rightarrow 1100$$

Decimal	Binary	Ex-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100

Also in Excess 3 only 10 of the possible 16 combinations of 4-Bit are used. The six invalid combinations are

0000, 0001, 0010

1101, 1110, 1111

4.9 Excess-3 Addition

Case (1)

In Excess-3 code whenever 2 Decimal digits are added whose sum is ≤ 9 , an Excess-6 number results. To return to Excess-3 form, subtract 3.

Example :	2	0101	Excess-3 of 2
	+5	1000	Excess-3 of 5
	7	1101	Excess-6 of 7
		-0011	Subtract 3
		1010	-> Excess-3 of 7.

Example :	43	0111	0110	Excess-3 of 43
	+56	1000	1001	Excess-3 of 56
	99	1111	1111	Excess-6 of 99
		-0011	-0011	Subtract 3 from each group
		1100	1100	Excess-3 of 99

Case (2)

Whenever the sum of 2 Decimal digits exceed or 9, there will be a carry from one group into the next. When this happens, the group that produced the carry will result to 8421 form. To restore the answer to Excess - 3 code. Three must be added to the group that produced the carry. If any group does not produce a carry, subtract 0011 from that group.

Example : 29	0101	1100	Excess-3 of 29
+39	0110	1100	Excess-3 of 39
68	1100	1000	First result
	-0011	+0011	Subtract and add 3
	1001	1011	Excess-3 of 68

4.10 Self-Complementing Codes

The BCD codes 4221, 2421 and Excess-3 are referred to as self-complementing codes. Because of having the special property that

“The 1’s complement of the Binary representation of these codes produce the 9’s complement of the decimal representation”

Example : 4221-code of 5 -> 0111
 1's comp of 0111 -> 1000
 Decimal Equivalent of 1000 -> 4
 (i.e) 4 is the 9's complement of 5.

4.11 Alphanumeric Codes

In addition to numerical data, a computer should recognize codes that represents letters of the Alphabet, Punctuation Marks and other special characters. These codes are called Alphanumeric codes. A complete Alphanumeric code would include the 26 lowercase letters, 26 uppercase letters, 10 numeric digits, 7 punctuation marks, and anywhere from 20 to 40 other characters, such as +, -, x, /, #, % and so on.

4.12 ASCII

American standard code for information interchange - code is alphanumeric code. This is a 7-Bit code. With 7-Bits 128 (= 2⁷) characters can be coded, which is enough for the full upper and lower-case alphabets, numbers, punctuations and control characters. The following shows some ASCII symbols and their corresponding decimal hex code/binary equivalent.

ASCIISymbol	Decimal	Hex	Binary	(7 Bit)
A	65	41	100	0001
B	66	42	100	0010
Z	90	54	101	1010
a	97	61	110	0001
b	98	62	110	0010
Z	122	74	111	1010
0	48	30	011	0000
1	49	31	011	0001
9	57	39	011	1001

4.13 EBCDIC

Extended BCD interchange code is another alphanumeric code. This is an 8-Bit Code. The following shows some EBCDIC symbols and their corresponding Decimal/Hex/Binary Equivalent.

EBCDIC

Symbol	Decimal	Hex-code	Binary	(8 Bit)
A	192	C1	1100	0001
B	194	C2	1100	0001
:				
I	201	C9	1100	1001
J	209	D1	1101	0001
K	210	D2	1101	0010
:				
R	217	D9	1101	1001
S	226	E2	1110	0010
T	227	E3	1110	0011
:				
Z	233	E9	1110	1001
a	129	81	1000	0001
b	130	82	1000	0010
:				
i	137	89	1000	1001
j	145	91	1001	0001
k	146	92	1001	0010
:				
r	153	99	1001	1001
s	162	A2	1010	0010
t	163	A3	1010	0011
:				
z	169	A9	1010	1001
0	240	F0	1111	0000
1	241	F1	1111	0001
:				
9	249	F9	1111	1001

Example : To represent “ASCII” in ASCII-code & EBCDIC-code

A	100	0001		1100	0001
S	101	0011		1110	0010
C	100	0011	ASCII	1100	0011
I	100	1001		1100	1001
I	100	1001		1100	1001
					EBCDIC

4.14 Parity Method for Error Detection

The movement of Binary data and codes from one location to another is the most frequent operation performed in Digital Systems.

- Example :
1. The storage and retrieval of data from external memory devices such as magnetic tape and disk.
 2. The transmission of information from a computer to a remote user terminal or another computer.

Whenever information is transmitted from one device (Transmitter) to another device (Receiver) there is possibility that errors can occur such that the receiver does not receive the identical information that was sent by the transmitter. The major cause of any transmission errors is Electrical Noise, which consists of spurious fluctuations in Voltage or current that are present in all electronic systems to varying degree.

The transmitter sends a relatively noise-free serial digital signal over a signal line to a receiver. However, by the time the signal reaches the receiver, it contains a certain degree of noise super imposed on the origin signal. When this occurs, the receiver may incorrectly interpret that bit as a logic 1, which is not what the transmitter has sent.

Many digital system employ some method for detecting the errors. One of the simplest and widely used schemes for error detection is the parity method.

Parity is the term used to identify whether a data word has an odd or even number of 1's. If a data word contains an odd number of 1's the word is said to have odd parity. The binary word 1101 1001 with five 1's has odd parity. The binary word 1100 0011 has four 1's and so has an even parity.

When transmitting a digital data a parity bit (0 or 1) is included so that the total number of 1's is always even (or odd). Suppose the transmitted binary data including parity bit has even parity. At the receiving end when parity check is done it should be even. Otherwise it is assumed, an error has occurred (assuming only one error has occurred). Depending on the input binary data a parity generator circuit produces a parity bit. As the receiving end a parity checker circuit checks the parity.

4.15 Summary

The digital data is represented, stored and transmitted as group of binary bits. This group is also called as **binary code**.

BCD is a way to express each of the decimal digits with a binary code. In the BCD, with four bits we can represent sixteen numbers (0000 to 1111).

Gray code is the non-weighted code and it is not arithmetic codes. That means there are no specific weights assigned to the bit position. It has a very special feature that, only one bit will change each time the decimal number is incremented

The Excess-3 code is also called as XS-3 code. It is non-weighted code used to express decimal numbers.

4.16 Model Questions

- (1) Represent $(123)_{10}$ in 8421 code.
- (2) Add $(23)_{10}$ and $(36)_{10}$ represent them in BCD.
- (3) Add $(38)_{10}$ and $(95)_{10}$ represent them in BCD.
- (4) Convert $(39)_{10}$ to Binary and Gray.
- (5) Represent $(35)_{10}$ and $(46)_{10}$ in excess 3 and add them.

Solutions

- (1) 0001 0010 0011
- (2) 0101 1001
- (3) 0001 0011 0011
- (4) 0010 0111 ; 0011 0100
- (5) 1011 0100

LESSON - 5

LOGIC GATES

Structure

- 5.0 Objectives**
- 5.1 Introduction**
- 5.2 The OR gate**
- 5.3 Truth Table**
- 5.4 Two State Operation**
- 5.5 Three Inputs**
- 5.6 Logic Symbols**
- 5.7 The AND gate**
- 5.8 Symbols**
- 5.9 Positive and Negative Logic**
- 5.10 The NOT gate**
- 5.11 NAND gate**
- 5.12 NOR gate**
- 5.13 Summary**
- 5.14 Model Questions**

5.0 Objectives

After studying this lesson you should be able to Understand

- OR gate using diode logic
- AND gate using diode logic
- NOT gate using transistor
- NAND and NOR gate

5.1 Introduction

In this Chapter we focus Logic gates. Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. logic gates are named as AND gate, OR gate, NOT gate etc.

5.2 The OR Gate

A gate is a logic circuit with one output and one or more inputs; an output signal occurs only for certain combinations of input signals.

The first kind of gate we study is the OR gate. Figure 1a shows a two-input OR gate where A and B are the inputs and Y is the output. For the moment, let us analyze this OR-gate by restricting the output voltages to either 0 or 1 V. There are only four possible cases to analyze.

- | | | |
|--------|---|---|
| Case 1 | : | A = 0 and B = 0. With both input voltages at zero the output voltage must be zero because no voltage exists anywhere in the circuit. Therefore, $y = 0$. |
| Case 2 | : | A = 0 and B = 1. The B battery forward-biases the lower diode, causing the output to be ideally 1 V. Since the A battery is 0 V, it looks like a short. Figure 1 (b) shows the circuit for this condition. The upper diode is off, the lower diode is on, and the output $y = 1$ V. |
| Case 3 | : | A = 1 and B = 0. Because of the symmetry of the circuit the argument is similar to case 2. The upper diode is on, the lower diode is off, and $y = 1$ V. |
| Case 4 | : | A = 1 and B = 1. With both inputs at 1 V both diodes are forward-biased. Since the voltages are in parallel, the output voltage is ideally 1 V. Therefore, $y = 1$ V as shown in Fig.1 (c) |

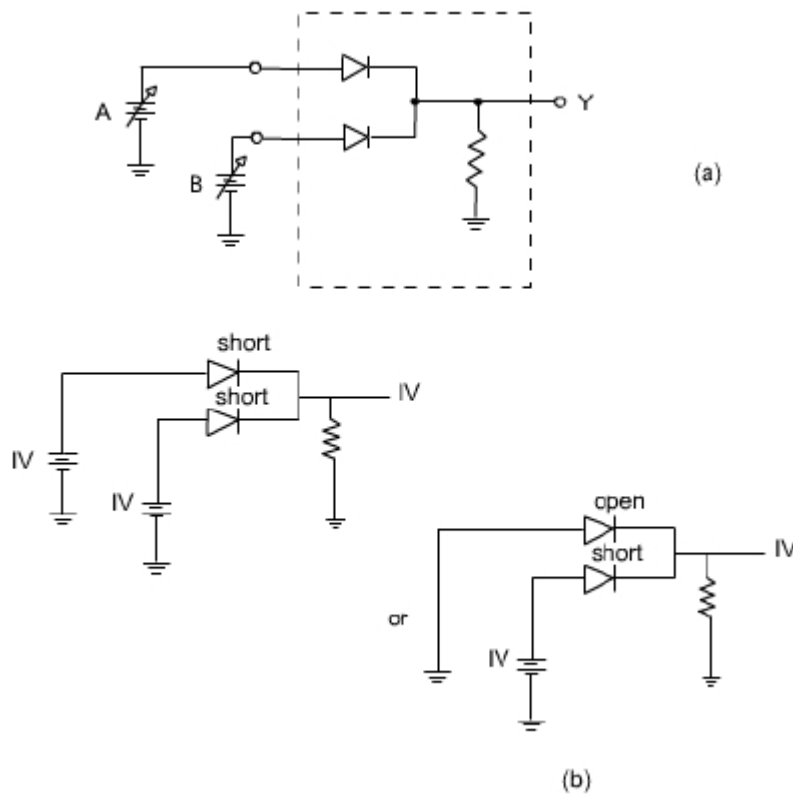


Fig. 1. Two-input OR gate circuit

5.3 Truth Table

Table 1 lists the input-output conditions of an OR gate. Examine this table carefully and memorize the following: the OR gate has a 1 output when either A or B or both are 1. In other words, the OR gate is an any-or-all gate; an output occurs when any or all of the inputs are present.

Table 1

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Incidentally, a truth table (sometimes called a table of combinations) is a table that shows all input/output possibilities for a logic circuit. Table 1 is one example of a truth table. All possible AB inputs, 00, 01, 10, and 11, are shown along with resulting outputs.

5.4 Two-State Operation

Most digital circuits use diodes and transistors as Switches to change from one voltage level to another. When we analyze digital circuits, we determine whether a voltage is low or high. The exact magnitude or value is unimportant, as long as the voltage is distinguishable as low or high.

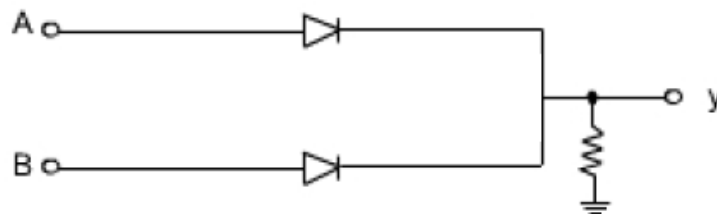


Fig.2. Inputs are either low (2V) or high (10V).

In digital circuits low and high voltages are often represented by 0 and 1, respectively. For instance, in the OR gate of Fig. 2 the input levels are either 2 or 10 V - low or high. The operation of this circuit is like that of Fig. 1; Table 2 shows the truth table. If we let 0 stand for 2 V and 1 for 10 V, we can show an equivalent truth table with 0's and 1's (Table 3). The point is that we can use the actual voltages, or we can use 0's and 1's to represent the low and high voltages; in either case, the OR gate has a high output when any or all inputs are high.

5.5 Three Inputs

Figure 3 shows a three-input OR gate. If A or B C is high, y will be high because the diode associated with the high input will turn on. Letting 0 stand for low and 1 for high, we get the truth table of Table 4.

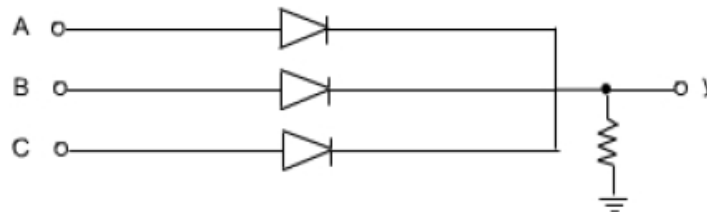


Fig.3. Three-input OR gate circuit

Table 2

A	B	Y
2 V	2 V	2 V
2 V	10 V	10 V
10 V	2 V	10 V
10 V	10 V	10 V

Table 3

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Table 4

A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Incidentally, the number of horizontal rows in a truth table equals 2^n where n is the number of inputs. For a two-input gate, the truth table has 2² or 4 rows. A three input gate will have a truth table with 2³ or 8 rows, while four-input gate results in 2⁴ or 16 rows, and so on. To include all possible entries, it helps to list the entries in a binary-number progression. In Table 4 we deliberately listed the ABC entries as 000, 001, 010, 011, following the binary-number count; this guarantees that we will not forget any possibility.

An OR gate can have as many inputs as desired; add one diode for each additional input. Six diodes result in six-input OR gate, nine diodes in a nine-input OR gate. No matter how many inputs, the action of any OR gate is summarized like this: One or more high inputs produce a high output.

5.6 Logic Symbols

Figure 4(a) shows the symbol for a two-inputs OR gate of any design. Whenever you see this symbol, remember the output is high if either input is high.

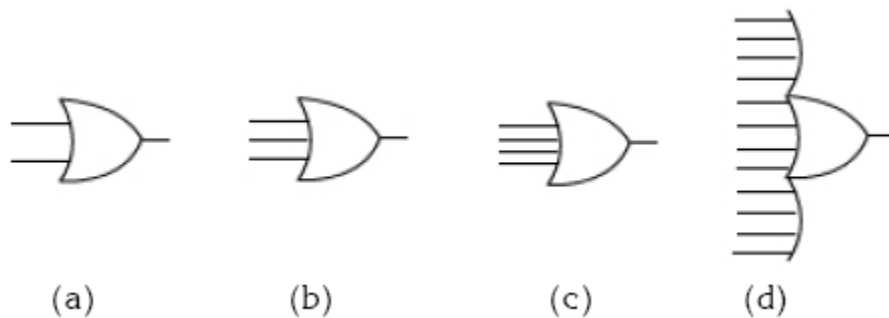


Fig. 4. OR-gate symbols.

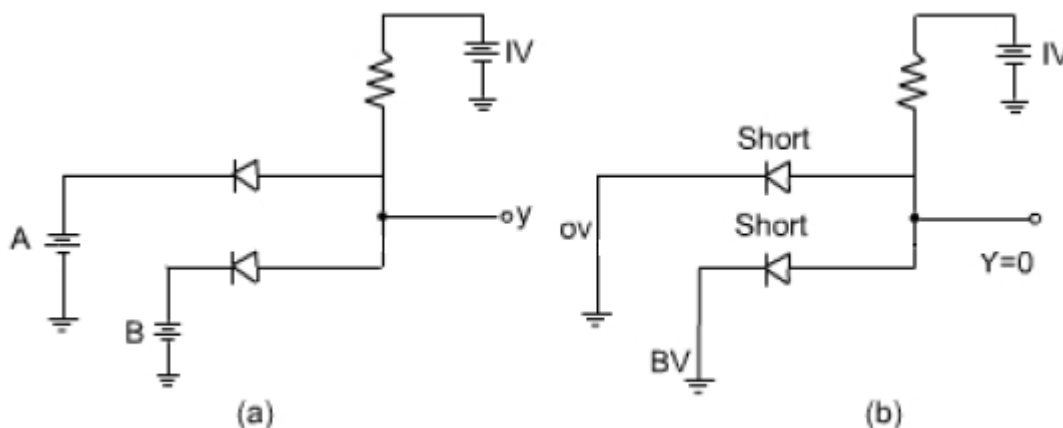
- | | |
|-----------------|------------------|
| (a) Two-input | (c) Four-input |
| (b) Three-input | (d) Twelve-input |

Shown in Fig.4b is the logic symbol for a three-input OR gate. Figure 4c is the symbol for a four-input OR gate. Fig.4d is the symbol for 12-input OR gate. Remember: The output is high when any input is high.

5.7 The AND Gate

The AND gate is another basic kind of digital circuit - it has an output only when all inputs are present. Consider the two input AND gate of Fig.5a. Again, use ideal diodes and restrict all voltages to either 0 or 1 V. There are four cases to analyze.

- Case 1 : $A = 0$ and $B = 0$. Since both input batteries are a 0V, they are like shorts (Fig.5b). The 1-V battery forces conventional current in the direction of each diode triangle; therefore, both diodes are on or shortened. Carefully examine Fig.5b and realize that the output is shorted to ground through the diodes and the batteries. Therefore, $y = 0$.
- Case 2 : $A = 0$ and $B = 1$. The upper diode is forward biased as shown in Fig.5c, the output is still shorted ground through the upper diode and the battery. Therefore, $y = 0$.
- Case 3 : $A = 1$ and $B = 0$. Because of symmetry the argument is similar to case 2, and $y = 0$.
- Case 4 : $A = 1$ and $B = 1$. No current flows in the circuit Fig.5d. With no current in R there is no voltage drop across R ; therefore, y must equal 1 V.



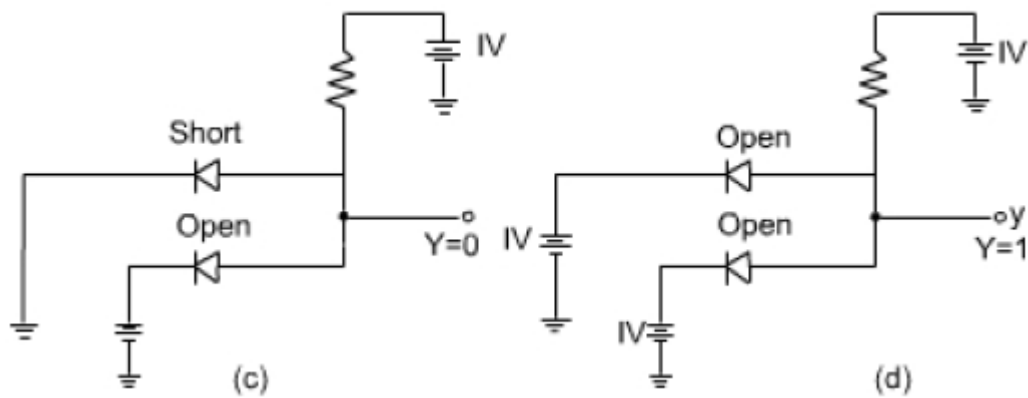


Fig. 5 - Two-input AND gate circuit

As usual, we can summarize the circuit action by a truth table (Table 5). Examine this table carefully and memorize the following : the AND gate has a 1 output when A and B are 1. In other words, the AND gate is an all-or-nothing gate; and output occurs only when all inputs are present.

The use of 0 and 1 V was only a convenience for analysis. We can use any two distinct voltages. In Fig. 5a suppose the 1-V battery is changed to a 10-V battery, and the inputs are either 2 or 10 V. If we use actual voltages, we get the truth table of Table 6.

Table 5

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Table 6

A	B	C
2V	2V	2V
2V	10V	2V
10V	2V	2V
10V	10V	10V

The AND gate can have more than two inputs. For instance Fig. 6. shows a three-input AND gate. Assume the two distinct voltage levels are 0 and 10V. If any input is at 0 V (grounded) the diode connected to that input is forward-biased or shorted ; therefore, the output will be shorted to ground. Therefore, $y = 0$ when any input is 0. The only way to make $y = 0$ is to have all inputs simultaneously equal to 10V. In this case no current flows through R, and the output voltage rises to the supply voltage. If we let 0 stand for low voltage and 1 for high voltage, we get the truth table of Table 7. AND gates can have as many inputs as desired ; add one diode for each additional input. For instance, 8 diodes result in an 8-input AND gate, 16 diodes in a 16-input AND gate. No matter how many inputs an AND gate has, the action can be summarized like this. This AND gate is an all-or-nothing gate; all inputs must be high to get a high output.

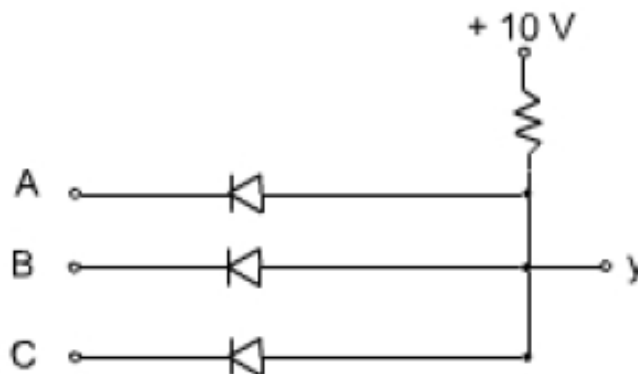


Fig.6. The input AND gate circuit

5.8 Symbols

Table 7

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Figure 7a is the symbol for a two-input AND gate of any design. Figure 7b shows the three-input AND gate, and Fig 7c shows the four input AND gate. For all these gates, all inputs must be high to get a high output.

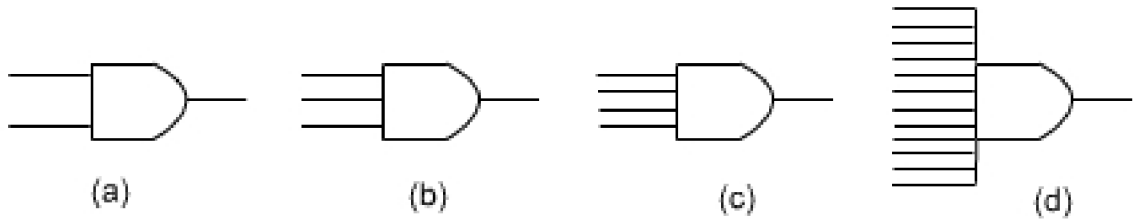


Fig.7 .AND gate symbols

- | | | | |
|-----|------------|-----|--------------|
| (a) | Two-input | (b) | Three-input |
| (c) | Four-input | (d) | Twelve-input |

5.9 Positive and Negative Logic

A positive logic system is a system in which a 1 represents the more positive of the two voltage levels; in a negative logic system the 1 stands for the more negative voltage. For instance, suppose a digital system has voltage level of +5 and 0 V. We are free to choose the definitions for a 0 and a 1. If we say a 1 stands for +5 V and a 0 for 0 V, the system becomes a positive logic system. On the other hand, if we let 1 denote 0 V and let 0 symbolize +5V, we have a negative logic system (up to now, we have been using positive logic).

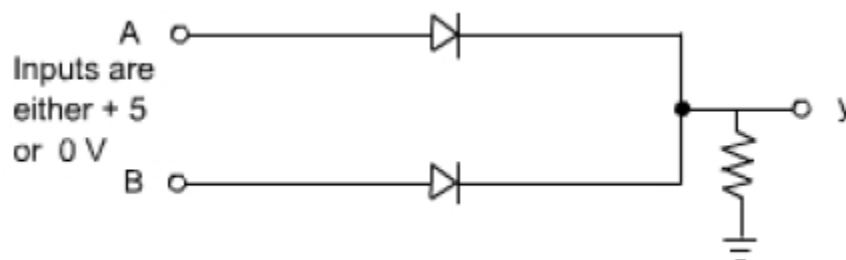


Fig.8. Positive OR gate versus negative AND gate

The distinction between positive and negative logic is important because an OR gate in a positive logic system becomes an AND gate in a negative logic system. To understand why, consider the circuit of Fig.8. The truth table for this circuit (Table - 8) gives actual voltage levels. The output is 5V when either A or B is 5V. Is this circuit an OR gate or an AND gate? The answer depends on how we define 0 and 1. If 1 stands for +5 V and 0 for 0 V, we have a positive logic

system and can show the truth table as in Table 9. This the truth table of an OR gate because there is a 1 output when A or B is 1.

On the other hand, if a 1 stands for 0 V and a 0 for +5 V, we have a negative logic system. By looking at Table 8, in writing a 1 for each 0V entry and 0 for each 5V entry, we get the truth table of Table 10. By inspection of this table a 1 output occurs only when A and B are 1; therefore, the circuit is an AND gate in a negative logic system.

Table 8

A	B	Y
0 V	0 V	0 V
0 V	5 V	5 V
5 V	0 V	5 V
5 V	5 V	5 V

Table 9

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Table 10

A	B	Y
1	1	1
1	0	0
0	1	0
0	0	0

A digital system can either be a positive or negative logic system; it all depends on how we define 0 and 1. The choice of which system to use is arbitrary - like defining the direction of current from + to - or from - to +. For consistency we shall use positive logic.

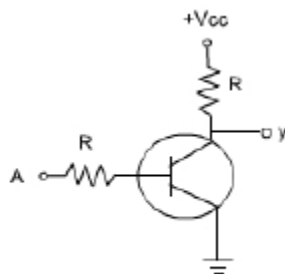
5.10 The NOT Gate

Another of the basic digital circuits is the NOT gate, also called an inverter. This circuit has one input and one output. All it does is invert the input signal; if the input is high, the output is low, and vice versa.

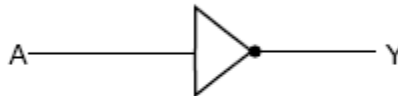
Figure 9a shows one way to build a NOT circuit. When the input voltage is high enough, the transistor saturates; therefore, the output is low. On the other hand, when the input voltage is low enough, the transistor cuts off, and the output voltage is high. The truth table of the NOT circuit is

Input	Output
0	1
1	0

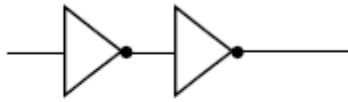
We call this circuit a NOT circuit because the output is not the same as the input.



(a) NOT circuit



(b) logic symbol



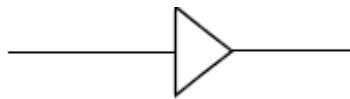
(c) cascaded inverters.

Fig. 9 NOT gate

Figure 9(b) shows the logic symbol for an inverter. Whenever you see this symbol, recall that the output is the complement (opposite) of the input.

If you cascade two inverters. [Fig.9(c)], you get a noninverting amplifier whose final output is identical to the input. Remember: Double inversion means the output is the same as the input.

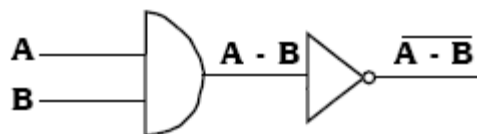
This is also called as a buffer with a symbol as shown in Fig. 9(d).



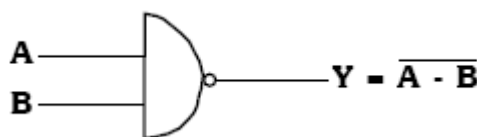
Note that there is no inverting bubble at the output.

5.11 NAND Gate

NAND is another popular gate which can be imagined as combination of AND gate followed by NOT gate.



This arrangement is available as a single gate with a logic symbol as shown :

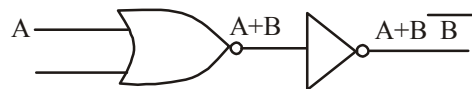


The corresponding truth table is :

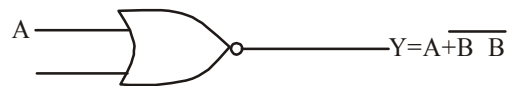
A	B	$Y = A-B$
0	0	1
0	1	1
1	0	1
1	1	0

5.12 NOR Gate

A NOR gate is a combination of OR gate followed by a NOT gate.



This arrangement also is available as a single gate with a logic symbol as shown.



The truth table is given

A	B	$Y = \overline{A+B}$
0	0	1
0	1	1
1	0	1
1	1	0

5.13 Summary

Logic gates are the basic building blocks of any digital system. It is an electronic circuit having one or more than one input and only one output. Logic gates are named as AND gate, OR gate, NOT gate etc.

NOT gate is also known as **Inverter**. It has one input and one output

A NOT-AND operation is known as NAND operation. It has n input ($n \geq 2$) and one output.

A NOT-OR operation is known as NOR operation. It has n input ($n \geq 2$) and one output.

5.14 Model Questions

1. Draw the OR gate with truth table
2. Draw the AND gate with truth table
3. Define NOR and NAND gate

LESSON - 6

BOOLEAN ALGEBRA AND LOGIC CIRCUITS

Structure

- 6.0 Objectives**
- 6.1 Introduction**
- 6.2 OR Addition**
- 6.3 AND Multiplication**
- 6.4 NOT Operation**
- 6.5 De Morgan's Theorem**
- 6.6 Circuit Implications of First Theorem**
- 6.7 Circuit Implication of Second Theorem**
- 6.8 Switching Circuit**
- 6.9 1-OF-16 DECODER**
- 6.10 The Universal Building Block**
- 6.11 Inverter**
- 6.12 AND gate**
- 6.13 OR gate**
- 6.14 Summary**
- 6.15 Model Questions**

6.0 Objectives

After studying this lesson you should be able to Understand

- Logic circuits using AND, OR, NOT gates
- De Morgan's theorem
- Encoders and Decoders
- NAND as a universal gate

6.1 Introduction

In this chapter we focus ordinary algebra when we solve an equation for its roots, we can get a real number - positive, negative, fractional, and so forth. In other words, the set of numbers in ordinary algebra is infinite. In Boolean algebra when we solve an equation, we get either a 0 or a 1. No other answers are possible, because the set of numbers includes only the binary digits 0 and 1.

6.2 OR Addition

Another difference about Boolean algebra is the meaning of the plus sign. To bring out this meaning, consider Fig.1. which shows a two-input OR gate with A and B inputs and a y output.

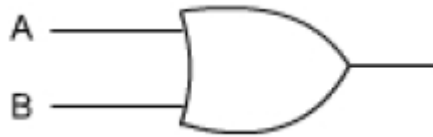


Fig.1. Two-input OR gate.

In Boolean algebra the + sign symbolizes the action of an OR gate. In other words, we may think of the OR gate as a device that combines A with B to give a result of y. In Boolean algebra when we write

$$y = A + B$$

We mean that A and B are to be combined in the same way that an OR gate combines A and B. Read the expression $y = A + B$ as y equals A or B. To repeat, the + sign does not stand for ordinary addition; it stands for OR addition whose rules are given by the OR truth table (Table 1).

To get used to OR addition, let us work out the value of $y = A + B$ for the four possible input conditions.

- | | | |
|--------|---|---|
| Case 1 | : | A = 0 and B = 0. We have $y = A + B = 0 + 0 = 0$ because an OR gate combines 0 with 0 to give 0. |
| Case 2 | : | A = 0 and B = 1. This gives $y = A + B = 0 + 1 = 1$ because an OR gate combines 0 with 1 to give 1. |

Case 3 : $A = 1$ and $B = 0$. This is like case 2. $y = A + B = 1 + 0 = 1$

Case 4 : $A = 1$ and $B = 1$. We get $y = A + B = 1 + 1 = 1$ because an OR gate combines 1 with 1 to give 1.

6.3 AND Multiplication

The multiplication sign has a new meaning in Boolean algebra. To understand this meaning, look at the AND gate of Fig.2.



Fig.2. AND-gate multiplication.

Think of an AND gate as a device that combines A and B to give a result of y. So in Boolean algebra when we write

$$y = A \cdot B$$

or

$$y = AB$$

we mean A and B are to be combined in the same way that an AND gate combines A with B to give a y-output. For practice, let us solve $y = AB$ for the four possible cases.

Case 1 : $A = 0$ and $B = 0$. This gives $y = AB = 0 \cdot 0 = 0$ because an AND gate combines 0 with 0 to give 0.

Case 2 : $A = 0$ and $B = 1$. We get $y = AB = 0 \cdot 1 = 0$ because an AND gate combines 0 with 1 to give 0.

Case 3 : $A = 1$ and $B = 0$. This is like case 2 : $y = AB = 1 \cdot 0 = 0$ because an AND gate combines 1 with 0 to give 0.

Case 4 : $A = 1$ and $B = 1$. This gives $y = AB = 1 \cdot 1 = 1$

These four results are easy to remember. Even though the dot does not mean multiplication in the ordinary sense, the results of AND multiplication are the same as for ordinary multiplication.

Example 1

What is the Boolean equation for the output of Fig.3?

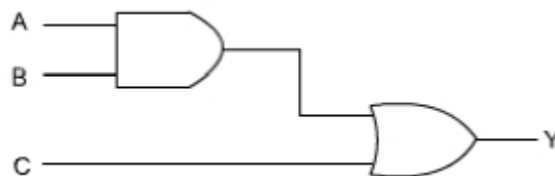


Fig.3

Solution

The output of the AND gate is AB . This goes into the OR gate as one of the inputs; the other input is C . Therefore, the final output is $y = AB + C$.

Example 2

Evaluate the Boolean equation of the preceding example for each of these input conditions:

- $A = 0, B = 1, C = 1$
- $A = 1, B = 0, C = 0$

Solution

- Substitute the given inputs and work out an answer as follows. $y = AB + C = 0 \cdot 1 + 1 = 0 + 1 = 1$

(Multiply before adding, just as in ordinary algebra)

- For $A = 1, B = 0, C = 0$
 $y = AB + C = 1 \cdot 0 + 0 = 0 + 0 = 0$

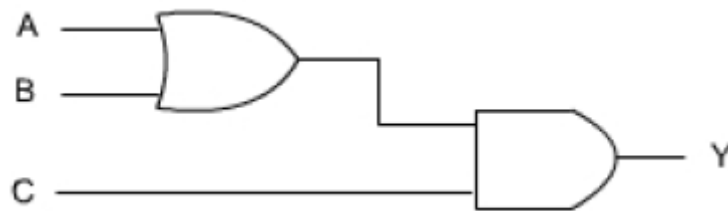
Example 3

Find the Boolean equation for the output of Fig.4, and evaluate the expression for $A = 0$, $B = 1$, and $C = 1$.

Solution

The output of the OR gate is $A + B$. This goes into the AND gate, along with C . The final output is

$$y = (A + B) C$$

**Fig.4**

(Note that the parentheses are also used to denote AND multiplication). When we substitute the given input values.

$$y = (A + B)C = (0 + 1) 1 = (1) 1 = 1.$$

Example 4

What is the Boolean equation for the output of Fig.5a?

Solution

The output of the first OR gate is $A + B$. This goes into the second OR gate, along with C , to produce a final output of

$$y = A + B + C$$

This output is identical to the output from a three-input OR gate like Fig.4. In other words, one way to build a three-input OR gates is by cascading two-input OR gates, as shown in Fig.5a.

You can expand the number of inputs by cascading more gates. For example, Fig.5b shows a cascade of three OR gates; the overall circuit is equivalent to a four-input OR gate with a Boolean equation of

$$y = A + B + C + D$$

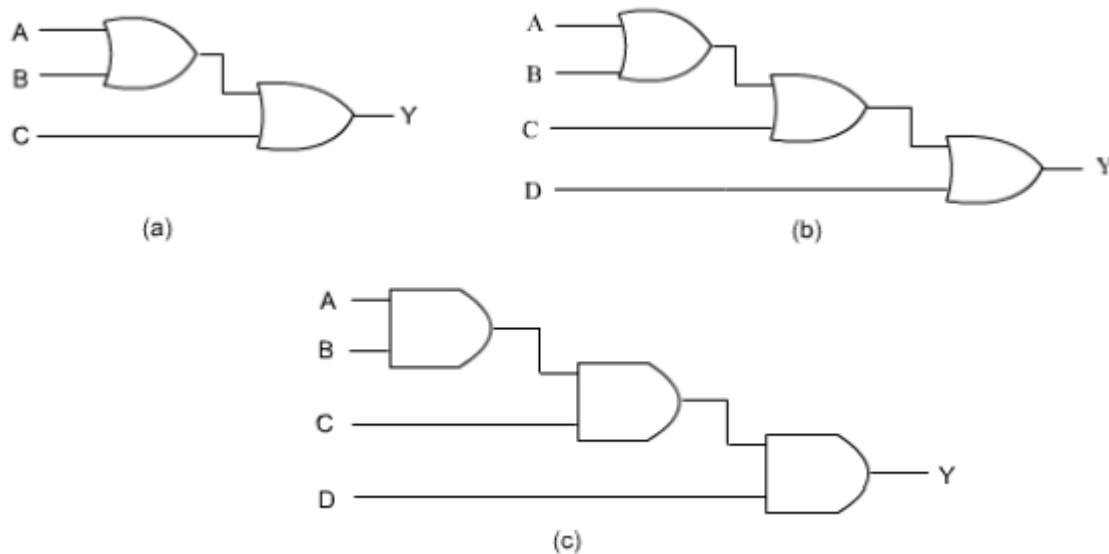


Fig.5.

A similar concept applied to AND gates. Cascading two-input AND gates results in an equivalent AND gate with more inputs. For example, Fig.5c shows three cascaded AND gates. The final output has a Boolean equation of

$$y = ABCD$$

This output is identical to the output from a four-input AND gate.

6.4 The NOT Operation

In Fig.6 the A input to the NOT gate is inverted: if a 0 goes in, a 1 comes out; if a 1 goes in, a 0 comes out. In Boolean algebra the expression means you are to change A in the same way a NOT gate changes A. Read the expression $y = \overline{A}$ as y equals NOT A. The bar over A means you change or complement the quantity to the alternative digit. In other words, when A = 0.

$$y = \overline{A}$$

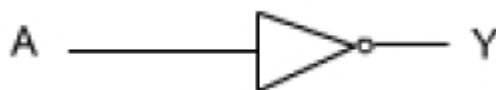


Fig.6

$$y = \overline{A} = \overline{0} = 1 \text{ because NOT 0 is 1}$$

When $A = 1$,

$$y = \overline{A} = \overline{1} = 0 \text{ because NOT 1 is 0}$$

The NOT operation is also called negation or inversion, and the prime is sometimes used instead of a bar to signify the NOT operation. That is,

$$y = \overline{A}$$

may be used instead of $y = A$.

Let us summarize the OR, AND and NOT rules:

OR	AND	NOT
$0 + 0 = 0$	$0 \cdot 0 = 0$	$\overline{0} = 1$
$0 + 1 = 1$	$0 \cdot 1 = 0$	$\overline{1} = 0$
$1 + 0 = 1$	$1 \cdot 0 = 0$	
$1 + 1 = 1$	$1 \cdot 1 = 1$	

Example

Write the Boolean equation for the output of Fig.7.

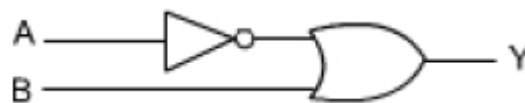


Fig.7

Solution

The OR gate has two inputs. One of them is A, and the other is B. Therefore, the output is

$$y = \overline{A} + B$$

Example

Evaluate the Boolean equation of the preceding example for all possible input conditions.

Solution

1. $A = 0$ and $B = 0$:

$$y = \overline{A} + B = \overline{0} + 0 = 1 + 0 = 1$$

2. $A = 0$ and $B = 1$:

$$y = \overline{A} + B = \overline{0} + 1 = 1 + 1 = 1$$

3. $A = 1$ and $B = 0$:

$$y = \overline{A} + B = \overline{1} + 0 = 0 + 0 = 0$$

4. $A = 1$ and $B = 1$:

$$y = \overline{A} + B = \overline{1} + 1 = 0 + 1 = 1$$

Example

Write the Boolean expression for the output of Fig.8(a).

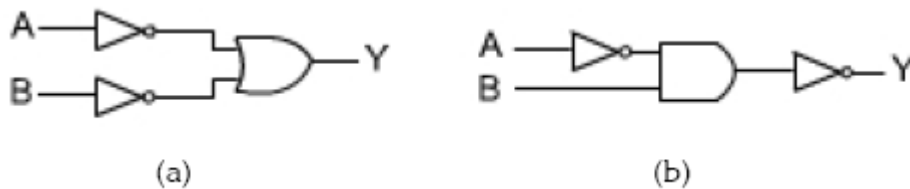


Fig. 8

Solution

One of the inputs to the OR gate is A; the other input is B. Therefore, the final output must be.

$$y = \overline{A} + \overline{B}$$

Example

Evaluate the Boolean expression of the preceding example for the four possible input combinations.

Solution

When $A = 0$ and $B = 0$

$$y = \overline{A} + \overline{B} = \overline{0} + \overline{0} = 1 + 1 = 1$$

When $A = 0$ and $B = 1$

$$y = \overline{A} + \overline{B} = \overline{0} + \overline{1} = 1 + 0 = 1$$

When $A = 1$ and $B = 0$

$$y = \overline{A} + \overline{B} = \overline{1} + \overline{0} = 0 + 1 = 1$$

Finally, when $A = 1$ and $B = 1$

$$y = \overline{A} + \overline{B} = \overline{1} + \overline{1} = 0 + 0 = 0$$

Example

- Find the Boolean expression for the output of Fig.8(b).
- Evaluate this Boolean expression for all possible input combinations.

Solution

- The inputs to the AND gate are A and B. Therefore, the input to the NOT circuits is AB . The final output is

$$y = \overline{\overline{AB}}$$

- When $A = 0$ and $B = 0$

$$y = \overline{\overline{AB}} = \overline{\overline{0.0}} = \overline{\overline{1.0}} = \overline{0} = 1$$

When $A = 0$ and $B = 1$

$$y = \overline{\overline{A}B} = \overline{0.0} = \overline{1.0} = \overline{0} = 1$$

When $A = 1$ and $B = 0$

$$y = \overline{\overline{A}B} = \overline{1.0} = \overline{0.0} = \overline{0} = 1$$

When $A = 1$ and $B = 1$

$$y = \overline{\overline{A}B} = \overline{1.1} = \overline{1.0} = \overline{0} = 1$$

As a general rule, when evaluating expressions like the foregoing.

1. Take the NOT of all individual terms first.
2. When a NOT is applied to more than one term (like 0.1), work out the AND or OR operation first, and then take the NOT of the result.

6.5 De Morgan's Theorems

1. $\overline{A + B} = \overline{A} \overline{B}$
2. $\overline{A B} = \overline{A} + \overline{B}$

The first equation says the complement of a sum equals the product of the complements. The second equation says the complement of a product equals the sum of the complements.

Proof of First Theorem

To prove

$$\overline{A + B} = \overline{A} \overline{B}$$

We need to show that the left side equals the right side for all possible values of A and B. Here are the four possible cases.

Case 1 : $A = 0$ and $B = 0$

$$\text{Left } \overline{A + B} = \overline{0 + 0} = \overline{0} = 1$$

$$\text{Right } \bar{A} \bar{B} = \bar{0} \cdot \bar{0} = 1 \cdot 1 = 1$$

Case 2 : $A = 0$ and $B = 1$

$$\overline{A + B} = \overline{0 + 1} = \overline{1} = 0$$

$$\bar{A} \bar{B} = \bar{0} \cdot \bar{1} = 1 \cdot 0 = 0$$

Case 3 : $A = 1$ and $B = 0$

$$\overline{A + B} = \overline{1 + 0} = \overline{1} = 0$$

$$\bar{A} \bar{B} = \bar{1} \cdot \bar{0} = 0 \cdot 1 = 0$$

Case 4 : $A = 1$ and $B = 1$

$$\overline{A + B} = \overline{1 + 1} = \overline{1} = 0$$

$$\bar{A} \bar{B} = \bar{1} \cdot \bar{1} = 0 \cdot 0 = 0$$

Since no other combinations of A and B exist, we have proved De Morgan's first theorem: $\overline{A + B} = \bar{A} \bar{B}$. To summarize the proof we can show the truth table for each expression (Tables 1 and 2). These tables show that

Table 1

A	B	$\overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

Table 2

A	B	$\overline{AB}$
0	0	1
0	1	0
1	0	0
1	1	0

$\overline{A+B}$ does equal $\overline{AB}$ for each case; therefore, the expressions are identical.

6.6 Circuit Implications of First Theorem

De Morgan's first theorem says

$$\overline{A+B} = \overline{A} \cdot \overline{B}$$

The left side of this equation represents an OR gate followed by an inverter (Fig.9a). Originally called a NOT-OR gate, this combination is now referred to as a NOR gate, and the abbreviated logic symbol of Fig.9b is widely used. Notice that the inverter triangle has been removed and the small circle or bubble moved to the OR-gate output. This bubble is a reminder of the inversion that follows the ORing.

The right side of De Morgan's first theorem is $\overline{A} \cdot \overline{B}$; this represents an AND gate whose inputs are inverted (Fig.9c). This combination is so widely used that the abbreviated logic symbol of Fig.9d had been adopted. Notice that the inverter triangles have been deleted and the bubbles moved to the AND-gate inputs. From now on, we will refer to the symbol of Fig.9d as a bubbled AND gate; the bubbles are a reminder of the inversion that takes place before ANDing.

Figure 9(e) summarize De Morgan's first theorem: A NOR gate and bubbled AND gate are equivalent. This means you can replace one by the other whenever desired. As shown later, the interchangeability of a NOR gate and a bubbled AND gate often leads to simpler logic circuits.

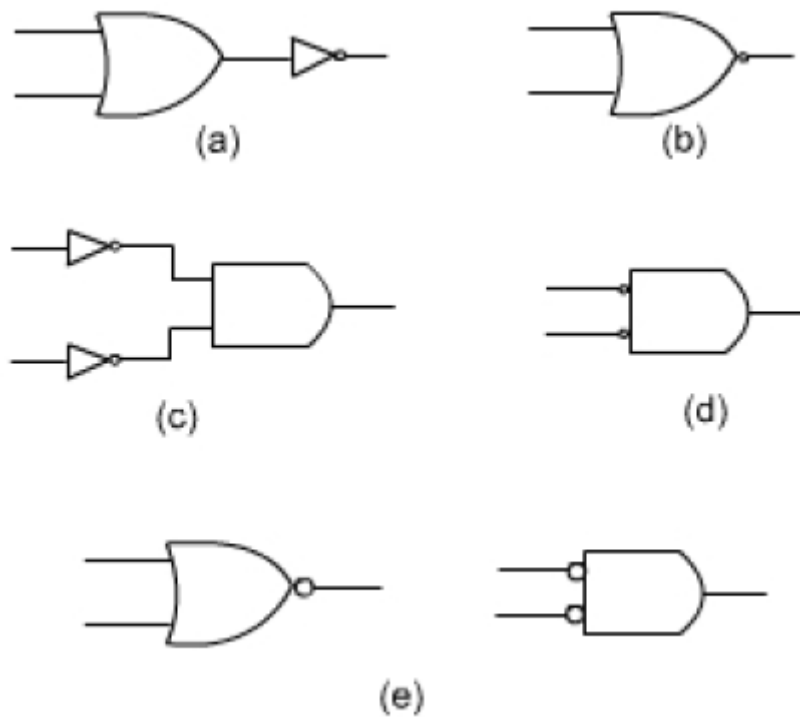


Fig.9.

Proof of Second Theorem

The second De Morgan theorem is

$$\overline{A + B} = \overline{A} + \overline{B}$$

(The complement of a product equals the sum of the complements). This theorem is easily proved Proof of second De Morgan theorem : $\overline{A + B} = \overline{A} + \overline{B}$.

When $A = 0$ and $B = 0$

$$\overline{AB} = \overline{0 \cdot 0} = \overline{0} = 1$$

$$\overline{A} + \overline{B} = \overline{0} + \overline{0} = 1 + 1 = 1$$

When $A = 0$ and $B = 1$

$$\overline{AB} = \overline{0 \cdot 1} = \overline{0} = 1$$

$$\overline{A} + \overline{B} = \overline{0} + \overline{1} = 1 + 0 = 1$$

When $A = 1$ and $B = 0$

$$\overline{AB} = \overline{1 \cdot 0} = \overline{0} = 1$$

$$\overline{A} + \overline{B} = \overline{1} + \overline{0} = 0 + 1 = 1$$

When $A = 1$ and $B = 1$

$$\overline{AB} = \overline{1 \cdot 1} = \overline{1} = 0$$

$$\overline{A} + \overline{B} = \overline{1} + \overline{1} = 0 + 0 = 0$$

Since there are no other input combinations, we have proved the second De Morgan theorem.

Table 3

A	B	$\overline{A B}$
0	0	1
0	1	1
1	0	1
1	1	0

Table 4

A	B	$\overline{A} + \overline{B}$
0	0	1
0	1	1
1	0	1
1	1	0

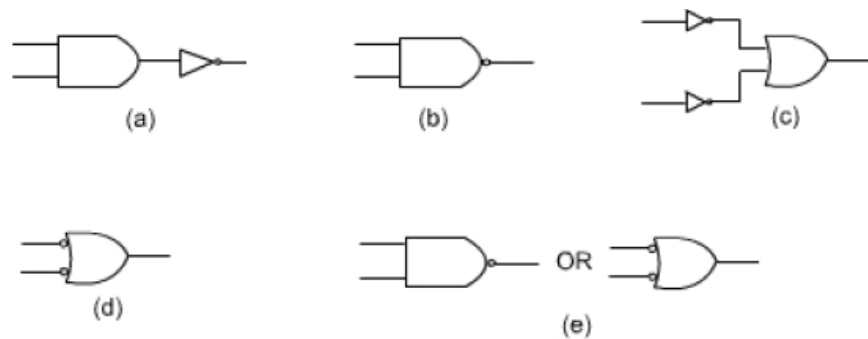
Table 3 and 4 give the truth tables for each expression. Note that the truth tables are identical. Since the truth tables are identical, the expressions are equivalent and the logic circuits represented by $\overline{A} \overline{B}$ and $\overline{A} + \overline{B}$ are interchangeable.

6.7 Circuit Implications of Second Theorem

Figure 10(a) represents the left side of De Morgan's second theorem. Originally called a NOT-AND gate, this combination is now referred to as a NAND gate and the abbreviated symbol of Fig.10(b) is used.

The right side of the second theorem implies that the inputs are inverted before reaching an OR gate [Fig.10(c)]. This combination is used so often that the abbreviated symbol of Fig.10(d) has come into use. Again, the triangles are deleted and the bubbles moved directly to the inputs. These bubbles remind us of the inversion that takes place before ANDing. From now on, we will call the symbol of Fig.10(d) a bubbled OR gate.

Figure 10(e) is a graphic summary of De Morgan's second theorem. A NAND gate and a bubbled OR gate are equivalent; therefore, we can replace one with the other whenever desired.

**Fig.10**

De Morgan's theorems are important and should be memorized. We repeat them again in words.

The complement of a sum equals the product of the complements.

The complement of a product equals the sum of the complements.

The theorems are useful in changing Boolean expressions to equivalent forms.

As already indicated, to apply the De Morgan theorems change plus signs to multiplications signs or vice versa, and take the complement of the individual terms rather than that of the entire expression. For instance, for $\overline{A + B}$.

1. Change the + sign to a sign to get AB .
2. Take the complement of each term to get $\overline{A} \overline{B}$.

The A and B can represent complicated expressions; we can still apply De Morgan's theorem. As an example, suppose

$$y = \overline{(C+DE)(CE+DF)}$$

This is the complement of a product, $(C + DE)$ dot $(CE + DF)$. Therefore, we can use the second De Morgan theroem and rewrite the expression as

$$y = \overline{(C+DE)} + \overline{(CE+DF)}. \text{ This can be further simplified.}$$

Example

Prove the following identity:

$$\overline{\overline{A}B} = A + \overline{B}$$

Solution

Use the second De Morgan theorem: the complement of a product equals the sum of the complements. First,

$\overline{\overline{A}B}$ means the complement of $\overline{A}B$. In turn $\overline{A}B$ is the product of $\overline{A}$ and B. Rewrite $\overline{\overline{A}B}$ as the sum of the complements of $\overline{A}$ and B.

$$\overline{\overline{AB}} = \overline{\overline{A}} + \overline{\overline{B}}$$

(We changed the sign to a + sign, and took the complements of A and B individually).

The final step is to recognize that the double complement of a variable is equal to the variable itself: $\overline{\overline{A}} = A$. If $A = 0$, $\overline{0} = \overline{1} = 0$. On the other hand, if $A = 1$, $\overline{1} = \overline{0} = 1$. Therefore,

$$\overline{\overline{AB}} = \overline{\overline{A}} + \overline{\overline{B}} = A + B$$

Example

Prove that Fig.11 (a) and (c) are equivalent.

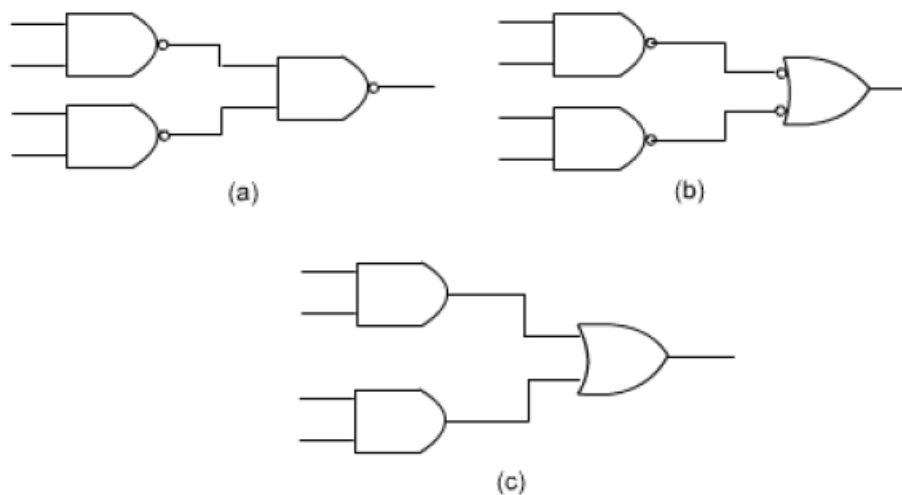


Fig.11

Solution

The final NAND gate of Fig.11(a) is equivalent to a bubbled OR gate; therefore, we can redraw the circuit as shown in Fig.11(b).

Double inversion results in the original signal, therefore, each double inversion in Fig.11(b) cancels out, leaving the simplified circuit of Fig.11c. As a result, Fig.11(a) and (c) are equivalent. This means we can use one in the place of the other.

All you have to remember for now is this: A group of NAND gates driving a NAND gate is equivalent to a group of AND gates driving an OR gate.

Example

Prove the three-input NAND gate of Fig.12(a) is interchangeable with the logic circuit of Fig.12(b).

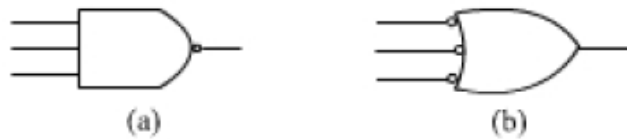


Fig. 12

Solution

The output of the NAND gate of Fig.12a is the NOT AND of the inputs. The AND of the inputs is ABC . The NOT of this is $\overline{ABC}$. So the output of the NAND gate is $\overline{ABC}$.

The output of Fig.12b is easily found. The OR gate has inputs of $\overline{A}$, $\overline{B}$ and $\overline{C}$. Therefore, the final output is $\overline{A} + \overline{B} + \overline{C}$.

To show the two systems are equivalent or interchangeable, we must prove

$$\overline{ABC} = \overline{AB \cdot C} = \overline{AB} + \overline{C}$$

We can do this with the second De Morgan theorem as follows. Think of $\overline{ABC}$ as the product of $\overline{AB}$ and $\overline{C}$. Now, apply the second De Morgan theorem to get

$$\overline{ABC} = \overline{AB \cdot C} = \overline{AB} + \overline{C}$$

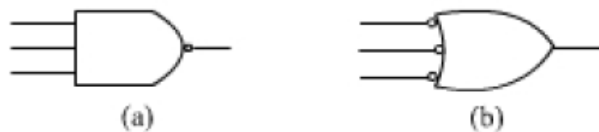


Fig. 13

Applying De Morgan's theorem again to $\overline{AB}$ gives

$$\overline{AB} + \overline{C} = \overline{A} + \overline{B} + \overline{C}$$

So you see, the three-input gates of Fig.12 are equivalent. A similar proof applies to any number of inputs. This means De Morgan's theorems are valid for any number of inputs. In

circuit terms, an n-input NAND gate is equivalent to an n-input bubbled OR gate. Similarly, an n-input NOR gate is equivalent to an n-input bubbled OR gate. Similarly, an n-input NOR gate is equivalent to an n-input bubbled AND gate.

$$1. \quad \overline{A_1 + A_2 + \dots + A_n} = \overline{A_1} \cdot \overline{A_2} \cdot \dots \cdot \overline{A_n}$$

$$2. \quad \overline{A_1 \cdot A_2 \cdot \dots \cdot A_n} = \overline{A_1} + \overline{A_2} \cdot \dots + \overline{A_n}$$

6.8 Switching Circuits

Hexa decimal Encoder

The circuit converts from hexadecimal to binary. Pressing each pushbutton switch results in a different output number. For instance, when pushbutton 3 is pressed, the two OR gates on the right have high inputs; therefore, the output from the overall circuits is 0011.

If pushbutton is pressed, only the OR gate on the left has a high output. In this case, the output becomes 1000.

If button F is pushed, all OR gates have high inputs, and the final output is 1111.

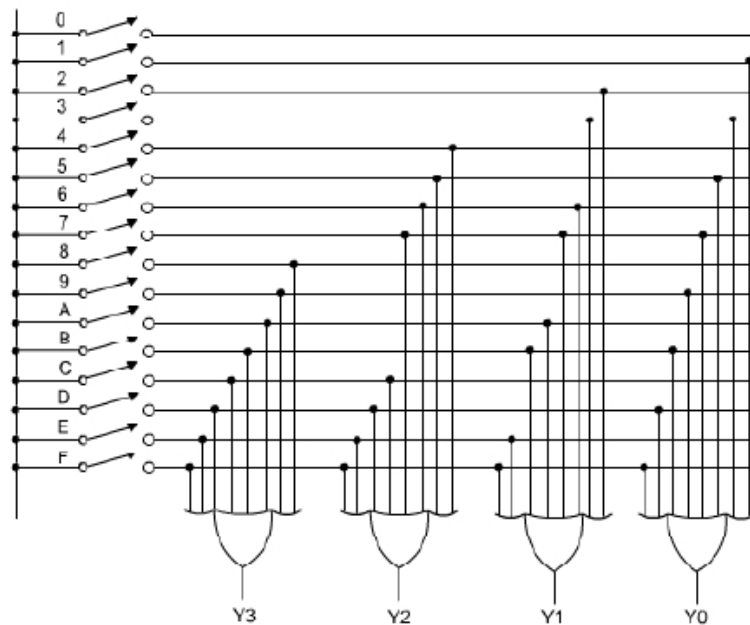


Fig. 14 Hexadecimal Encoder

Check the other inputs switches to convince yourself that the output always equals the binary equivalent of the switch being pressed.

6.9 1-OF-16 Decoder

To begin with, a register is a group of devices that stores a binary number. The 4-bit register shown can store any binary number from 0000 to 1111.

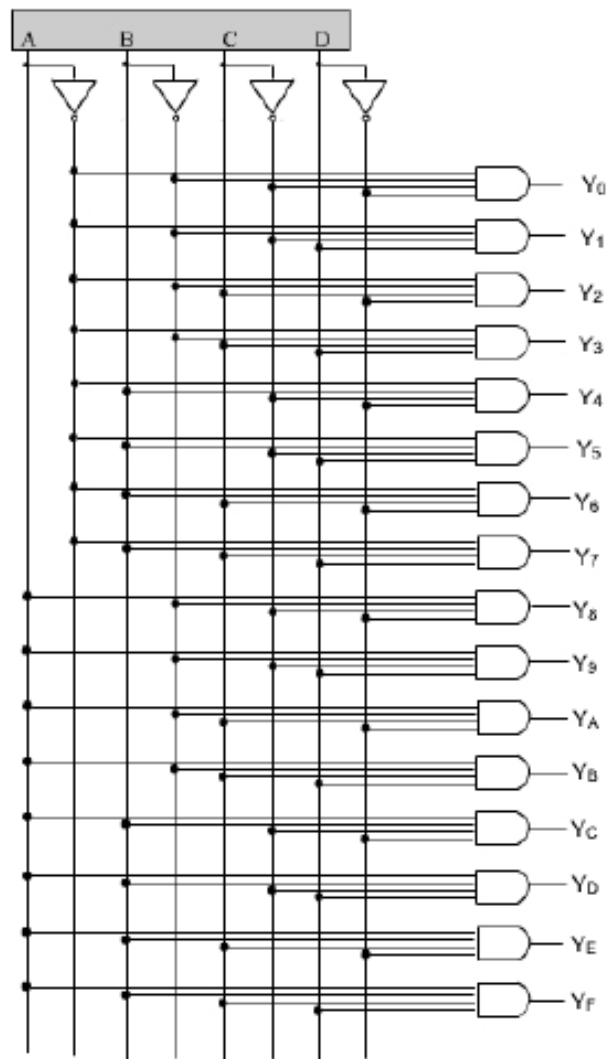


Fig. 15 shows a 1 of 16 decoder

The overall circuit is a binary-to-hexadecimal converter. For instance, when the register contents are 0011, only the y_3 AND gate has all high inputs; therefore, only y_3 is high. If the register contents change to 1000, only y_8 AND gate has all high inputs. As a result, only y_8 is high; all other outputs are low. When the register contents change to 1100, they y_0 AND gate has all high inputs. So, only y_0 is high while all other outputs are low.

If you check the other input possibilities, you will find the subscript of the high output always equals the hexadecimal equivalent of the register contents. This is why the circuit is called a binary-to-hexadecimal decoder. An alternative name is a 1-of-16 decoder.

6.10 The Universal Building Block

Given any Boolean expression, we can build a logic circuit for it. Conversely, given a logic circuit, we can write a Boolean expression. As an example, the expression $(A + B)C$ suggests a logic circuit in which A and B are first ORed, and then ANDed with the complement of C. Figure 16 shows the logic circuit.

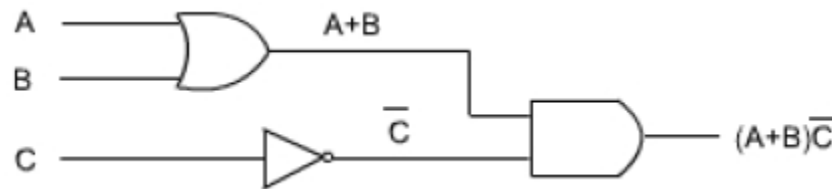


Fig. 16

To build the logic circuit associated with any Boolean expression, we can use OR gates for the + signs, AND gates for dot signs, and NOT gates for the overhead bars. In other words, OR, AND, and NOT gates are the basic building blocks of all logic circuits.

The NAND gate has an interesting property: it can be used to build an OR gate or an AND gate or a NOT gate. Because of this, NAND gates are all we need to build any logic circuit. Therefore, the NAND gate is a universal building block. (A similar statement can be made for the NOR gate).

6.11 Inverter

A NOT gate can be made out of a NAND gate by connecting all inputs together as shown in Fig. 17a. If A is 0, the output of the NAND gate is

$$\overline{AA} = \overline{0 \cdot 0} = \overline{0} = 1 = \overline{A}$$

And if A is 1, the output is

$$\overline{AA} = \overline{1 \cdot 1} = \overline{1} = 0 = \overline{A}$$

From now on, when using a NAND gate as a NOT gate, we shall show the symbol of Fig. (17b) - it will be understood that all inputs are connected together.

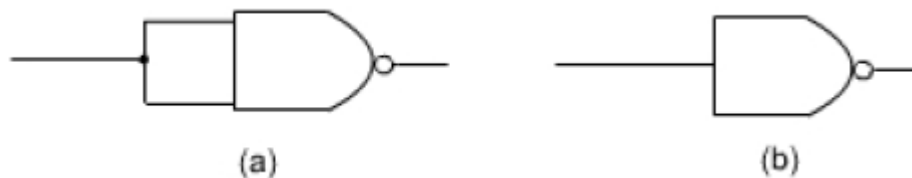


Fig. 17

6.12 AND Gate

We can use NAND gates to make an AND gate. Figure 15 shows how. The output of the first NAND gate is $\overline{AB}$. The second NAND gate then complements this, producing $\overline{\overline{AB}}$. Since the double complement of a quantity is the quantity itself, $AB = \overline{\overline{AB}}$. Therefore, NAND gates can be connected to perform the AND function.

6.13 OR Gate

We can also make an OR gate using NAND gates. Figure 16 shows how to connect the NAND gates. The first two NAND gates invert A and B to produce $\overline{A}$ and $\overline{B}$. The two input NAND gates then produces an output of $\overline{\overline{A}\overline{B}}$. With De Morgan's second theorem.

$$\overline{\overline{A}\overline{B}} = \overline{\overline{A}} + \overline{\overline{B}} = A + B$$

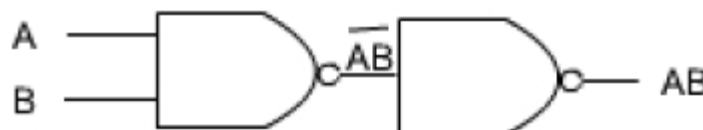


Fig.18

Since the final output is $A \text{ OR } B$, we have shown that NAND gates can be used to get the OR function.

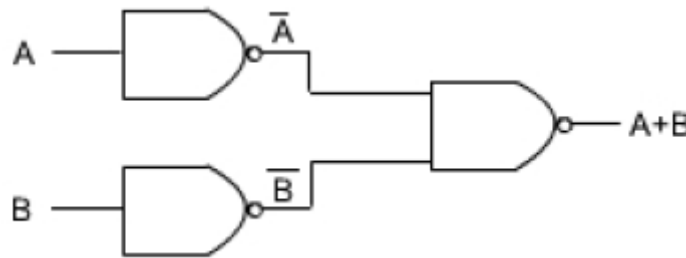


Fig. 19

The point is that we can use NAND gates to build OR, AND, and NOT circuits. This, in turn, means we can build any logic circuit using only NAND gates. (A similar statement applies to the NOR gate).

6.14 Summary

Boolean Algebra is used to analyze and simplify the digital (logic) circuits. It uses only the binary numbers i.e. 0 and 1. It is also called as **Binary Algebra** or **logical Algebra**.

This theorem is useful in finding the **complement of Boolean function**. It states that the complement of logical OR of at least two Boolean variables is equal to the logical AND of each complemented variable.

Decoder is a combinational circuit that has 'n' input lines and maximum of 2^n output lines

An **Encoder** is a combinational circuit that performs the reverse operation of Decoder. It has maximum of 2^n input lines and 'n' output lines

6.15 Model Questions

- (1) Draw the logic circuit for $y = \overline{A} \overline{B} + A \overline{B}$ using AND, OR and NOT gates. Implement the same circuit using NAND gates only.
- (2) Show that the complement of $\overline{A} \overline{B} + AB = \overline{A} B + A \overline{B}$ using De Morgan's theorem.
- (3) Show that NOR is a universal gate.

LESSON - 7

LAWS AND THEOREMS OF BOOLEAN ALGEBRA

Structure

- 7.0 Objectives
- 7.1 Introduction
- 7.2 Basic Laws
- 7.3 New Operators
- 7.4 Simplification
- 7.5 Duality
- 7.6 Summary
- 7.7 Model Questions

7.0 Objectives

After studying this lesson you should be able to Understand

- Boolean Laws
- Duality
- Simplification

7.1 Introduction

In this chapter we focus Boolean Laws such as Commutative law, Associative law and Distributive law. Duality theorem states that the dual of the Boolean function is obtained by interchanging the logical AND operator with logical OR operator and zeros with ones. For every Boolean function, there will be a corresponding Dual function

7.2 Basic Laws

Here are some basic laws and theorems of Boolean algebra.

The first group of laws in Boolean algebra is the same as in ordinary algebra.

Commutative laws :

$$A + B = B + A \quad (1)$$

$$A \cdot B = B \cdot A \quad (2)$$

Associative laws :

$$A + (B+C) = (A+B) + C \quad (3)$$

$$A(BC) = (AB)C \quad (4)$$

Distributive law :

$$A(B+C) = AB + AC \quad (5)$$

The commutative laws indicate that the order of adding or multiplying is unimportant. In other words, we get the same answer by adding A to B as we do by adding B to A. Likewise, the same answer results whether we multiply A by B or B by A.

The associative laws say you can group any two terms of a sum or any two factors of a product. In other words, given $A + B + C$, you can first add B and C, and then add the result to A; or, you can first add A and B, and then add the result to A; or, you can first add A and B, and then add the result to C. A similar statement applies to multiplication.

The distributive law, Eq. (5), indicates that we can expand expression by multiplying term by term as in ordinary algebra. This law also implies that we can factor expression. In other words, given a sum of two terms, each containing a common variable, we can factor this common variable. (This is like $AB + AC$. Each term contains A, so we can factor the A to get $A(B + C)$).

7.3 New Operators

The next group of laws is the backbone of Boolean algebra. First,

$$A + 0 = A \quad (6)$$

$$A \cdot 0 = 0 \quad (7)$$

These should be thought of in terms of OR and AND gates. $A + 0 = A$ should remind us of an OR gate with inputs of A and 0 as shown in Fig. 1(a). The output of the OR gate depends on

A only; if A is 0, the output must be 0; if A is 1, the output must be 1. Similarly, $A \cdot 0 = 0$ suggests the AND gate of Fig. 1(a). Since the AND gate is an all-or-nothing gate, the output must be 0. The remaining identities in the group are:

$$A + 1 = 1 \quad (8)$$

$$A \cdot 1 = A \quad (9)$$

$$A + A = A \quad (10)$$

$$A \cdot A = A \quad (11)$$

$$A + \bar{A} = 1 \quad (12)$$

$$A \cdot \bar{A} = 0 \quad (13)$$

Again, visualize these formulas as logic circuits. Figure 1 shows the circuits for each. If there is any-doubt, you can prove each formula by substituting the two possible values of A; 0 and 1. In each case, the left side of the formula equals the right side.

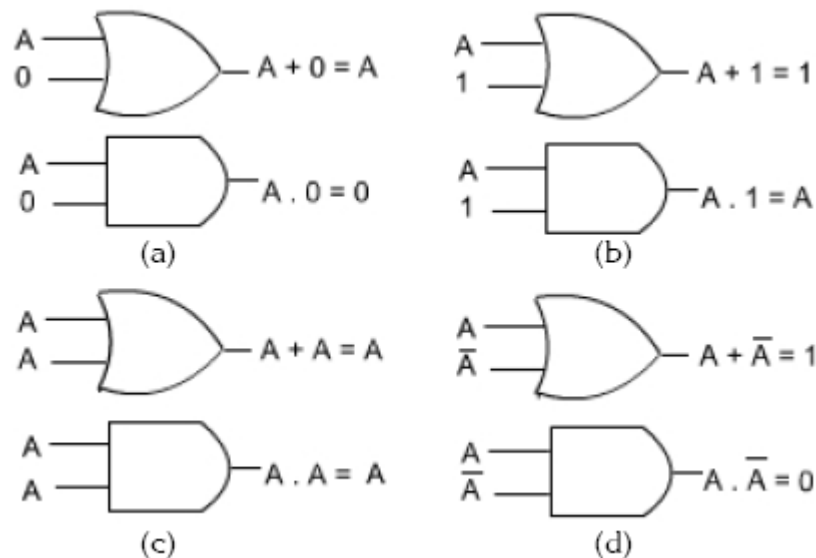


Fig. 1

The next three-formulas are already familiar: the double-complement theorem and De Morgan's theorems.

$$\overline{\overline{A}} = A \quad (14)$$

$$\overline{A + B} = \overline{A} \cdot \overline{B} \quad (15)$$

$$\overline{A} \cdot \overline{B} = \overline{A + B} \quad (16)$$

$$(A+B) (A+C) = A + BC \quad (17)$$

$$A + \overline{A} B = A + B \quad (18)$$

$$A(\overline{A} + B) = AB \quad (19)$$

$$(A+B) (\overline{A} + C) = AC + \overline{A} B \quad (20)$$

$$AB + \overline{A} C = (A + C) (\overline{A} + B) \quad (21)$$

$$A (A+B) = A \quad (23)$$

7.4 Simplification

Often, you can simplify a complicated Boolean expression by using the foregoing laws and theorems. This is important; it means you can build a simpler circuit instead of a more complicated one. For instance, suppose a digital circuit has an output given by

$$y = AC + ABC$$

This equation says the output y is obtained by

1. ANDing A and C
2. ANDing A , B and C
3. ORing AC and ABC

Figure 2(a) shows the digital circuit $y = AC + ABC$, it uses two AND gates and an OR gate. This circuit is more complicated than it has to be because the original expression simplifies as follows:

$$\begin{aligned} y &= AC + ABC \\ &= AC (1+B) \\ &= AC \end{aligned}$$

Figure 2(b) illustrates the circuit for $y = AC$; it uses only one AND gate. The point is clear. Whenever you can simplify a Boolean expression, do so; this results in less hardware when the circuit is built.

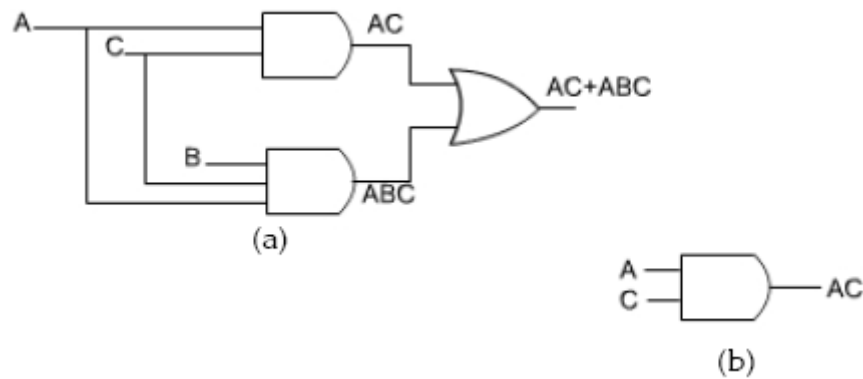


Fig. 2

When we simplify a complicated Boolean expression, we are changing a complicated digital circuit into a simpler one. For instance, to build the circuit for

$$y = ABC + A\bar{B}C + AB\bar{C}$$

we need the following.

A three-input OR gate to add ABC , $A\bar{B}C$ and $AB\bar{C}$. Three three-input AND gates to produce ABC , $A\bar{B}C$, and $AB\bar{C}$.

Two NOT gates to produce $\bar{B}$ and $\bar{C}$

Figure 3(a) shows the proper connection of these elements. By using Boolean laws and theorems, we can simplify the original expression.

$$\begin{aligned}
 y &= ABC + A\bar{B}C + AB\bar{C} \\
 &= AC(B + \bar{B}) + AB\bar{C} \\
 &= AC + AB\bar{C} \\
 &= A(C + B\bar{C})
 \end{aligned}$$

$$= A(C+B)$$

$$= A(B+C)$$

and get

$$y = A(B+C)$$

This leads to a simpler logic circuit. Only one two-input AND gate and one two-input OR gate are needed, as shown in Fig. 3(b).

The logic circuits of Fig. 3 are identical as far as input-output operation is concerned; in other words, they are interchangeable. The circuit of Fig. 19(b) is preferred since it uses less hardware and is easier to construct.

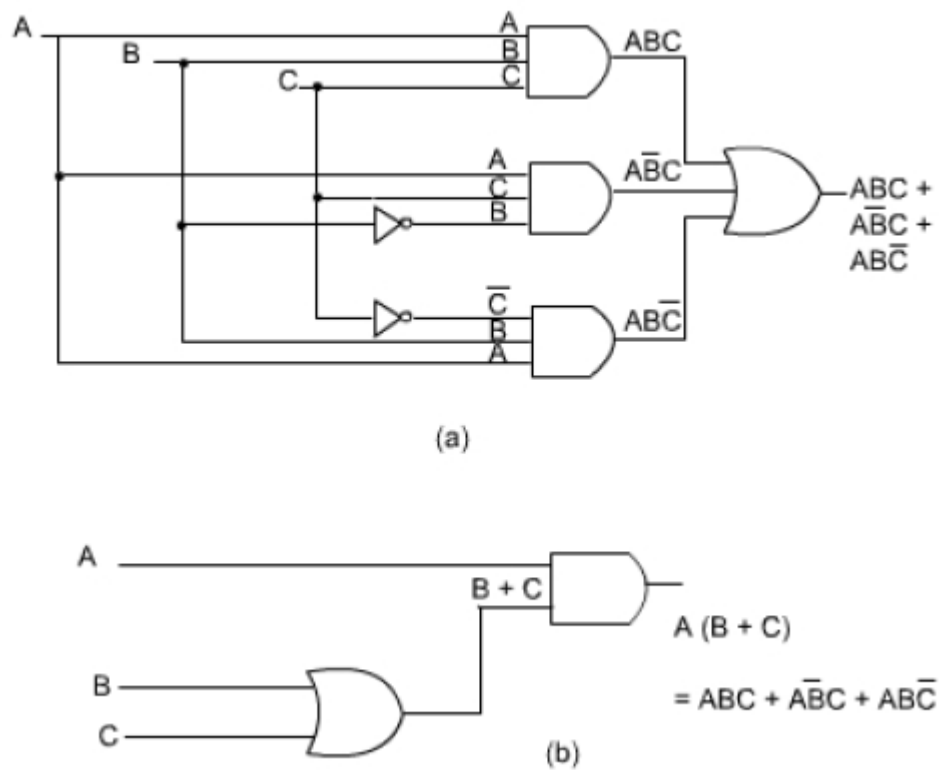


Fig. 3

7.5 Duality

Positive and negative logic give rise to basic duality in all the identities. When changing from one logic system to another, 0 becomes 1 and vice versa. Furthermore, AND gates become OR gates, and OR gates become AND gates. This means that, given any Boolean identity we can produce a dual identity by

Changing + signs to dot signs, and vice versa

Complementing all 0s and 1s

For instance, give Eq (6),

$$A + 0 = A$$

complement the + sign and 0 to get the dual identity, Eq(9):

$$A \cdot 1 = A$$

As another example, Eq (12) is

$$A + \overline{A} = 1$$

To get the dual identity, complement the + sign and the 1 to get

$$A \cdot \overline{A} = 0$$

This property of duality is helpful.

Example

Prove the following:

a. $A + A = A$

b. $A \cdot A = A$

Solution

a. Prove this by substituting the possible values of A, which are 0 and 1. When A = 0

$$A + A = A \qquad 0 + 0 = 0 \qquad 0 = 0$$

When A = 1

$$A + A = A \qquad 1 + 1 = 1 \qquad 1 = 1$$

- b. Prove that $A \cdot A = A$ in the same way. When $A = 0$.

$$A \cdot A = A \qquad 0 \cdot 0 = 0 \qquad 0 = 0$$

When $A = 1$,

$$A \cdot A = A \qquad 1 \cdot 1 = 1 \qquad 1 = 1$$

Example

Case 1 : When $A = 0$ and $B = 0$

$$A + AB = A$$

$$0 + 0 \cdot 0 = 0$$

$$0 + 0 = 0$$

$$0 = 0$$

Case 2 : When $A = 0$ and $B = 1$,

$$A + AB = A$$

$$0 + 0 \cdot 1 = 0$$

$$0 + 0 = 0$$

$$0 = 0$$

Case 3 : When $A = 1$ and $B = 0$,

$$A + AB = A$$

$$0 + 1 \cdot 0 = 1$$

$$1 + 0 = 1$$

$$1 = 1$$

Case 4 : When $A = 1$ and $B = 1$,

$$A + AB = A$$

$$1 + 1 \cdot 1 = 1$$

$$1 + 1 = 1$$

$$1 = 1$$

A different way to prove $A + AB = A$ is the following:

$$A + AB + A(1+B) = A(1) = A$$

This approach uses Eqs. (5) and (8)

7.6 Summary

If any logical operation of two Boolean variables give the same result irrespective of the order of those two variables, then that logical operation is said to be **Commutative Law**.

If a logical operation of any two Boolean variables is performed first and then the same operation is performed with the remaining variable gives the same result, then that logical operation is said to be **Associative**.

If any logical operation can be distributed to all the terms present in the Boolean function, then that logical operation is said to be **Distributive**.

Duality theorem states that the **dual** of the Boolean function is obtained by interchanging the logical AND operator with logical OR operator and zeros with ones. For every Boolean function, there will be a corresponding Dual function.

7.7 Model Questions

- (1) Show that $A + AB = A + B$ using Boolean laws and truth table.
- (2) Show that $AB + AC + BC = AB + AC$
- (3) Simplify $Y = ABC + ABC + ABC + ABC$

LESSON - 8

ARITHMETIC BUILDING BLOCKS

Structure

- 8.0 Objectives**
- 8.1 Introduction**
- 8.2 XOR and XNOR GATE**
- 8.3 Logic Symbol**
- 8.4 XNOR Gate**
- 8.5 Application of XOR**
- 8.6 Word Comparator**
- 8.7 The Half-Adder**
- 8.8 The Full-Adder**
- 8.9 A Parallel Binary Adder**
- 8.10 An 8421 Adder**
- 8.11 Forbidden Groups**
- 8.12 Rules for BCD Addition**
- 8.13 Half-Subtractor and Full-Subtractor**
- 8.14 Summary**
- 8.15 Model Questions**

8.0 Objectives

To introduce :

- XOR & XNOR gates
- Parity checker, binary to Gray code conversion, etc.
- Half Adder / Full Adder
- Parallel binary adder / subtractor
- Half subtractor / Full subtractor

8.1 Introduction

In this chapter we focus Ex-OR gate is Exclusive-OR gate. Its function is same as that of OR gate except for some cases, when the inputs having even number of ones. Half adder is a combinational circuit, which performs the addition of two binary numbers A and B are of single bit. It produces two outputs sum, S & carry, C. The full subtractor is a combinational circuit with three inputs A, B, C and two output D and C'. A is the 'minuend', B is 'subtrahend', C is the 'borrow' produced by the previous stage.

8.2 XOR and XNOR Gates

Figure 1 shows an exclusive - OR gate. It has two inputs and one output. Each input goes into an inverter; the outputs of the inverters are $\overline{A}$ and $\overline{B}$. As shown in Fig. 1. $\overline{A}$ and B go into the upper AND gate, so its output is $\overline{A}B$. Likewise, $A\overline{B}$ comes out of the lower AND gate. The OR gate has inputs of $A\overline{B}$ and $\overline{A}B$, so the final output is

$$y = A\overline{B} + \overline{A}B$$

1. When A = 0 and B = 0,

$$y = 0 \cdot \overline{0} + \overline{0} \cdot 0 = 0 \cdot 1 + 1 \cdot 0 = 0 + 0 = 0$$

2. When A = 0 and B = 1,

$$y = 0 \cdot \overline{1} + \overline{0} \cdot 1 = 0 \cdot 0 + 1 \cdot 1 = 0 + 1 = 1$$

3. When A = 1 and B = 0,

$$y = 1 \cdot \overline{0} + \overline{1} \cdot 0 = 1 \cdot 1 + 0 \cdot 0 = 1 + 0 = 1$$

4. When A = 1 and B = 1,

$$y = 1 \cdot \overline{1} + \overline{1} \cdot 1 = 1 \cdot 0 + 0 \cdot 1 = 0 + 0 = 0$$

These results are summarized in the truth table of Table 1.

The reason for the name exclusive-OR is this: a 1 output occurs when A or B is 1, but not both. Stated another way, the exclusive-OR gate has a 1 output only when the inputs are different. The output is 0 when the inputs are the same.

Incidentally, XOR is an abbreviation for exclusive-OR.

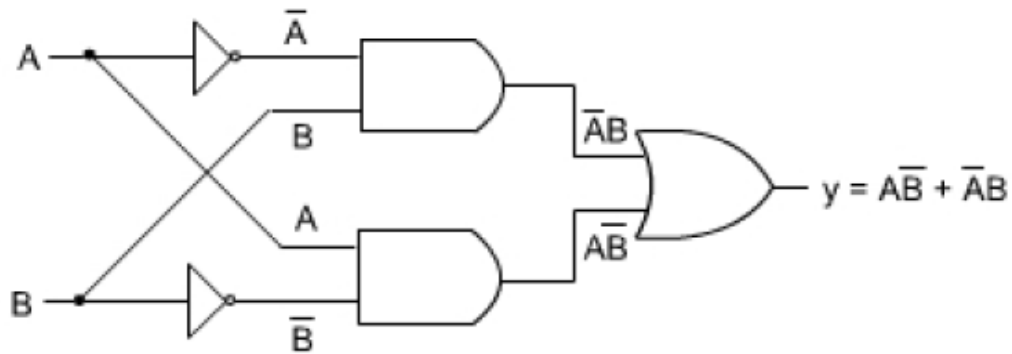


Figure 1. Exclusive-OR gate.

Table 1 : Exclusive OR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

The XOR gate gives us a new kind of function to work with. We will use the symbol $\oplus$ to stand for this function. To describe the action of an XOR gate, we can write

$$y = A \oplus B$$

Whenever we see $y = A \oplus B$, we shall know the output is given by the truth table of Table. 1.

The exclusive-OR operation is sometimes called mod-2 addition.

Here are the rules for this addition:

$$0 \oplus 0 = 0$$

$$0 \oplus 1 = 1$$

$$1 \oplus 0 = 1$$

$$1 \oplus 1 = 0$$

Mod-2 addition is the same as binary addition, provided we disregard carries.

8.3 Logic Symbol

Figure 2 is the logic symbol for a two-input XOR gate. Whenever you see this symbol, remember the action: The inputs must be different to get a high output.



Figure 2. Logic symbol for exclusive-OR gate.

8.4 XNOR Gate

The exclusive-NOR gate, abbreviated XNOR, is logically equivalent to an XOR gate followed by an inverter (Fig.3a). Because of the inversion, the truth table of XNOR gate is the complement of an XOR truth table. For this reason, the output of an XNOR gate is high only when the two inputs are the same.

Figure 3(b) is the logic symbol for an XNOR gate. Remember how it works: Both inputs must be the same to get a high output. Because of its action, the XNOR gate can be used to build a comparator (Example 3).



(a)



(b)

Figure 3

8.5 Applications for XOR

Example 1

One way of building a parity checker for a 4-bit word.

Figure 4(a) shows a way to check the parity of the word ABCD. The circuit adds the bits of ABCD; a final sum of 0 implies even parity; a sum of 1 means odd parity. For instance, suppose ABCD = 1001. Figure 4(b) shows the circuit for this condition. Note that the final sum is 0, which means even parity.

As another example, the circuit for ABCE = 1110 is shown in Fig.4(c).

The final output is 1, denoting odd parity.

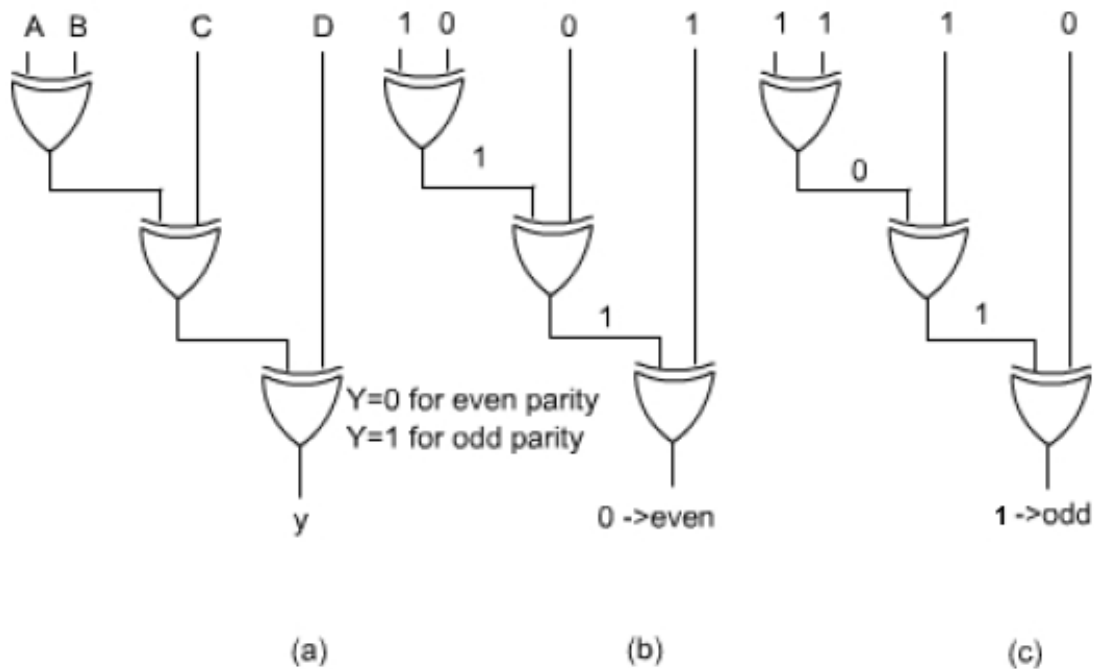


Figure 4. Checking parity with exclusive-OR gate.

Example 2

A way of building of a 6-bit binary-to-Gray converter.

Figure 5(a) shows a 6-bit binary-to-Gray converter. The binary number is ABCDEF. The circuit adds the adjacent bits using mod 2 addition and produces the Gray equivalent.

As a numerical example, suppose ABCDEF = 100110. Figure 5(b) shows the circuit for this input. The circuit adds the bits, two at a time, to get the Gray equivalent, 110101.

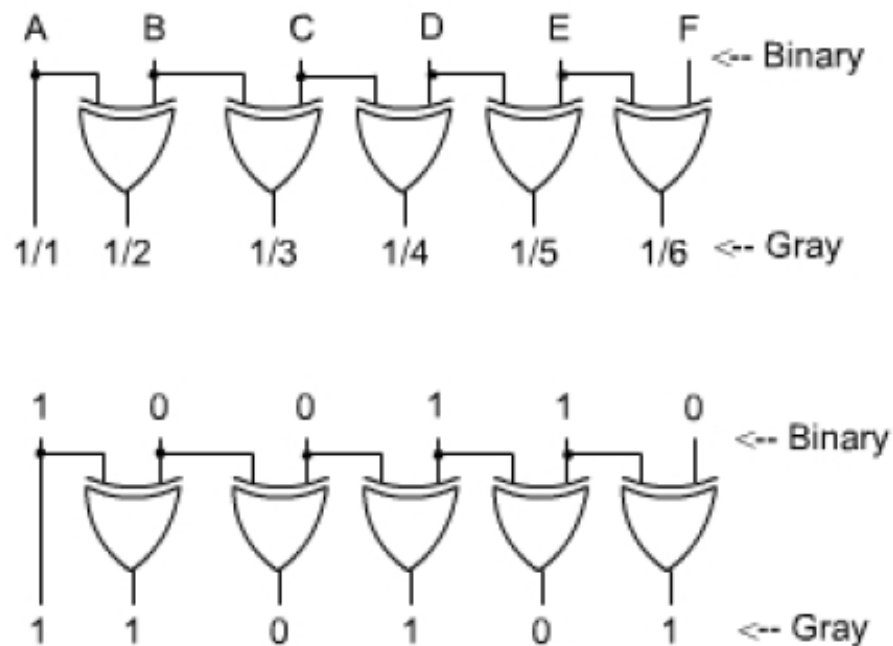


Figure 5. Converting from binary to Gray code

8.6 Word Comparator

The leftmost XNOR gate compares bits A_3 and B_3 ; if they are the same, y_3 is a 1. The second XNOR gate compares A_2 and B_2 ; if they are the same, y_2 is a 1. The remaining XNOR gates compare the bits that are left, producing a 1 output for equal bits and a 0 output for unequal bits.

If the binary numbers stored in registers A and B are identical, all XNOR gates have high outputs and the AND has a high EQUAL. If the stored numbers differ in one or more bit positions, the AND gate has a low EQUAL.

One more point. A group of bits does not always represent a number. In Fig.6, the contents of the registers may represent coded instructions or data of any type. To convey this variety of meaning, a group of bits is often called a binary word or simply a word. We can summarize the action of Fig.6 by saying the circuit is a word comparator; it produces a high output when the two input words are identical.

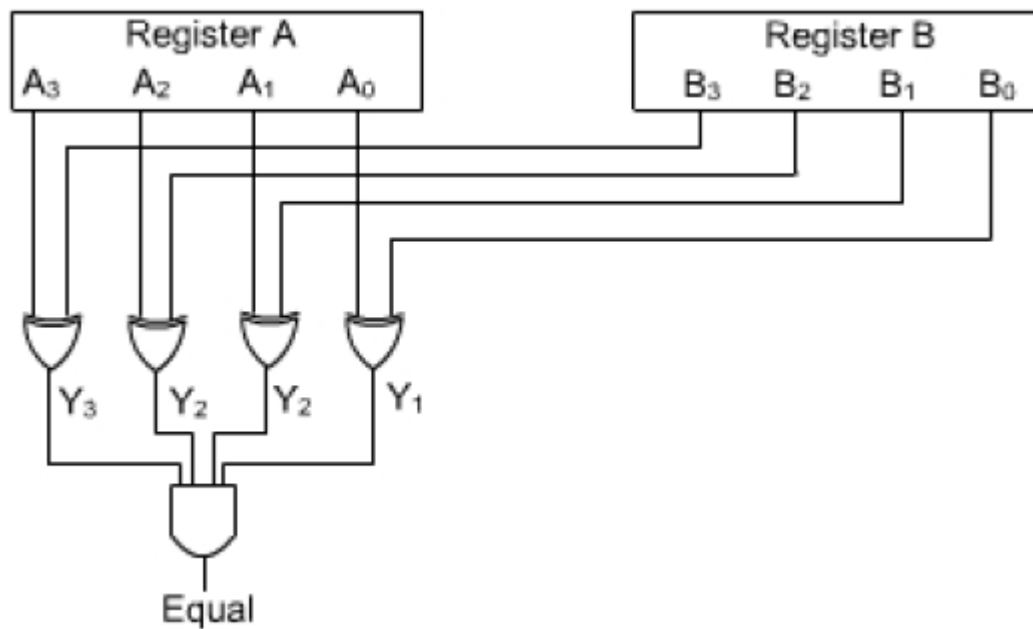


Figure 6. Comparing two binary words

Example

Controlled Inverter

Fig.7 shows a controlled inverter (sometime called a programmed inverter). When INVERT is low, it transmits the register word to the output. But when INVERT is high, it transmits the 1's complement.

For instance, if the register contents are

$A = 1\ 1\ 0\ 0\ 0\ 1\ 1\ 1$

Then a low INVERT produces

$y = 1\ 1\ 0\ 0\ 0\ 1\ 1\ 1$

But a high INVERT results in

$y = 0\ 0\ 1\ 1\ 1\ 0\ 0\ 0$

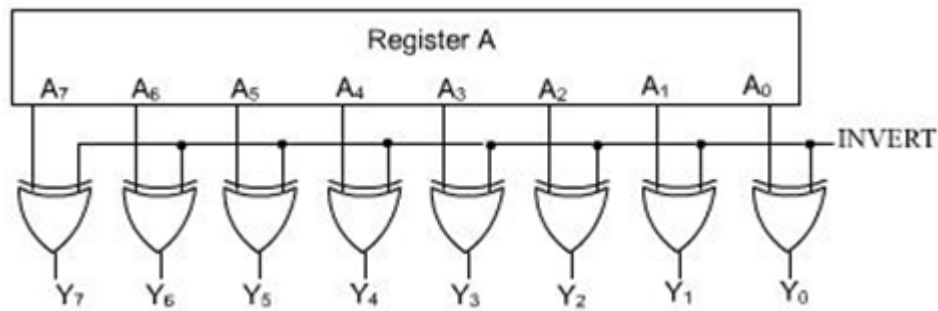


Figure 7. Controlled inverter

The controlled inverter is important. Later, we will use it for subtraction. All you need to remember is that a low INVERT transmits the original word and a high INVERT produces the 1's complement.

8.7 The Half-Adder

The half-adder adds 2 binary digits at a time. Figure 8 shows how to make a half-adder. The output of the exclusive-OR gate is the sum, and the output of the AND gate is the carry. As with any two-input circuit, there are four distinct cases. These are:

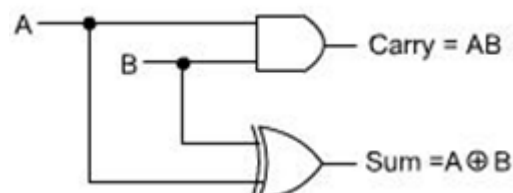


Figure 8. Half-adder

1. When $A = 0$ and $B = 0$
 $\text{Sum} = A \oplus B = 0 \oplus 0 = 0$
 $\text{Carry} = AB = 0 \cdot 0 = 0$
2. When $A = 0$ and $B = 1$
 $\text{Sum} = 0 \oplus 1 = 1$
 $\text{Carry} = 0 \cdot 1 = 0$
3. When $A = 1$ and $B = 0$
 $\text{Sum} = 1 \oplus 0 = 1$

$$\text{Carry} = 1 \cdot 0 = 0$$

4. When $A = 1$ and $B = 1$

$$\text{Sum} = 1 \oplus 1 = 0$$

$$\text{Carry} = 1 \cdot 1 = 1$$

Table 2 summarises these results.

The half-adder of Fig.8 performs binary addition. It does what we do mentally when we add 2 binary digits. For instance, 0 combines with 0 gives 0 carry a 0. 1 combines with 1 gives 0 carry a 1.

There are many ways to build half-adders. The important thing to remember is the half-adder adds 2 binary digits. It is an elementary circuit, but it does take the first step toward circuits capable of more difficult arithmetic.

Table 2

A	B	Carry	Sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

8.8 The Full-Adder

When adding two binary numbers, you may have carry form one column to the next. For instance,

$$\begin{array}{r} 111 \\ +101 \\ \hline 1100 \end{array}$$

In the least significant column.

$$1 + 1 = 0 \text{ carry a } 1$$

In the next column, you must add 3 digits because of the carry.

$$1 + 0 + 1 = 0 \text{ carry a } 1$$

In the last column, you again must add 3 digits because of the carry

$$1 + 1 + 1 = 1 \text{ carry a } 1$$

To add binary numbers electronically, we need a circuit that can handle 3 digits at a time. By connecting two half-adders and an OR gate, we get a full-adder, a circuit that can add 3 digits at a time. Figure 9 shows a full-adder. The boxes labeled HA are half-adders. Since we know how a half-adder and OR gate work, we can easily determine the final output.

For instance, suppose $A = 1$, $B = 1$ and $C = 0$. Figure 10a shows the full-adder with these inputs. The first half-adder has a sum of 0 with a carry of 1. The second half-adder has a sum of 0 with a carry of 0. Therefore, the final output is a sum of 0 with a carry of 1.

Table 3

A	B	C	Carry	Sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

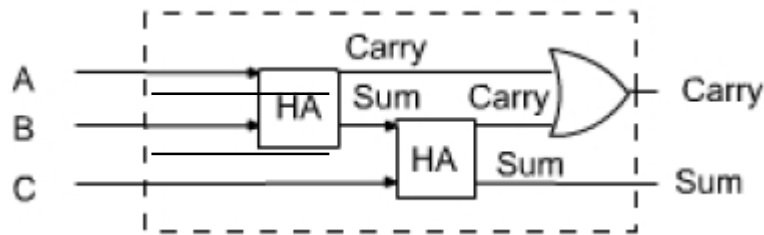


Fig 9. Full-adder

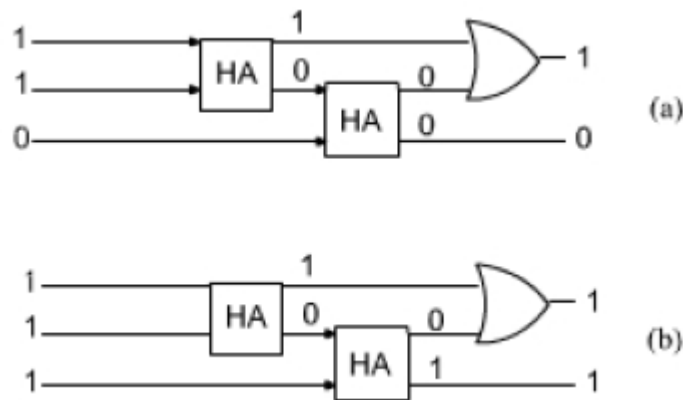


Fig 9. Examples of how a full-adder works

If the inputs are $A = 1$, $B = 1$ and $C = 1$, we get a sum of 1 with its carry of 1. Figure 10b shows the half-adder with these inputs. Each half-adder has the outputs shown, so the final output is sum of 1 with a carry of 1.

If we work out the sum and carry outputs in the same way for the other input combinations, we get the truth table of Table 3.

Remember the Key idea. The full-adder adds 3 binary digits at a time. The next section shows how to use full-adders to get the sum of binary numbers with more than 1 bit.

8.9 A Parallel Binary Adder

We can connect adders as shown in Fig. 10 to add two binary numbers. (The boxes labeled FA are full-adders). The binary numbers being added are $A_3 A_2 A_1 A_0$ and $B_3 B_2 B_1 B_0$. The answer is

$$\begin{array}{r}
 A_3 A_2 A_1 A_0 \\
 + \quad B_3 B_2 B_1 B_0 \\
 \hline
 S_3 S_2 S_1 S_0
 \end{array}$$

The first column requires only a half-adder. For any column above the first, there may be a carry from the preceding column; therefore, we must use a full-adder for each column above the first.

As an example of how the adder of Fig. 10 works, suppose we want to add decimal numbers 11 and 7. The binary equivalent of decimal 11 is 1011, and the binary equivalent of decimal 7 is 0111. Figure 12 shows the binary adder with these inputs. Starting with the half-adder, we know the sum must be 0 with a carry of 1 as shown. The carry goes into the first full-adder, which adds 1 + 1 + 1 to get a sum of 1 with a carry of 1. This carry goes into the next full-adder, which adds 0 + 1 + 1 to get a sum of 0 with a carry of 1. The last full-adder adds 1 + 0 + 1 to get sum of 0 with a carry of 1. The final output of the system is 10010. The decimal equivalent of this is 18. Which is the correct decimal sum of 11 and 7. Therefore the parallel binary adder of Fig.11 gives the binary sum of two 4-bit numbers.

$C_3 \quad C_2 \quad C_1 \quad C_0$

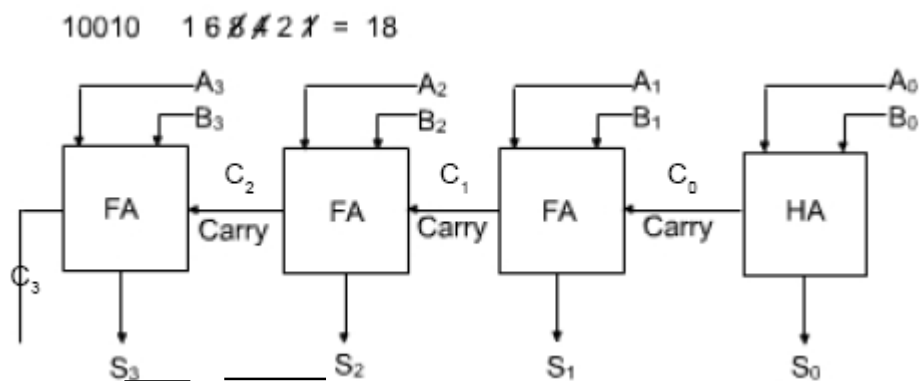


Fig. 10 Parallel 4-bit binary adder

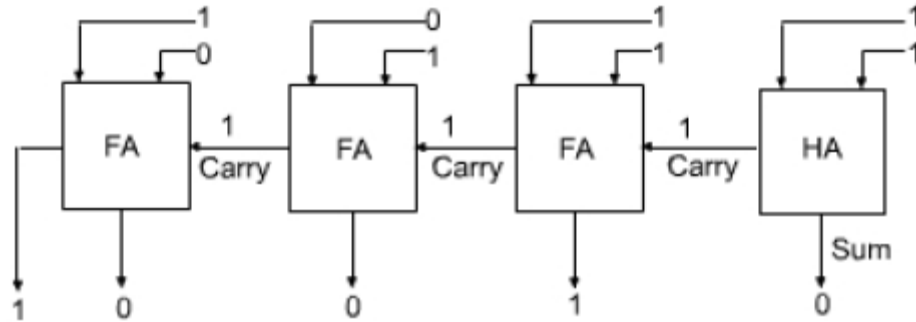


Figure 11. Example of adding 4-bit binary numbers

The adder of Fig. 10 has limited capacity. The largest binary numbers that can be added are 1111 and 1111. So, the maximum capacity is

$$\begin{array}{r}
 15 \quad 1111 \\
 +15 \quad +1111 \\
 \hline
 30 \quad 11110
 \end{array}$$

To increase the capacity, more full-adders can be connected to the left end of the system. For instance, to add 6-bit numbers, connect two more full-adders. How about subtraction? Figure 12 shows a circuit that subtracts $\overline{B}_3 \overline{B}_2 \overline{B}_1 \overline{B}_0$. The full-adders now add $A_3 A_2 A_1 A_0$, $\overline{B}_3 \overline{B}_2 \overline{B}_1 \overline{B}_0$ and the end around carry. In other words, the circuit does what we do in our minds when we use the 1's complement to subtract.

Figure 13 shows an adder/subtractor based on the 2's complement. When SUB is low, the B bits pass through the controlled inverter to the full-adders. Therefore, the full-adders produce the sum of A and B. For instance, if A = 1001 and B = 0011, the decimal and binary sums are

$$\begin{array}{r}
 9 \quad 1001 \\
 +3 \quad +0011 \\
 \hline
 12 \quad 1100
 \end{array}$$

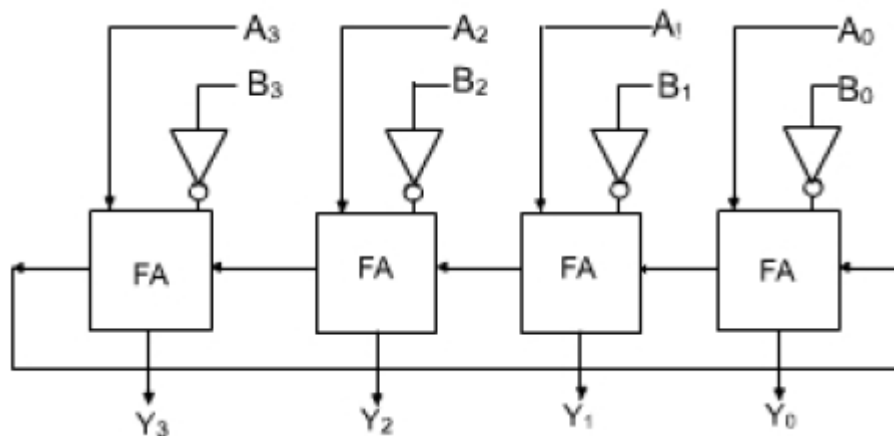


Figure 12. Parallel 4-bit binary subtractor

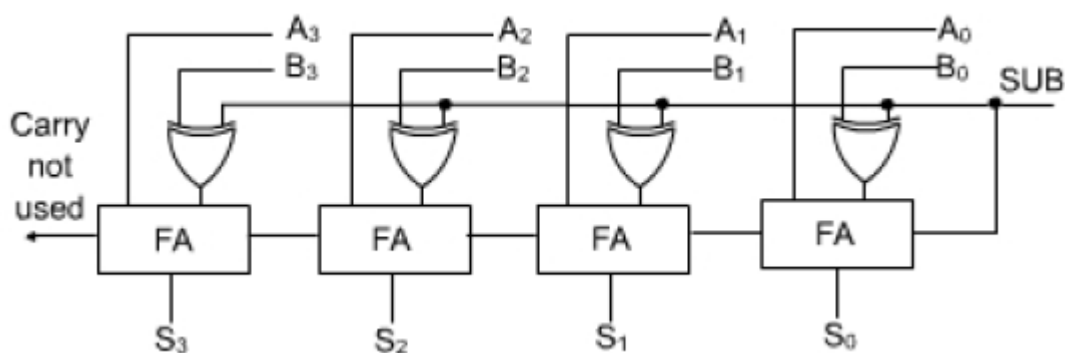


Figure 13. 2's complement adder / subtractor

When SUB is high, the B bits are inverted before reaching the full-adders. Also, a high SUB adds a 1 to the first full-adder. The addition of 1 forms the 2's complement of B. In other words, the controlled inverter produces the 1's complement, and adding 1 results in the 2's complement. Therefore, the output of the full-adders is the difference of A and B. For instance, with the same values of A and B used in the preceding example, the decimal and binary sums look like this when SUB is high:

9	1001
+3	+1101
6	0110

(As usual, disregard the final carry).

At last we are beginning to remove some of the mystery about digital computers. We have just seen how logic circuits can be connected to perform binary addition and subtraction. There are other aspects of computer operation such as storing numbers, bringing the numbers out memory into the arithmetic section, and putting the answer back into memory.

8.10 An 8421 Adder

Some digital systems use binary numbers throughout. But there are many machines that use BCD numbers. As mentioned earlier, the 8421 code is the most popular BCD code; it is often called “the BCD code”.

8.11 Forbidden Groups

In the 8421 code each decimal digit is converted to its 4-bit equivalent. For instance, decimal 539 becomes

5	3	9	Decimal equivalent
0101	0011	1001	8421

In the 8421 code there is no 4-bit group above 1001; combinations like 1010, 1011, 1100, 1101, 1110 and 1111 do not exist. Because of this, we run into trouble when trying to add 8421 numbers whose sum exceeds 9. For instance, if we add 8 and 5 using binary addition, we get

8	1000
+5	+0101
13	1101

The answer 1101 is fine in straight binary code, but it is meaningless. In the 8421, answer should be 0001 0011.

The problem arises because the 8421 code uses only 10 of 16 possible 4-bit groups. When the decimal sum exceeds 9, we must somehow skip the six forbidden groups to restore the answer to 8421 form. To skip these six forbidden combinations, we can add 6(0110) to the sum. For instance, if we add 8 and 5 using 8421 numbers, we get

	1000	
	+0101	
	1101	binary equivalent of 3
	+ 0110	add 6 to return to 8421 form
0001	0011	8421 equivalent of 13

The final answer is the 8421 equivalent of 13. As another example, we add 9 and 3.

9	1001	
+3	+0011	
12	1100	binary answer
	+0110	add 6 to return to 8421 form
0001	0010	8421 answer

8.12 Rules for BCD Addition

We can summarize the rules for 8421 additions as follows:

Add 8421 numbers using the rules of binary addition. If the sum of the 4-bit groups is greater than decimal 9, add 0110 to that sum to return to 8421 form.

If the sum is 9 or less, leave it alone because it is in 8421 form.

Fig. 15 shows an 8421 adder. This system adds 8421 digits using binary addition, and adds the correction of 0110 when the sum exceeds 9.

As an example of how the 8421 adder of Fig. 14 works, let us trace the addition for 8 and 5. At the top of the figure the two input BCD digits are $A_3A_2A_1A_0$ (1000) and $B_3B_2B_1B_0$ (0101). The upper row of adders combines these two inputs to get

$A_3A_2A_1A_0$	1000
+ $B_3B_2B_1B_0$	+ 0101
$S_3S_2S_1S_0$	01101

The sum is greater than 9, detected by feeding S_3 and S_2 into the upper AND gate. Both the inputs are 1s, so the AND gate produces a 1 output; this 1 output is fed into the OR gate, so a 1 comes out of the OR gate.

Next, to restore $S_3S_2S_1S_0$ (1101) to 8421 form, 0110 is added to it:

$$\begin{array}{r}
 S_3 S_2 S_1 S_0 \\
 + \quad 0 \ 1 \ 1 \ 0 \\
 \hline
 Y_3 Y_2 Y_1 Y_0
 \end{array}$$

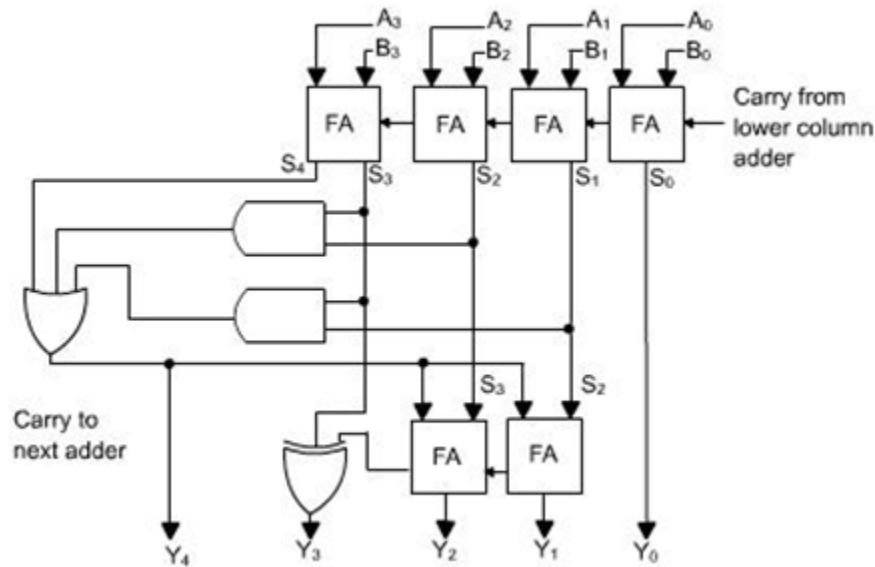


Figure 14. Four-bit 8421 adder

In the least significant column, y_0 must equal S_0 so S_0 comes down to the final output, as shown in Fig.15. The bottom row of adders adds the 1 out of the OR gate to S_2 and S_1 . This restores the answer to 8421 form.

The system of Fig. 14 will correctly add any two 8421 BCD digits. It defects when the decimal sum is greater than 9-this is the purpose of the two AND gates and the OR gate-and it adds 0110 to the answer. When the sum is 9 or less, 0000 is added.

Remember: The system of Fig.15 takes care of any one column of decimal digits. To add decimal numbers with several digits, use several 8421adders, one for each decimal column. For instance to add 531 and 326, three 8421 adders like that of Fig.15 are needed, one adder for each decimal column. By cascading 8421 adders, you can add numbers of any size.

8.13 Half and Full-Subtractors

Instead of using complements to subtract, circuits can subtract binary numbers directly. Recall the rules for binary subtraction:

$$0 - 0 = 0 \quad \text{with a borrow of 0}$$

$$0 - 1 = 1 \quad \text{with a borrow of 1}$$

$$1 - 0 = 1 \quad \text{with a borrow of 0}$$

$$1 - 1 = 0 \quad \text{with a borrow of 0}$$

Table 4 summarizes these results by listing the subtraction rules for A - B.

A	B	Borrow	Difference
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

What kind of logic circuit has a truth table like Table 4? First, the difference output is 0 whenever the inputs A and B are the same; the different. So, we can use an exclusive-OR gate to produce the difference output. Second, the borrow output is 1 only when A is 0 and B is 1. We can get this borrow output by ANDing $\overline{A}$ and B.

Figure 15 shows one way to build a half-subtractor, a circuit that subtracts one binary digit from another. The circuit of Fig. 15 has a truth table identical to Table 4. You can see there will be borrow only when A = 0 and B = 1. Further, the difference output will be correct for each of the four possible A - B combinations.

The half-subtractor handles only 2 bits at a time and can be used for the least significant column of a subtraction problem. To take care of a higher-order column, we need a full-subtractor. Figure 16 shows a full-subtractor; It uses two half-subtractors and an OR gate.

Half and full-subtractors are analogous to half-and full-adders; by cascading half and full-subtractors as shown in Fig.17, we have a system that directly subtracts $B_3B_2B_1B_0$ from $A_3A_2A_1A_0$.

The adders and subtractors give us the basic circuits needed for binary arithmetic; multiplication and division can be done by repeated additions and subtractions.

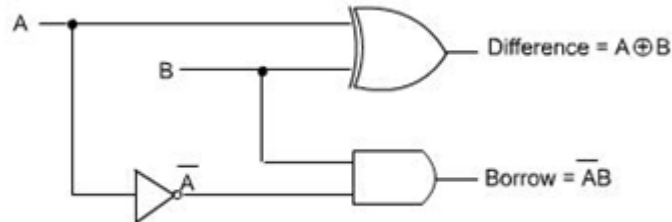


Figure 15. Half-Subtractor

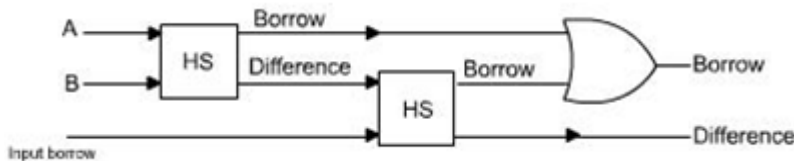


Figure 16. Full-Subtractor

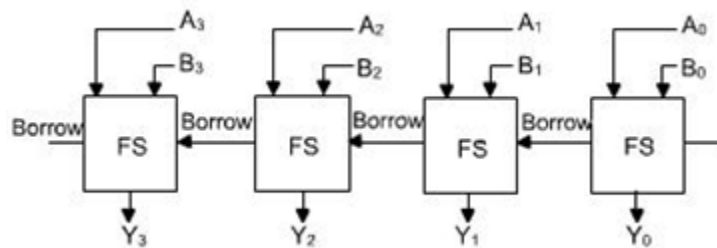


Figure 17. Parallel 4-bits binary subtractor

8.14 Summary

The full form of Ex-OR gate is **Exclusive-OR** gate. Its function is same as that of OR gate except for some cases, when the inputs having even number of ones.

The full form of Ex-NOR gate is **Exclusive-NOR** gate. Its function is same as that of NOR gate except for some cases, when the inputs having even number of ones.

Half adder is a combinational circuit, which performs the addition of two binary numbers A and B are of **single bit**. It produces two outputs sum, S & carry, C.

Full adder is a combinational circuit, which performs the **addition of three bits** A, B and C_{in} . Where, A & B are the two parallel significant bits and C_{in} is the carry bit, which is generated from previous stage.

Half subtractor is a combination circuit with two inputs and two outputs (difference and borrow). The disadvantage of a half subtractor is overcome by full subtractor. The full subtractor is a combinational circuit with three inputs A, B, C and two outputs D and C'. A is the 'minuend', B is 'subtrahend', C is the 'borrow' produced by the previous stage.

8.15 Model Questions

- (1) Explain how you will add and subtract two 4-bit numbers using a 4-bit adder and XOR gates.
- (2) Give logic expression for difference and borrow for the full subtractor.
- (3) Implement half-adder and full-adder using NAND gates only.
- (4) Explain the 4-bit 8421 adder circuit.

Solution

(2) Difference = $A \oplus B \oplus C$

$$\text{Borrow} = \overline{A}B + (A \oplus B)C$$

LESSON - 9

SIMPLIFYING LOGIC CIRCUITS

Structure

- 9.0 Objectives**
- 9.1 Introduction**
- 9.2 Fundamental Products**
- 9.3 Two Input Signal**
- 9.4 Three Input Signal**
- 9.5 Four Input Signal**
- 9.6 Sum of Products**
- 9.7 AND-OR-Networks**
- 9.8 Algebra Simplification**
- 9.9 Summary**
- 9.10 Model Questions**

9.0 Objectives

To introduce :

- Fundamental products
- Sum of products
- AND-OR-Networks
- Simplifying AND-OR networks

9.1 Introduction

In this chapter we focus SOP and POS. Standard SOP means Standard Sum of Products. In this form, each product term need not contain all literals. Standard POS means Standard Product of Sums. In this form, each sum term need not contain all literals. So, the sum terms may or may not be the Max terms.

9.2 Fundamental Products

In digital computers and most other digital systems, signals are available in either complemented or uncomplemented form. For instance, if two variables A and B exist, their complement $\bar{A}$ and $\bar{B}$ normally are also present.

9.3 Two Input Signals

Figure 1 shows the four possible ways to AND two input signals that are in complemented or uncomplemented form. Look at Fig. 1a and note the inputs are $\bar{A}$ and $\bar{B}$. Therefore, the output is

$$y = \bar{A} \cdot \bar{B}$$

It's clear the output can equal a 1 only when $\bar{A} = 1$ and $\bar{B} = 1$. Equivalently, the output is a 1 only when

$$A = 0 \text{ and } B = 0$$

From now on, we are going to abbreviate the foregoing input condition by

$$AB = 00$$

Figure 1b shows another possibility. Here the inputs are A and $\bar{B}$. So the output is

$$y = \bar{A} B$$

This output can equal a 1 only when $AB = 01$

Any other input condition produces an output of 0. In Fig. 1c, the inputs are A and B. The output

$$y = A \bar{B}$$

equals a 1 only for the input condition

$$AB = 10$$

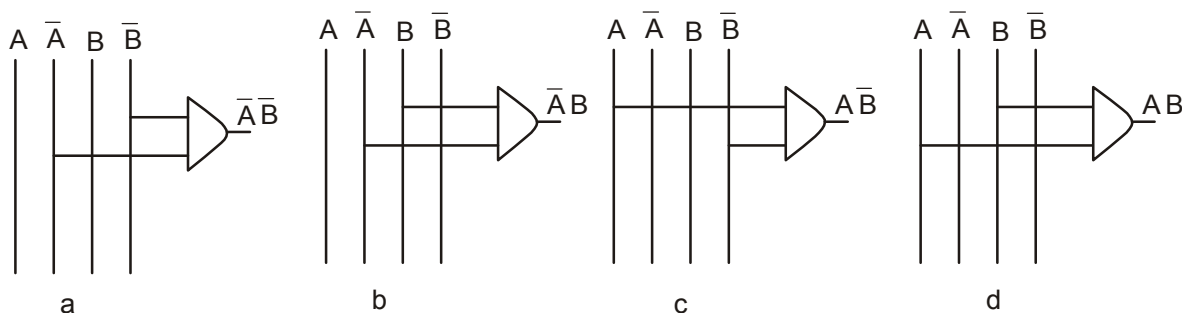


Fig. 1. Four possible ways of AND two input signal

Finally, in Fig.1d, the inputs are A and B, The input

$$y = AB$$

can equal a 1 only when

$$AB = 11$$

Table 1 : Fundamental Products

A	B	$\overline{A} \overline{B}$	$\overline{A} B$	$A \overline{B}$	AB
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Table 1 summarizes the four possible ways of ANDing two signals in complemented or uncomplemented form. The four logical products are $\overline{A} \overline{B}$, $\overline{A} B$, $A \overline{B}$ and AB . These products are called fundamental products because they represent the basic ways to combine two signals with an AND gate.

9.4 Three Input Signals

A similar idea applies to three signals in complemented and uncomplemented form. Let $\overline{A}$, $\overline{B}$ and $\overline{C}$ represent the three signals in complemented form, and let A, and C stand for the three signals in uncomplemented form. Then there are eight ways to combine these three signals with an AND gate: $\overline{A} \overline{B} \overline{C}$, $\overline{A} \overline{B} C$, $\overline{A} B \overline{C}$, $\overline{A} B C$, $A \overline{B} \overline{C}$, $A \overline{B} C$, $A B \overline{C}$ and ABC . Again, we refer to these basic possibilities as fundamental products.

Table 2 lists the fundamental products for three input signals. For convenience, each fundamental product appears adjacent to the input condition resulting in an output of 1. For instance, $\overline{A} \overline{B} \overline{C}$ equals 1 only when

$$A B C = 0 0 0$$

The fundamental product $\overline{A} \overline{B} C$ is a 1 only for an input condition of

$$A B C = 0 0 1$$

Likewise, $\overline{A} B \overline{C}$ equals 1 only for

$$A B C = 0 1 0$$

and so forth

Table 2: Fundamental Products of Three Variables

A	B	C	Fundamental Product
0	0	0	$\overline{A} \overline{B} \overline{C}$
0	0	1	$\overline{A} \overline{B} C$
0	1	0	$\overline{A} B \overline{C}$
0	1	1	$\overline{A} B C$
1	0	0	$A \overline{B} \overline{C}$
1	0	1	$A \overline{B} C$
1	1	0	$A B \overline{C}$
1	1	1	$A B C$

9.5 Four Input Signals

When there are four input variables ($A B C D$), there are 16 possible input conditions. 0 0 0 0 through 1 1 1 1. The corresponding fundamental products are from $\overline{A} \overline{B} \overline{C} \overline{D}$ through $A B C D$. A quick method for finding the fundamental product corresponding to any input condition is this: Whenever an input variable is a 0, the same variable is complemented in the fundamental product. For instance, if the input condition is

$$A B C D = 0 1 1 0$$

The corresponding fundamental product is $\overline{A} B C \overline{D}$; this is the product the results in a 1 output for the given input condition. If you want the fundamental product for

$$A B C D = 0 1 0 0$$

You have to complement A, C and D to get ABCD.

9.6 Sum of Products

Given a truth table, you can find the Boolean equation for the output by ORing the fundamental products that produce 1 outputs. The following examples show you how.

9.6.1 Two Input Signals

Suppose we're given a truth table like Table 3 and we want to find the Boolean equation corresponding to this table. To get a 1 output for the input condition;

$$A B = 1 0$$

we need a fundamental product of AB. Similarly, to get a 1 output for an input of

$$A B = 1 1$$

Table 3

A	B	Y
0	0	0
0	1	0
1	0	$1 \rightarrow A\bar{B}$
1	1	$1 \rightarrow AB$

we need a fundamental product of AB. ORing these products gives

$$y = A\bar{B} + AB \quad (1)$$

This is the Boolean equation for y because it results in a 1 for the input conditions $AB = 1 0$ and $AB = 1 1$; the equation produces a 0 for the other input conditions.

As another example of getting a Boolean equation from a truth table, look at Table 4. In this case, the fundamental products that produce the 1 outputs are $\bar{A}B$ and $A\bar{B}$. Therefore, the Boolean equation is

$$Y = \overline{A}B + A\overline{B} \quad (2)$$

As a check, you can substitute the four input conditions of Table 4; if you do, you will find y equals a 1 only for these input conditions.

$$AB = 01$$

$$AB = 10$$

Table 4

A	B	Y
0	0	0
0	1	$1 \rightarrow \overline{A}B$
1	0	$1 \rightarrow A\overline{B}$
1	1	0

9.6.2 Three Input Signals

When three input variables are involved, you again OR the fundamental products. For instance, suppose we're given a truth table like Table 5. The fundamental products that result in 1 outputs are listed. By ORing these products, we get the Boolean equation of the output:

$$y = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC \quad (3)$$

This is the equation that results in a 1 output only for the input conditions:

$$\overline{A}BC = 011$$

$$A\overline{B}C = 101$$

$$AB\overline{C} = 110$$

$$ABC = 111$$

Table 5

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	$1 \rightarrow \overline{A}BC$
1	0	0	0
1	0	1	$1 \rightarrow A\overline{B}C$
1	1	0	$1 \rightarrow AB\overline{C}$
1	1	1	$1 \rightarrow ABC$

In each of the foregoing examples, the same method is used

1. Find the fundamental products corresponding to each 1 output in the truth table.
2. OR the fundamental products.

These two steps always results in a Boolean equation that is a logical sum of fundamental products. For this reason, this method of getting the Boolean equation is called the “Sum-of-products” method.

Example

A truth table has four input variables. A 1 output occurs only for these input conditions.

A B C D = 0 0 1 0

A B C D = 1 0 0 1

A B C D = 1 1 0 1

What is the Boolean equation for the output?

Solution

The fundamental products for the given inputs are $\overline{A} \overline{B} C \overline{D}$, $A \overline{B} \overline{C} D$ and $AB \overline{C} D$, ORing these products gives

$$y = \overline{A} \overline{B} C \overline{D} + A \overline{B} \overline{C} D + AB \overline{C} D$$

9.7 AND-OR-Networks

The sum-of-products method results in a Boolean equation where the inputs are first ANDed, then ORed. Because of this, the logic circuit corresponding to a sum-of-products equation is a group of AND gates working into an OR gate.

For instance, Table 3 led to the Boolean equation

$$y = A \overline{B} + AB$$

This implies that A is ANDed with $\overline{B}$, A is ANDed with B and AB is ORed with $A \overline{B}$. Figure 2a shows the logic circuit to use when variables and their complements are available from other circuits. If complements are not available, you have to add an inverter as shown in Fig.2b. In most digital circuits, variables and their complements are available from other digital circuits.

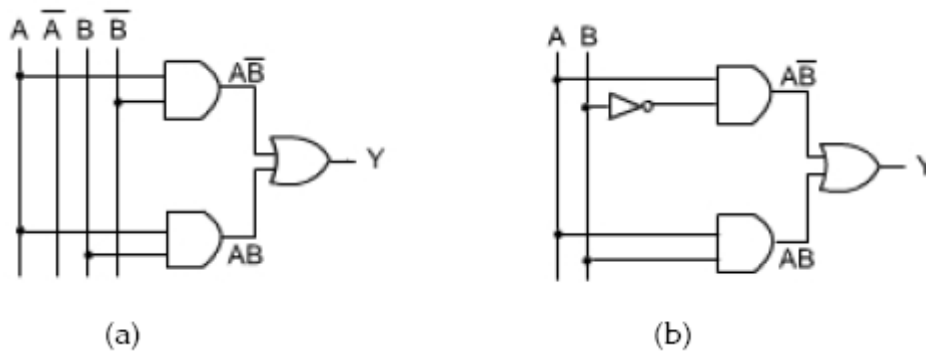


Fig.2 Sum-of-products logic diagrams

Here's another example of implementing a Boolean equation. Table 4 led to this Boolean equation.

$$y = \overline{A} B + A \overline{B}$$

Figure-3 shows the corresponding logic circuit. It produces a 1 output only for

$A B = 0 1$

$A B = 1 0$

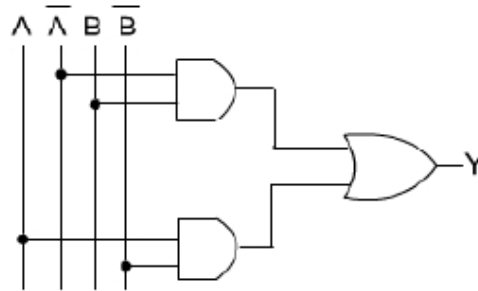


Figure.3 Sum-of-product circuit

Another example: Applying the sum-of-products method to Table 5 gave

$$y = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

To implement this, we need four three-input AND gates working into a four input OR gate as shown in Fig.4.

In summary given a truth table, we can find its Boolean equation by ORing the fundamental products producing 1 outputs. Because this equation is always a sum of products, the corresponding logic circuit consists of a group of AND gates driving on OR gate. The logic circuits like Figure-4 are called as AND-OR Networks.

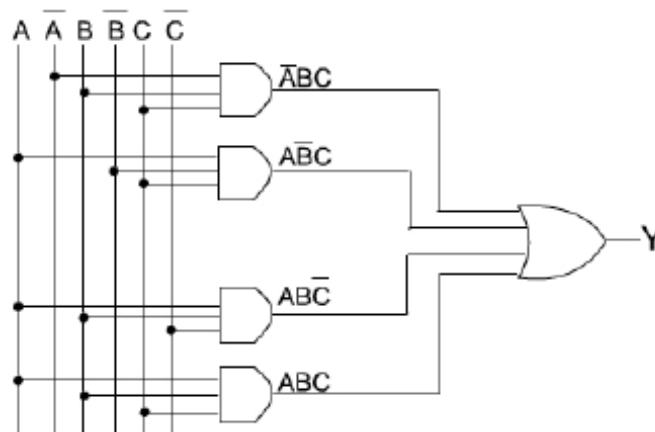


Fig.4. AND-OR networks

Example

Suppose a truth table leads to this Boolean equation:

$$y = A \bar{B} \bar{C} D + ABCD$$

What is the corresponding AND-OR network?

Solution

Figure-5 shows the AND-OR network. The logic circuit consists of 2 four-input AND gates and one two-input OR gate.

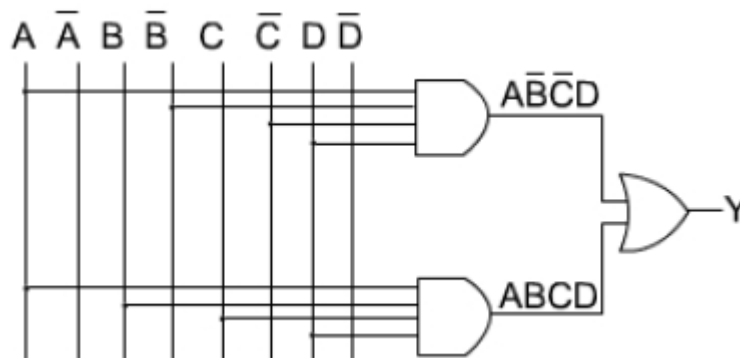


Figure.5 Example of AND-OR network

9.8 Algebra Simplification

The sum-of-products method always works. Given a truth table, you can always find a Boolean equation by ORing the appropriate fundamental products; then you can implement the equation with an AND-OR network. On small production runs, an AND-OR network like Fig.4 may be all right to use as is. But on large production runs, the cost of each gate contributes significantly to the overall cost of production. In this case, it pays to look for a simpler logic circuit.

A preliminary guide for comparing one logic circuit with another is to count the number of input gate leads; the circuit with fewer input gate leads is usually less expensive. For instance, the AND-OR network of Fig.5 has a total of 10 input gate leads (four on each AND gate and two on the OR gate). The AND-OR network of Fig.4 on the other hand, has a total of 16 input gate leads (three on each AND gate and four on the OR gate). The AND-OR network of Fig.5 will cost less to build than the AND-OR network of Fig.4.

Factoring to Simplify

One way to reduce the number of input gate leads is to factor the Boolean equation, if possible. For instance, Table 3 led to the Boolean equation

$$y = A\bar{B} + AB$$

Figure 6a shows the AND-OR network; it has six input gate leads. By factoring Eq.(4), we obtain

$$y = A(\bar{B} + B)$$

The logic circuit for this is shown in Fig.6b; only four input gate leads are required, so this circuit would cost less than the AND-OR network of Fig.6a.

As a matter of fact, recall that a variable ORed with its complement always equals 1; therefore,

$$y = A(\bar{B} + B) = A \cdot 1 = A$$

This equation says the output equals the value of A. This being the case, all we need is a connecting wire from input to output as shown in Fig.6c. In other words, we don't need any gates at all to implement Eq.4.

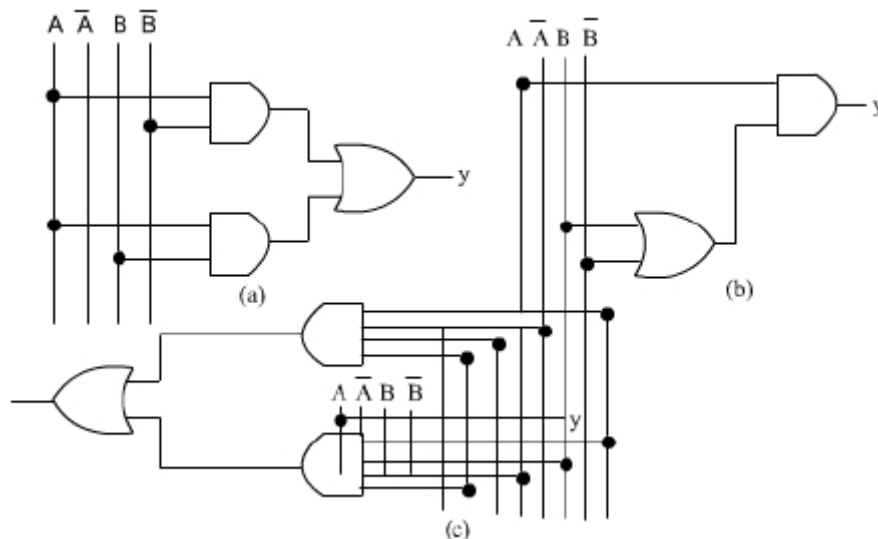


Figure 6. Equivalent logic circuits

Another Example

Suppose a truth table has a 1 output for these input conditions.

$$A B C = 0 1 0$$

$$A B C = 1 1 0$$

$$A B C = 1 1 1$$

Figure 7a shows the AND-OR network; it has 12 input gate leads. In this the least expensive logic circuit?

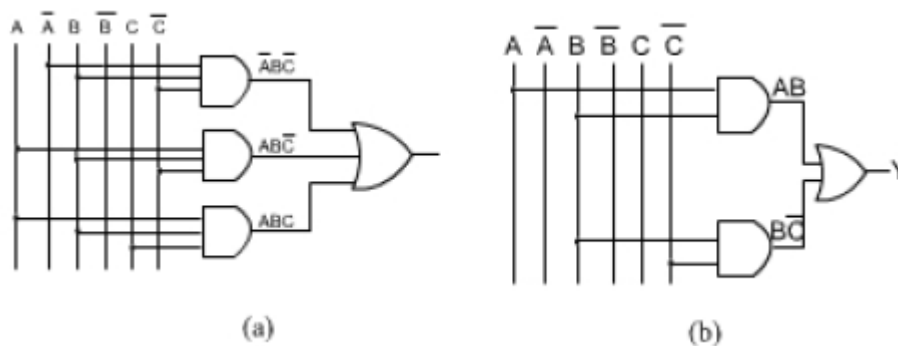


Figure.7. Simplifying an AND-OR network

Recall that a variable ORed with itself equals the variable; that is,

$$A + A = A$$

In a similar way, it's valid to write.

$$AB\bar{C} + AB\bar{C} = AB\bar{C}$$

Which is equivalent to

$$AB\bar{C} = AB\bar{C} + AB\bar{C}$$

Substitute this into Eq.5 to get

$$y = \bar{A}B\bar{C} + AB\bar{C} + AB\bar{C} + ABC$$

Next, factor to get

$$Y = (\bar{A} + A) B\bar{C} + AB(\bar{C} + C)$$

But a variable ORed with its complement always equals 1; there the foregoing equation reduces to

$$y = B\overline{C} + AB$$

Figure 7b is the AND-OR network for this simplified equation; it has only six input gate leads.

The two circuits of Fig.7 are equivalent, meaning they produce the same output for each input condition. In other words, both circuits have the same truth table. The circuit on the left has 12 input gate leads, while the one on the right has only six input gate leads. There's no question about which circuit is simpler.

This example brings out some important ideas. First, factoring a Boolean equation may lead to simpler equation and a less expensive logic circuit. Second, there appear to be algebraic tricks that lead to simpler equations and logic circuits. It is possible to become an expert at manipulating equations using the algebra rules. But most engineers and technicians don't simplify equations with algebra. Instead, they use the Karnaugh map.

Example

What is the AND-OR network for

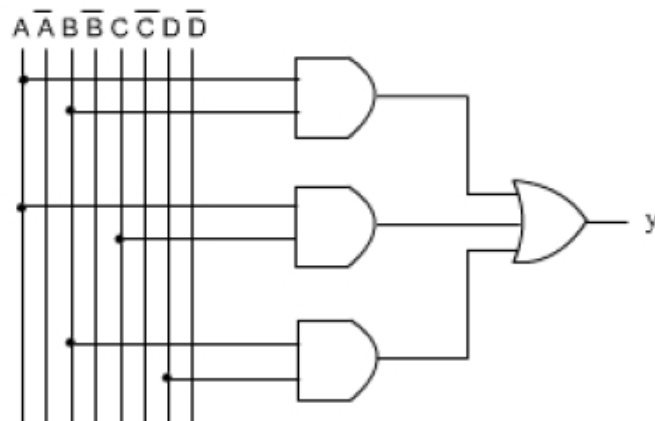
$$y = AB + AC + BD$$

If this equation is factored, what is the corresponding logic circuit?

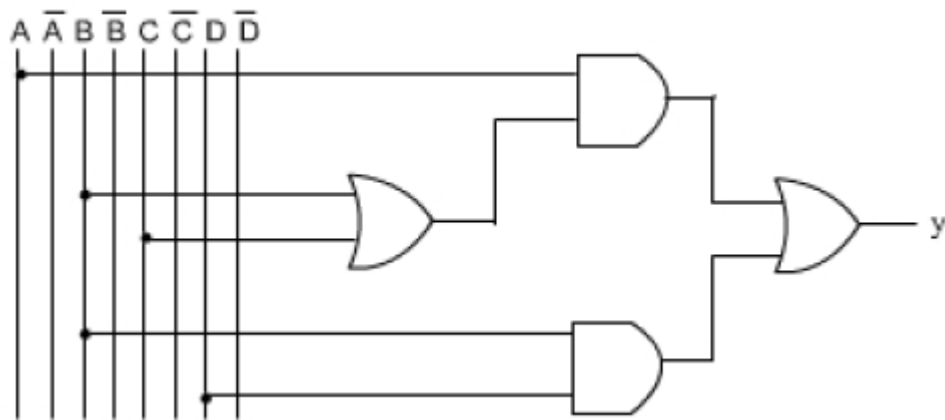
Solution

Figure 8a shows the AND-OR network; it has nine input gate leads. Factoring the given equation gives

$$y = A(B+C) + BD$$



(a)



(b)

Figure 8. Factoring to reduce input gate leads.

The corresponding logic circuit is shown in Fig.8b this has only eight input gate leads.

Even though Fig.8b has one fewer input gate lead than Fig.8a, it may be better to use Fig.8a. Here's one reason. In the worst case, the overall propagation delay time of Fig.8a is the sum of only two gate delays, whereas the overall propagation delay time of Fig.8b may be the sum of as many as three gate delays. For instance, if each gate has a t_p of 10ns, the overall propagation delay time is 20ns for Fig.8a and 30ns for Fig.8b in the worst case. If speed is important, Fig.8a is preferred in spite of its extra input gate lead.

9.9 Summary

Standard SoP form means **Standard Sum of Products** form. In this form, each product term need not contain all literals. So, the product terms may or may not be the min terms.

Standard PoS form means **Standard Product of Sums** form. In this form, each sum term need not contain all literals. So, the sum terms may or may not be the Max terms.

9.10 Model Questions

(1) Simplify the following expressions and draw the logic circuit using AND-OR gates.

a. $Y = \overline{A} \overline{B} \overline{C} + \overline{A} \overline{B} C + AB \overline{C} + ABC$

b. $Y = A \overline{B} + AB + \overline{A} \overline{B} C + ABC$

c. $Y = \overline{A} BC + A \overline{B} C + AB \overline{C} + ABC$

LESSON - 10

KARNAUGH MAP SIMPLIFICATION - I

Structure

- 10.0 Objectives**
- 10.1 Introduction**
- 10.2 Truth Table to Karnaugh Map**
- 10.3 Pairs, Quads, and Octets**
- 10.4 Karnaugh Simplifications**
- 10.5 Rolling The Map**
- 10.6 More Examples**
- 10.7 Eliminating Redundant Groups**
- 10.8 Boolean Equation Simplification**
- 10.9 Don't- Care Condition**
- 10.10 Nand-Nand Networks**
- 10.11 Product of Sums Solution**
- 10.12 Complemented Karnaugh Map**
- 10.13 Apply De Morgan's Theorems**
- 10.14 Summary**
- 10.15 Model Questions**

10.0 Objectives

- To draw Karnaugh map.
- 2-variable, 3-variable and 4-variable Karnaugh map
- Simplification using pairs, quads and octets
- Don't care conditions
- NAND-NAND / NOR-NOR networks

10.1 Introduction

Boolean expressions are used to design a digital electronic system. But the process becomes tedious and error-prone as the number of variables involved increases. This necessitates the use of Karnaugh Map. (K-Map) It is a graphical technique of simplifying boolean expressions. In this chapter, we focus on K-maps and the simplification of K-maps.

10.2 Truth Table to Karnaugh Map

Suppose you're given a truth table like Table 1. Here's how to construct the Karnaugh map for this truth table. Begin by drawing Fig. 1a. Note the order of the variables and their complements: the vertical column has $\overline{A}$ followed by A, and the horizontal row has B followed by $\overline{B}$.

10.2.1 Plotting 1's and 0's

Next, look for output 1's in Table 1. This first output 1 to appear is for an input of

$$A B = 1 0$$

Table 1

A	B	Y
0	0	0
0	1	0
1	0	1
1	1	1

The fundamental product for this is $A\overline{B}$. Now, enter a 1 on the Karnaugh map as shown in Fig. 1b. This 1 represents the product $A\overline{B}$ because the 1 is in the A row and $\overline{B}$ column.

Similarly, Table 1 has an output 1 appearing for an input of $AB = 11$

The fundamental product for this is AB . When you enter a 1 on the Karnaugh map to represent AB , you get the map of Fig. 1c.

The final step in the construction of the Karnaugh map is to enter 0s in the remaining spaces. These zeros mean fundamental products are not needed for the corresponding inputs.

$$A B = 0 0$$

$$A B = 0 1$$

Figure 1d shows how the Karnaugh map looks in final form.

How does a Karnaugh map differ from a truth table? A truth table shows the output for each input condition. A Karnaugh map, on the other hand, shows the fundamental products needed to produce the output 1's for corresponding input conditions. Later sections will tell you why this is important and how to use a Karnaugh map to simplify the logic circuit.

Here's another example of constructing a two-variable Karnaugh map. In the truth of Table 2, notice that output 1's appear for inputs of

$$A B = 0 1$$

$$A B = 1 0$$

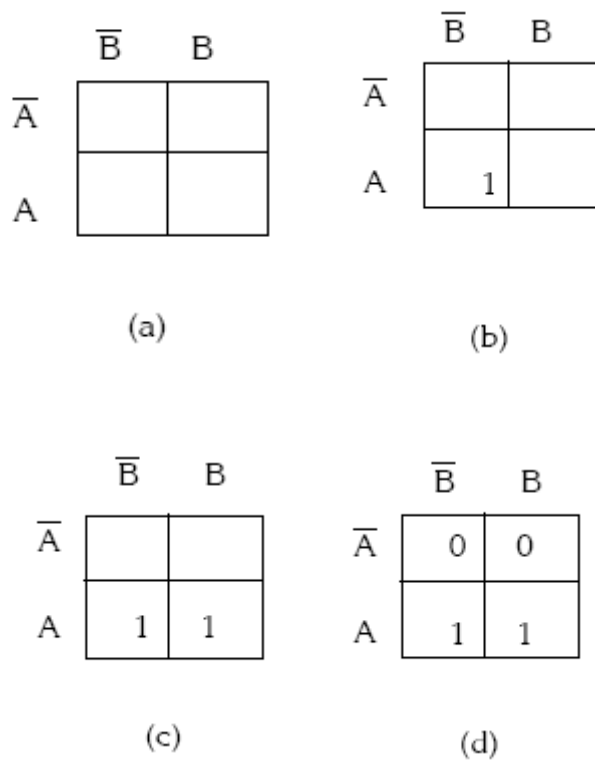


Figure.1. Constructing a Karnaugh map.

Table 2

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

	$\overline{B}$	B
$\overline{A}$	0	1
A	1	0

Figure 2

The fundamental products for these inputs are $A'B$ and AB' . When 1s are entered on the Karnaugh map for these products and 0s in the remaining spaces, the completed map looks like Fig.2.

10.2.2 Three Variable Map

Here's how to construct a Karnaugh map when there are three input variable: A, B, C. Suppose the truth table is that of Table 3. First, draw a blank Karnaugh map like Fig.3a. It is especially important to notice the order of the variables and complements. The vertical column is labeled $\overline{A}\overline{B}$, $\overline{A}B$, AB and $A\overline{B}$. This order is not a binary progression; rather, it follows the Gray-code progression of 00, 01, 11 and 10. The reason for this explained in the derivation of the Karnaugh - map method; briefly, it's done so that only one variable will change from complemented to uncomplemented form (or vice versa). Whenever you construct a Karnaugh map, you must use the Gray-code order of $\overline{A}\overline{B}$, $\overline{A}B$, AB and $A\overline{B}$.

Next, look for output 1s in Table 3. Output 1s appear for inputs of

$A B C = 0 1 0$

$A B C = 1 1 0$

$$A B C = 1 \ 1 \ 1$$

The fundamental products for these inputs are $\bar{A} B \bar{C}$, $A B \bar{C}$, and $A B C$.

Enter 1s for these products on the karnaugh map (Fig.3b). Each entry should be clear.

The 1 for $\bar{A} B \bar{C}$ appears

	$\bar{C}$	C
$\bar{A}\bar{B}$		
$\bar{A}B$		
AB		
$A\bar{B}$		

(a)

	$\bar{C}$	C
$\bar{A}\bar{B}$		
$\bar{A}B$	1	
AB	1	1
$A\bar{B}$		

(b)

	$\bar{C}$	C
$\bar{A}\bar{B}$	0	0
$\bar{A}B$	1	0
AB	1	1
$A\bar{B}$	0	0

(c)

Figure.3. Three-variable Karnaugh map.

Table 3

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

To the right of $A\bar{B}$ and below $\bar{C}$; the 1 for $AB\bar{C}$ is right of AB and under $\bar{C}$ and the 1 for ABC is right of AB and below C.

The final step is to enter 0s in the remaining spaces (Fig.3c). This complemented Karnaugh map is useful because it shows at a glance which fundamental products are needed to implement the Boolean equation corresponding to the truth table.

10.2.3 Four-Variable Karnaugh Maps

Table 4

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0

0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Here's an example of constructing a four variable Karnaugh map. Suppose you're given a truth table like table. 4 The first step in constructing the Karnaugh map is to draw the blank map of Fig.4a. Again, the order is crucial; the vertical column is labeled $\overline{A}\overline{B}$, $\overline{A}B$, AB and $A\overline{B}$; the horizontal row is labeled $\overline{C}\overline{D}$, $\overline{C}D$, $C\overline{D}$ and CD . You must always follow this Gray-code progression when constructing Karnaugh maps.

In Table 4, output 1s appear for these inputs:

$ABCD = 0001$

$ABCD = 0110$

$ABCD = 0111$

$ABCD = 1110$

The fundamental products corresponding to these inputs are $\overline{A}\overline{B}\overline{C}D$, $\overline{A}B\overline{C}\overline{D}$, $\overline{A}B\overline{C}D$ and $ABC\overline{D}$. After entering 1s on the Karnaugh map, we have fig.4b. The final step of filling in 0s results in the completed map of Fig.4c

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$				
$\overline{A}B$				
AB				
$A\overline{B}$				

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$		1		
$\overline{A}B$			1	1
AB				1
$A\overline{B}$				

(b)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	1	0	0
$\overline{A}B$	0	0	1	1
AB	0	0	0	1
$A\overline{B}$	0	0	0	0

(c)

Figure.4. Constructing a four variable Karnaugh map.

10.3 Pairs, Quads and Octets

Look at Fig.5a. The map contains a pair of 1s that are horizontally adjacent (next to each other). The first 1 represents the product $ABCD$; the second 1 stands for the product $ABC\bar{D}$. As we move from the first 1 to the second 1, only one variable goes from uncomplemented to complemented form (D to $\bar{D}$);

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	0	0	0	0
AB	0	0	1	1
$A\bar{B}$	0	0	0	0

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	0	0	0	0
AB	0	0	1	1
$A\bar{B}$	0	0	0	0

Figure.5. Horizontally adjacent 1's

The other variables don't change form (A , B and C remain uncomplemented). Whenever this happens, you can eliminate the variable that changes form.

PROOF

The sum-of-products equation corresponding of Fig.5a is $y = ABCD + ABC\bar{D}$

Which factors into

$$y = ABC (D + \bar{D})$$

Since D is ORed with $\overline{D}$, the equation reduces to

$$y = ABC$$

A pair of horizontally adjacent 1s like those of Fig.5a always means the sum-of-products equation will have a factorable variable and a complement that drop out as shown above.

For easy identification, it is customary to encircle a pair of adjacent 1s as shown in fig.5b. Then, when you look at the map, you can tell at a glance that one variable and its complement will drop out of the Boolean equation. In other words, an encircled pair of 1s like those of Fig.5b no longer stand for the ORing of two separate products, $ABCD$ and $ABC\overline{D}$; rather, the encircled pair should be visualized as representing a single reduced product ABC .

Figure 6a shows a pair of 1s that are vertically adjacent. These 1s correspond to products of $ABC\overline{D}$ and $A\overline{B}C\overline{D}$. Notice that only one variable changes from uncomplemented to complemented form (B to $\overline{B}$) all other variables retain their original form. Therefore, we can predict that B and $\overline{B}$ can be factored and eliminated algebraically.

Here's the proof. The sum-of-products equation is

$$y = ABC\overline{D} + A\overline{B}C\overline{D}$$

which factors into

$$y = AC\overline{D} (B + \overline{B})$$

This reduces too

$$y = AC\overline{D}$$

Therefore, the encircled pair of 1s in Fig.6a represents the simplified product $AC\overline{D}$.

From now on, we don't have to bother with algebra. Whenever we see a pair of horizontally or vertically adjacent 1s, we can eliminate the variable that goes from complemented to uncomplemented form (or vice versa). The remaining variables or their complements will be the only ones appearing in the single product term corresponding to the pair of 1s. For instance, a glance at Fig.6b indicates the B goes from complemented to uncomplemented form when we move from the upper to the lower 1; the other variables ($\overline{A}$, C and D) remain the same. Therefore, the encircled pair of 1s in Fig.6b represents the product $\overline{A}CD$.

Likewise, given the pair of 1s in Fig.6c, the only change is from D to $\bar{D}$. So the encircled pair of 1s stands for the product $A\bar{B}\bar{C}$.

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	0	0	0	0
AB	0	0	0	1
$A\bar{B}$	0	0	0	1

(a)

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	1	0
$\bar{A}B$	0	0	1	0
AB	0	0	0	0
$A\bar{B}$	0	0	0	0

(b)

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	0	0	0	0
AB	0	0	0	0
$A\bar{B}$	1	1	0	0

(c)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	1	1	0
AB	1	0	0	0
$A\overline{B}$	1	0	0	0

(d)

Figure 6

If more than one pair exists on a Karnaugh map, OR the simplified products to get the Boolean equation. For instance, the lower pair of Fig.14d represents the simplified product $A\overline{C}\overline{D}$; the upper pair stands for $\overline{A}BD$. The corresponding Boolean equation for this map is

$$y = A\overline{C}\overline{D} + \overline{A}BD$$

From now on, a pair means two 1s that are horizontally or vertically adjacent. Because one variable changes form, a pair always represents a simpler product. (Diagonally adjacent 1s are worthless; they lead to no simplifications).

THE QUAD

A quad is a group of four 1s that are horizontally or vertically adjacent. The 1s may be end-to-end as shown in

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	0	0
AB	1	1	1	1
$A\overline{B}$	0	0	0	1

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	0	0
AB	0	0	1	1
$A\overline{B}$	0	0	1	1

(b)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	0	0
AB	1	1	1	1
$A\overline{B}$	0	0	0	0

(c)

Figure 7. Examples of Quads

Fig 7a or in the form of a square as in Fig.7b. When you see a quad, always encircle it because it leads to a simpler product. In fact, a quad means two variables and their complements drop out of the Boolean equation.

Here's why a quad eliminates two variables and their complements. Visualize the four 1s of Fig.7 as two pairs (see Fig.7c). The first pair represents $AB\overline{C}$; the second pair stands for ABC . The Boolean equation for these two pairs is

$$y = AB\overline{C} + ABC$$

This factors into

$$y = AB (\bar{C} + C)$$

But this reduces to

$$y = AB$$

So, the quad of Fig.7a represents a product where two variables and their complements have dropped out.

A similar proof applies to any quad. You can visualize it as two pairs whose Boolean equation leads to a single product involving only two variables or their complements. There's no need to go through the algebra any more. Merely step through the different 1s in the quad and determine which two variables go from complemented to uncomplemented form (or vice versa); these are the variables that drop out.

For instance, look at the quad of Fig.7b. Pick any 1 as a starting point. When you move horizontally, D is the variable that changes form; when you move vertically, B changes form. Therefore, the remaining variables (A and C) are the only ones appearing in the simplified product. In other words, the simplified equation for the quad of Fig.7b.

$$y = AC$$

The Octet

Besides pairs and quads, there's one more group of adjacent 1s to look for: the octet. This is a group of eight 1s like those of Fig.8a. An octet like this eliminates three variables and their complements. Here's why. Visualize the octet as two quads (see Fig.8b). The equation for these two quads is

$$y = A\bar{C} + AC$$

After factoring,

$$y = A (\bar{C} + C)$$

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	0	0
AB	1	1	1	1
$A\overline{B}$	1	1	1	1

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	0	0
AB	1	1	1	1
$A\overline{B}$	1	1	1	1

(b)

Figure 8. Examples of Octets

But this reduces to

$$y = A$$

So the octet of Fig.8a means three variables and their complements drop out of the corresponding product.

A similar proof applied to any octet. So, from now on, don't bother with the algebra. Merely step through the 1s of the octet and determine which three variables change form. These are the variables that drop out.

10.4 Karnaugh Simplifications

You've just learned that a pair eliminates one variable and its complement, a quad eliminates two variables and their complements, and an octet eliminates three variables and their complements. Because of this, after you construct a Karnaugh map, you should encircle the octets first, the quads second, and the pairs last. In this way, the greatest simplification results.

Here's an example. Suppose you've translated a truth table into the Karnaugh map shown in Fig.9a. To find the simplified Boolean equation, look for octets first. There are none in Fig.9a. Next, look for quads. When you find them, encircle them. Finally, look for and encircle pairs. If you do this correctly, you arrive at Fig.9b.

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	1	1	1
$\overline{A}B$	0	0	0	1
AB	1	1	0	1
$A\overline{B}$	1	1	0	1

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	1	1	1
$\overline{A}B$	0	0	0	1
AB	1	1	0	1
$A\overline{B}$	1	1	0	1

(b)

Figure 9. Encircling octets, quads and pairs.

The pair represents the simplified product $\overline{A} \overline{B} D$, the lower quad stands for $A \overline{C}$, and the quad on the right represents $C \overline{D}$. By Oring these simplified products, we get the Boolean equation corresponding to the entire Karnaugh map:

$$y = \overline{A} \overline{B} D + A \overline{C} + C \overline{D}$$

Overlapping Groups

When you encircle groups, you are allowed to use the same 1 more than once. In other words, the same 1 can be common to two or more groups. Figure 10a illustrates this idea. The 1 representing the fundamental product $AB \overline{C} D$ is part of the pair and part of the octet. The simplified equation for the overlapping groups is

$$y = A + B \overline{C} D \quad (2)$$

It is valid to encircle the 1s as shown in Fig.10b, but then the isolated results in a more complicated equation:

$$y = A + \overline{A} B \overline{C} D$$

This will require a more complicated logic circuit than Eq.(2). So always overlap groups if possible; that is, use the 1s more than once to get the largest groups you can.

	$\overline{C} \overline{D}$	$\overline{C} D$	$C D$	$C \overline{D}$
$\overline{A} \overline{B}$	0	0	0	0
$\overline{A} B$	0	1	0	0
$A B$	1	1	1	1
$A \overline{B}$	1	1	1	1

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	1	0	0
AB	1	1	1	1
$A\overline{B}$	1	1	1	1

(b)

Figure 10. Overlapping groups

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	1	0	0	1
AB	1	0	0	1
$A\overline{B}$	0	0	0	0

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	1	0	0	1
AB	1	0	0	1
$A\overline{B}$	0	0	0	0

(b)

Figure 11. Rolling the map

10.5 Rolling the Map

Another thing to know about is rolling. Look at Fig. 11a. The pairs result in this equation:

$$y = B\overline{C}\overline{D} + BC\overline{D}$$

Visualize picking up the Karnaugh map and rolling it so that the left side touches the right side. If you're visualizing correctly, you will realize the two pairs actually form a quad. To indicate this, draw half circles around each pair as shown in Fig. 11b. From this viewpoint, the quad of Fig. 11b has the equation.

$$y = B\overline{D}$$

This final equation is the one that represents a rolled quad like Fig. 11b. Therefore, 1s on the edges of a Karnaugh map can be grouped with 1s on opposite edges.

10.6 More Examples

If possible, roll and overlap to get the largest groups you can find. For instance, Fig. 12a shows an inefficient way to encircle groups. The octet and pair have a Boolean equation of

$$y = \overline{C} + BC\overline{D}$$

You can do better by rolling and overlapping as shown in Fig. 12b; the Boolean equation now becomes

$$y = \overline{C} + B\overline{D}$$

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	1	1	0	0
$\overline{A}B$	1	1	0	1
AB	1	1	0	1
$A\overline{B}$	1	1	0	0

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	1	1	0	0
$\overline{A}B$	1	1	0	1
AB	1	1	0	1
$A\overline{B}$	1	1	0	0

(b)

Figure 12. Rolling and overlapping

Here's another example. Figure 13a shows an inefficient grouping of 1s; the corresponding equation is

$$y = \overline{C} + \overline{A}C\overline{D} + A\overline{B}C\overline{D}$$

If we roll and overlap as shown in Fig.13b, the equation is simpler:

$$y = C + \overline{A}\overline{D} + A\overline{B}\overline{D} \quad (5)$$

It's possible to group the 1s as shown in Fig.13c. The equation now becomes

$$y = \overline{C} + \overline{A}\overline{D} + \overline{B}C\overline{D} \quad (6)$$

Compare this with Eq.(5). As you can see, the equations are comparable in simplicity. Because of this, the corresponding logic circuits have the same number of input gate leads. Either grouping (Fig.13b or c) is valid therefore, you can use whichever you like.

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	1	1	0	1
$\overline{A}B$	1	1	0	1
AB	1	1	0	0
$A\overline{B}$	1	1	0	1

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	1	1	0	1
$\overline{A}B$	1	1	0	1
AB	1	1	0	0
$A\overline{B}$	1	1	0	1

(b)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	1	1	0	1
$\overline{A}B$	1	1	0	1
AB	1	1	0	0
$A\overline{B}$	1	1	0	1

(c)

Figure 13

10.7 Eliminating Redundant Groups

After you've finished encircling groups, there's one more thing you should do before writing the simplified Boolean equation: eliminate any groups whose 1s are completely overlapped by other groups. Here's an example. Given a Karnaugh map like Fig. 14a, the first thing you can do is encircle the quad in the center of the map to get Fig. 14b. Next, you can group the remaining 1s into pairs by overlapping (Fig. 14c). The final step is to review the encircled groups to see if any group is redundant (all of its 1s overlapped by other groups). In Fig. 14c, each 1 in the quad is overlapped by a pair. Because of this, the quad is redundant and can be eliminated to get Fig. 14d. The equation for Fig. 14d will contain one less product than the equation for Fig. 14c; therefore, Fig. 14d is the most efficient way to group the 1s.

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	1	0
$\overline{A}B$	1	1	1	0
AB	0	1	1	1
$A\overline{B}$	0	1	0	0

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	1	0
$\overline{A}B$	1	1	1	0
AB	0	1	1	1
$A\overline{B}$	0	1	0	0

(b)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	1	0
$\overline{A}B$	1	1	1	0
AB	0	1	1	1
$A\overline{B}$	0	1	0	0

(c)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	1	0
$\overline{A}B$	1	1	1	0
AB	0	1	1	1
$A\overline{B}$	0	1	0	0

(d)

Figure 14

10.8 Boolean Equation Simplification

Here's a summary of the Karnaugh-map method for simplifying Boolean equations

1. Enter a 1 on the Karnaugh map for each fundamental product that produces a 1 output in the truth table. Enter 0s elsewhere.
2. Encircle the octets, quads, and pairs. Remember to roll and overlap to get the largest groups possible.
3. If any isolated 1s remain, encircle each.
4. Review the groups, and eliminate any redundant group.
5. Write the Boolean equation by ORing the products corresponding to the encircled groups.

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	1	1	1	1
$A\overline{B}$	0	1	1	1

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	1	1	1	1
$A\overline{B}$	0	1	1	1

(b)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	1	1	1	1
$A\overline{B}$	0	1	1	1

(c)

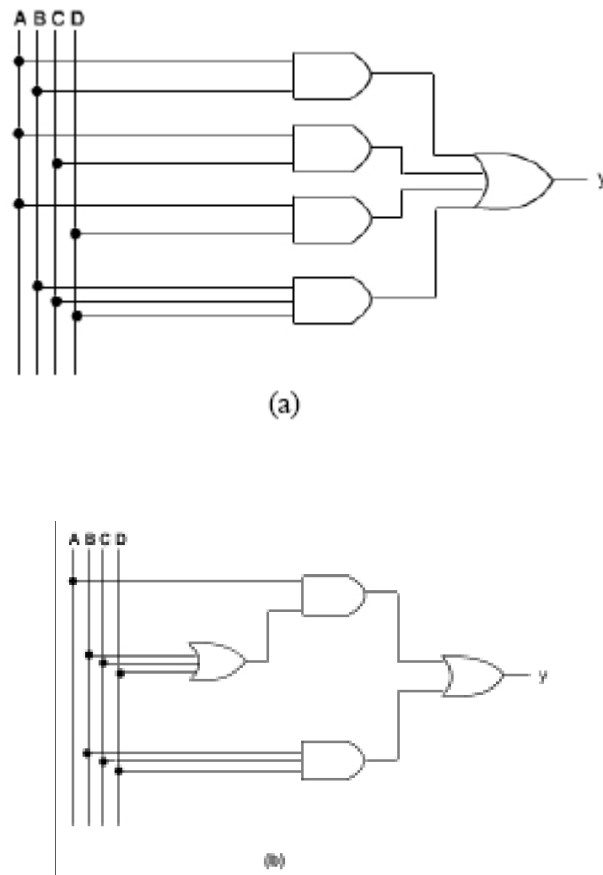
	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	1	1	1	1
$A\overline{B}$	0	1	1	1

(d)

Figure 15

Example

What is the simplified Boolean equation corresponding to the Karnaugh map of Fig.15a?

Solution**Fig.16**

There are no octets, but there is a quad as shown in Fig.15b. By overlapping, we can find two more quads (see Fig.15c). We can encircle the remaining 1 by making it part of an overlapped pair (Fig.15d). Finally, there are no redundant groups.

The horizontal quad of Fig.15d corresponds to a simplified product AB . The square quad on the right corresponds to AC , while the one on the left stands for AD . The pair represents BCD . By ORing these products, we get the simplified Boolean equation:

$$y = AB + AC + AD + BCD \quad (7)$$

Example

What is the AND-OR network corresponding to the simplified Boolean equation of the preceding example? If the equation is factored, what is the resulting logic circuit?

Solution

Figure 16a shows the AND-OR network for Eq.(7). Notice it has 13 input gate leads.

When Eq.(7) is factored, it becomes

$$y = A(B + C + D) + BCD$$

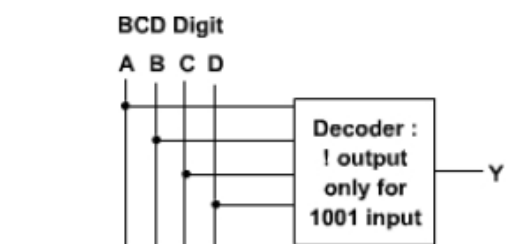
Figure 16b shows the corresponding logic circuit; it has only 10 input gate leads.

Figure 16b is simpler than Fig. 16a it might be preferred because it has a smaller propagation delay time (the sum of two gate delays instead of three). The AND-OR network can be converted into a NAND-NAND network (a group of NAND gates working into a final NAND) gate. A NAND gate is the least expensive TTL gate. Because of this, the AND-OR network of Fig. 16a can be built with fewer NAND gates than Fig. 16b.

The point of the example is this. Even though it may be possible to factor a simplified sum-of-products equation like Eq.(7) and reduce the number of input gate leads, this is not normally done because it increases the overall propagation delay time and may cost more to implement the logic circuit with NAND gates.

10.9 Don't Care Conditions

In Fig. 17a, a BCD input drives a decoder. As indicated, the decoder produces a 1 output only for a BCD input of 1001 (equivalent to decimal 9). As you already know, BCD digits are restricted to the 4-bit numbers from 0000.



(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	X	X	X	X
$A\overline{B}$	0	1	X	X

(b)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	x	x	x	x
$A\overline{B}$	0	1	x	x

(c)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	x	x	x	x
$A\overline{B}$	0	1	x	x

(d)

Figure 17. Don't care conditions

through 1001:1010 through 1111 cannot occur for normal operation.

This is way the truth table for the decoder of Fig.17 a lists only the inputs from 0000 through 1001 (see Table 5).

What is the logic circuit inside the decoder of Fig.17a?

Let's use the Karnaugh-map method to find the answer. To begin with, Tables 5 has a 1 output for only one input condition.

$$A B C D = 1 0 0 1$$

The fundamental product for this is ABCD. Figure 17b shows the 1 representing this fundamental product. Also shown are the 0s corresponding to the other input conditions of Table 5.

Table 5

A	B	C	D	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1

The empty spaces on the map are for the forbidden BCD inputs that are not listed in truth table.

Since forbidden BCD inputs don't occur under normal operating conditions, the empty spaces in Fig. 17b can be treated as 0s or 1s, whichever is more convenient. To indicate this, we mark X's as shown in Fig. 17c. These X's are called don't cares because they can be treated either as 0s or as 1s. (Don't cares are like wild cards in poker; you can let them stand for whatever you like).

Figure 17d shows the most efficient way to encircle the 1. Notice two crucial ideas. First, the 1 is included in a quad, the largest group you can find if you visualize all X's as 1s. Second, after the 1 has been encircled, all X's outside the quad are visualized as 0s. In this way, the X's are used to the best possible advantage. As already mentioned, you are free to do this because don't cares correspond to BCD inputs that cannot occur under normal operating conditions.

The quad of Fig. 17d results in the Boolean equation

$$y = AD$$

The logic circuit inside the decoder is therefore nothing more than an AND gate (Fig. 18). When A and D are both 1s (which only happens for a BCD input of 1001), the output y equals a 1.

In summary, don't cares can simplify the logic circuit if you remember these ideas.

1. Enter a 1 on the Karnaugh map for the fundamental products that produce output 1s in the truth table. Enter 0s for the other inputs listed in the truth table, and enter X's for the forbidden inputs (those that cannot occur during normal operation).
2. Encircle the actual 1s on the Karnaugh map in the largest groups you can find by treating the don't cares as 1s.

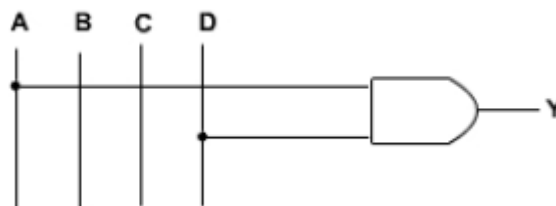


Fig. 18 Logic circuit for decoding 1001

3. After the actual 1s have been included in groups, disregard the remaining don't cares by visualizing them as 0s.

Example

What is the simplest logic circuit for a decoder that produces a 1 output when the BCD input is 0000?

Solution

The truth table has a 1 output only for the input condition

$$ABCD = 0000$$

The corresponding fundamental product is $\overline{A} \overline{B} \overline{C} \overline{D}$. Figure 19a shows the Karnaugh map with a 1 for this product, 0s for the other inputs of the truth table, and X's for the forbidden inputs (1010 through 1111). In this case, the don't cares are of no help. The best we can do is encircle the isolated 1, while treating the don't cares as 0s. So the Boolean equation is

$$y = \overline{A} \overline{B} \overline{C} \overline{D}$$

Figure 19b shows the corresponding logic circuit. This four-input AND gate produces a 1 output only for the input condition.

$$ABCD = 0000$$

Example

What's the simplest logic circuit for decoding a BCD input of 0111 as a 1 output?

Solution

Figure 20a is the Karnaugh map. The most efficient encircling is to group the 1 into a pair using the don't care as shown. Since this is the largest group possible, all remaining don't cares are treated as 0s. The equation for the pair is $y = BCD$ and Fig.20b is the logic circuit. This three-input AND gate produces a 1 output only for a BCD input of 0111.

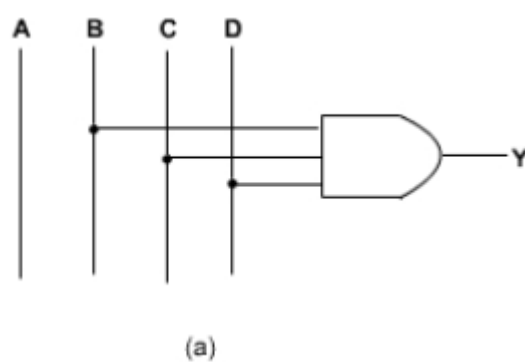
	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	1	0	0	0
$\overline{A}B$	0	0	0	0
AB	x	x	x	x
$A\overline{B}$	0	0	x	x

(a)

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	0	0	0	0
$\overline{A}B$	0	0	1	0
AB	x	x	x	x
$A\overline{B}$	0	0	x	x

(b)

Figure 19. Decoding 0000



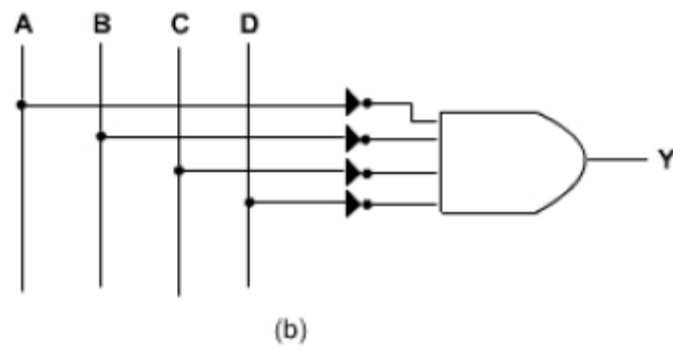


Figure 20

Example

Show one way of building a BCD-to-decimal decoder that produces a 1 output on one of 10 output lines.

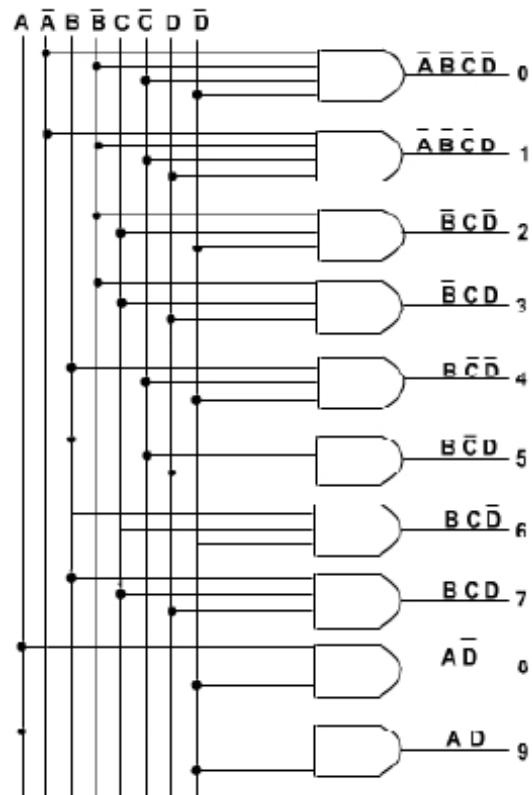
Solution

Fig. 21 BCD-to-Decimal decoder

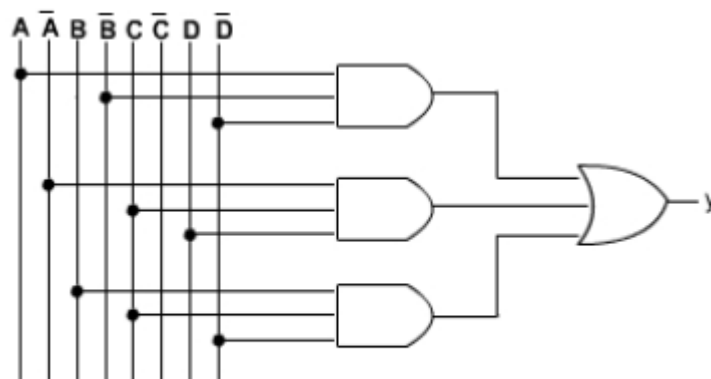
One way to do this is to use a decoder for each 1 output. In other words, we can use an AND gate like Fig.19b to decode a BCD input of 0000, an AND gate like Fig.20b for the 0111 input, an AND gate like Fig.18 for the 1001 input, and so forth. To get the AND gates for other inputs, we'd have to construct a Karnaugh map for each of the remaining inputs, as in Examples. When we finished, there would be one AND gate for each input to be decoded.

Figure 21 shows the final result. Given any BCD input from 0000 through 1001, only one AND gate will be on. Therefore, a 1 appears on only one of the 10 output lines.

10.10 NAND-NAND Networks

The NAND gate is the backbone of the 7400 TTL series. Not only is it the least expensive gate in the line, but other circuits in the 7400 series often use the NAND gate as a basic building block. 7400 series NAND gates are available as quad two-input gates, triple three-input gates, dual four-input gates, and single eight-input gates.

Any AND-OR network can be converted into a NAND-NAND network by replacing each AND gate by a NAND gate, and the final OR gate by a NAND gate. For example, if we have an AND-OR network like Fig.22a, we can replace it with the NAND-NAND network of Fig.22b. Both network have the same truth table.



(a)

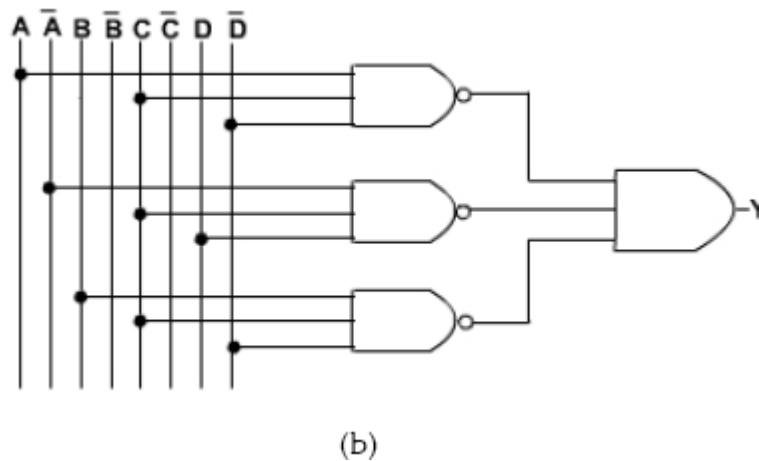


Figure.22. Converting AND-OR to NAND-NAND.

Here's the proof. To begin with, we need two ideas discussed earlier. Recall that an AND gate followed by an inverter is equivalent to a NAND gate, as shown in Fig.23a. Also, if you review the discussion of De Morgan's second theorem, you will agree that an OR gate preceded by inverters is equivalent to a NAND gate as shown in Fig.23b.

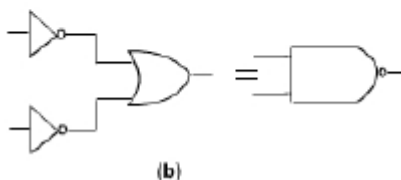
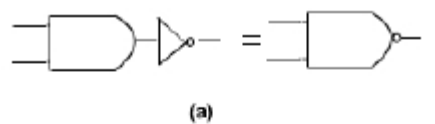


Figure.23 Equivalent Logic Circuits.

With the foregoing equivalent circuits we can now prove that any AND-OR network is replaceable by a NAND-NAND network. Consider the simple AND-OR network of Fig.24a. We can double complement each signal path as shown in Fig.24b without changing the value of the final output. Because of the equivalent circuits given in Fig.23, the circled portions of Fig.24c can be replaced by NAND gates as shown in Fig.24d.

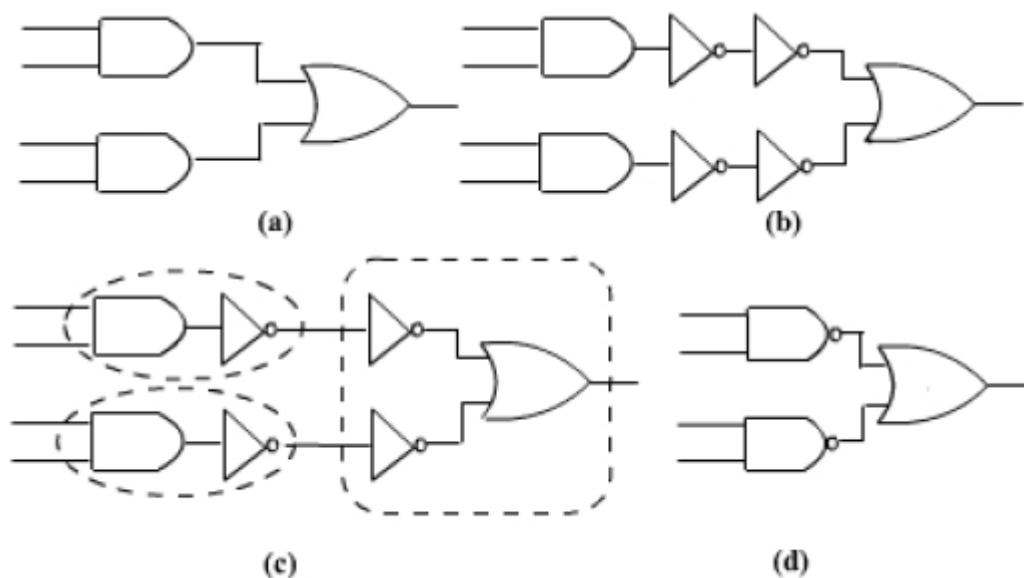


Figure.24. Providing a NAND-NAND is equivalent to an AND-OR

A similar proof applies to any number of AND gates driving on OR gate. Because of this, any AND-OR network can be replaced by a NAND-NAND network; simply replace each AND gate by a NAND gate, and replace the final OR gate by a NAND gate.

Now we have a way of getting from a truth table to an inexpensive NAND-NAND network. First, the truth table is translated into a Karnaugh map. Second, the 1s are grouped as efficiently as possible. Third, we write a simplified Boolean equation. Fourth, the corresponding AND-OR network is drawn. Fifth, we replaced each AND gate by a NAND gate, and the final OR gate by a NAND gate.

Incidentally, since a NAND gate is equivalent to an AND gate followed by an inverter (Fig.23a), a wire-AND connection and an inverter can replace the final NAND gate of a NAND-NAND network. For instance, suppose we want to implement

$$y = AB + CD + EF$$

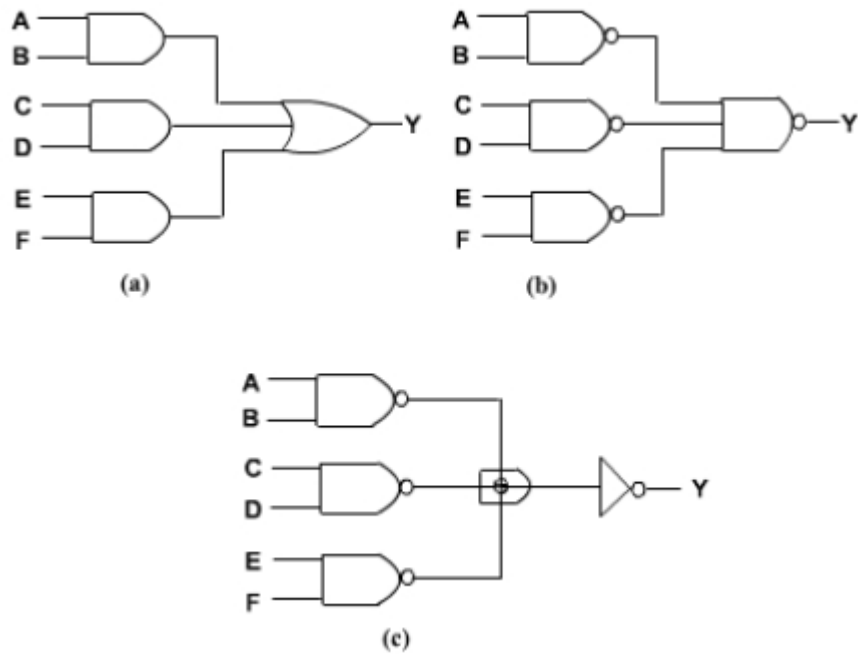


Figure 25. Wire-AND and inverter replace final NAND gate.

Figure 25a shows the AND-OR network for this, and Fig.25b is the equivalent NAND-NAND network. But the NAND function is the same as the AND function followed by inversion. For this reason, Fig.25c is equivalent to Fig.25b. Therefore, you can implement Eq. with any of the three circuits shown in Fig.25.

Example

Use De Morgan's first theorem to prove above Eq. gives the output of Fig.25c.

Solution

Let y_1 stand for the input to the inverter. Then

$$y_1 = AB \cdot CD \cdot EF$$

With De Morgan's first theorem, this becomes

$$y_1 = AB + CD + EF$$

The output of the inverter is the complement of y_1 therefore,

$$y = y_1 = AB + CD + EF$$

or

$$y = AB + CD + EF$$

By including an inverter after a wire-AND connection, we wind up with the ORing of logical products. This is why the wire- AND connection is often referred to as a wire-OR connection.

10.11 Product of Sums Solution

We have emphasized the sum-of-products method in previous sections. A sum-of-products solution always results in a group of AND gates driving a final OR gate. As you know, this is equivalent to a NAND-NAND network. Since the NAND gate is the most common gate in the TTL series, the sum-of-products solution is the most widespread way to implement a logic circuit.

But there is an alternative. It's known as the product-of-sums solution and results in a group of OR gates working into a final AND gate. Because of De Morgan's theorems, this is equivalent to NOR gates driving a NOR gate. Since the NOR gate is the most common gate in the ECL series, the product-of-sums solution may be preferred when ECL gates are being used.

10.12 Complemented Karnaugh Map

Figure 26a shows a Karnaugh map with three quads. The sum of products equation is

$$y = \overline{A} \overline{B} + AB + AC$$

Figure 26b is the logic circuit. it has nine input gate leads.

Here's how to work out the product-of-sums solution. First, complement the 0s and the 1s on the original map (Fig.26a) to get the

	$\overline{C}\overline{D}$	$\overline{C}D$	CD	$C\overline{D}$
$\overline{A}\overline{B}$	1	1	1	1
$\overline{A}B$	0	0	0	0
AB	1	1	1	1
$A\overline{B}$	0	0	1	1

(a)

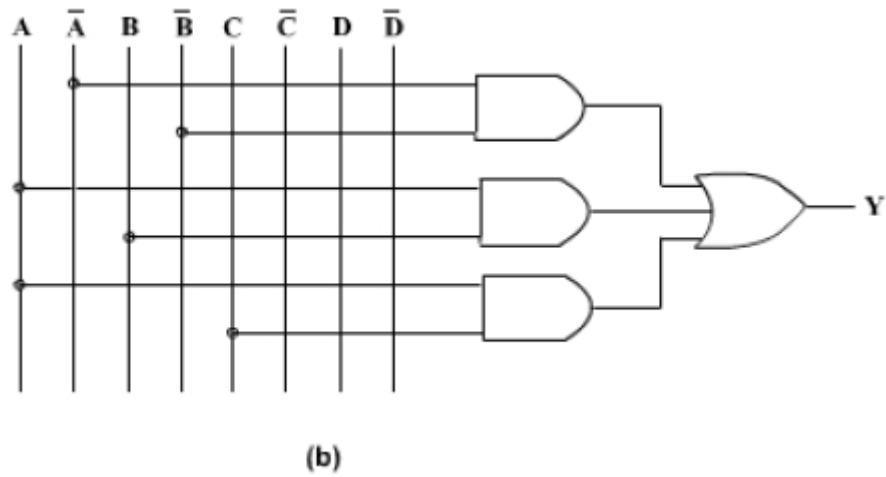


Figure 26

new map shown in Fig.27a. This is the Karnaugh map for y Because of the quad and the pair, the simplified equation is

$$Y = \overline{A} B + A \overline{B} \overline{C}$$

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	1	1	1	1
AB	0	0	0	0
$A\bar{B}$	1	1	0	0

(a)

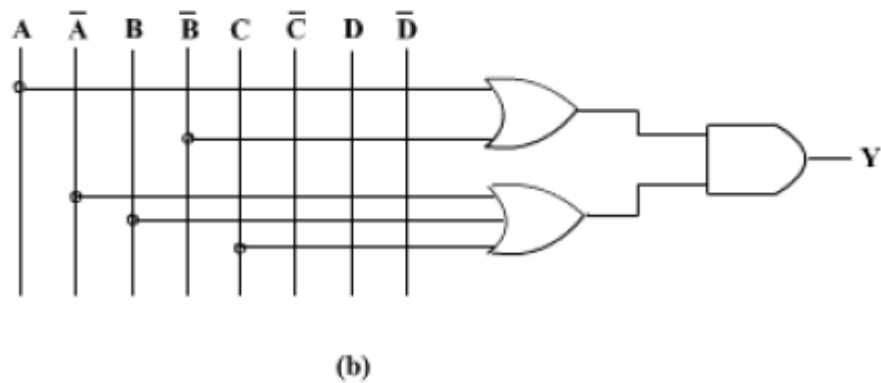


Figure 27

10.13 Apply De Morgan's Theorems

Next take the complement of theorems, we arrive at

$$y = \bar{A}B + A\bar{B}\bar{C}$$

By applying De Morgan's theorems, we arrive at

$$y = (A + \bar{B})(\bar{A} + B + C)$$

This final expression is in the form of a product of sums, and this is why it's called the product-of-sums solution.

Figure 27b shows the logic circuit. This time, we have OR gates working into a final AND gate. Notice that the circuit has seven input gate leads and is equivalent to the sum-of-products circuit found earlier (Fig.27b). Because of this equivalence, we have the option of using either circuit.

NOR-NOR Network

It is proved that my AND-OR network can be converted to a NAND-NAND network. A similar proof applies to OR-AND networks; they are equivalent to NOR-NOR networks. In other words, given a product-of-sums circuit like Fig.27b, we can replace all gates by NOR gates.

10.14 Summary

Here's how to get the product-of-sums solution:

1. Given the truth table for y, draw the Karnaugh map for $\bar{y}$.
2. Simplify in the usual way to get a sum-of-products equation for $\bar{y}$.
3. Apply the De Morgan theorems to get a product-of-sums equation.
4. Draw the logic circuit, which consists of OR gates driving at AND gate.

10.15 Model Questions

(1) Simplify the following equation using Karnaugh map $Y = AB + ABC + \bar{A}B + AC$

(2) Use K-map to simplify

$Y = AB\bar{C}\bar{D} + A\bar{B}CD + ABCD + ABC\bar{D} + \bar{A}\bar{B}\bar{C}D + \bar{A}BCD$ and draw the circuit for the simplified expression.

Solutions

(1) $Y = B$

(2) $Y = \bar{A}BD + ACD + AB\bar{D}$

LESSON - 11

KARNAUGH MAP SIMPLIFICATION - II

Structure

- 11.0 Objectives**
- 11.1 Introduction**
- 11.2 Minterms**
- 11.3 Maxterms**
- 11.4 5 Variable Map**
- 11.5 6 Variable K-Map**
- 11.6 Summary**
- 11.7 Model Question**

11.0 Objectives

After studying this lesson you should be able to:

- Minterms, Maxterms
- 5-variable K-map.
- 6-variable K-map.

11.1 Introduction

Karnaugh introduced a method for simplification of Boolean functions in an easy way. K-Map method is most suitable for minimizing Boolean functions of 2 variables to 5 variables.

11.2 Minterms

A binary variable may appear either in its normal form A or in its complement form $\bar{A}$. Consider three variables A, B, C . Each variable may appear in either form. There are eight possible combination which can be combined in an AND gate. Each AND term is called a minterm and represented by m_j Where j denotes the decimal equivalent of the binary number of the minterm designated.

Minterms

A B C	Term	Designation
0 0 0	ABC	m_0
0 0 1	ABC	m_1
0 1 0	ABC	m_2
0 1 1	ABC	m_3
1 0 0	ABC	m_4
1 0 1	ABC	m_5
1 1 0	ABC	m_6
1 1 1	ABC	m_7

Using the concept of minterm we can express logical functions. For example, if the expression is

$$Y = \bar{A} \bar{B} \bar{C} + A \bar{B} \bar{C} + AB \bar{C} \text{ can be represented as}$$

$$Y = m_0 + m_1 + m_3 \text{ or } Y = m_{(0,1,3)}$$

In a three variable K-map , each cell represents a minterm as shown in fig 11.1.

BC		00	01	11	10
A	0	m_0	m_1	m_3	m_2
	1	m_4	m_5	m_7	m_6

Similarly for a four variable map there will be 16 minterms as shown in Figure 11.2.

	CD	00	01	11	10
AB	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}

Therefore using the techniques introduced in lesson 10 - K-map I, logic equation represented in minterms can be simplified. Problems in K-map can be given as:

Simplify

$$F(A,B,C) = \sum m(0,1,3,5,7)$$

$$F(A,B,C,D) = \sum m(1,3,4,5,6,7,8,9,11,13,15)$$

11.3 Maxterms

Just as a minterm represents a standard product (AND), a maxterm represents a standard sum (OR). A maxterm is represented by M_j .

Maxterms

A B C	Terms	Designation
0 0 0	$A + B + C$	M_0
0 0 1	$A + B + C$	M_1
0 1 0	$A + B + C$	M_2
0 1 1	$A + B + C$	M_3
1 0 0	$A + B + C$	M_4
1 0 1	$A + B + C$	M_5
1 1 0	$A + B + C$	M_6
1 1 1	$A + B + C$	M_7

A Boolean function can be expressed as a sum of minterms or product of maxterms. They are said to be in canonical forms. An expression in one canonical form can be converted into another canonical form.

A product-of-sum expression given below,

$$Y = p (A+B+C) (A+B+C)(A+B+C)$$

can be represented as

$$Y = p (0,2,7)$$

11.4 5-variable map

A 5-variable map requires 32 squares. The rows and columns are represented in reflexed binary coded sequence. But forming groups is difficult and to avoid confusion, a 5-variable map 32 squares is represented as two 4-variable maps of 16 squares each.

Suppose the variables are A,B,C,D and E. The first map is drawn using the 4 variables B, C, D, E with A=0 and the second map using the variables B, C, D, E with A=1. This is shown below.

		DE				A = 0
BC		00	01	11	10	
00		m_0	m_1	m_3	m_2	
01		m_4	m_5	m_7	m_6	
11		m_{12}	m_{13}	m_{15}	m_{14}	
10		m_8	m_9	m_{11}	m_{10}	

		DE				
		00	01	11	10	
BC	00	m_{16}	m_{17}	m_{19}	m_{18}	$A = 1$
	01	m_{20}	m_{21}	m_{23}	m_{22}	
	11	m_{28}	m_{29}	m_{31}	m_{30}	
	10	m_{24}	m_{25}	m_{27}	m_{26}	

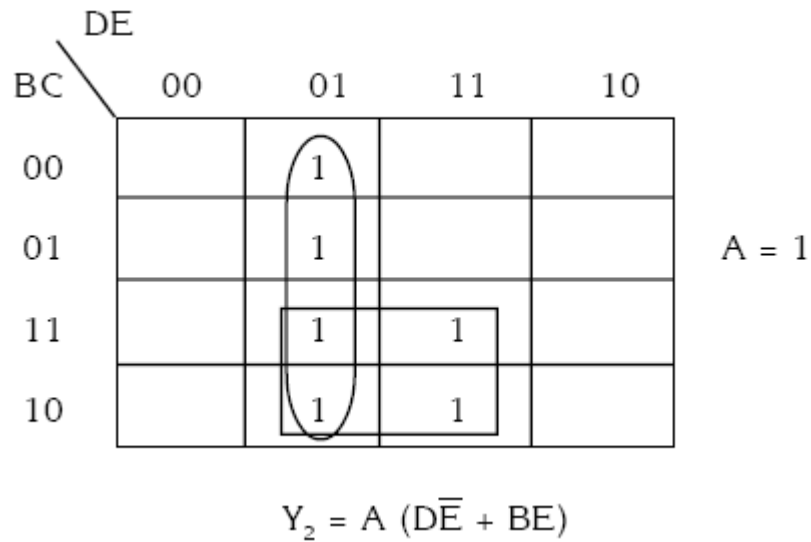
The two maps are simplified using the usual procedure and the final solution is given as a logical sum of the solutions of the two maps. Care should be taken to note $A = 0$ for the first map and $A = 1$ for the second map.

Example :

$$\text{Simplify } F(A,B,C,D,E) = (0,2,4,6,9,11,13,15,17,21,25,27,29,31)$$

		DE				
		00	01	11	10	
BC	00	1			1	$A = 0$
	01	1			1	
	11		1	1		
	10		1	1		

$$Y_1 = \bar{A} (\bar{B}\bar{E} + BE)$$



$$\begin{aligned}
 Y &= Y_1 + Y_2 \\
 &= \overline{A} (\overline{B} \overline{E} + BE) + A (\overline{D} \overline{E} + BE) \\
 &= \overline{A} \overline{B} \overline{E} + \overline{A} BE + A \overline{D} \overline{E} + ABE \\
 &= \overline{A} \overline{B} \overline{E} + BE(\overline{A} + A) + A \overline{D} \overline{E} \\
 Y &= \overline{A} \overline{B} \overline{E} + BE + A \overline{D} \overline{E}
 \end{aligned}$$

Two cells in the two maps can be combined when they occupy the same relative position on their respective maps.

11.5 6-variable K-map

A 6-variable map will have 64 squares representing the minterms m_0 to m_{63} and it will be very tedious to identify groups for simplifications. But a 6-variable map can be represented as four 4-variable maps (i.e) if the variables are A, B, C, D, E and F, each 4-variable is drawn with variables C, D, E, F, the first map with AB = 00, the second map with AB=01, the third map with AB=10 and the fourth map with AB = 11. The four maps with the corresponding minterms is shown below.

		EF			
		00	01	11	10
CD	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{17}
	10	m_8	m_9	m_{11}	m_{10}

$$A = 0 \quad B = 0$$

		EF			
		00	01	11	10
CD	00	m_{16}	m_{17}	m_{19}	m_{18}
	01	m_{20}	m_{21}	m_{23}	m_{22}
	11	m_{28}	m_{29}	m_{31}	m_{30}
	10	m_{24}	m_{25}	m_{27}	m_{26}

$$A = 0 \quad B = 1$$

		EF			
		00	01	11	10
CD	00	m_{32}	m_{33}	m_{35}	m_{34}
	01	m_{26}	m_{37}	m_{39}	m_{38}
	11	m_{44}	m_{45}	m_{47}	m_{46}
	10	m_{40}	m_{41}	m_{43}	m_{42}

$$A = 1 \quad B = 0$$

		EF			
		00	01	11	10
CD	00	m_{48}	m_{49}	m_{51}	m_{50}
	01	m_{52}	m_{53}	m_{55}	m_{54}
	11	m_{60}	m_{61}	m_{63}	m_{62}
	10	m_{56}	m_{57}	m_{59}	m_{58}

$$A = 1 \quad B = 1$$

11.6 Summary

Karnaugh introduced a method for simplification of Boolean functions in an easy way.

K-Map method is most suitable for minimizing Boolean functions of 2 variables to 5 variables. The number of cells in 2 variable K-map is four, since the number of variables is two.

The number of cells in 3 variable K-map is eight, since the number of variables is three.

The number of cells in 4 variable K-map is sixteen, since the number of variables is four.

The number of cells in 5 variable K-map is thirty-two, since the number of variables is 5.

11.7 Model Questions

Simplify using K-map

1. $F(A,B,C,D) = S(7,13,14,15)$
2. $F(A,B,C,D) = S(2,3,12,13,14,15)$
3. $F(A,B,C,D,E) = S(0,1,4,5,16,17,21,25,29)$
4. $F(A,B,C,D) = p(0,1,2,3,4,10,11)$

Solutions

1. $F = ABD + ABC + BCD$
2. $F = AB + \overline{A} \overline{B} C$
3. $F = \overline{A} \overline{B} \overline{D} + \overline{B} \overline{C} \overline{D} + A \overline{D} E$
4. $F = (A+B)(B+\overline{C})(A+C+D)$

LESSON - 12

TABULATION METHOD

Structure

- 12.0 Objectives
- 12.1 Introduction
- 12.2 Determination of prime-implicants
- 12.3 Summary
- 12.4 Model Question

12.0 Objectives

To introduce

- Tabulation method
- Determination of prime-implicants
- Selection of prime-implicants

Karnaugh method of simplification becomes very difficult (to identify groups) when the number of variables becomes more than 5 or 6. The tabulation method overcomes this difficulty. This method is also called as Quine-McClusky method. It is a specific step-by-step procedure that is guaranteed to produce a simplified standard form expression for a function.

The tabular method of simplification consists of two parts. The first is to find by exhaustive search all the terms that are candidates for inclusion in the simplified function. These terms are called prime-implicants. The second operation is to choose among the prime-implicants those that give an expression with the least number of literals.

12.1 Introduction

Karnaugh method of simplification becomes very difficult (to identify groups) when the number of variables becomes more than 5 or 6. The tabulation method overcomes this difficulty. This method is also called as Quine-McClusky method. It is a specific step-by-step procedure that is guaranteed to produce a simplified standard form expression for a function.

The tabular method of simplification consists of two parts. The first is to find by exhaustive search all the terms that are candidates for inclusion in the simplified function. These terms are called prime-implicants. The second operation is to choose among the prime-implicants those that give an expression with the least number of literals.

12.2 Determination of prime-implicants

First list the minterms that specify the function. Then find the prime implicants by using a matching process. This process compares each minterm with every other minterm. If two minterms differ in only one variable, that variable is removed and a term with one less literal is found. This process is repeated for every minterm until the exhaustive search is completed. The matching-process cycle is repeated for those new terms just found. The cycle can be repeated till elimination of literals is no longer possible. The remaining terms and all the terms that did not match during the process comprise the prime-implicants. The method is best understood with an example.

Example 1

Simplify by tabulation method

$$F(A,B,C,D) = S(0,1,2,8,10,11,14,15)$$

Step 1

Represent each minterm in binary as below :

0	–	0000
1	–	0001
2	–	0010
8	–	1000
10	–	1010
11	–	1011
14	–	1110
15	–	1111

Group the minterms into groups so that the first group contains no 1's, the second group contains one 1, the third group contains two 1's and so on. The decimal equivalents of the minterms are also included. (This is shown in column (a) of table 12.1)

Step 2

Any two minterms which differ from each other by only one variable can be combined and the unmatched variable removed. Two minterm numbers fit into this category if they both have the same bit value in all positions except one. From column (a) minterms in one section is compared with the minterms in the next section which is just one step down (i.e.) in this example the minterm m_0 is compared with the minterms m_1, m_2 and m_8 only. Similarly, the minterms m_1, m_2 and m_8 are compared with the minterm m_{10} only. If any two numbers are the same in every bit position but one, a check is placed to the right of both minterms to show that they have been used.

Tabulation method for $F(A, B, C, D) = S(0, 1, 2, 8, 10, 11, 14, 15)$

(a)						(b)						(c)					
A B C D						A B C D						A B C D					
0	0	0	0	0	✓	0,1	0	0	0	--	0,2,8,10	--	0	--	0		
1	0	0	0	1	✓	0,2	0	0	--	0	✓	0,8,2,10	--	0	--	0	
2	0	0	1	0		0,8	--	0	0	0	✓	10,11,14,15	1	--	1	--	
8	1	0	0	0		2,10	--	0	1	0	✓	10,14,11,15	1	--	1	--	
10	1	0	1	0	✓	8,10	1	0	--	0	✓						
11	1	0	1	1	✓	10,11	1	0	1	--	✓						
14	1	1	1	0	✓	10,14	1	--	1	0	✓						
15	1	1	1	1	✓	11,15	1	--	1	1	✓						
						14,15	1	1	1	--	✓						

Table 1 Determination of Prime-implicants

The minterm $m_0 = 0000$ is compared with the minterm $m_1 = 0001$. The variable eliminated during the matching process is denoted by a dash in its original position (i.e.) m_0 and m_1 can combine to give $000-$. The minterms m_0 and m_2 (000 and 0010) combine to give $00-0$. Similarly m_0 and m_8 gives -000 . This is continued for other minterms in column (a). The results are tabulated in column (b).

Step 3

The terms of column (b) have only three variables. A '1' under the variable means it is unprimed, a '0' means it is primed and a dash means the variable is not included in the term. The searching and comparing process is repeated for the terms in column (b) to form two variable terms in column (c). Again, terms in each section need to be compared only if they have dashes in the same position. Note that the term $(000-)$ does not match with any other term. Therefore, it has no check mark at its right. The decimal equivalents are written on the left hand side of each entry for identification purposes. The comparing process should be carried out again in column (c) and in subsequent columns as long as proper matching is encountered. In present example, the operation stops at the third column.

Step 4

The unchecked terms in the table form the prime-implicants. In this example, we have the term, $A'B'C'$ ($000-$) in column (b), and the terms $B'D'$ ($-0-0$) and AC ($1-1-$) in column (c). Note that each term in column (c) appears twice in the table, and as long as the term forms a prime-implicant, it is unnecessary to use the same term twice. The sum of the prime-implicants, it is unnecessary to use the same term twice. The sum of the prime-implicants gives a simplified expression for the function. This is because each checked term in the table has been taken into account by an entry of a simpler term in a subsequent column. Therefore, the unchecked entries are the terms left to formulate the function. For the present example, the sum of prime-implicants gives the minimized function in sum of products.

$$F = A'B'C' + B'D' + AC$$

It is worth comparing this answer with that obtained by the map method. Fig. 12.1 shows the map simplification of this function.

		CD			
		00	01	11	10
AB	00	1	1		1
	01				
	11			1	1
	10	1		1	1

Fig. 1 $F = \overline{A} \overline{B} \overline{C} + \overline{B} \overline{D} + AC$

Karnaugh map for $F(A,B,C,D) = S(0,1,2,8,10,11,14,15)$

$$F = \overline{A} \overline{B} \overline{C} + \overline{B} \overline{D} + AC$$

The combinations of adjacent squares give the three prime-implicants of the function. The sum of these three terms is the simplified expression in sum of products. The minterms m_0 and m_1 form a pair giving $A' B' C'$. The minterms m_{10} , m_{11} , m_{14} and m_{15} form a quad giving AC . The minterms m_0 , m_2 , m_8 and m_{10} (the four corner 1's) form another quad giving $B' D'$. Therefore the simplified expression using K-map is

$$F = A' B' C' + AC + B' D'$$

This is the same as the expression obtained in tabulation method.

It is important to point out that example 12.1 was purposely chosen to give the simplified function from the sum of prime-implicants. In most other cases, the sum of prime-implicants does not necessarily form the expression with the minimum number of terms. This we will see in the later example.

The tedious manipulation that one must undergo when using the tabulation method is reduced if the comparing is done with the decimal numbers instead of binary. A method will now be shown that uses subtraction of decimal numbers instead of the comparing and matching of binary numbers. We note that each 1 in a binary number represents the coefficient multiplied by a power of 2. When two minterms are the same in every position except are, the minterm with

the extra 1 must be larger than the number of the other minterm by a power of 2. Therefore two minterms can be combined if the number of the first minterm differs by a power of 2 from a second larger number in the next section down the table. We shall illustrate this procedure by repeating example 12.1.

As shown in Table 12.1a, column (a) the minterms are arranged in section as before except that now only the decimal equivalents of the minterms are listed. The process of comparing minterms is as listed. The process of comparing minterms is as follows : Inspect every two decimal numbers in adjacent sections of the table. If the number in the section below is greater than the number in the section above by a power of 2. (i.e. 1, 2, 4, 8, 16, etc.). Check both numbers to show that they have been used and write them down in column (b). The pair of numbers transferred to column (b). The pair of numbers transferred to column (b) includes a third number in parentheses tells us the position of the dash in the binary notation. The result of all comparisons of column (a) is shown in column (b).

The comparison before adjacent sections is column (b) is carried out in a similar fashion, except that only those terms with the same number in parentheses are compared. The pair of numbers in one section must differ by a power of 2 from the pair of numbers combination to take place. In column (c), write all four decimal numbers with the two numbers in parentheses designating the positions of the dashes. A comparison of Tables 12-1 and 12-2 may be helpful in understanding. The derivations in table 12-2.

The prime-implicants are those terms not checked in the table. These are the same as before, except that they are given in decimal notation. To convert from decimal notation to binary; convert all decimal numbers in the term to binary and then insert a dash in those positions designated by the numbers in parentheses. Thus 011 (1) is converted to binary as 0000, 0001 ; a dash in the first position of either number results in (000 -).

$$F = A' B' C' + B' D' + A C$$

Let us now do another problem of the same type.

Example 2

1. Simplify by tabulation method :

$F(A, B, C, D) = (0, 1, 4, 5, 10, 11, 14, 15)$

Solution :

By Karnaugh map method, we get

AB \ CD	CD			
	CD	CD	CD	CD
00	1	1		
01	1	1		
11			1	1
10			1	1

Fig. 2 $F = \overline{A}\overline{C} + AC$

Table 2 (a) Determination of Prime-implicants

(a)	(b)	(c)
0 ✓	0.1 (1)	0,2,8,10 (2,8)
1 ✓	0.2 (2) ✓	0,2,8,10 (2,8)
2 ✓	0.8 (8) ✓	10,11,14,15 (1,4)
8 ✓	2,10 (8) ✓	10,11,14,15 (1,4)
10 ✓	8,10 (2) ✓	
11 ✓	10,11 (1) ✓	
14 ✓	10,14 (4) ✓	
15 ✓	11,15 (4) ✓	
	14,15 (1) ✓	

**Table 2 (b) $F(A,B,C,D) = (0, 1, 4, 5, 10, 11, 14, 15)$
 $F = \overline{AC} + AC$**

(a)		(b)		(c)	
0	0 0 0 0 ✓	(0,1)	0 0 0 --	(0,1,4,5)	0 -- 0 --
		(0,4)	0 -- 0 0	(0,1,4,5)	0 -- 0 --
1	0 0 0 1 ✓	(1,5)	0 -- 0 1	(10,11,14,15)	1 -- 1 --
4	0 1 0 0 ✓	(4,5)	0 1 0 --	(10,14,15)	1 -- 1 --
5	0 1 0 1 ✓	(10,11)	0 1 0 --		
10	1 0 1 0 ✓	(10,14)	0 -- 1 0		
11	1 0 1 1 ✓	(11,15)	1 -- 1 1		
14	1 1 1 0 ✓	(14,15)	1 1 1 --		
15	1 1 1 1 ✓				

As already mentioned the sum of prime implicant does not necessarily form the expression with the minimum number of terms. This is shown in the next example.

Example 3

Determine the prime implicant of the function :

$$F(W, X, Y, Z) = (1, 4, 6, 7, 8, 9, 10, 11, 15)$$

The minterm numbers are grouped in sections as done earlier. The binary equivalent of the minterms is included for the purpose of counting the number of 1's. The binary numbers in the first section have only one 1 ; the second section two 1's etc. The minterm numbers are compared by the decimal method and a match is found if the number in the section below is greater than that in the section above. If the number in the section below is smaller than the one above a match is not recorded even if the two numbers differ by a power of 2.

The exhaustive search in column (a) results in the term of column (b), with all the minterm in column (a) checked. There are only two matches of term in column (b). 8, 9 (1) joining with 10, 11 (1) to give 8, 9, 10, 11, (1, 2). 8, 10 (2) joins with 9, 11 (2) to give 8, 9, 10, 11 (1, 2).

Refer table 12.3

The prime implicant selection table is as shown in Table 12.3a.

There are six rows one for each prime-implicant. There are one columns each representing one minterm of the function. Crosses are placed in each row to indicate the minterms contained in the prime implicant of that row. For example, the two crosses in the first row indicate that minterms 1 and 9 are contained in the prime implicant $X'Y'Z$. Decimal equivalent of minterms used to get the prime-implicant is also included in the table for convenience.

Table 3 Determination of Prime-implicants

(a)		(b)		(c)	
0 0 0 1	1 ✓	1,9	(8)	8,9,10,11	(1,2)
0 1 0 0	4 ✓	4,6	(2)	8,9,10,11	(1,2)
1 0 0 0	8 ✓	8,9	(1) ✓		
0 1 1 0	6 ✓	8,10	(2) ✓		
1 0 0 1	9 ✓	6,7	(1)		
1 0 1 0	10 ✓	9,11	(2) ✓		
0 1 1 1	7 ✓	10,11	(1) ✓		
1 0 1 1	11 ✓	7,15	(8)		
1 1 1 1	15 ✓	11,15	(4)		

		yz		y	
wx		00	01	11	10
	00		1		
	01	1		1	1
	11			1	
	10	1	1	1	1
		z			

Fig. 3 $F = wx' + xyz + x'y'z + w'xz'$

After all the crosses have been marked, we can proceed to select a minimum number of prime-implicants.

The completed prime-implicant selection table is inspected for columns containing only a single cross. In this example, there are four minterms whose columns have a single cross 1, 4, 8 and 10. Minterms 1 is covered by prime-implicant $X'Y'Z$, i.e., the selection of prime-implicant $X'Y'Z$ guarantees that minterm 1 is included in the function. Similarly, minterm 4 is covered by prime-implicant $W'XZ'$, and minterms 9 and 10, by prime-implicant WX' . Prime-implicants that cover minterms with a single cross in their column are called essential prime-implicants. To enable the final simplified expression to contain all the minterms, we have no alternative but to include essential prime-implicants. A check mark is placed in the table next to the essential prime-implicants to indicate that they have been selected.

Prime implicants are collected from all the unchecked terms.

PRIME IMPLICANTS

Decimal	Binary	Term
	A B C D	
1,9(8)	-- 0 0 1	$X'Y'Z$
4,6(2)	0 1 -- 0	$W'XZ'$
6,7(1)	-- 1 1 1	$W'XY$
7,15(8)	-- 1 1 1	XYZ
11,15(4)	1 -- 1 1	WYZ
8,9,10,11(1,2)	1 0 -- --	WX'

The prime implicant selection table is shown in 12.3a.

Table 4 Prime implication Selection

		1	4	6	7	8	9	10	11	15
✓	$x'y'z$	1,9	x				x			
✓	$w'xz'$	4,6	x	x						
	$w'xy$	6,7		x	x					
	xyz	7,15			x					x
	wyz	11,15							x	x
✓	wx'	8,9,10,11				x	x	x	x	
			✓	✓	✓	✓	✓	✓	✓	

7 and 15 are not covered. The prime-implicant xyz covers both the minterms 7 & 15. Therefore, xyz is also covered.

Next we check each column whose minterm is covered by the prime-implicants. For example, the selected prime-implicant $x'y'z$ covers minterms 1 and 9. A check is inserted in the bottom of the columns.

Similarly, prime-implicant $w'xz'$ covers minterms 4 and 6 and wx' covers minterms 8, 9, 10 and 11. Inspection of the prime-implicants covers all the minterms of the function except 7 and 15. These two minterms must be included by the selection of one or more prime-implicants. In this example it is clear that prime-implicant xyz covers both minterms and is therefore the one to be selected. We have thus found the minimum set of prime-implicants whose sum gives the required minimized function :

$$F = x'y'z + w'xz' + wx' + xyz$$

12.3 Summary

In previous chapter, we discussed K-map method, which is a convenient method for minimizing Boolean functions up to 5 variables. But, it is difficult to simplify the Boolean functions having more than 5 variables by using this method.

Quine-McClukey tabular method is a tabular method based on the concept of prime implicants. We know that **prime implicant** is a product (or sum) term, which can't be further reduced by combining with any other product (or sum) terms of the given Boolean function.

12.4 Model Questions

- (1) Represent $(-37)_{10}$ in (a) sign magnitude (b) signed and 1's complement (c) signed and 2's complement
- (2) Add $(+22)_{10}$ with $(-13)_{10}$ using 9's complement and 10's complement.
- (3) Add $(+34)_{10}$ with $(-56)_{10}$ representing them binary and using 1's complement and 2's complement.

Solutions

- (1) 1010 0101 ; 1101 1010 ; 1101 1011
- (2) 09
- (3) 1110 1001; 1110 1010

LESSON - 13

FLIP-FLOPS

Structure

- 13.0 Objectives**
- 13.1 Introduction**
- 13.2 Flip-Flops**
- 13.3 RS Flip-Flop**
- 13.4 Clocked RS Flip-Flop**
- 13.5 D Flip-Flop**
- 13.6 Edge-Triggered D Flip-Flop**
- 13.7 JK Flip-Flop**
- 13.8 JK Master-Slave Flip-Flop**
- 13.9 Summary**
- 13.10 Model Questions**

13.0 Objectives

To introduce :

- RS Flip-flop using NOR gates and NAND gates.
- Clocked RS Flip-flop
- Present and clear functions
- D Flip-flop
- J.K and J.K Master slave flip-flops

13.1 Introduction

We can implement flip-flops in two methods. In first method, cascade two latches in such a way that the first latch is enabled for every positive clock pulse and second latch is enabled for every negative clock pulse. So that the combination of these two latches become a flip-flop. In second method, we can directly implement the flip-flop, which is edge sensitive.

13.2 Flip-Flops

A flip-flop is a bistable electronic circuit that has two stable states—that is, its output is either 0 or +5 V dc as shown in Fig. 1 & 2. The flip-flop also has memory since its output will remain as set until something is done to change it. As such, the flip-flop can be regarded as a memory device. In fact, any bistable device can be used to store one binary digit (bit). For instance, when the flip-flop has its output set at 0 V dc, it can be regarded as storing a logic 0 and when its output is set at +5 V dc, as storing a logic 1.

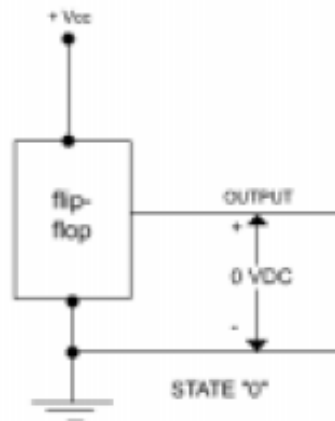


Fig. 1

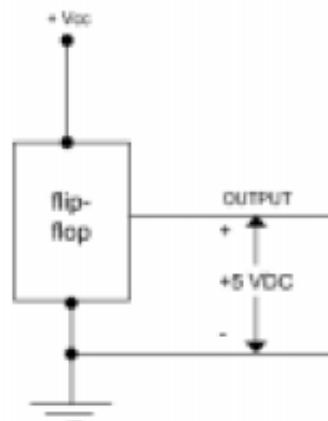


Fig. 2

13.3 RS Flip-Flops

The basic flip-flop is shown in Fig. 3. The flip-flop actually has two outputs, defined in more general terms as Q and $\bar{Q}$. It should be clear that regardless of the value of Q , its complement is $\bar{Q}$. There are two inputs to the flip-flop defined as R and S .

The input/output possibilities for this RS flip-flop are summarized in the truth table in Fig. 4.

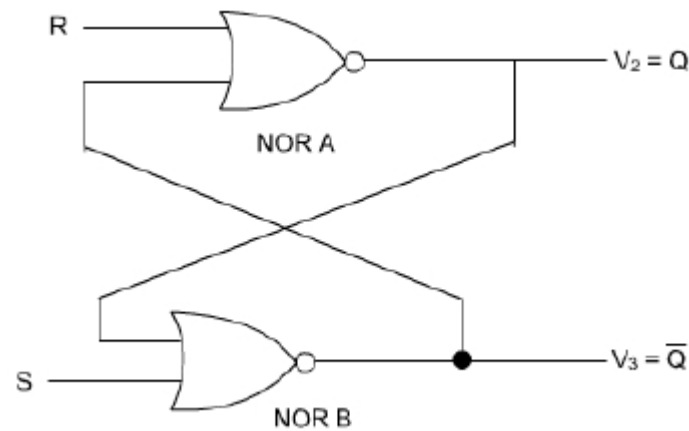


Fig.3 Nor-Gate Flip-Flop

R	S	Q	Action
0	0	Last Value	No change
0	1	1	Set
1	0	0	Reset
1	1	?	Forbidden

Fig.4 Truth Table for a NOR-gate RS flip-flop

The first input condition in the truth table is $R = 0$ and $S = 0$. Since a 0 at the input of a NOR gate has no effect on its output, the flip-flop simply remains in its present state, that is Q remains unchanged.

The second input condition $R = 0$ and $S = 1$ forces the output of NOR gate B low. Both inputs to NOR gate A are now low, and the NOR-gate output must be high. Thus a 1 at the S input is said to SET the flip-flop, and it switches to the stable state where $Q = 1$.

The third input condition is $R = 1$ and $S = 0$. This condition forces the output of NOR gate A low, and since both inputs to NOR gate B are now low, the output must be high. Thus a 1 at the R input is said to RESET the flip-flop, and it switches to the stable state where $Q = 0$ (or $\bar{Q} = 1$).

The last input condition in the table, $R = 1$ and $S = 1$, is forbidden, as it forces the outputs of both NOR gates to the low state. In other words, both $Q = 0$ and $\bar{Q} = 0$ at the same time! But this violates the basic definition of a flip-flop that requires Q to be the complement of $\bar{Q}$, and so it is generally agreed never to impose this input condition. Incidentally, if this condition is for some reason imposed, the resulting state of Q is not predictable. That's why the truth table entry is a ?.

The standard logic symbol for an RS flip-flop is shown in Fig 5 and 6 along with its truth table. The truth table is necessary since it describes exactly how the flip-flop functions. An RS flip-flop is also called a "latch," or a "bistable multivibrator".

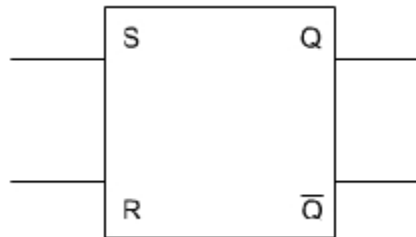


Fig.5 Logic Symbol

R	S	Q
0	0	Last State
0	1	1
1	0	0
1	1	? (Forbidden)

Fig 6: Truth table for RS Flipping

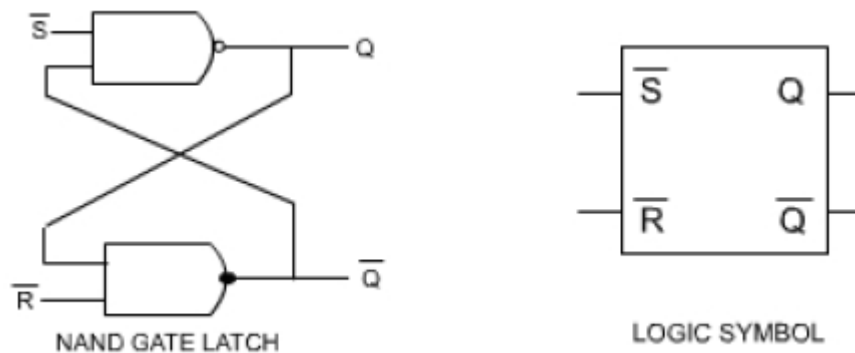


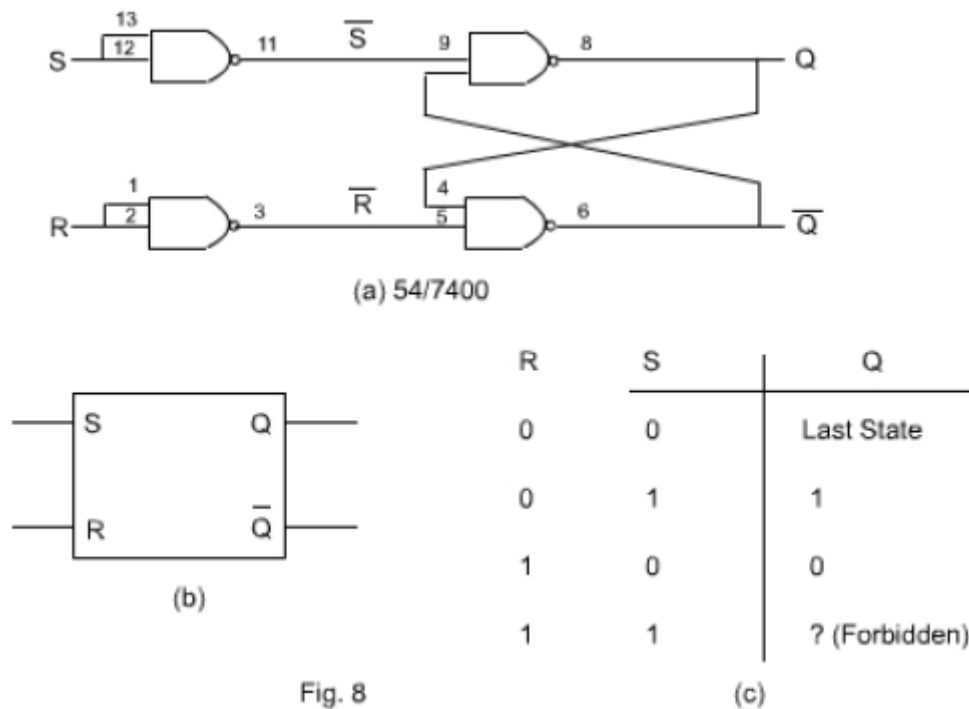
Fig. 7: Logic Circuit for RS Flip-Flop

A slightly different latch can be constructed by using NAND gates as shown in Fig.7. The truth tables for this NAND-gate latch is different from that for the NOR-gate latch. We will call this latch an RS flip-flop. To understand how this circuit function, recall that a low on any input to a NAND gate will force its output high. Thus a low on the S input will set the latch ($Q = 1$ and $Q = 0$). A low on the R input will reset it ($Q = 0$). If both R and S are high, the flip-flop will remain in its previous state. Setting both R and S low simultaneously is forbidden since this forces both Q and $\bar{Q}$ high. Fig.8 shows how to convert the RS flip-flop into an RS flip-flop by placing an inverter at each input, the 2 inputs are now R and S, and the resulting circuit behaves exactly as the RS flip-flop in Fig.6.

Simple latches can be constructed from NAND or NOR gates or obtained as medium-scale integrated circuits (MSI).

13.4 Clocked RS Flip-Flop

Two different methods for constructing an RS flip-flop were discussed. The NOR-gate realization in Fig.3 is an exact equivalent of the NAND-gate realization in Fig.8a, and they both have the exact same symbol and truth tables as given in Fig.5 & 6. Both of these RS flip-flops, or latches, are said to be “transparent”; that is, any change in input information at R or S is transmitted immediately to the output at Q and $\bar{Q}$ according to the truth table.



The addition of two AND gates at the R and S inputs as shown in Fig.9 will result in a flip-flop that can be enabled or disabled. When the ENABLE input is low, the AND gate outputs must both be low and changes in neither R nor S will have any effect on the flip-flop output Q. The latch is said to be disabled.

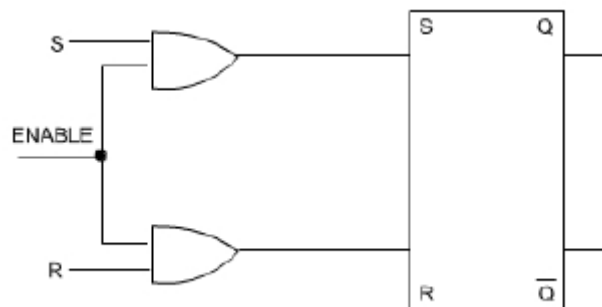


Fig.9 Logic Diagram Clocked RS Flip-Flop

When the ENABLE input is high, information at the R and S inputs will be transmitted directly to the outputs the latch is said to be enabled the output will change in response to input changes as long as the ENABLE is high. When the ENABLE input goes low, the output will retain the information that was present on the input when the high-to-low transition takes place.

In this fashion, it is possible to stroke or clock the flip-flop in order to store information (set it or reset it) at any time, and then hold the stored information for any desired period of time. This flip-flop is called a clocked RS flip-flop. The proper symbol and truth table are given in Fig.10. Notice that there are now three inputs-R, S, and the ENABLE or CLOCK input, labelled CLK. Notice also that the truth-table output is not simply Q, but Q_{n+1} . This is because we must consider two different instants in time: the time before the ENABLE goes low Q_n and the time just after ENABLE goes low Q_{n+1} .

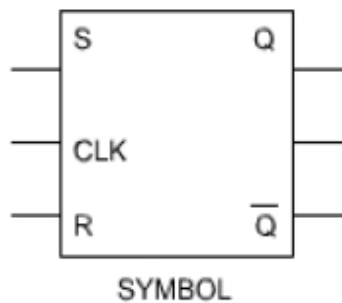


Fig.10

For the flip-flop to operate properly, there must be a transition from low to high on the CLK input. While CLK is high, the information on R and S causes the latch to set or reset. Then when CLK transition back to low, this information is retained in the latch. When this high-to-low transition occurred, both R and S inputs were low (0), and thus there was no change of state. In other words, the value of Q at time $n+1$ is the same as it was at time n . Remember that time n occurs just before the high-to-low transition on CLK and time $n+1$, just after the transition.

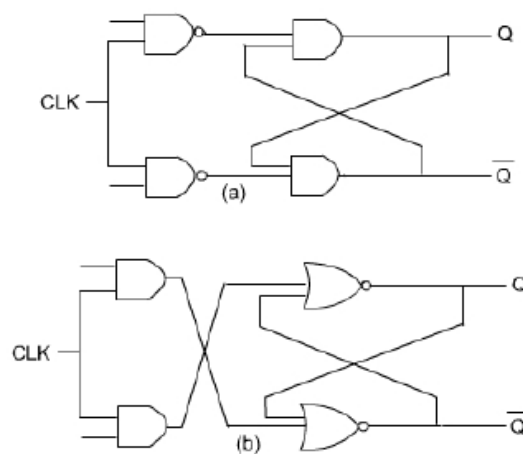


Fig.11 Two Different realizations for clocked RS Flip-flop

The logic diagrams shown in Fig. 11a and b illustrate two different methods for realizing a clocked RS flip-flop. Both realizations are widely used in medium and large-scale integrated circuits.

Figure.12 shows the input waveforms R,S,CLK applied to a clocked RS flip-flop and the output waveform Q.

Initially, the flip-flop is reset ($Q = 0$). At time t_1 CLK goes high; the flip-flop is now enabled and it is immediately set ($Q = 1$) since $R = 0$ and $S = 1$. At time t_2 CLK goes low, the flip-flop is still disabled and latches in the stable state $Q = 1$.

Between t_2 and t_3 both R and S change states, but since CLK is low, the flip-flop is still disabled and Q remains at 1.

Between t_3 and t_6 , the flip-flop will respond to any change in R and S since CLK is high. Thus at t_3 Q goes low, and at t_4 it goes back high. At t_6 the value $Q = 1$ is latched and no changes in Q occur between t_6 and t_7 even though both R and S change.

Between t_7 and t_8 no change in Q occurs since both R and S are low.

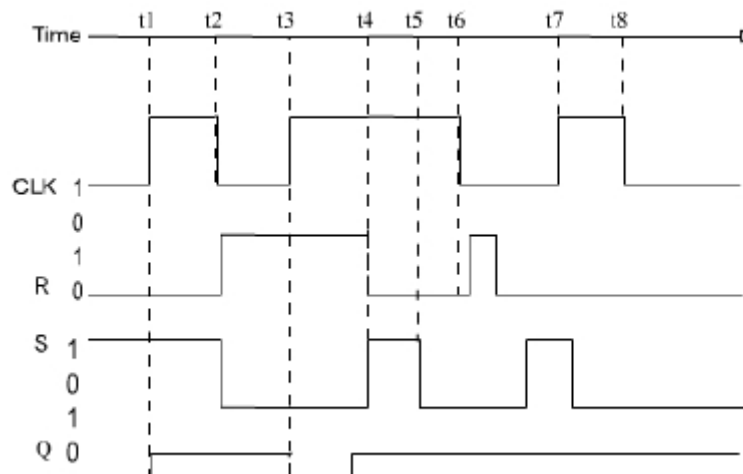


Fig.12

13.5 D Flip-Flop

The RS flip-flop has two data inputs, R and S. To store a high bit, you need a high S; to store a low bit, you need a high R. Generation of two signals to drive a flip-flop is a disadvantage

in many applications. Furthermore, the forbidden condition of both R and S high may occur inadvertently. This has led to the D flip-flop, a circuit that needs only a single data input.

Figure.13 shows a simple way to build a delay D flip-flop. This kind of flip-flop prevents the value of D from reaching the Q output until a clock pulse occurs. The action of the circuit is straightforward, as follows. When the clock is low, both AND gates are disabled; therefore, D can change value without affecting the value of Q. On the other hand, when the clock is high, both AND gates are enabled. In this case, Q is forced to equal the value of D. When the clock again goes low, Q retains or stores the last value of D.

There are many ways to design D flip-flop. In general, a D flip-flop is a bistable circuit whose D input is transferred to the output after a clock pulse is received. Figure.14 shows the logic symbol used for any type of D flip-flop.

In this D flip-flop in which Q can follow the value of D while the clock is high. In other words, if the data bit changes while the clock is high, the last value of D before the clock returns low is the value of D that is stored. This kind of D flip-flop is often called a D latch.

Figure.14b shows the truth table for a D latch. While the clock (CLK) is low, D is don't care; Q will remain latched in its last state. When the clock is high, Q takes on the last value of D. If D is changing while the clock is high, it is the last value of D that is stored.

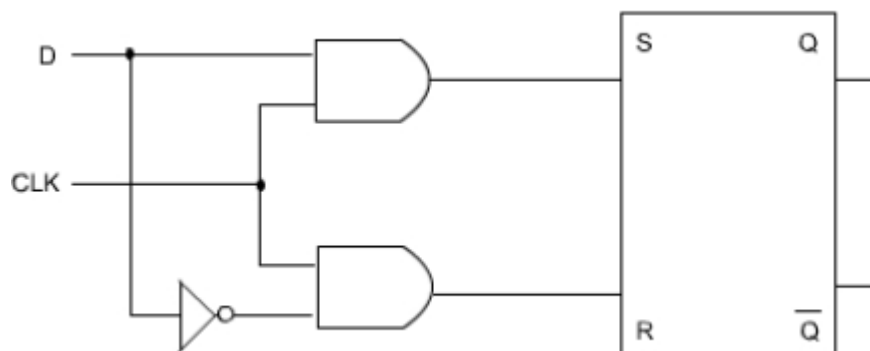


Fig.13 A D Flip-Flop

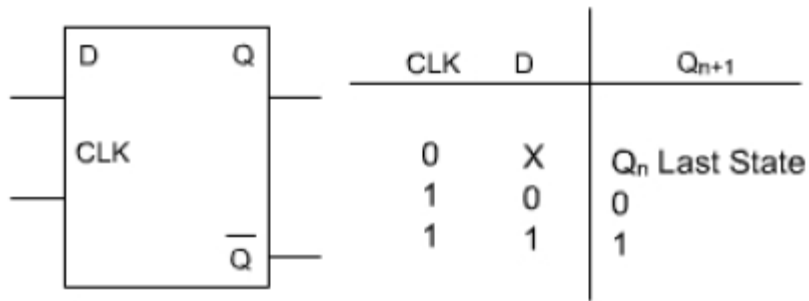


Fig.14 D Flip-Flop Logic Symbol

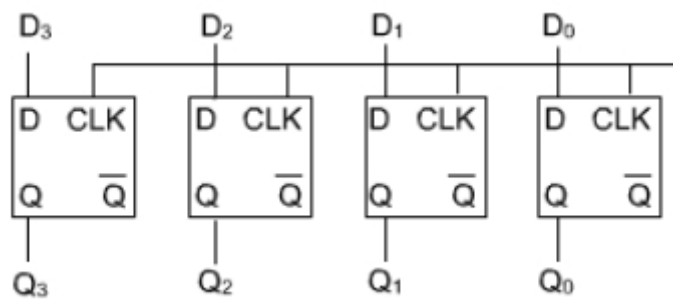


Fig.15 Storing a 4-Bit Word

The idea of data storage is illustrated in Fig.15. Four D latches are driven by the same clock signal. When the clock goes high, input data is loaded into the flip-flops and appears at the output. Then when the clock goes low, the output retains the data. For instance, suppose that the data input is

$$D_3 D_2 D_1 D_0 = 0111$$

When the clock goes high, this word is loaded into the D latches, resulting in an output of

$$Q_3 Q_2 Q_1 Q_0 = 0111$$

After the clock goes low, the output data is retained, or stored. As long as the clock is low, the D values can change without affecting the Q values.

13.6 Edge-Triggered D Flip-Flop

Although the D latch is used for temporary storage in electronic instruments, an even more popular kind of D flip-flop is used in digital computers and systems. This kind of flip-flop samples the data bit of at a unique point in time.

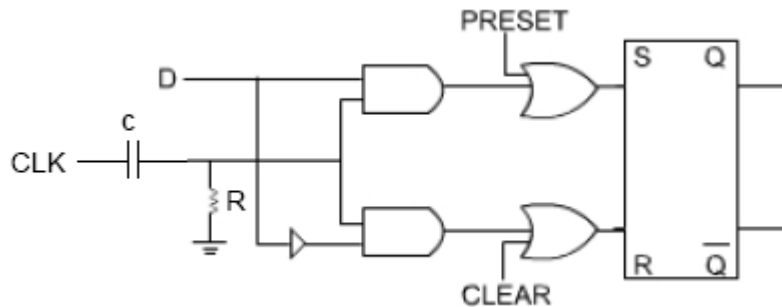


Fig. 16 RC Differential Clock Input

Figure 16 shows an RC circuit at the input of a D latch. By deliberate design, the RC time constant is much smaller than the clock's pulse width. Because of this, the capacitor can charge fully when the clock goes high; this exponential charging produces a narrow positive voltage spike across the resistor. Later, the trailing edge of the pulse results in narrow negative spike. (This is called RC differentiator)

CLK	D	Q_{n+1}
0	x	Q_n Last State
↑	0	0
↑	1	1
1	x	Q_n Last State

Fig. 17 Truth table

The narrow positive spike enables the AND gates for an instant; the narrow negative spike does nothing. The effect is to activate the AND gates during the positive spike, equivalent to sampling the value of D for an instant. At this unique point in time, D and its complement hit the flip-flop inputs, forcing Q to set or to reset (unless Q already equals D)

This kind of operation is called edge triggering because the flip-flop responds only when the clock is in transition between its two voltage states. The triggering in Fig.16 occurs on the positive-going edge of the clock; this is why it's referred to as positive-edge triggering.

The truth table in Fig.17 summarizes the action of a positive-edge-triggered D flip-flop. When the clock is low, D is a don't care and Q is latched in its last state. On the leading edge of the clock, designated by the up arrow, the data bit is loaded into the flip-flop and Q takes on the values of D. On the trailing edge of the clock (the down arrow), D is a don't care and Q remains in its last state.

When power is first applied, flip-flops come up in random states. To get some computers started, an operator has to push a RESET button. This sends a CLEAR or RESET signal to all flip-flop. Also, it's necessary in some digital systems to preset (synonymous with set) certain flip-flop.

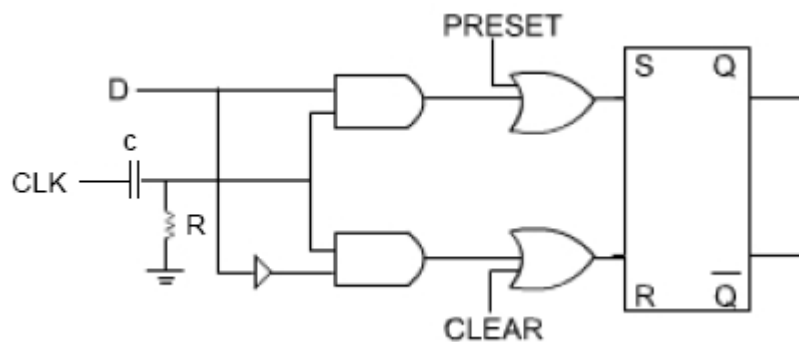
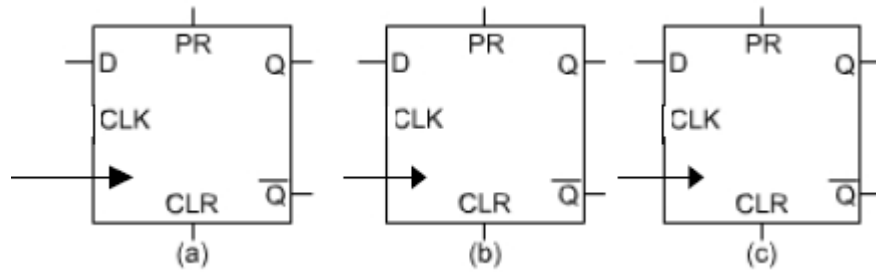


Fig.18 Preset and Clear Functions

Figure-18 shows how to include both functions in a D flip-flop. The edge triggering is the same as previously described. Furthermore, the OR gates allow us to slip in a high PRESET or a high CLEAR when desired. A high PRESET forces Q to equal 1; a high CLEAR resets Q to 0.

The PRESET and CLEAR are called asynchronous inputs because they activate the flip-flop independently of the clock. On the other hand, the D input is a synchronous input because it has an effect only when a clock edge occurs.

Figure-19a is the symbol for a positive-edge-triggered D flip-flop. The clock input has a small triangle to serve as a reminder of edge triggering. When you see this symbol, remember what it means; the D input is sampled and stored on the leading edge of the clock.



- (a) Positive edge triggered (b) Negative Edge triggered
(c) Positive edge triggered with active low preset and clear

Fig.19

Sometimes, triggering on the negative edge is better suited to the application. In this case, an internal inverter can complement the clock pulse before it reaches the AND gates. This means that the trailing edge of the clock activates the gates, allowing data to be stored. Figure-19b is the symbol for a negative-edge-triggered D flip-flop. The bubble and triangle symbolize the negative-edge triggering.

Figure-19a and b also include preset (PR) and clear (CLR) inputs. Figure-19c is another commercially available D flip-flop. Besides having positive-edge triggering, it has an inverted CLEAR input. This means that a low CLR resets it.

13.7 JK Flip-Flop

Flip-Flops can be used to build a counter, a circuit that counts the number of positive or negative clock edges driving its clock input. For purposes of counting, the JK flip-flop is the ideal element to use.

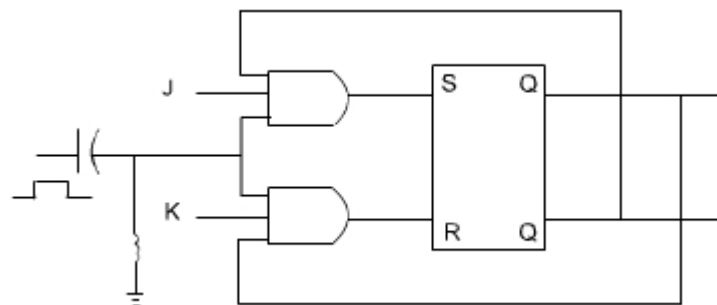


Fig.20 A JK flip-flop

One way to implement a JK flip-flop

CLK	J	K	Q_{n+1}
x	0	0	Q_n Last State
1	0	1	0
1	1	0	1
1	1	1	Q_n Last State

Fig 21 Truth Table

Figure-20 shows one way to build a JK flip-flop, the variables J and K are called control inputs because they determine what the flip-flop does when a positive clock edge arrives. As before, the RC circuit has a short time constant, thus converting the rectangular clock pulse into narrow spikes. Because of the AND gates, the circuit is positive-edge-triggered.

When J and K are both low, both AND gates are disabled. Therefore, clock pulse have no effect. This first possibility is the initial entry in the truth table. As shown, when J and K are both 0s, Q retains its last value.

When J is low and K is high, the upper gate is disabled, so there's no way to set the flip-flop. The only possibility is reset. When Q is high, the lower gate passes a RESET trigger as soon as the next positive clock edge arrives. This forces Q to become low (the second entry in the truth table). Therefore, J = 0 and K = 1 means that the next positive clock edge resets the flip-flop (unless Q is already reset).

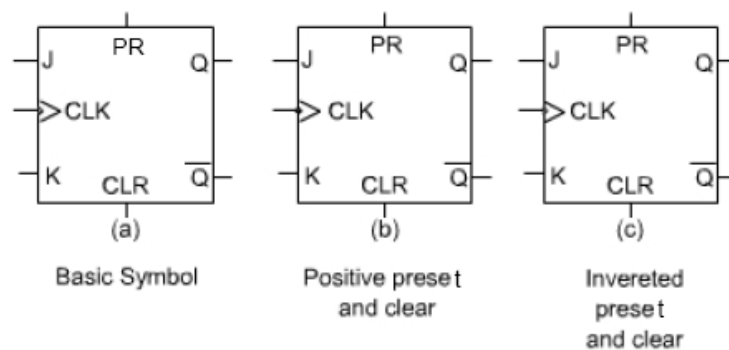


Fig.22

When J is high and K is low, the lower gate is disabled, so it's impossible to reset the flip-flop. But you can set the flip-flop as follows. When Q is low, Q is high; therefore, the upper gate passes a SET trigger on the next positive clock edge. This drives Q into the high state (the third entry in the truth table). As you can see, $J = 1$ and $K = 0$ means that the next positive clock edge sets the flip-flop (unless Q is already high).

When J and K are both high (notice that this is the forbidden state with an RS flip-flop), it's possible to set or reset the flip-flop. If Q is high, the lower gate passes a RESET trigger on the next positive clock edge. On the other hand, when Q is low, the upper gate passes a SET trigger on the next positive clock edge. Either way, Q changes to the complement of the last state (see the truth table). Therefore, $J = 1$ and $K = 1$ means the flip-flop will toggle on the next positive clock edge. "Toggle" means to switch to the opposite state.

Figure-22a shows a symbol for a JK flip-flop of any design. When you see this on a schematic diagram, remember that on the next positive clock edge, (1) low J and K have no effect, (2) low J and high K produce a reset, high J and low K produce a set, and high J and K result in a toggle.

You can include OR gates in the design to accommodate PRESET and CLEAR as was done earlier. Figure-22b gives the symbol for a JK flip-flop with PR and CLR. Notice that it is positive-edge-triggered and requires a high PR to set it or a high CLR to reset it.

Figure-22c is another commercially available JK flip-flop. It is negative-edge-triggered and requires a low PR to set it or a low CLR to reset it.

13.8 JK Master-Slave Flip-Flop

Figure-23 shows one way to build a JK master-slave flip-flop. It provides another way to avoid racing. Here's how it works. To begin with, the master is positive-edge-triggered and the slave, negative-edge-triggered. Therefore, the master responds to its J and K inputs before the slave. If $J = 1$ and $K = 0$, the master sets on the positive clock edge. The high Q output of the master drives the J input of the slave, so when the negative clock edge hits, the slave sets, copying the action of the master.

If $J = 0$ and $K = 1$, the master resets on the leading edge of the clock. The high Q output of the master goes to the kind K input of the slave. Therefore, the arrival of the clock's trailing edge forces the slave to reset. Again, the slave has copied the master.

If the master's J and K inputs are both high, it toggles on the positive clock edge and the slave then toggles on the negative clock edge. Regardless of what the master does, therefore, the slave copies it: if the master sets, the slave sets; if the master resets, the slave resets.

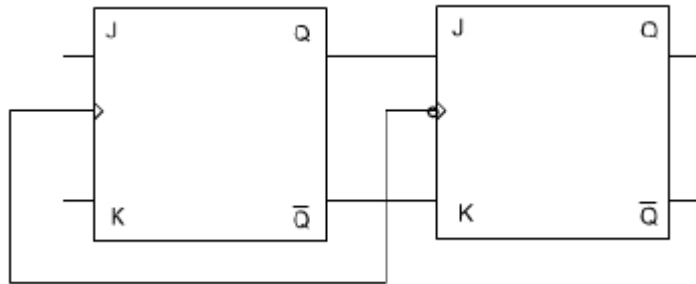


Fig-23 Master-slave flip-flop

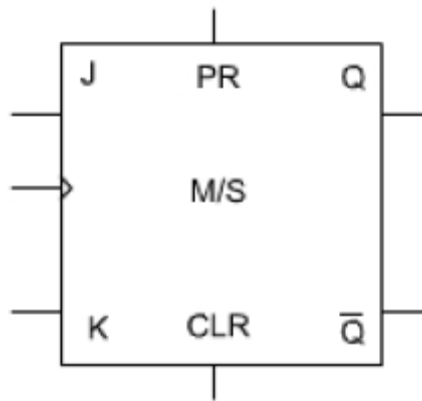


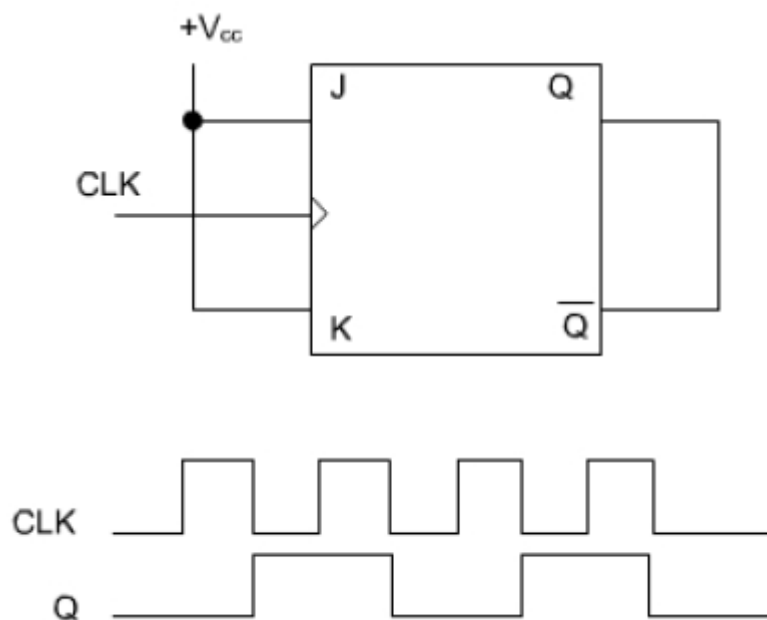
Fig. 24 - JK master-slave flip-flop Symbol

Other designs are possible for a JK master-slave flip-flop. The basic idea remains the same: the slave copies the master on the negative clock edge. Figure-24 show the symbol we will use for a JK master-slave flip-flop; it includes PRESET and CLEAR functions. The bubble on the clock input is a reminder that the output changes state on the negative clock edge. Its truth table is given in Fig. 25.

Inputs at t_n		Output at t_{n+1}
J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	$\overline{Q_n}$

Fig-25 Truth Table

The JK master-slave flip-flop in Fig-26 has its J and K input tied to +Vcc, and a series of pulses (actually a square wave) are applied to its CLK input. The waveform at Q is explained as follows:

**Fig. 26 JK Master-Slave Flip-Flop and Waveform.**

Since $J = K = 1$, the flip-flop simply toggles each time the clock goes low. The waveform at Q has a period twice that of the CLK waveform. In other words, the frequency of Q is only one-half that of CLK. This circuit acts as a frequency divider-the output frequency is equal to the input frequency divided by 2.

13.9 Summary

We can implement flip-flops in two methods.

In first method, **cascade two latches** in such a way that the first latch is enabled for every positive clock pulse and second latch is enabled for every negative clock pulse. So that the combination of these two latches become a flip-flop.

In second method, we can directly implement the flip-flop, which is edge sensitive.

SR flip-flop operates with only positive clock transitions or negative clock transitions.

D flip-flop operates with only positive clock transitions or negative clock transitions.

JK flip-flop is the modified version of SR flip-flop. It operates with only positive clock transitions or negative clock transitions.

13.10 Model Questions

- (1) Draw the circuit for R.S. flip-flop using NAND/NOR gates and give the truth table.
- (2) Draw the circuit for D flip-flop and explain its action.
- (3) Describe the working of a J.K. master slave flip-flop.

LESSON - 14

REGISTERS

Structure

- 14.0 Objective**
- 14.1 Introduction**
- 14.2 4 Bit Shift Registers**
- 14.3 Bi-directional Shift Registers**
- 14.4 Summary**
- 14.5 Model Question**

14.0 Objectives

- To introduce shift register
- Shift right shift register – serial and parallel data transfer
- Shift left shift register
- Shift left shift right shift register

14.1 Introduction

A flip flop can store a single bit and is sometimes called as a one bit register. If we require n bits to be stored we require n flip-flops. An array of flip flops connected in such a way to store n bits is called a register. If the bits can be shifted right or left then the register is called a Shift Register.

14.2 4 bit shift register (shift right)

A 4 bit shift register constructed with D flip-flops is shown in the figure. Inter connection between flip-flops are such that (except FF_0) the logic level at a data input is determined by the state of the preceding flip flop. For each flip-flop D is the data input and Q the output. Input is transferred to output on application of a clock pulse.

We can discuss the operation of the shift register with reference to the circuit (Fig 14.1).

Let all the outputs Q_0, Q_1, Q_2, Q_3 be initially equal to 0000. Let us send a binary combination 1011 through serial input D_R of the first flip-flop. On applying clock pulse synchronously to each flip-flop, D_R shifts to Q_0 , Q_0 shifts to Q_1 , Q_1 shifts to Q_2 , and Q_2 shifts to Q_3 . Now the data available at the output of the register is 1000. Now the new input at $D_R = 1$. On applying the next pulse each bit shifts to the right and the output becomes 1100. For 4 clock pulses the sequence is as shown in fig.

CK	D_R	Q_0	Q_1	Q_2	Q_3
0	1	0	0	0	0
1	1	1	0	0	0
2	0	1	1	0	0
3	1	0	1	1	0
4		1	0	1	1

Since each bit is shifted right this arrangement is called shift right shift register.

When the input data is sent in through D_R and collected as a group of four bits on Q_0, Q_1, Q_2, Q_3 output pins, this arrangement becomes a Serial in Parallel Out shift register. If the output is along a single line Q_3 with the inputs from D_R then this arrangement is a serial in-serial out shift register.

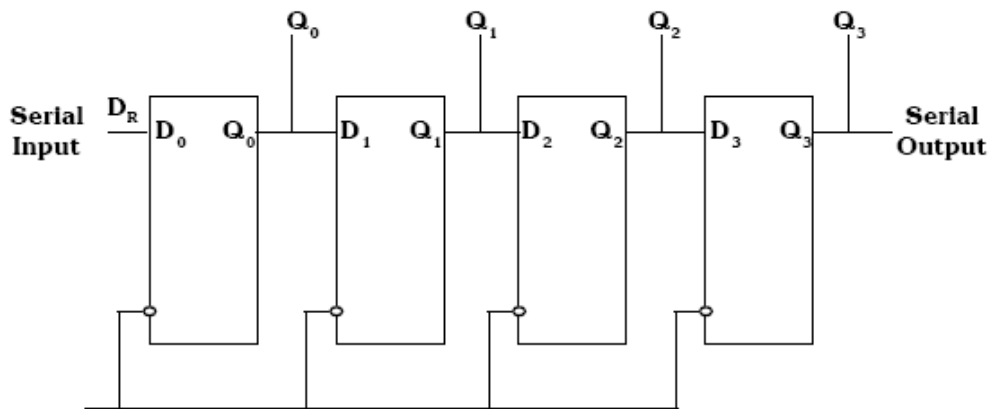


Fig. 1 Shift Register (Shift Right)

If the bits are shifted from right to left we get a shift left shift register (Fig 2). The data input D_L is given to D_3 and on application of a clock pulse D_L comes out in Q_3 . Q_3 is connected as an input to D_2 and on application of a clock pulse Q_3 shifts left and is available on Q_2 . By a similar arrangement Q_2 shifts to Q_1 and Q_1 shifts to Q_0 . Since the bits are shifted left from D_L to Q_3 to Q_2 to Q_1 to Q_0 . This arrangement is called as a shift left shift register. (Fig. 2)

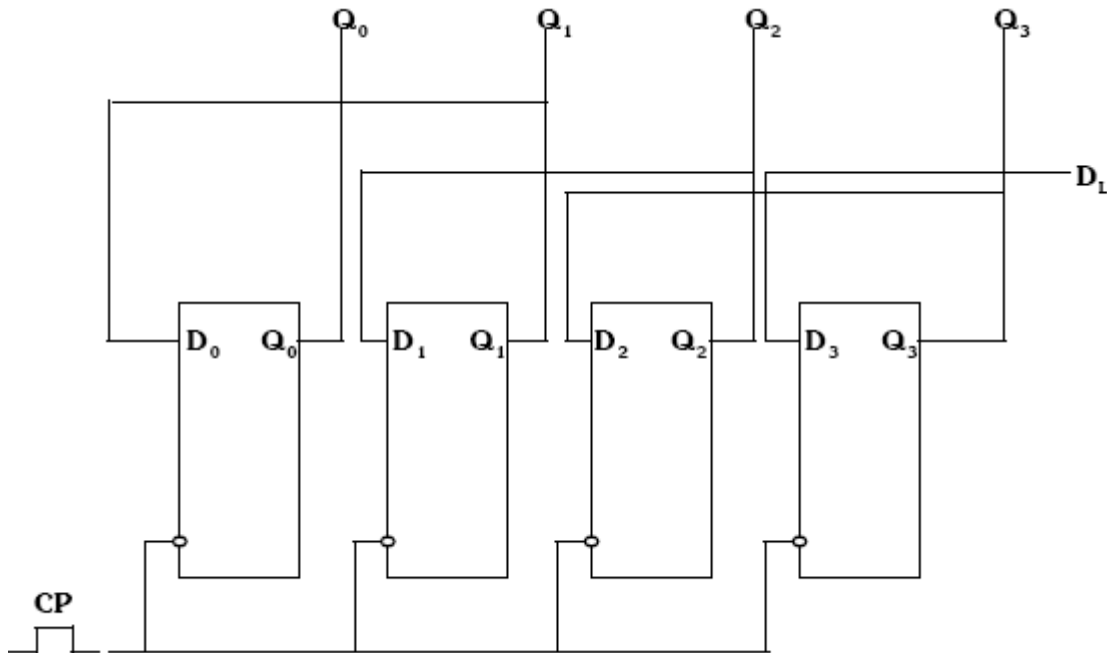


Fig. 2 Shift Register (Shift Left)

14.3 Bi-directional shift Register

Combining a shift right shift register circuit and a shift left shift register a bi-directional shift register can be constructed as shown. This arrangement consists of four D flip-flops and a AND-OR logic gate arrangement between the flip-flops. A mode control M is kept at a logic 1 ($M=0$) to shift the bits from D_R to D_0 , Q_0 to D_1 , Q_1 to D_2 and Q_2 to D_3 i.e., shift right operation takes place. Similarly when mode control $M=0$ ($M=1$), D_L is given to D_3 , Q_3 to D_2 , Q_2 to D_1 and Q_1 to D_0 i.e., shift left operation takes place. (Fig. 3)

The D flip-flops can be combined with multiplexers so that shift left, shift right, clearing the output to zero and parallel load operations and performed.

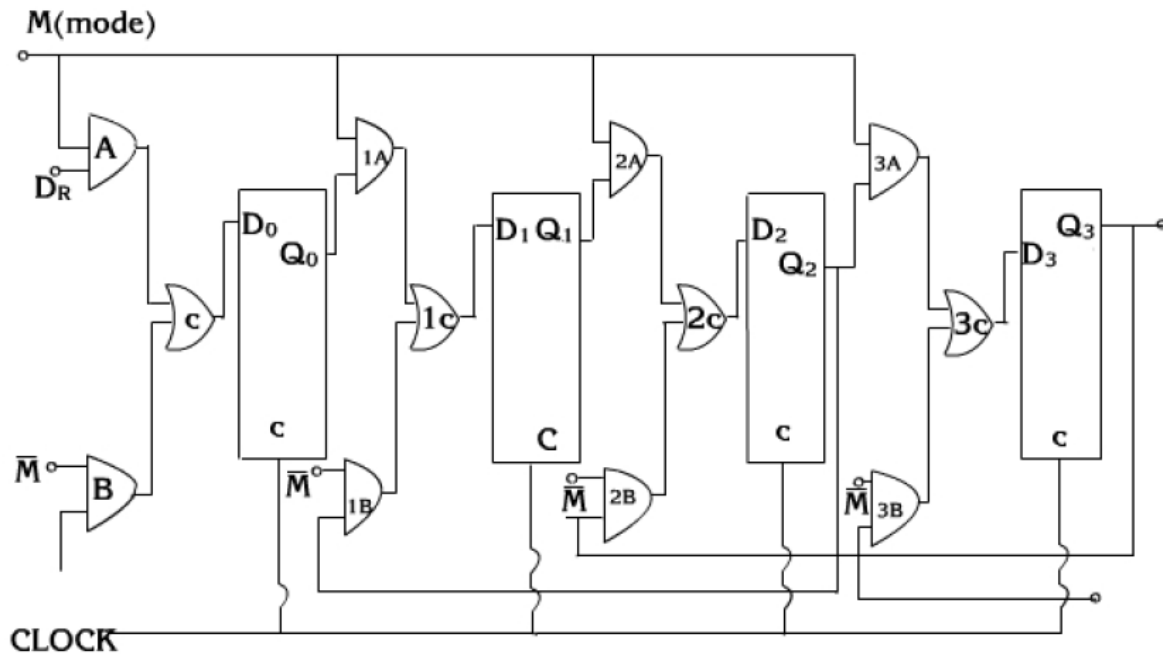


Fig. 3

14.4 Summary

Flip-flop is a 1 bit memory cell which can be used for storing the digital data. To increase the storage capacity in terms of number of bits, we have to use a group of flip-flop. Such a group of flip-flop is known as a **Register**.

If the register is capable of shifting bits either towards right hand side or towards left hand side is known as **shift register**.

14.5 Model Questions

- (1) Describe the working of a 4-bit shift register.
- (2) With circuit explain the working of a bi-directional shift register.

LESSON - 15

COUNTERS

Structure

- 15.0 Objectives
- 15.1 Introduction
- 15.2 Asynchronous or Ripple Counter
- 15.3 Synchronous (or Parallel) Counters
- 15.4 Mod-3, Mod-6 and Mod-12 Counters
- 15.5 Mod-6 Counters
- 15.6 Mod-12 Counters
- 15.7 Mod-5 and Mod-10 Counters
- 15.8 Mod-10 Counter
- 15.9 Presettable Counter
- 15.10 Up Counter and Down Counter
- 15.11 Controlled Counter
- 15.12 Ring Counter
- 15.13 Shift Counter
- 15.14 Summary
- 15.15 Model Questions

15.0 OBJECTIVES

To study :

- Asynchronous ripple counter
- Synchronous counter
- Mod 3 counter
- Mod 6 counter
- Mod 10 counter
- Up and down counter

15.1 Introduction

Digital counter consist of a number of flip-flops. Their function is to count electrical pulses. To count certain event, electrical pulses proportional to the event are produced for the counting purpose. Counters can also be used to count time interval or frequency. For such purposes clock pulses are applied to the counter. As the clock pulses occur at certain known intervals, the number of pulses are proportional to time. Therefore, time interval can easily be measured. The frequency is inversely proportional to time. So frequency can also be measured.

There are basically two types of counters: synchronous and asynchronous. In a synchronous counter all flip-flops are clocked simultaneously. On the other hand in asynchronous counters the flip-flops are not clocked simultaneously. Each flip-flop is triggered by the previous flip-flop.

15.2 Asynchronous or Ripple Counters

A binary asynchronous counter can be realized by the use of edge-triggered J-K flip-flops as shown in Fig 1(a). The flip-flops are connected in series. The Q output of one flip-flop is connected to the clock terminal of the next flip-flop. So it is also called serial counter. It is inherently slow in operation due to series connection. The triggers move through the flip-flops like a ripple in water. So it is called ripple counter.

Flip-flops are connected to operate as T flip-flops. When J and K inputs of all flip-flops are made high (i.e. when count enable is high); each flip-flop will change its state (toggle) on the receipt of the negative edge of its input pulse. Suppose that the output of T flip-flop is 0. When clock is applied its output will change from 0 to 1 on the receipt of one clock pulse. When clock goes low, the output will remain high. On the receipt of the second clock pulse, its output will change from 1 to 0. Thus a flip-flop gives an output of one pulse on the receipt of two pulses at its input. Fig. 1(a) shows a binary ripple counter consisting of 3 flip-flop. If at the input terminal of the first flip-flop 8 clock pulses are applied, first flip-flop will give 4 pulses at its output terminal. Now Q acts as the clock input for the 3rd flip-flop. At the input terminal of the 3rd flip-flop 2 pulses are applied. The third flip-flop will give one pulse at its output terminal.

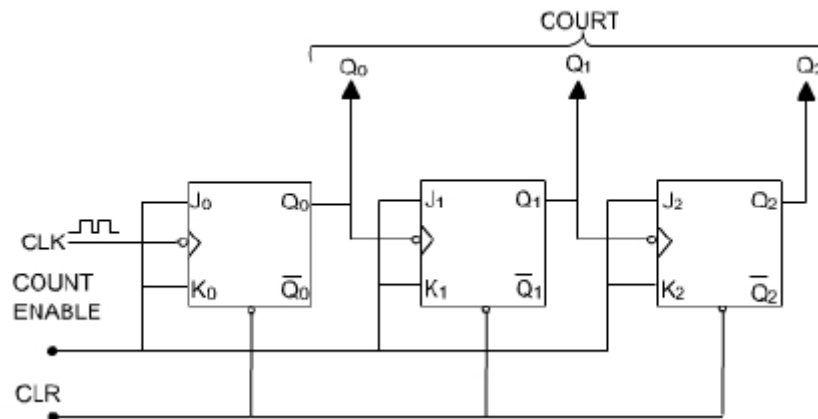


Fig.1 (a) Binary asynchronous (Ripple) counter

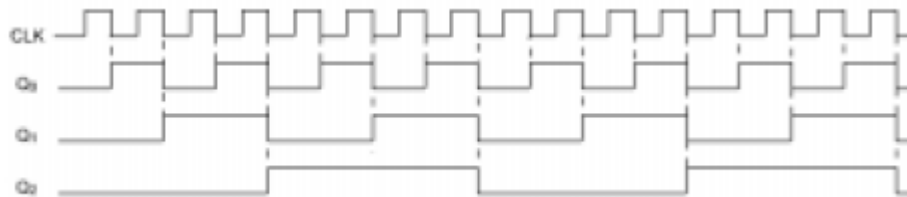


Fig.1 (b) Timing diagram

Table-1 shows how the counter contents advance on the receipt of clock pulses. It is clear from the table that counter content advances one count on the receipt of each clock pulse.

On the receipt of 7 clock pulses all the three flip-flops show high outputs. On the receipt of 8th clock pulse the 3rd flip-flop gives an output of one pulse and all flip-flops show 0 states. A three flip-flop counter counts from 0 to 7., It has 8 states. It is called modulus-8 (or mod-8) counter. Similarly, a four flip-flop counter counts from 0 to 15. So it is a mod-16 counter. A six flip-flop counter is a mod-64 counter. Thus the number of output states of a counter is called modulus of a counter.

In a ripple counter a decoding gate is used, which detects a given state of the counter and produces an output when the state is detected.. Is

Clock Pulses	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

The ripple counters are very simple to build but they are not suitable for high frequency applications. They are used as frequency dividers. Their operation at lower frequency is satisfactory. Each flip-flop has a delay time. In a ripple counter the delay time of all the flip-flops are added resulting in greater settling time. Another problem with ripple counters is that glitches (unwanted pulses) occur at the output of the gates. Both of these problems are eliminated in synchronous counters discussed below.

15.3 Synchronous (or Parallel) Counters

In synchronous counters all the flip-flops are directly clocked at the same time so that all the flip-flop change their output state at the same time. Fig.2 shows the schematic diagram of synchronous counter. Examples of synchronous counters are :

74161 – 4- bit binary UP counters.

74162 – Decade Up counters.

74192 – Decade UP/DOWN Counters.

74193 – 4-bit UP/DOWN counters

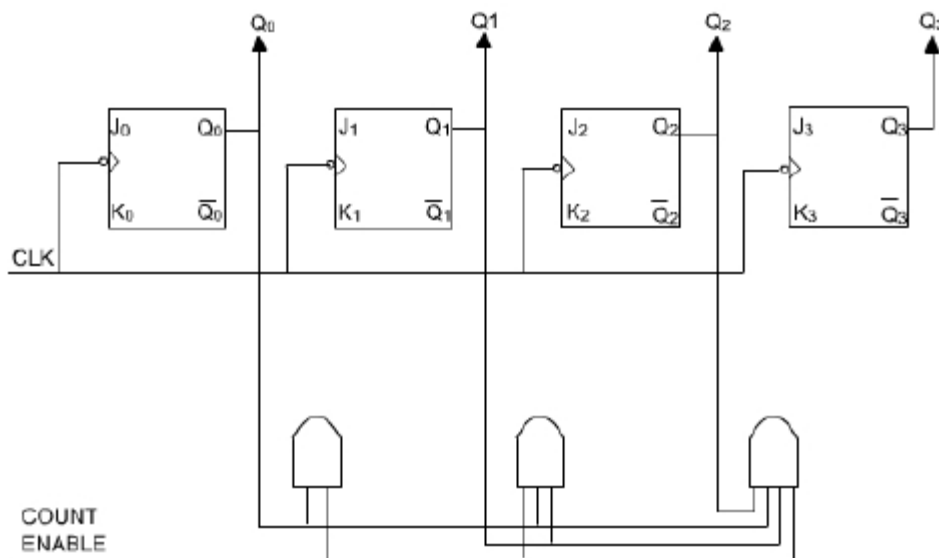


Fig. 2 Synchronous Counter

15.4 Mod-3, Mod-6 and Mod-12 Counters

The synchronous and asynchronous counter discussed earlier have a modulus of 2^n , where n is the number of flip-flops in the counter. The count 2^n is called natural count.

The counters consisting of 1,2,3 and 4 flip-flops have their natural count 2,3,8 and 16 respectively. These counters progress one count at a time in binary progression. Sometimes it is desirable to have the modulus of 3, 5 etc., other than 2^n . Such modulus are called modified count. A counter of smaller modulus can be realized from a counter of larger modulus by skipping certain states. Fig.3(a) shows a counter of modulus 3. Its truth table is shown in table-2. The timing diagram is shown in Fig.3(b)

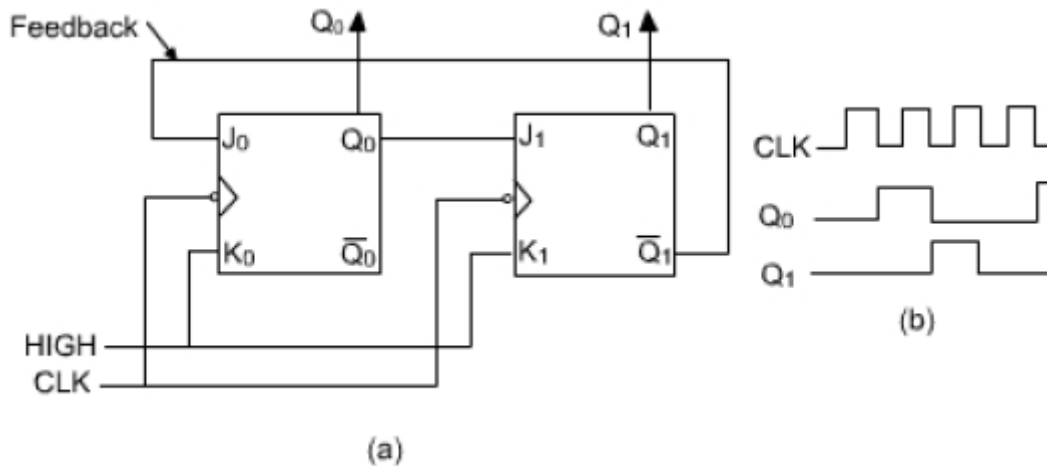


Fig.3 Mod-3 counters (a) schematic diagram (b) Timing diagram

The counter progresses one count at a time upto 2. After two clock pulses Q_0 is low and Q_1 is high. Q_1 is fed back to J_0 . When Q_1 is high $Q_1 = 0$. So $J_0 = 0$ and $K_0 = 1$. On the arrival of the 3rd clock pulse the first flip-flop will remain in the reset state. So Q_0 will remain 0. At this moment $J_1 = 0$ and $K = 1$. Hence the second flip-flop will also reset. Thus Q_1 will also become 0. Thus due to feedback of Q_1 to J_0 , the counter resets to 00 after receiving 3 clock pulses.

Table - 2. Truth Table of Mod-3 Counter

Q_1	Q_0	Count
0	0	0
0	1	1
1	0	2
0	0	2

15.5 Mod-6 Counters. LS2

To realize mod-6 counter a flip-flop is added to Mod-3 counter as shown in Fig.4

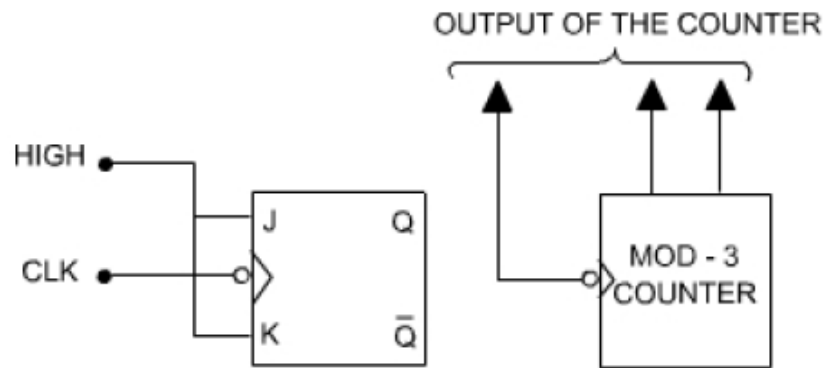


Fig.4 Mod-6 counter

15.6 Mod-12 Counters

Similarly, using 2 flip-flops and one mod-3 counter ($2 \times 3 \times 2$) we can realize a mod-12 counter.

The 7492A is an asynchronous divide by 12 counter (divide by 2 and divided by 6). It consists of 4 master-slave J-K flip-flops 74LS924 is also a divide by 12 (2×6) counter.

15.7 Mod-5 and Mod-10 (Decade or Bcd) Counters

Fig.5 shows a mod-5 counter. Q_2 is fed back to J_0 , Q_0 , Q_1 are applied to J_2 through an AND gate. Table-3 shows the truth table of a mod-5 counter.

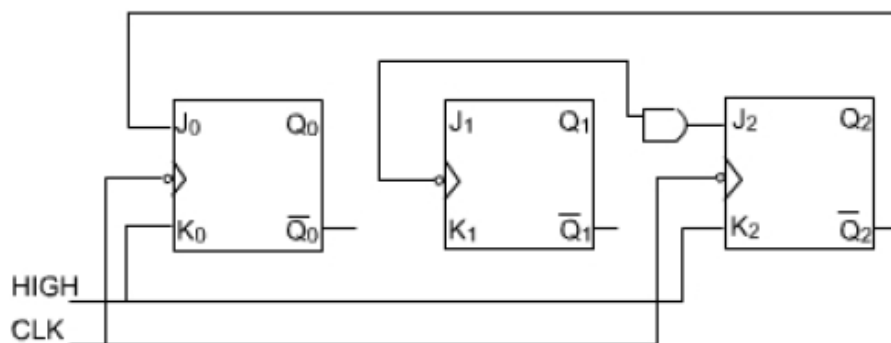


Fig.5 Mod-5 counter

The counter progresses on count at a time upto 4. At this moment, $Q_2 = 1$, $Q_1 = 0$. As Q_2 is fed back to J_0 , $J_0 = 0$. K_0 is always 1. So the first flip-flop will remain in reset state. $Q_0 = 0$ is also applied to J_2 through the AND gate resulting in $J_2 = 0$. Therefore, on receipt of 5th clock the third flip-flop will now reset changing Q_2 from 1 to 0. Q_1 was already 0. Thus the counter's output is 000. Thus the counter resets at 000 after receiving 5 clock pulses..Is

Table-3 Truth Table of a Mod-5 Counter. Is 2

Q_2	Q_1	Q_0	Count
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
0	0	0	5

15.8 Mod-10 (Decade of BCD) Counters

A decade counter can be realized using a flip-flop and a mod-5 counter ($2 \times 5 = 10$). Fig.6 shows a decade counter. Table-4 shows its truth table.

The ICS, 7490, 74L90, 74LS90, 74290, 74LS290, 74390, 74490 are asynchronous decade counters. They are 2 x 5 counters. 74190, 74LS190, 74L192 and 74LS192 are BCD synchronous counters..Is

Table-4 Truth Table of a Decade Counter

Q_3	Q_2	Q_2	Q_1	Count
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2

15.10 Up Counters, Down Counters and Up/Down Counters

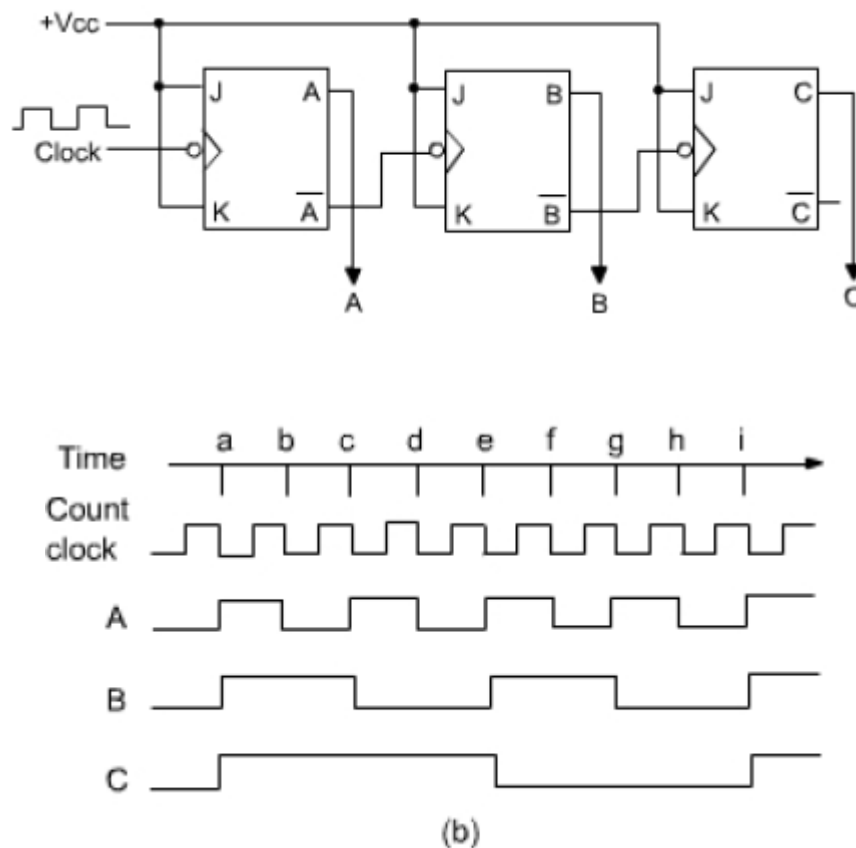
15.10.1 Up Counters

If a counter counts upward towards higher number, it is called up counter. The asynchronous counter shown in figure 1(a) is the example for up counters. The examples of up counters are:

74160, 74162 Decade Up counters

15.10.2 Down Counters

The counters which count downward towards lower numbers are down counters. For example, the binary asynchronous ripple counter in figure-1 (a) can be modified and realized as shown in figure.

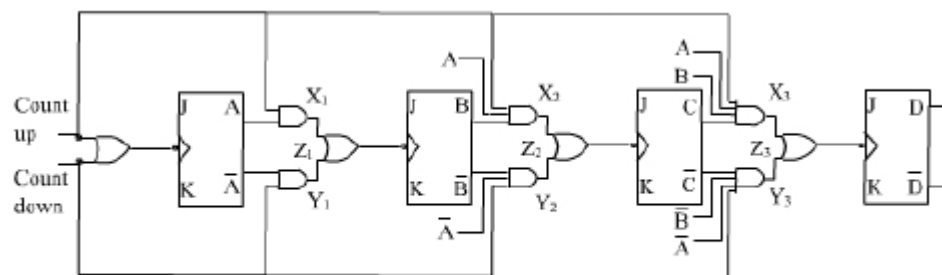


State	C	B	A
7	1	1	1
6	1	1	0
5	1	0	1
4	1	0	0
3	0	1	1
2	0	1	0
1	0	0	1
7	1	1	1

Figure 7: Binary ripple down counter.

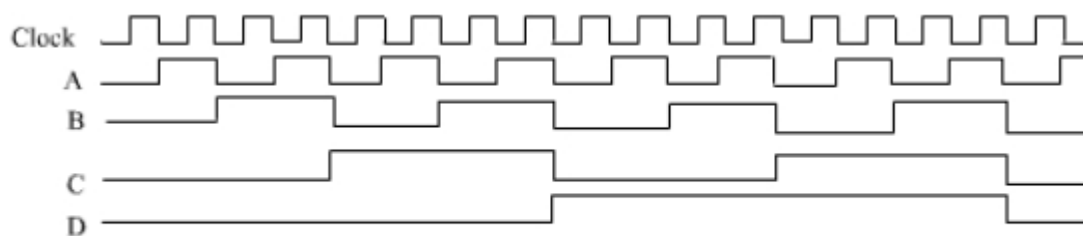
15.10.3 Up/Down Counters

A counter which is designed to count either towards higher numbers or towards lower numbers is called an up/down counter. An U/D terminal is provided to control up/down counting. If U/D is high the counter becomes an up counter. If it is low the counter becomes a down counter. The examples of UP/DOWN counters are:



Note: All J and K inputs are used to + Vcc

(a) Logic diagram



Count up waveform

(b)

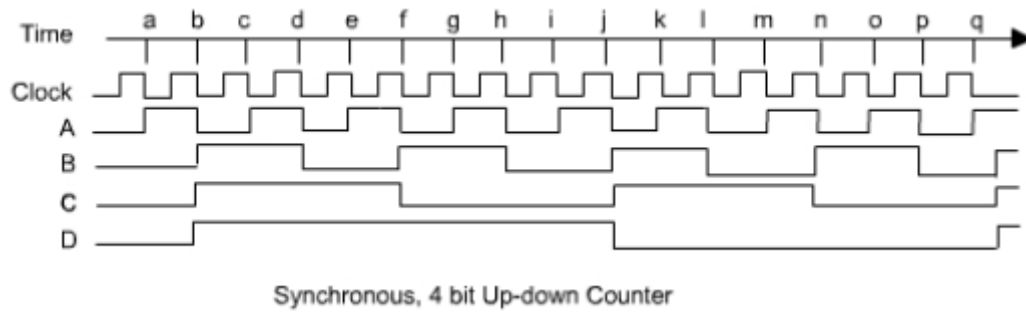


Figure 8: Synchronous, 4-bit Up-down Counter

74168, 74190, 74192 Decade UP/DOWN synchronous counters.

74169, 174191, 74193, 4-bit binary UP/DOWN synchronous counters. The following figure shows, the binary up/down synchronous counter.

15.11 Controlled Counters

A Controlled counter counts electrical pulses when it is asked to do so. There is a control terminal COUNT to control counting. When it is high, the counter counts pulses applied to it. When COUNT is low it does not do counting even though pulse may remain applied to it.

15.12 Ring Counters

Instead of counting numbers a ring counter uses a binary word which has only a single high-bit. This word is shifted within the counter. Fig.9 shows a 4-bit ring counter. In a ring counter flip-flops are connected in series. The output Q of the one stage is connected to D input

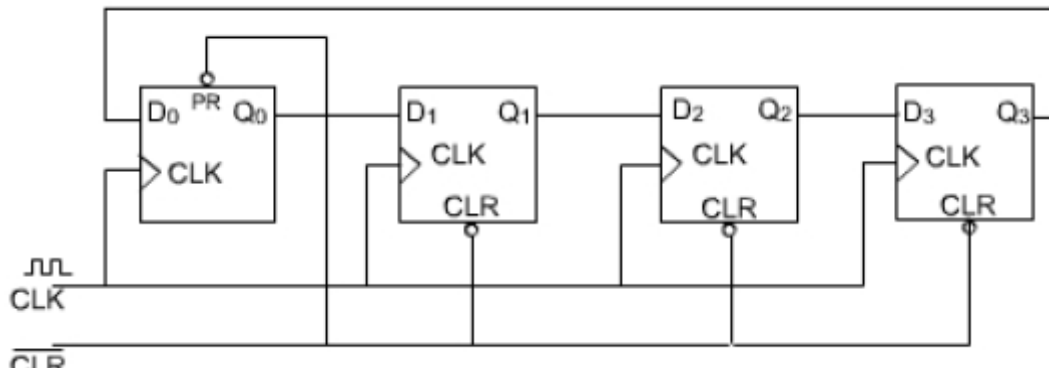


Fig. 9 Ring counter

A ring counter is employed to control a number of operations in a sequence. The devices may be connected to $Q_0, Q_1, Q_2, \dots, Q_n$ outputs of the counter. The outputs $Q_0, Q_1, Q_2, \dots, Q_n$ go high in a sequence.

For initial setting a binary word 0000 is set in a 4-bit ring counter. First CLR goes low, Q_0 is made high. The CLR becomes high. The device connected to Q_0 is activated. Now clock pulses applied to CLK terminal activate devices. On the receipt of the first clock pulse Q_1 becomes high, and the device connected to Q_1 is activated. The second clock pulse makes Q_2 high, the 3rd clock pulse makes Q_3 high and so on. At a time only one output becomes high and all others remain low. As the outputs $Q_0, Q_1, Q_2,$

..... Q_n are made high in a sequence, the devices connected to these outputs are activated in a sequence. Ring counters are used in computers to control certain operations in a sequence, their applications are to control stepper motors, encoding the outputs of electronic keyboards etc.

15.13 Shift Counter (Johnson or Moebius Counter)

It is similar to a ring counter. In a ring counter the Q output of the last stage is fed back to the D input of the 1st stage whereas in a Johnson counter the complemented output Q of the stage is fed back to D input of the 1st stage. If J-K flip-flops are used in the counter Q is fed back to J input and $\bar{Q}$ is fed back to K input of the first stage. It is also called a walking-ring, a twisted-ring or switched tail counter.

Table-5. Truth Table for 3-bit Johnson Counter

Q_2	Q_1	Q_0	Count	State
0	0	1	1	1
0	1	1	3	2
1	1	1	7	3
1	1	0	6	4
1	0	0	4	5
0	0	0	0	6
0	0	1	1	

Table-5 and 6 show the truth table for 3-bit and Mod-10 shift counters respectively. Such counters do not count in binary or decimal progression. Their counting sequence is shown in the truth tables. In counting sequence one flip-flops gives high output, then two flip-flops give high outputs, then three and so on..ls

Table-6. Truth Table for a Mod-10 Johnson Counter

Q_4	Q_3	Q_2	Q_1	Q_0	Count	State
0	0	0	0	1	1	1
0	0	0	1	1	3	2
0	0	1	1	1	7	3
0	1	1	1	1	15	4
1	1	1	1	1	31	5
1	1	1	1	0	30	6
1	1	1	0	0	28	7
1	1	0	0	0	24	8
1	0	0	0	0	16	9
0	0	0	0	0	0	10
0	0	0	0	1	1	

At one stage all flip-flops give high outputs. After this one flip-flop give 0 output, then two flip-flops give 0 output, then three and so on. Finally, all flip-flops give 0 output.

Mod-10 shift counters employ five flip-flops. For 5 flip-flop counters there should be 32 states. But there are only 10 states. Other are invalid states.

15.14 Summary

Counter is a sequential circuit. A digital circuit which is used for a counting pulses is known counter. Counter is the widest application of flip-flops. It is a group of flip-flops with a clock signal applied.

If the flip-flops do not receive the same clock signal, then that counter is called as **Asynchronous counter**.

If all the flip-flops receive the same clock signal, then that counter is called as **Synchronous counter**

The 2-bit ripple counter is called as MOD-4 counter and 3-bit ripple counter is called as MOD-8 counter. So in general, an n-bit ripple counter is called as modulo-N counter.

15.15 Model Questions

- (1) Draw the circuit of 4 bit ripple counter and draw the output waveform.
- (2) Describe the working of mod-10 counter.

LESSON - 16

DESIGN OF SEQUENTIAL CIRCUITS WITH STATE EQUATIONS

Structure

- 16.0 Objectives**
- 16.1 Introduction**
- 16.2 Excitation Table**
- 16.3 Excitation Table for JK Flip-Flop**
- 16.4 State Table**
- 16.5 State Diagram**
- 16.6 Design of 2 bit Binary Counter**
- 16.7 Summary**
- 16.8 Model Questions**

16.0 Objectives

To learn

- Excitation table of flip-flops
- State equation and state table
- Design with state equation

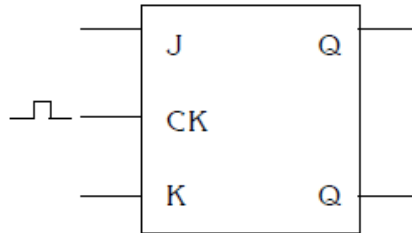
In previous lesson, we have learnt the fundamentals of S-R, D, J-K and T flip-flops. To design sequential circuits using these flip-flops we have to go little deeper. We need excitation tables and states tables. We will go through them one by one.

16.1 Introduction

In previous lesson, we have learnt the fundamentals of S-R, D, J-K and T flip-flops. To design sequential circuits using these flip-flops we have to go little deeper. We need excitation tables and states tables. We will go through them one by one.

16.2 Excitation Table

Let us take the case of J-K flip-flop.



The truth table/characteristic table is shown.

J	K	$Q_{(t+1)}$	
0	0	$Q_{(t)}$	No change
0	1	0	Clear to 0
1	0	1	Set to 1
1	1	$Q'_{(t)}$	Toggle (complement)

where $Q_{(t)}$ is the binary state at a given time (present state). $Q_{(t+1)}$ is the binary state after application of a clock pulse (next state).

In design of sequential circuits we usually know the required transitions from the present state to the next state and wish to find the flip-flop input conditions that will cause the required transition. For this reason we need a table that lists the required input combinations for a given change of state. Such a table is called as flip-flop excitation table.

16.3 Excitation Table for J-K Flip-Flop

For J-K flip-flop, if $Q_{(t)}$ is 0 and $Q_{(t+1)}$ is also 0, it means the output is 0 even after the application of clock pulse. This can happen when $J = 0$ and $K = 0$ (No change condition) or $J = 0$ and $K = 1$ (Reset condition). Therefore if $Q_{(t)} = 0$ and $Q_{(t+1)} = 0$, then $J = 0$ and $K = 0$ or 1 i.e., $K = X$.

Similarly when $Q_{(t)}=0$ and $Q_{(t+1)}=1$, it means that the output changes from 0 to 1 on application of a clock pulse. This can mean a set condition or toggle condition. For set condition, $J=1$ and $K=0$. For toggle, $J=1$ and $K=1$. Therefore for $Q_{(t)}=0$ and $Q_{(t+1)}=1$, $J=1$ and $K=X$.

Similarly, for $Q_{(t)}=1$ and $Q_{(t+1)}=0$, $J=X$ and $K=1$ and

for $Q_{(t)}=1$ and $Q_{(t+1)}=1$, $J=X$ and $K=0$.

Therefore the excitation table for J-K flip-flop is

$Q_{(t)}$	$Q_{(t+1)}$	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

The block diagram, the characteristic table and the excitation table for D flip-flop are given below.

Characteristic table for D flip-flop:

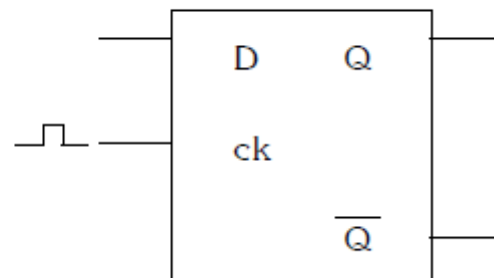
D $Q_{(t+1)}$

0 0

Clear to 0

1 1

Set to 1

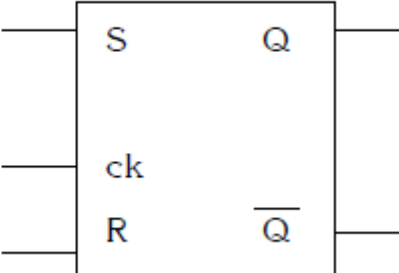


Excitation table for D flip-flop

$Q(t)$	$Q(t+1)$	D
0	0	0
0	1	1
1	0	0
1	1	1

The block diagram, the characteristic table and the excitation table for S-R flip-flop are given below:

Characteristic table for S-R flip-flop:

S	R	$Q(t)$		
0	0	$Q(t)$	No change	
0	1	0	Clear to 0	
1	0	1	Set to 1	
1	1	?	Not allowed	

Excitation table for S-R flip-flop:

$Q(t)$	$Q(t+1)$	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

16.4 State table

The behaviour of a sequential circuit is determined from the inputs, the outputs and the state of the flip-flop. Both the outputs and next state are a function of the inputs and the present

state. A sequential circuit is specified by a state table that relates outputs and next states as a function of inputs and present state. In clocked sequential circuits, the transition from present state to next state is activated by the presence of a clock signal.

Let us consider two D flip flop named A and B. Let the input to the flip-flop be D_A and D_B . Let the output of A flip flop be A and A' and that of B flip flop be B and B'. Using two D flip flops, and input variable x and an output variable y. Let us construct a synchronous sequential circuit obeying the following equation. (Fig. 1)

$$D_A = Ax + Bx$$

$$D_B = A'x$$

$$y = Ax' + Bx'$$

The connections from the two outputs of the flip-flops to the input of gates is not shown to improve clarity.

Table 16.1

Present State		Input	Next state		Output
A	B	x	A	B	y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

The table consists of four sections labelled present state, input, next state and output. The present state section shows the state of flip-flops A and B at any given time t . The input section gives a value of x for each possible present state. The next state section shows the state of the flip-flop one clock period late at a time $t+1$. The output section gives the value of y for each present state and input condition.

The derivation of state table consists of first listing all possible binary combination of present state and inputs. In this case we have eight binary combinations from 000 to 111. The next state values are then determined from the logic diagram or from input equations.

the equation (as already mentioned) as

$$D_A = Ax + Bx$$

$$D_B = A'x$$

$$y = Ax' + Bx'$$

Using the above equations binary values of the next state and outputs are obtained.

Using the above equations binary values of the next state and outputs are obtained.

16.5 State Diagram

The information available in a state table can be represented graphically in as state diagram. In this type of diagram, a state is represented by a circle, and the transition between states is indicated by directed lines connecting the circles. The state diagram of the sequential circuit or Fig. 2. The state diagram provides the same information as the state table and is obtained directly from Table. The binary number inside each circle identifies the state of the flip-flops. The directed lines are labelled with two binary numbers separated by a slash. The input value during the present state is labelled first and the number after the slash gives the output during the present state. For example, the directed line from state 00 and the inputs is 1, the outputs is 0. After a clock transition, the circuit goes to the next state 01. The same clock transition may change the input value. If the input changes to 0, the output becomes 1, but if the input remains at 1, the output stays at 0. This information is obtained from the state diagram along the two directed lines emanating from the circle representing state 01. A directed line connecting a circle with itself indicates that no change of state occurs.

There is no difference between a state table and a state diagram except in the manner of representation. The state table is easier to derive from a given logic diagram and the state diagram follows directly from the state table. The state diagram gives a pictorial view of state transitions and is the form suitable for human interpretation of the circuit operation. For example, the state diagram of Fig. 2, clearly shows that starting from state 00, the output is 0 as long as the input stays at 1. The first 0 input after a string of 1's gives an output of 1 and transfers the circuit back to the initial state 00.

16.6 Design of a 2 bit-binary counter

The design procedure consists of first translating the circuit specifications into a state diagram. The state diagram is then converted into a state table. From the state table we obtain the information for obtaining the logic circuit diagram.

We wish to design a clocked sequential circuit that goes through a sequence of repeated binary state 00, 01, 10 and 11 when an external input x is equal to 1. The state of the circuit remains unchanged because the state sequence is identical specifies when the count should proceed.

The binary counter needs two flip-flops to represent the two bits. The state diagram for the sequential circuit is shown in Fig. 3. The diagram is drawn to show that the states of the circuit follow the binary count as long as $x = 1$. The state following 11 is 00, which causes the count to be repeated. If $x = 0$, the state of the

$$D_A = Ax + Bx$$

$$D_B = A'x$$

$$y = Ax' + Bx'$$

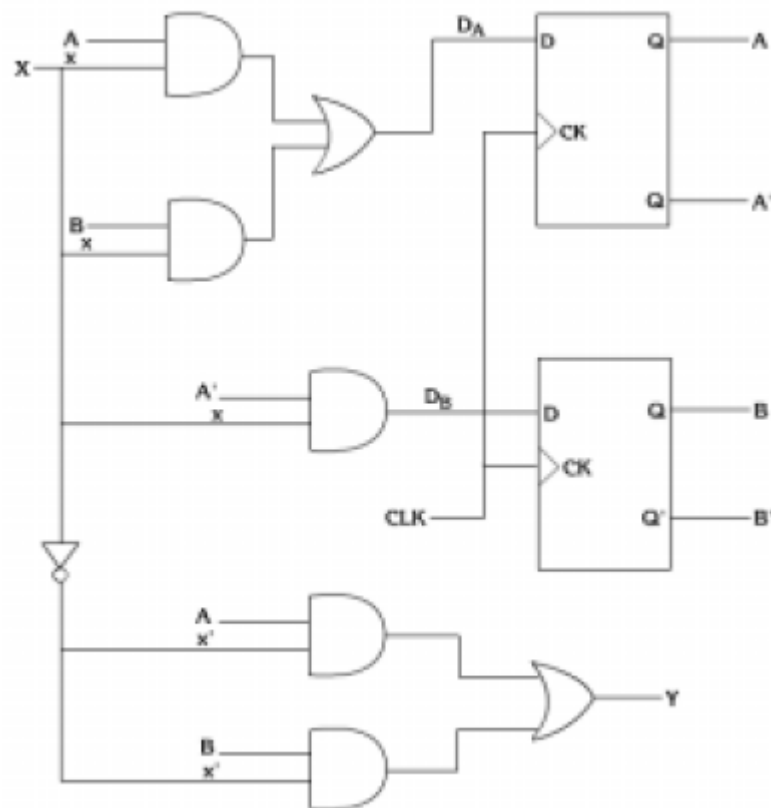


Figure 1

circuit remains unchanged. This sequential circuit has no external outputs, and therefore only the input value is labelled in the diagram. The state of the flip-flop is considered as the outputs of the counter.

We have already assigned the symbol x to the input variable. We now assign the symbols A and B to the two flip-flop outputs. The next state of A and B , as a function of the present state and input x , can be transferred from the state diagram into a state table. The first five columns of Table 2 constitute the state table. The entries for this table are obtained directly from the state diagram.

The excitation table of a sequential circuit is an extension of the state table. This extension consists of a list of flip-flop input excitations that will cause the required state transitions. The

flip-flop input conditions are a function of the type of flip-flop used. If we employ JK flip-flops, we need columns for the J and K inputs of each flip-flop. We denote the inputs of flip-flop A by J_A and K_A , and those of flip-flop B by J_B and K_B .

The excitation table for the JK flip-flop is now used to derive the excitation table of the sequential circuit. For example, in the first row of Table we have a transition for flip-flop A from 0 in the present state to 0 in the next state. In Table 2 we find that a transition of state from $Q(t)=0$ to $Q(t+1)=0$ in a JK flip-flop requires that input $J=0$ and input $K=x$. So 0 and x copied in the first row under J_A and K_A , respectively. Since the first row also shows a transition for flip-flop B from 0 in the present state to 0 in the exnt state, 0 and x are copied in the first row under J_B and K_B . The second row of Table 2 shows a transition for flip-flop B from 0 in the present state to 1 in the next state. From Table 2 we find that a transition from $Q(t)=0$ to $Q(t+1)=1$ requires that input $J=1$ and input $K=x$. So 1 and x are copied in the second row under J_B and K_B , respectively. This process is continued for each row of the table and for each flip-flop, with the input conditions as specified in Table being copied into the proper row of the particular flip-flop being considered.

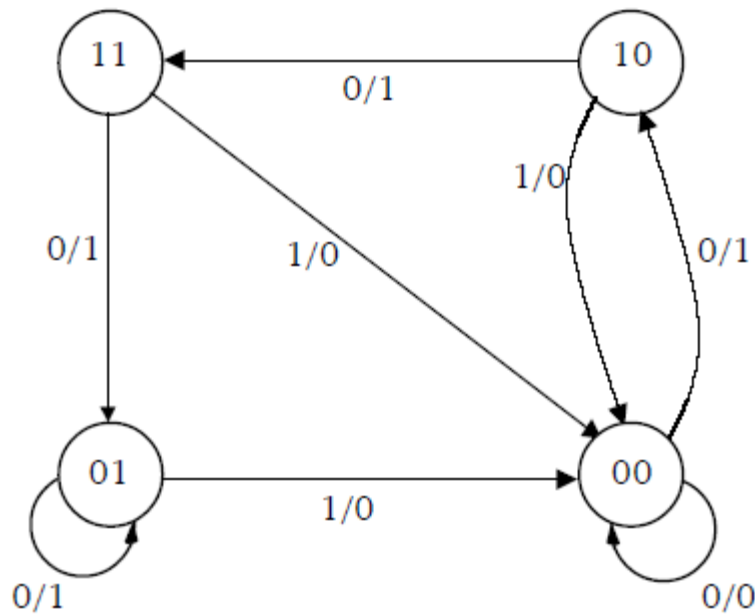


Fig. 2

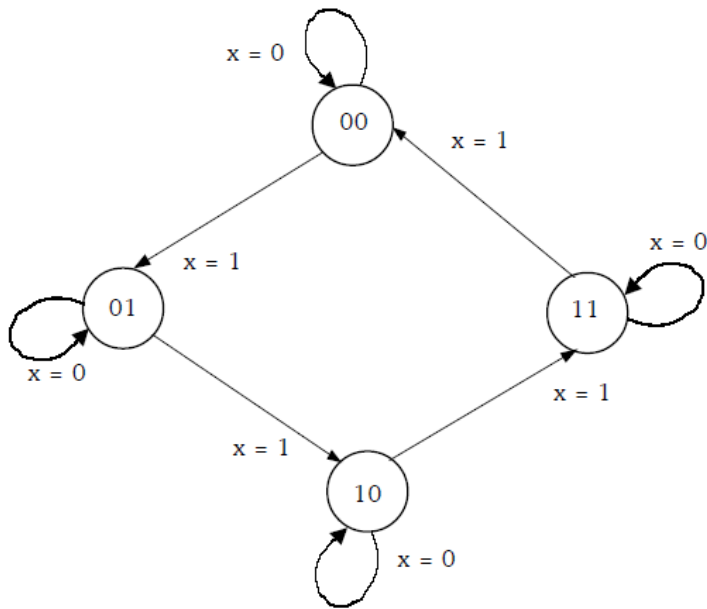


Fig. 3

Table 2 - Excitation Table for Binary Counter

Present state		Input	Next state		Flip-Flop Inputs			
A	B		A	B	J _A	K _A	J _B	K _B
0	0	0	0	0	0	X	0	X
0	0	1	0	1	0	X	1	X
0	1	0	0	1	0	X	X	0
0	1	1	1	0	1	X	X	1
1	0	0	1	0	X	0	0	X
1	0	1	1	1	X	0	1	X
1	1	0	1	1	X	0	X	0
1	1	1	0	0	X	1	X	1

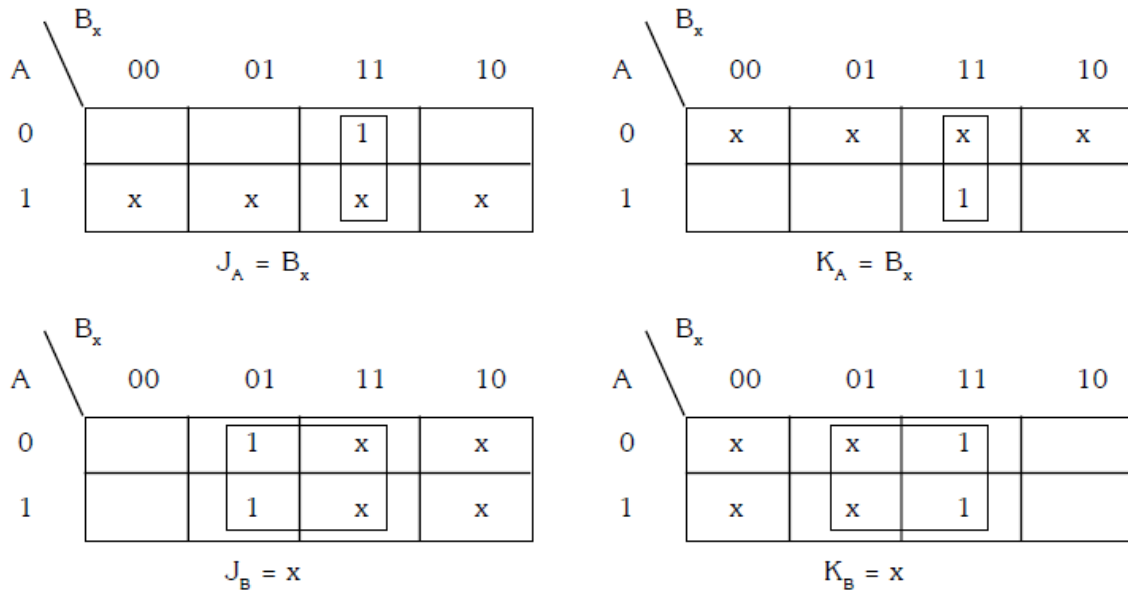


Fig. 4 - Maps for Combinational Circuit of Counter

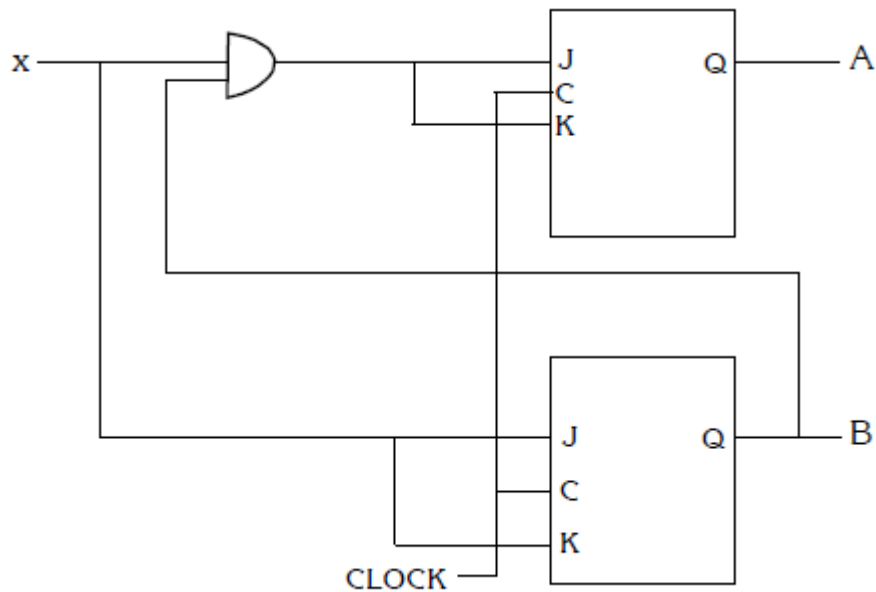


Fig. 5 - Logic Diagram of a 2-Bit Binary Counter

Let us now consider the information available in an excitation table such as Table 16.2. We know that a sequential circuit consists of a number of flip-flops and a combinational circuit. From the block diagram of Fig. 16.2, we note that the outputs J_A , K_A , J_B and K_B . The inputs to the combinational circuit are the external input x and the present-state values of flip-flops A and B .

Moreover, the Boolean functions that specify a combinational circuit are derived from a truth table that shows the input-output relationship of the circuit. The entries that list the combinational circuit inputs are specified under the “present state” and “input” columns in the excitation table. The combinational circuit outputs are specified under the “flip-flop inputs” columns. Thus an excitation table transforms a state diagram to a truth table needed for the design of the combinational circuit part of the sequential circuit.

The simplified Boolean functions for the combinational circuit can now be derived. The inputs are the variable A, B and x. The outputs are the variables J_A , K_A , J_B and K_B . The information from the excitation table is transferred into the maps of Fig. 16.4, where the four simplified flip-flop input equations are derived:

$$J_A = Bx \quad K_A = Bx$$

$$J_B = x \quad K_B = x$$

The logic diagram is drawn in Fig. 16.5 and consists of two JK flip-flops and an AND gate. Note that inputs J and K determine the next state of the counter when a clock signal occurs. If both J and K are equal to 0, a clock signal will have no effect; that is, the state of the flip-flops will not change. Thus when $x = 0$, all four inputs of the flip-flops are equal to 0 and the state of the flip-flops remains unchanged even though clock pulses are applied continuously.

16.7 Summary

Excitation table is similar to the truth table, showing the input variable states that are necessary to generate a particular next state when the current state is known

The state table representation of a sequential circuit consists of three sections labeled *present state*, *next state* and *output*. The present state designates the state of flip-flops before the occurrence of a clock pulse. The next state shows the states of flip-flops after the clock pulse, and the output section lists the value of the output variables during the present state.

16.8 Model Questions

- 1) Discuss about State Diagram
- 2) Define Excitation Table

LESSON - 17

MEMORY ORGANIZATION

Structure

- 17.0 Objectives**
- 17.1 Introduction**
- 17.2 Ram and Rom**
- 17.3 Random Access Memories**
- 17.4 Linear Select Memory Organization**
- 17.5 Decoder**
- 17.6 Read Only Memories**
- 17.7 Summary**
- 17.8 Model Questions**

17.0 Objectives

To introduce :

- Random Access Memory (RAM)
- Read-only memory (ROM)
- EPROM-EEPROM

17.1 Introduction

In this chapter we focus RAM and ROM. RAM constitutes the internal memory of the CPU for storing data, program and program result. It is read/write memory. It is called random access memory (RAM). ROM stands for Read Only Memory. The memory from which we can only read but cannot write on it.

17.2 RAM and ROM

Memory cells are distributed throughout a computer. The faster memory devices are in the central processing unit which uses flip-flops. The main memory then uses IC circuits (or

cores), but in arrays which are slower than the very high-speed flip-flops used in the CPU. The backup, or secondary, memory uses magnetic devices such as disks or drum.

17.3 Random-Access Memories

The main memory of a computer is organized in a way which is particularly desirable. Figure 1 shows that a high-speed main memory in a computer is organized into words of fixed length. As the figure indicates, a given memory is divided into N words, when N generally is some power of 2, and each word is assigned an address, or location, in the memory. Each word has the same number of bits, called the word length. And if we read, for instance, the word at location 71, we receive a word from the memory with this word length.

The addresses, or address numbers, in the memory run consecutively, starting with the address 0 and running up to the largest address. Thus at address 0 we find a word, at address 1 a second word, at address 2 a third word, and so on up to the final word at the largest address.

Generally, the computer can read a word from or write a word into each location in the memory. For a memory with an 8-bit word, if we write the word 01001011 into memory address 17 and later read from this same address, we shall read the word 01001011. If we again read from the address at a later time (and have not written another word in), then the word 01001011 will be read again. This means the memory is a nondestructive read, in that reading does not destroy or change a stored word.

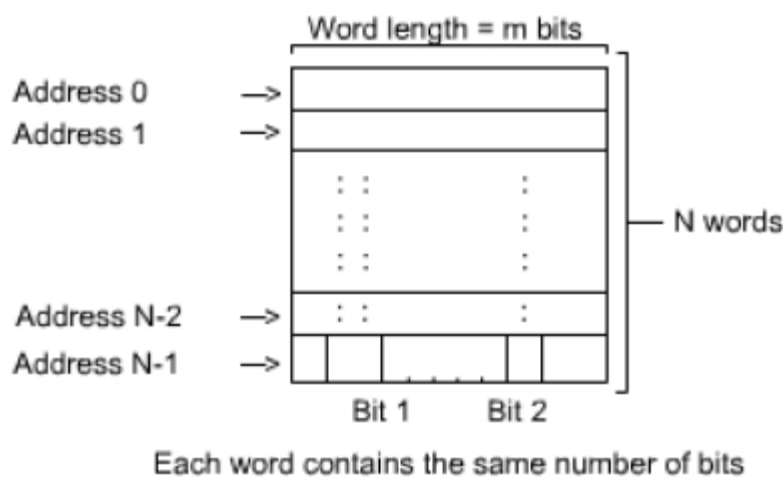


Figure - 1 : Words in high speed memory

It is important to understand the difference between the contents of a memory address and the address itself. A memory is like a large cabinet containing as many drawers as there are addresses in memory. In each drawer is a word, and the address of each word is written on the outside of the drawer. If we write or store a word at address 17, it is like placing the word in the drawer labeled 17. Later, reading from address 17 is like looking in that drawer to see its contents. We do not remove the word at an address when we read. We change the contents at an address only when we store or write a new word.

From an exterior viewpoint, a high-speed main memory has very much like a “black box” with a number of locations or addresses into which data can be stored or from which data can be read. Each address, or location, contains a fixed number of binary bits, the number being called the word length for the memory. A memory with 4096 locations, each with a different address and with each location storing 16 bits, is called a 4096-word 16-bit memory, or in the vernacular of the computer trade, a 4K 16-bit memory. (Since memories generally come with a number of words equal to 2^n some n , if a memory has $2^{14} = 16,384$ words, computer literature and jargon would refer to it as a 16K memory, because it is always understood that the full 2^n words actually occur in the memory. Thus a 2^{15} -word 16-bit memory is called a 32K 16-bit memory).

Memories can be read from (that is, data can be taken out) or written into (data can be entered into the memory). Memories which can be both read from and written into are called read-write memories. Some memories have programs or data permanently stored and are called read-only memories.

A block diagram of a read-write memory is shown in Fig-2. The computer places the address of the location into which the data are to be read into the memory address register. This register consists of n binary devices (generally flip-flops), where 2^n is the number of words that can be stored in the memory. The data to be written into the memory are placed in the memory buffer register, which has as many binary storage devices as there are bits in each memory word. The memory is told to write by means of a 1 signal on the WRITE line. Then the memory will store the contents of the memory buffer register in the location specified by the memory address register.

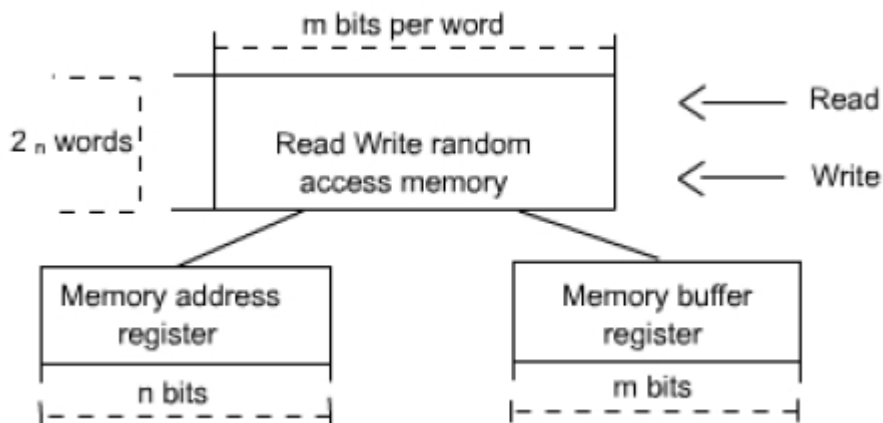


Fig-2: Read-write random access memory.

Words are read by placing the address of the location to be read from into the memory address register. Then a 1 signal is placed on the READ line, and the contents of that location are placed by the memory in the memory buffer register.

As can be seen, the computer communicates with the memory by means of the memory address register, the memory buffer register, and the READ and WRITE inputs. Memories are generally packaged in separate modules or packages. It is possible to buy a memory module of a specified size from a number of different manufacturers. For instance, a 64K 16-bit memory module can be purchased on a circuit board ready for use. Similarly, if a computer is purchased with a certain amount of main memory, generally more memory can be added later by purchasing additional modules and “plugging them in.”

If it is possible to read from or write into any location “at once”, that is, if there is no more delay in reaching one location than another, the memory is called a random-access memory (RAM). Computers almost invariably use random-access read-write memories for their high-speed main memory and use backup or slower-speed memories to hold auxiliary data.

17.4 Linear-Select Memory Organization

The most used random-access memories are IC memories. To present the basic principles, an idealized IC memory is shown.

In any memory there must be a basic memory cell. Figure-3 shows a basic memory cell consisting of an RS flip-flop with associated control circuitry. To use this cell in a memory,

however, a technique for selecting those cells addressed by the memory address register must be used, as a method to control whether the selected cells are written into or read from.

Figure 4 shows the basic memory organization for a linear-select IC memory. This is a four-address memory with 3 bits per word. The memory address register (MAR) selects the memory cells (flip-flops) to be read from or written into through a decoder which selects three flip-flops for each address that can be in the memory address register.

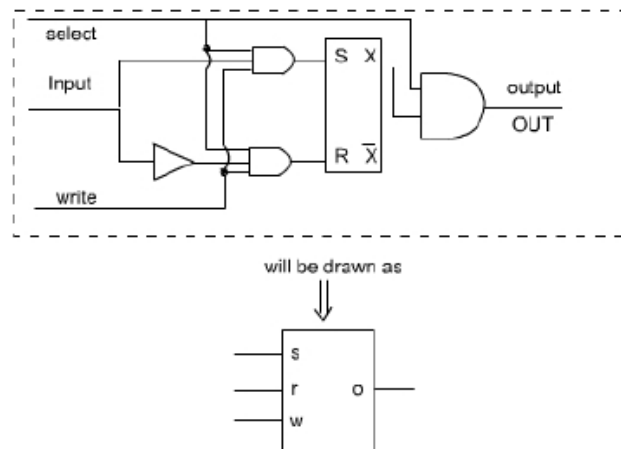


Figure - 3 : Basic Memory Cell

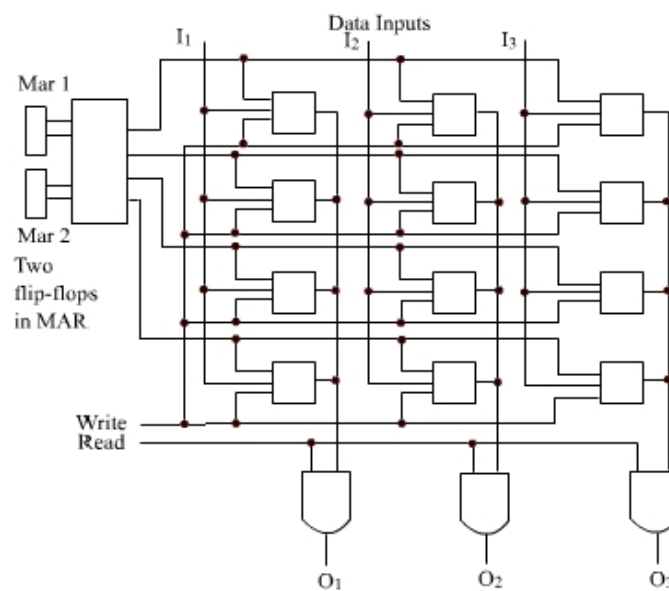


Figure-4: Linear-select ic memory.

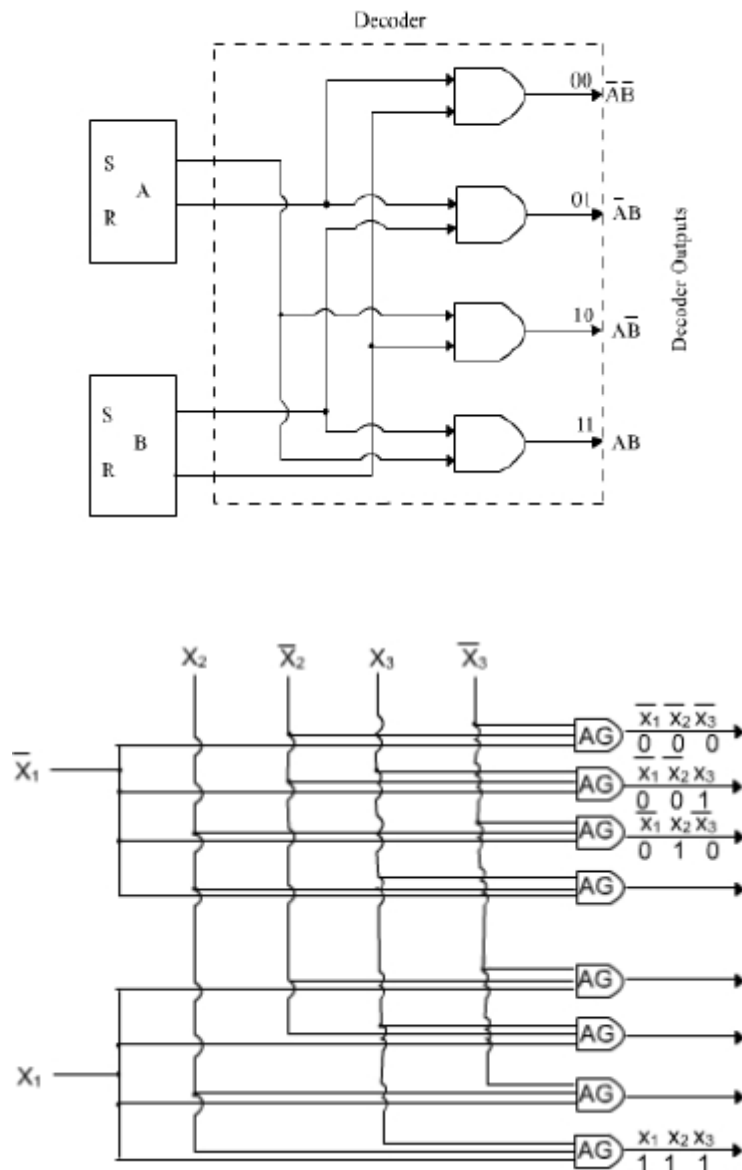


Figure-5: (a) Four-output decoder. (b) Parallel decoder

Figure 5(a) shows the decoder in expanded form. It has an input from each flip-flop (bit) to be decoded. If there are 2 input bits as in Fig.5(a), then there will be four output lines, one for each state (value) the input register can take. For instance, if the MAR contains 0 in both flip-flops, then the upper line of the decoder will be a 1 and the remaining three lines a 0. Similarly, if both memory cells contain a 1, the lowest output line will be a 1 and the remaining three lines a 0. Similar reasoning will show that there will be a single output line with a 1 output for each possible input state, and the remaining lines will always be a 0.

Figure 5(b) shows a decoder for three inputs. The decoder has eight output lines. In general, for n input bits a decoder will have 2^n output lines.

The decoder in Fig.5(b) operates in the same manner as that in Fig.5(a). For each input state the decoder will select a particular output line, placing a 1 on the selected line and a 0 on the remaining lines.

Returning to Fig.4, we now see that corresponding to each value can be placed in the MAR, a particular output line from the decoder will be selected and carry a 1 value. The remaining output lines from the decoder will contain 0s, not selecting the AND gates at the inputs and outputs of the flip-flops for these rows.

The memory in Fig.4 is organized as follows: There are four words, and each row of three memory cells comprises a word. At any given time the MAR selects a word in memory. If the READ line is a 1, the contents of three cells in the selected word are read out on the O_1 , O_2 and O_3 lines. If the WRITE line is a 1, the values on I_1 , I_2 and I_3 are read into the memory.

The AND gates connected to the OUT lines on the memory cells in fig.3 must have the property that when a number of AND gate output lines are connected, the output goes to the highest level. (If any OUT is a 1, the line goes to 1; otherwise, it is a 0).

This called a wired OR. In Fig.4 all four memory cells in the first column are wire-ORed; so if any output line is a 1, the entire line will be a 1. (Memory cells in IC memories are constructed in this manner).

Now if the READ line is a 1 in Fig.4, then the output values for the flip-flops in the selected row will all be gated onto the output line for each bit in the memory.

For example, if the second row in the memory contains 110 in the three memory cells, and if the MAR contains 01, then the second output line from the decoder (marked 01) will be a 1, and the input gates and output gates to these three memory cells will be selected. If the READ line is a 1, then the outputs from the three memory cells in the second row will be 110 to the AND gates at the bottom of the figure, which will transmit the value 110 as an output from the memory.

If the WRITE line is a 1 and the MAR again contains 01, the second row of flip-flops will have selected inputs. Then the input values on I_1 , I_2 and I_3 will be read into the flip-flops in the second row.

As may be seen, this is a complete memory, fully capable of reading and writing. The memory will store data for an indefinite period and will operate as fast as the gates and flip-flops will permit. There is only one problem with the memory-its complexity. The basic memory cell (the flip-flop with its associated circuitry) is complicated, and for large memories the decoder will be large.

17.5 Decoders

An important part of the system which selects the cells to be read from and written into is the decoder. This particular circuit is called a many-to-one decoder, a decoder matrix, or simply a decoder. It has the characteristic that for each of the possible 2^n binary input numbers which can be taken by the input cells, the matrix will have a unique one of its 2^n output lines selected.

Figure 5(b) shows a decoder which is completely parallel in construction and designed to decode three flip-flops. There are $2^3 = 8$ output lines; and for each of the eight states which the three input (flip-flops) may take, a unique output line will be selected.

17.6 Read-Only Memories

A type of storage device called a read-only memory (ROM) is widely used. ROMs have the unique characteristic that they can be read from, but not written into. Thus the information stored in these memories is introduced into the memory in some manner such that the information is semipermanent or permanent. Sometimes the information stored in a ROM is placed in the memory at the time of construction, and sometimes devices are used in which the information can be changed. In this section we study several types of ROMs. These are characteristic of this particular class of memory devices, and most devices are variations on the principles presented.

Basically a ROM is a device with several input and output lines such that for each input value there is a unique output value. Thus a ROM physically realize a truth table, or table of combinations. A typical (small) table is shown in Table.1. This list of input-output values is actually a list of binary-to-Gray code values. It is important to see that the list can be looked at in two ways: (1) as a table for a gating network with four inputs and four outputs and (2) as a list of addresses from 0 to 15, given by the X values, and the contents of each address, given by the values of Z. Thus we might construct a gating network as in Fig.7 which would give the correct Z output for each X input. (The boxes with $\oplus$ are mod 2 adders).

Table – 1: Binary-to-Gray code Values

Input				Output			
X_1	X_2	X_3	X_4	Z_1	Z_2	Z_3	Z_4
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1

Input				Output			
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

Table 1 could also be realized by a 16-word 4-bit-per-word IC memory into which we had read 0000 at address 0; 0001 at address 1; 0011 in the next address; and so on until 1000 at the last address. If we never wrote into this memory afterward, it would be a ROM memory and would serve the same purpose as the gating network in Fig.7.

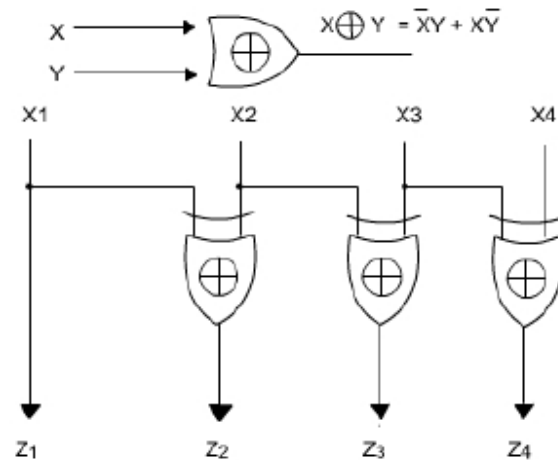


Figure – 7: Combinational network for binary-to-gray code.

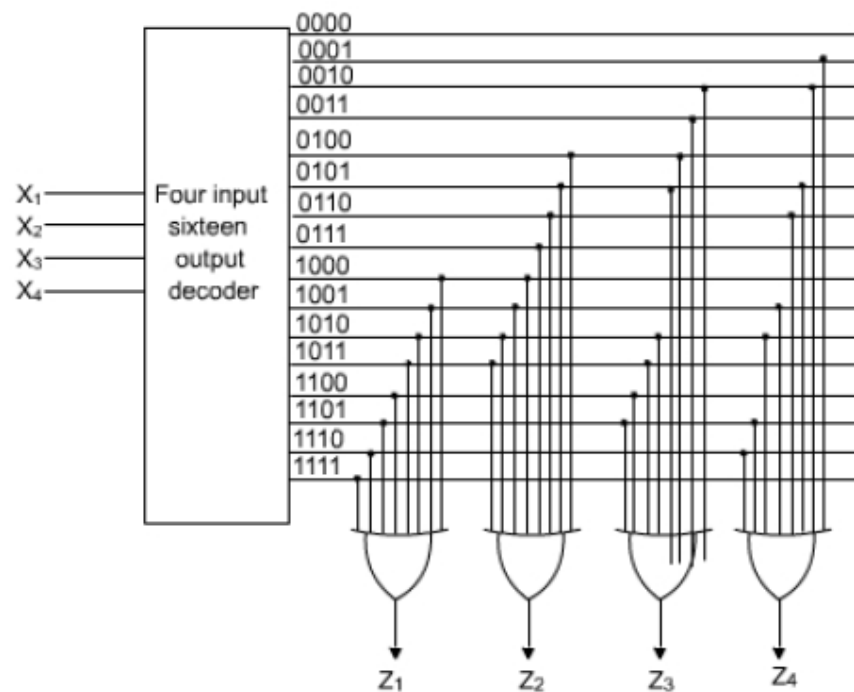


Figure-8 : (a) Read-only memory layout.

Figure 8(a) shows a scheme for implementing Table 1 by using a decoder network with four inputs X_1 , X_2 , X_3 and X_4 and four OR gates. With a given input combination (Or address), a single output line from the decoder will be high. Let us assume that the input value is $X_1 = 0$, X_2

The entire scheme outlined above realizes the ROM with OR gates. By using LSI techniques, arrays of this sort can be inexpensively fabricated in small containers at low prices. The 4K-word 8-bit memories are of about average size for the large-scale integration ROMs; these memories effectively store 32K bits in all.

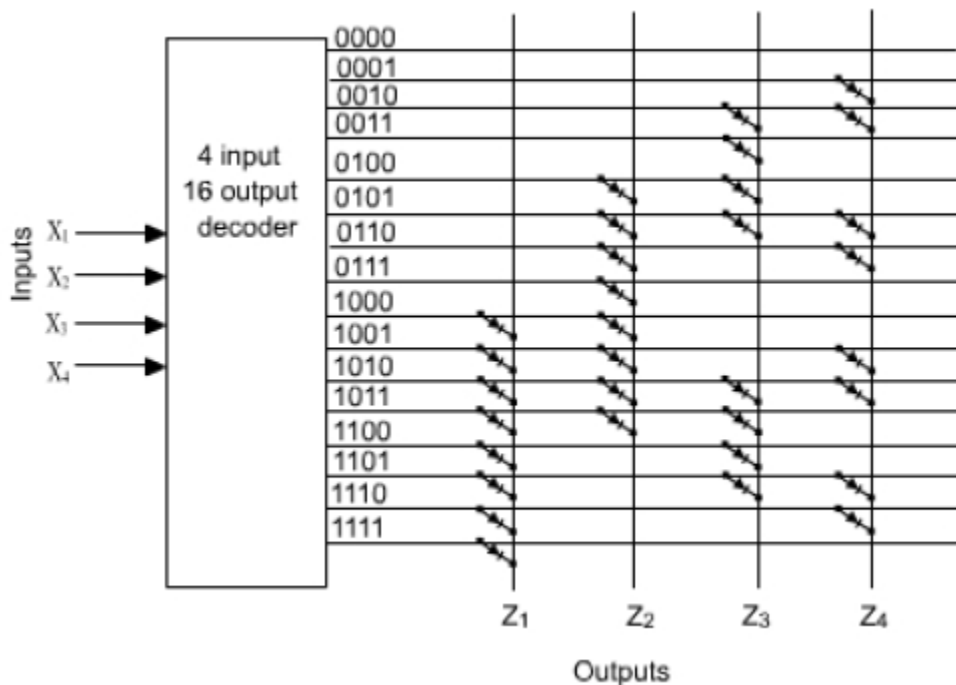


Figure – 8 : (b) Diode ROM

When a ROM is constructed so that the user can electrically (or by using other techniques) write in the contents of the memory, the memory is called a programmable ROM or PROM. Often a scheme is used in which a memory chip is delivered with 1s in every position, but 0s can be introduced at give positions by placing an address on the input lines and then raising each output line which is to be 0 to a specified voltage, thus destroying a connection to the selected cell. (Sometimes the memories contain all 0s, and 1s are written in by the user). Devices are also manufactured which program PROMs by reading paper tapes, magnetic tapes, punched cards, etc., and placing their contents into the PROM.

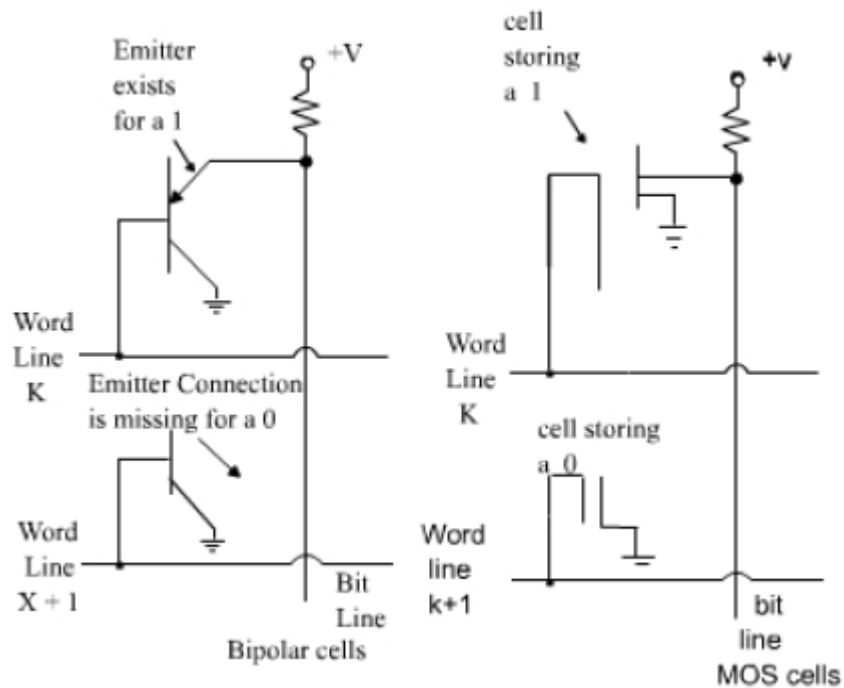


Figure – 8 : (c) Rom cells using MOS and bipolar transistors

When a ROM is manufactured so that the memory's contents can be set as desired by the user and the memory can later have the contents erased and new values written in, the ROM is said to be erasable and reprogrammable and is often called an EPROM.

For example, some memory chips are made with a transparent lid. Exposing the semiconductor chip (through the lid) to ultraviolet light will erase the pattern on the chip, and a new pattern can be written in electrically. This can be repeated as often as desired.

Some bipolar ROMs and of several electrically programmable ROMs (EPROMs) which can have their contents erased by exposure to ultraviolet light and can be programmed (written into) by placing designated voltage on inputs. Electrically alterable ROMs (EAROMs) which can have selected contents rewritten while in place in a circuit by means of properly applied input voltages and EEPROMs (electrically erasable programmable ROMs). The difference between EAROMs and EEPROMs is that EAROMs can have their contents altered while in place in the circuit and EEPROMs must be erased and rewritten.

Several companies make devices for programming PROMs and EPROMs. Some of these devices are operated from a keyboard, others from tape, and still others from external inputs such as microprocessors.

When an EPROM, EEPROM, EAPROM, or a PROM chip is to be programmed, there is generally a write enable to be raised, which makes output lines able to accept data. Then address lines are set to the location to be written into. Next, for some chips, the output lines to have 1s are raised to high voltages (or a sequence of large-amplitude pulses are applied to them); or, for some chips, the normal logical levels are placed on the output lines, and a special program input is placed with a sequence of high-voltage (25-V) pulses. In either case, each memory location must be written into by setting the address lines and then writing the desired contents into the output lines. Applying an erase (for example, applying an ultraviolet light to the lid for some specified time) generally erases all the contents of the memory except for EAROMs, in which the individual locations can be changed.

17.7 Summary

A RAM constitutes the internal memory of the CPU for storing data, program and program result. It is read/write memory. It is called random access memory (RAM).

ROM stands for Read Only Memory. The memory from which we can only read but cannot write on it. This type of memory is non-volatile. The information is stored permanently in such memories during manufacture.

EPROM -Erasable and Programmable Read Only Memory

EEPROM -Electrically Erasable and Programmable Read Only Memory

17.8 Model Questions

- (1) Explain the organisation for ROM / RAM.
- (2) Distinguish between ROM-EPROM-EEPROM.

LESSON - 18

INSTRUCTION FORMATS AND ADDRESSING MODES

Structure

- 18.0 Objectives**
- 18.1 Introduction**
- 18.2 Three Address Instructions**
- 18.3 Two Address Instructions**
- 18.4 One Address Instructions**
- 18.5 Zero Address Instructions**
- 18.6 Addressing Modes**
- 18.7 Implied Mode**
- 18.8 Immediate Mode**
- 18.9 Register Mode**
- 18.10 Register Indirect Mode**
- 18.11 Auto Increment/Decrement Mode**
- 18.12 Direct Address Mode**
- 18.13 Indirect Address Mode**
- 18.14 Relative Addressing Mode**
- 18.15 Indexed Addressing Mode**
- 18.16 Base Register Addressing Mode**
- 18.17 Summary**
- 18.18 Model Questions**

18.0 Objectives

To introduce :

- Instruction set of the computer
- Addressing mode of instructions

18.1 Introduction

A computer will usually have a variety of instruction code formats. It is the function of the control unit within the CPU to interpret each instruction code and provide the necessary control functions needed to process the instruction.

The format of a instruction is usually depicted in a rectangular box symbolizing the bits of the instruction as they appear in memory words or in a control register. The bits of the instruction are divided into groups called fields. The most common fields found in instructions formats are:

1. An operation code field that specified the operation to be performed.
2. An address field that designates a memory address or a processor register.
3. A mode field that specifies the way the operand or the effective address is determined.

Other special fields are sometimes employed under certain circumstances, as for example a field that gives the number of shifts in a shift-type instruction.

The operation code field of an instruction is a group of bits that define various processor operations, such as add, subtract, complement, and shift. The bits that define the mode field of an instruction code specify a variety of alternatives for choosing the operands from the given address. In this section we will be concerned with the address field of an instruction format and consider the effect of including multiple address fields in an instruction.

Operations specified by computer instructions are executed on some data stored in memory or processor registers. Operands residing in memory are specified by their memory address. Operands residing in processor registers are specified with a register address. A register address is a binary number of k bits that defines one of 2^k registers in the CPU. Thus, a CPU with 16 processor registers R0 through R15 will have a register address field of four bits. The binary number 0101, for example, will designate register R5.

Computers may have instructions of several different lengths containing varying number of addresses. The number of address fields in the instruction format of a computer depends upon the internal organization of its registers.

All operations are performed with an implied accumulator register. The instruction format in this type of computer used one address field. For example, the instruction that specifies an arithmetic addition is defined by an assembly language instruction as

ADD X

Where X is the address of the operand. The ADD instruction in this case results in the operation $AC \leftarrow AC + M[X]$. AC is the accumulator register and $M[X]$ symbolized the memory word located at address X.

The instruction format in a general register type of computer needs three register address fields. Thus, the instruction for an arithmetic addition may be written in an assembly language as

ADD R1, R2, R3

To denote the operation $R1 \leftarrow R2 + R3$. The number of address fields in the instruction can be reduced from three to two if the destination register is the same as one of the source register. Thus, the instruction

ADD R1, R2

Would denote the operation $R1 \leftarrow R1 + R2$. Only register address for R1 and R2 need be specified in this instruction.

Computers with multiple processor registers use the move instruction with a mnemonic MOV to symbolize a transfer instruction. Thus, the instruction

MOV R1, R2

Denotes the transfer $R1 \leftarrow R2$ (or $R2 \leftarrow R1$, depending on the particular computer). Thus, transfer-type instruction need two address fields to specify the source and the destination. General register-type computers employ two or three address fields in their instruction format. Each address field may specify a processor register or a memory word. An instruction symbolized by

ADD R1, X

would specify the operation $R1 \leftarrow R1 + M[X]$. It has two address fields, one for register R1 and the other for the memory address X.

Computers with stack organization would have PUSH and POP instructions which require an address field. Thus, the instruction.

PUSH X

will push the word at address X to the top of the stack. The stack pointer is automatically updated. Operation-type instructions do not need an address field in stack-organized computers. This is because the operation is performed on the two items which are on top of the stack. The instruction

ADD

In a stack computer consists of an operation code only with no address field. This operation has the effect of popping the two top numbers from the stack, adding the numbers and pushing the sum into. There is no need to specify operands with an address field since all operands are implied to be in the stack.

Most computers fall in one of the three types of organizations that have just been described. Some computers combine features from more than one organizational structure. For example the Intel 8080 microprocessor has seven CPU registers, one of which is an accumulator register. As a consequence, the processor has some of the characteristics of a general register type and some of the characteristics of an accumulator register, so these instructions that transfer data among the seven processor registers have a format that contains two register address fields. Moreover, the Intel 8080 processor has a stack pointer and instructions to push and pop from a memory stack. The processor, however, does not have the zero-address-type instruction which are characteristic of stack-organized CPUs.

To illustrate the influence of the number of address fields on computer programs, we will evaluate the arithmetic statement.

$$X = (A + B) * (C + D)$$

using zero, one, two or three address instructions.

We will use the symbols ADD, SUB, MUL and DIV for the four arithmetic operations; MOV for the transfer-type operation; and LOAD and STORE for transfers to and from memory and AC register. We will assume that the operands are in memory addresses A, B, C and D and the result must be stored in memory at address X.

18.2 Three-Address Instructions

Computers with three-address instruction formats can use each address field to specify either a processor register or a memory operand. The program in assembly language that evaluates $X = (A+B) * (C+D)$ is shown below, together with comments that explain the register transfer operation of each instruction.

ADD	R1, A, B	$R1 \leftarrow M[A] + M[B]$
ADD	R2, C, D	$R2 \leftarrow M[C] + M[D]$
MUL	X, R1, R2	$M[X] \leftarrow R1 * R2$

It is assumed that the computer has two processor registers R1 and R2. The symbol $M[A]$ denotes the operand at memory address symbolized by A.

The advantage of the three-address format is that it results in short programs when evaluating arithmetic expressions. The advantage is that the binary-coded instructions require too many bits to specify three addresses.

An example of a commercial computer that uses three-address instructions is the Cyber 170. The instruction formats in the Cyber 170 computer are restricted to either three register address fields or two register address fields and one memory address field.

18.3 Two Address Instructions

Two-address instructions are the most common in commercial computers. Here again each address field specify either a processor register or a memory word. The program to evaluate

$X = (A + B) * (C + B)$ is as follow:

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R1, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$

MOV X,R1 $M[X] \leftarrow R1$

The MOV instruction moves or transfers the operands to and from memory and processor registers. The first symbol listed in an instruction is assumed to be both a source and the destination where the result of the operation is transferred.

An example of a commercial computer that uses two-address instruction is the PDP-11. The two address fields in the PDP-11 computer instructions always specify processor registers. However, depending on the addressing mode the specified register may contain the operand or may contain the address of a memory operand.

18.4 One-Address Instructions

One-address instructions use an implied accumulator (AC) register for all data manipulation. For multiplication and division there is a need for a second register. However, here we will neglect the second register and assume that the AC contains the result of all operations. The program to evaluate $X = (A + B) * (C + D)$ is

LOAD	A	$AC \leftarrow AC + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

All operations are done between the AC register and a memory operand. T is the address of a temporary memory location required for storing the intermediate result. An example of a commercial that uses one-address instructions is the PDP-8.

18.5 Zero-Address Instruction

A stack-organized computer does not use an address field for the instruction ADD and MUL. The PUSH and POP instructions, however, need an address field to specify the operand that communicates with the stack. The following program shows how $X = (A + B) * (C + D)$ will be written for a stack-organized computer, (TOS stands for top of stack).

PUSH	A	$TOS \leftarrow A$
PUSH	B	$TOS \leftarrow B$
ADD		$TOS \leftarrow (A + B)$
PUSH	C	$TOS \leftarrow C$
PUSH	D	$TOS \leftarrow D$
ADD		$TOS \leftarrow (C + D)$
MUL		$TOS \leftarrow (C + D) * (A + B)$
POP	X	$M[X] \leftarrow TOS$

In order to evaluate arithmetic expressions in a stack computer, it is necessary to convert the expression in to reverse Polish notation. The name zero-address is given to this type of computer because of the absence of an address field in the computational instructions. An example of a stack-organized commercial computer is the Burrough-6700.

18.6 Addressing Modes

The operation field of an instruction specifies the operation to be performed. This operation must be executed on some data stored in computer registers or memory words. The way the operands are chosen during program execution is dependent on the addressing mode of the instruction. The addressing mode specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually referenced.

Computers use addressing mode techniques for the purpose of accommodating one or both the following provisions:

1. To give programming versatility to the used by providing such facilities as pointer, counters, indexing and program relocation.
2. To reduce the number of bits in the address field of the instruction.

To the inexperienced user, the variety of addressing modes in some computers may seem excessively complicated. However the availability of different addressing scheme gives the experienced programmer flexibility for writing programs that are more efficient with respect to the number of instruction and execution time. Although most addressing modes modify the

address field of the instruction, there are two modes that need not address field at all. These are the implied and implied and immediate mode.

18.7 Implied Mode

In this mode the operands are specified implicitly in the definition of the instruction. In fact, all register reference instructions that use an accumulator are implied-mode instructions. Zero-address instructions in a stack-organized computer are implied-mode instructions since the operands are implied to be on top of the stack.

18.8 Immediate Mode

In this mode the operand is specified in the instruction itself. In other words, an immediate-mode instruction has an operand field rather than an address field. The operand field contains the actual operand to be used in conjunction with the operation specified in the instruction. Immediate-mode instructions are useful for initializing registers to a constant value.

It was mentioned previously that the address field of an instruction may specify either a memory word or a processor register. When the address field specifies a processor register, the instruction is said to be in the register mode.

18.9 Register Mode

In the mode the operands are in registers which reside within the CPU. The particular register is selected from a register field in the instruction. A k-bit field can specify any one of 2^k registers.

18.10 Register-Indirect Mode

In this mode the instruction specifies a register in the CPU whose contents give the address of the operand in memory. In other words the selected register contains the address of the operand rather than the operand itself. Before using a register-indirect mode instruction, the programmer must ensure that the memory address of the operand is placed in the processor register with a previous instruction. A reference to the register is then equivalent to specifying a memory address. The advantages of a register-indirect mode instructional is that the address field of the instruction uses fewer bits to select a register than would have been required to specify a memory address directly.

18.11 Auto Increment or Auto Decrement Mode

This is similar to the register-indirect mode except that the register is incremented or decremented after (or before) its value is used to access memory when the address stored in the register is used. To enable data increment it is necessary to increment or decrement the register after every access to memory. This can be achieved by using the increment or decrement instruction.

However, because it is such a common requirement, some computers incorporate a special mode which automatically increments or decrements the content of the register after data access.

The address field of an instruction is used by the control unit in the CPU to obtain the operand from memory. Sometimes the value given in the address field is the address of the operand, but sometimes it is just an address from which the address of the operand is calculated. To differentiate among the various addressing modes it is necessary to distinguish between the address part of the instruction and the effective address used by the control when executing the instruction. The effective address is defined to be the memory address obtained from the computation dictated by the given addressing mode. The effective address is the address of the operand in a computational-type instruction. It is the address where control branches in response to a branch-type instruction.

18.12 Direct-Address Mode

In the mode the effective address is equal to the address part of the instruction. The operand resides in memory and its address is given directly by the address field of the instruction. In a branch-type instruction the address field specifies the actual branch address.

18.13 Indirect-Address Mode

In this mode the address field of the instruction gives the address where the effective address is stored in memory. Control fetches the instruction from memory and uses its address part to access memory again to read the effective address.

A few addressing modes require that the address field of the instruction be added to the content of a specific register in the CPU.

The effective address in these modes is obtained from the following computation.

$$\begin{aligned}\text{Effective address} &= \text{Address part of instruction} \\ &\quad + \text{Content of CPU register}\end{aligned}$$

The CPU register used in the computation may be the program count, an index register, or a base register. In either case we have a different addressing mode which is used for a different application.

18.14 Relative Address Mode

In this mode the content of the program counter is added to the address part of the instruction in order to obtain the effective address. The address part of the instruction is usually a signed number (in 2's complement representation) which can be either positive or negative. When this number is added to the content of the program counter, the result produces an effective address whose position in memory is relative to the address of the next instruction. To clarify with an example, assume that the program counter contains the number 825 and the address part of the instruction contains the number 24. The instruction at location 825 is read from memory during the fetch cycle and the program counter is then incremented by one to 826. The effective address computation for the relative address mode is $826 + 24 = 850$. This is 24 memory locations forward from the address of the next instruction. Relative addressing is often used with branch-type instruction when the branch address is in the area surrounding the instruction word itself. It results in a shorter address field in the instruction format since the relative address can be specified with a smaller number of bits compared to the number of bits required to designate the entire memory address.

18.15 Indexed Addressing Mode

In this mode the content of an index register is added to the address part of the instruction to obtain the effective address. The index register is a special CPU register that contains an index value. The address field of the instruction defines the beginning address of a data array in memory. Each operand in the array is stored in memory relative to the beginning address. The distance between the beginning address and the address of the operand is the index value stored in the index register. Any operand in the array can be accessed with the same instruction provided the index register contains the correct index value. The index register can be

incremented to facilitate the access to consecutive operands. Note that if an index-type instruction does not include an address field in its format the instruction converts to the register-indirect mode of operation.

Some computers dedicate one CPU register to function solely as an index register. This register is involved implicitly when the index-mode instruction is used. In computers with many processor registers, any one of the CPU registers can contain the index number. In such a case, the register must be specified explicitly in a register field within the instruction format.

18.16 Base Register Addressing Mode

In this mode the content of a base register is added to the address part of the instruction to obtain the effective address. This is similar to the indexed addressing mode except that the register is now called a base register instead of an index register. The difference between the two modes is in the way they are used, rather than in the way that they are computed. An index register is assumed to hold an index number which is relative to the address part of the instruction. A base register is assumed to hold a base address and the address field of the instruction gives a displacement relative to this base address. The base register addressing mode is used in computers to facilitate the relocation of programs in memory. When programs and data are moved from one segment of memory to another, as required in multiprogramming systems the address values of instructions must reflect this change of position. With a base register, the displacement values of instructions do not have to change. Only the value of the base register requires updating to reflect the beginning of a new memory segment.

In some computers, the addressing mode of the instruction is specified with a distinct binary code, just like the operation code is specified. Other computers use a single binary code that designates both the operation and the mode of the instruction. Instructions may be defined with a variety of addressing modes. Sometimes two or more addressing modes are combined in one instruction.

18.17 Summary

Computers may have instructions of several different lengths containing varying number of addresses. The number of address fields in the instruction format of a computer depends on the internal organization of its registers.

Computers with threeaddress instruction formats can use each address field to specify either a processor register or a memory operand.

Two address instructions are the most common in commercial computers. Here again each address field can specify either a processor register or a memory word.

Oneaddress instructions use an implied accumulator (AC) register for all data manipulation.

The different ways in which a source operand is denoted in an instruction is known as **addressing modes**

Register addressing mode: It means that the register is the source of an operand for an instruction.

Direct addressing mode : The addressing mode in which the effective address of the memory location is written directly in the instruction.

Immediate addressing mode : The addressing mode in which the data operand is a part of the instruction itself is known as immediate addressing mode.

18.18 Model Questions

1. Discuss the different addressing modes with examples in the instruction set of a computer.

LESSON - 19

INSTRUCTION SET

Structure

- 19.0 Objectives**
- 19.1 Introduction**
- 19.2 Data Transfer and Manipulation**
- 19.3 Data Transfer Instruction**
- 19.4 Data Manipulation Instruction**
- 19.5 Arithmetic Instruction**
- 19.6 Logic and bit manipulation Instruction**
- 19.7 Shift Instruction**
- 19.8 Program Control**
- 19.9 Status Bit Condition**
- 19.10 Conditional Branch Instruction**
- 19.11 Subroutine Call and Return**
- 19.12 Program Interrupt**
- 19.13 Summary**
- 19.14 Model Questions**

19.0 Objectives

To introduce

- Data transfer instructions
- Arithmetic instructions.
- Logical instructions.
- Branch Instructions

19.1 Introduction

In this chapter we focus Data Transfer Instruction, Data Manipulation Instruction, Shift Instruction, Branching Instruction, Program Control and Program Interrupt Data Transfer Instructions: These instructions are used to transfer the data from the source operand to the destination operand.

19.2 Data Transfer and Manipulation

Computer Provide an extensive set of instructions to give the user the flexibility to carry out various computation tasks. The instruction set of different computers differ from each mostly in the way the operands are determined from the address and mode fields. The actual operation available in the instruction set are not very different form one computer to another. It so happens that the binary code assignments in the operation field is different in different computers, even for the same operation. It may also happen that the symbolic name given to instructions in the assembly language notation is different in different computers, even for the same instruction. Nevertheless, there is a set of basic operations that most, if not all, computers include in their instruction repertorie. The basic set of operations available in a typical computer is given in the next section.

Most computer instructions can be classified into three categories:

1. Data transfer instructions.
2. Data manipulation instructions.
3. Program control instructions.

Data transfer instructions cause transfer of data from one location to another without changing the binary information content. Data manipulation instructions are those that perform arithmetic, logic, and shift operations. Program control instructions provide decision-making capabilities and change the path taken by the computer determines the register-transfer operations and control decisions that are available to the user.

19.3 Data Transfer Instructions

Data transfer instructions move data from one place to another without changing the data content. The most common transfer are between memory and processor registers, between

processor register and input & output, and between the processor registers themselves. Table 1 gives the list of eight data transfer instructions used in many computers.

Table 1 Typical Date Transfer Instruction

NAME	MNEMONIC
LOAD	LDA
STORE	STA
MOVE	MOV
EXCHANGE	XCH
INPUT	IN
OUTPUT	OUT
PUSH	PUSH
POP	POP

Accompanying each instruction is a mnemonic symbol. This is the assembly language abbreviation recommended by a proposed standard. It must be realized that different computers use different mnemonic for the same instruction name.

The load instruction has been used mostly to designate a transfer from memory to a processor register, usually an accumulator. The store instruction designates a transfer from a processor register into memory. The move instruction has been used in computers with multiple CPU registers to designate a transfer from one register to another. It has been also used from data transfer between CPU register and memory or between two memory words. The exchanging instruction swaps informations between two register of a register and a memory word. The input and output instructions transfer data among processor register and input terminals. The push and pop instructions transfer data between processor register and a memory stack.

It must be realized that the instructions listed in Table 1, as well as in the subsequential tables in the section, are often associated with a variety of addressing modes. Some assembly-language conventions modify the mnemonic symbol to differentiate between the different addressing modes. For example, the mnemonic for the load immediate becomes LDI. Other assembly language conventions use a special character to designate the addressing mode. For example, the immediate mode is recognized from a pound sign # placed before the operand.

In any case the important thing is to realize that each instruction can occur with a variety of addressing modes. As an example, consider modes the load to accumulator instruction when used with eight different addressing modes. Table 2 shows the recommended assembly-language convention and the actual transfer accomplished in each case. ADR stands for an address,

Table 2 Eight Addressing Modes for the Load Instruction

Mode	Assembly Convention	Register Transfer
Direct address	LD ADR	$AC \leftarrow M[ADR]$
Indirect address	LD @ADR	$AC \leftarrow M[M[ADR]]$
Relative address	LD \$ADR	$AC \leftarrow M[PC+ADR]$
Immediate operand	LD #NBR	$AC \leftarrow NBR$
Index address	LD ADR(X)	$AC \leftarrow M[ADR+ER]$
Register	LD R1	$AC \leftarrow R1$
Register Indirect	LD (R1)	$AC \leftarrow M[R1]$
Autoincrement	LD (R1)+	$AC \leftarrow M[R1] \quad R1 \leftarrow R1+1$

NBR is a number or operand X is an index register, R1 is a processor register and AC is accumulator register. The @ character symbols a direct address. The \$ character before an address makes the address relative to the program counter PC. The # character precedes the operand in an immediate-mode is symbolized by giving the name of a processor register.

In the register-indirect mode the name or register that hold the memory address is enclosed in parenthesis. The autoincrement mode is distinguished from the register-indirect mode by placing in plus after the parenthesized register. The autodecrement mode would use a minus instead. To be able to write assembly-language programs for a computer, it is necessary to know the type of instructions available and also to be familiar with the addressing mode used in the particular computer.

19.4 Data Manipulation Instructions

Data manipulation instructions perform operations data and provide the computational capabilities for the computer. The data manipulation instruction in a typical computer are usually divided into three basic types.

1. Arithmetic instructions.
2. Logical and bit manipulation instructions.
3. Shift instructions.

A list of data manipulation instruction will look very much like list of micro operation. If must be realized, however, that each instruction when executed in the computer must go through the fetch cycle to read its binary code value from memory. The operands must also be brought into processor registers according to the rules of introduction addressing mode. The last step is to execute the instruction in the processor. This last step is implemented by means of micro-operations or through an ALU and shifter. Some of the arithmetic instructions need special circuits for their implementation.

19.5 Arithmetic Instructions

The four basic arithmetic operations are addition, subtraction, multiplication, and division. Most computers provide instructions for all four operations. Some small computers have only addition and possibly subtraction instructions. The multiplication and division must then be generated by means of software subroutines. The four basic arithmetic operations are sufficient for formulating solutions to scientific problems when expressed in terms of numerical analysis methods.

A list of typical arithmetic instructions is given in Table 3 the increment.

Table 3: Typical Arithmetic Instructions

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB

Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's complement)	NEG

The increment instruction adds 1 to the value stored in a register or memory word. One common characteristic of the increment operations when executed in processor registers is that a binary number of all 1's when incremented produces a result of all 0s. The decrement instruction subtracts 1 from a value stored in a register or memory word. A number with all 1s when decremented, produces a number with all 0s.

The add, subtract, multiply, and divide instructions may be available for different types of data. The data type assumed to be in processor registers during the execution of these arithmetic operations is included in the definition of the operation code. An arithmetic instruction may specify fixed-point or floating-point data binary or decimal data, single-precision or double precision data.

It is not uncommon to find computers with three or more add instructions; one for binary integers, one for floating point operands, and one for decimal operands. The mnemonics for three add instructions that specify different data types are shown below.

ADDI	Add two binary integer numbers
ADDF	Add two floating-point numbers
ADDD	Add two decimal numbers in BCD.

The number of bits in any register is of finite length and therefore the results of arithmetic operations are of finite precision. Some computers provide hardware double-precision operations where the length of each operand is taken to be the length of two memory words. Most small computers provide special instructions to facilitate double-precision arithmetic. A special carry flip-flop is used to store the carry from an operation. The instruction add with carry performs the addition on two operands plus the value of the carry from the previous computations. Similarly, the subtract with borrow instruction subtracts two words and a borrow which may have resulted from a previous subtract operation. The negate instruction forms the 2's complement of a

number effectively reversing the sign of an integer with represented in the signed – 2's complement form.

19.6 Logical and Bit Manipulation Instruction

Logical instructions perform binary operations on strings of bits stored in registers. They are useful for manipulating individual bits or a group of bits that represent binary-coded information. The logical instructions consider each bit of the operand separately and treat it as a Boolean variable. By proper application of the logical instructions it is possible to change bit values, to clear a group of bits, or to insert new bit values into operands stored in registers of memory words.

Some typical logical and bit manipulation instructions are listed in Table.4.

Table : 4 Typical Logical and Bit Manipulation Instructions

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive-OR	XOR
Clear carry	CLRC
Set carry	SETC
Complement carry	COMC
Enable	interruptEI
Disable	interruptDI

The clear instructions causes the specific operand to be replaced by 0's. The complement instruction produces the 1's complement by inverting the bits of the operand. The AND, OR and XOR instructions produce the corresponding logical operations on individual of the operands. Although they perform Boolean operations, when used in computer instructions, the logical instructions should be considered as performing bit manipulation operations. There are three bit manipulation operations possible a selected bit can be cleared to 0, or can be set to 1, or can be complemented. Three logical instructions are usually applied to do just that.

The AND instruction is used to clear a bit or a selected group or bits of an operand. For any Boolean variable x , the relationships $x.0 = 0$ and $x.1 = x$ dictate that a binary variable ANDed with 0 produces a 0; but the variable does not change in value when ANDed with a 1. Therefore, the AND instruction can be used to selectively clear bits of an operand by Anding the operand with another operand that has 0's in the bit positions that must be clear. The AND instruction is also called a mask because it marks or inserts 0's in a selected portion of an operand.

The OR instruction is used to set a bit or selected group of bits of an operand. For any Boolean variable x , the relationship $x+1=1$ and $x + 0 =x$ dictate that a binary variables ORed with a 1 produces a 1; but the variables does not change when ORed with a 0. Therefore, the OR instruction can be used to selectively set bits of an operand by ORing it with another operand with 1's in the bit positions that must be set to 1.

Similarly, the XOR instruction is used to selectively complement bits of an operand. This is because of the Boolean relationships, $X+1 = 1$, and $x+0=x$. Thus, a binary variable is complemented when XORed with a 1 but does not change in value when XORed with 0.

A few other bit manipulation instructions are included in Table 4 individual bits such as a carry can be cleared, set, or complemented when XORed with 1 but does not change in value when XORed with at 0.

A few other bit manipulations, instructions are included in Table 5 individual bits such as a carry can be cleared, set, or complemented with appropriate instructions. Another example is a flipt-flop that controls the interrupt facility and is either enabled or disabled by means of bit manipulation instructions.

19.7 Shift Instructions

Instructions to shift the content of an operand are quite useful and are often provided in several variations, shifts are operations in which the bits of a words are moved to the left or right. The bit shifted in at the end of the word determines the type of shift used. Shift interactions may specify either logical shifts arithmetic shifts, or rotate-type operations. In either case the shift may be to the right or to the left.

Table 5 lists four types of shift instructions. The logical shift inserts 0 to the end bit of position. The end position is the left most bit for shift right and the right most bit position for the

shift left. Arithmetic shifts usually conform with the rules of signed, Z's complement numbers. These rules are given in Table 4-6. The arithmetic shift-right instruction must preserve the sign bit in the left-most position. The sign bit is shifted to the right together with the rest of the number, but the sign bit itself remains unchanged. This is a shift-right instruction, inserts 0 to the end position and is identical to the logical shift instruction. For this reason many computers do not provide a distinct arithmetic shift-left instruction when the logical shift-left instruction is already available.

Table 5 : Typical Shift Instructions

Name	Mnemonic
Logical Shift right	SHR
Logical Shift left	SHL
Arithmetic shift right	SHRA
Arithmetic Shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

The rotate instructions produce a circular shift. Bits shifted out at one end of the word are not lost as in logical shift but are circulated back into the other end. The rotate through carry instruction treats carry bit as an extension of the register whose word is being rotated. Thus, a rotate left through carry instruction transfers the carry bit into the right-most bit position of the register, transfers the leftmost bit position into the carry, and at the same time shifts the entire register to the left.

Some computers have a multiple-field format for the shift instructions. One field contains the operation code and the others specify the type of shift and the number of times that an operand is to be shifted. A possible instruction code format of a shift instruction may include five fields as follows :

OP REG TYPE RL COUNT

Here OP is the operation code field; REG is a register address that specifies the location of the operand; TYPE is a 2-bit field specifying the four different types of shifts; RL is a 1-bit field specifying shift right or left; and COUNT is a k-bit specifying up to $k-1$ shifts. With such a format it is possible to specify the type of shift, the direction, and the number of shifts, all in one instruction.

19.8 Program Control

Instructions are always stored in successive memory locations. When processed in the CPU, the instructions are fetched from consecutive memory locations and executed. Each time an instruction is fetched from memory, the program counter is incremented so that it contains the address of the next instruction in sequence. After the execution of a data transfer or data manipulation instruction, control returns to the fetch cycle with the program counter containing the address of the instruction, next in sequence. On the other hand, a program control-type instruction, when executed, may change the address value in the program counter and cause the flow of control to be altered. In other words, program control instruction specify conditions for altering the content of the program counter, while data transfer and manipulation instruction specify conditions for data processing operation. The change in value of the program counter as result of the execution of a program control instruction causes a break in the sequence of instruction execution. This is an important feature in digital computers, as it provides control over the flow of program, execution and a capability for branching to different program segments.

Some typical program control instructions listed in Table 6. The branch and jump instructions are used interchangeably to mean the same thing, but sometimes they are used to denote different addressing modes. The branch is usually a one-address instruction. It is written

Table 6 Typical Program Control Instruction

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
Call	CALL

Return	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TST

in assembly language as BR ADR, where ADR is symbolic name for an address. When executed, the branch instruction causes a transfer of the value of ADR into the program counter. Since the program counter contains the address of the instruction to be executed, the next instruction will come from location ADR.

Branch and jump instruction may be conditional or unconditional. An unconditional branch instruction causes branch to the specified address without any conditions. The conditional branch instruction specifies a condition for the branch such as branch if positive or branch if zero. If the condition is met, the program counter is loaded with the branch address and the next instruction is taken from this address. If the condition is not met, the program counter is not changed and the next instruction is taken from the next location in sequence.

The skip instruction does not need an address field and is, therefore, a zero-address instruction. A conditional skip instruction will skip the next instruction if the condition is met. This is accomplished by incrementing the program counter during the executive cycle in addition to its being incremented during the fetch cycle. If the condition is not met, control proceeds with the next instruction in sequence where the programmer inserts an unconditional branch instruction. Thus, a skip branch pair of instructions causes a branch if the condition is not met, while a single conditional branch instruction causes a branch if the condition is met.

The call and return instructions are used in conjunction with subroutines. Their performance and implementation is discussed later in this section. The compare and test instructions do not change the program sequence directly. They are listed in Table 6 because of their application in setting conditions for subsequent conditional branch instructions. The compare instruction performs a subtraction between two operands but the result of the operation is not retained. However, certain status bit conditions are set as a result of the operation. Similarly, the test instruction performs the logical AND of two operands and updates certain status bits without retaining the results or changing the operands. The status bits of interest are the carry bit, the sign, a zero indication, and an overflow condition. The generation of these status bits will be discussed first and then we will show how they are used in conditional branch instruction.

19.9 Status Bit Conditions

It is sometimes convenient to supplement the ALU circuit in the CPU with a status register where status bit conditions can be stored for further analysis. Status bits are also called condition code bits or flag bits. Figure 1 shows the block diagram of an 8 bit ALU with a 4-bit status register. The four status bits are symbolized by C,S,Z and V. The bits are set or cleared as a result of an operation performed in the ALU.

1. Bit C is set if the output carry of the ALU is 1. It is cleared if the output carry is 0.
2. Bit S is set if the highest – order bit of the result in the output the ALU (the sign bit) is 1. It is cleared if the highest-order bit is 0.
3. Bit Z is set if the output of the ALU contains all 0's and cleared otherwise Z = 1 if the result is zero, and Z=0 if the result nonzero.
4. Bit V is set if the exclusive – OR of C and C is 1cleared otherwise. This is the condition for overflow when the numbers are in signed 2's complement representation. For the 8-bit ALU, V is set if the result is greater than 127 or less than -128.

The status bit can be checked after an ALU operation to determine certain relationships that exist between the values of A and B. If bit V is set after the addition of two signed numbers, it indicates an overflow condition. If z is set after an exclusive OR operation, it indicates A = B. This is so because $X \oplus X = 0$, and the exclusive – OR of two equal operands given an all – 0's result which sets the Z bit. A single bit in A can be checked to determine if it is 0 or 1 by masking all bits except the bit in question and then checking the Z status bit. For example, let A = 101x 100 Where x is the bit to be checked. The AND operation of A with B = 00010000 produces a result 000 x 0000. If x = 0, the Z status bit is set, but if X = 1, the Z bit cleared since the result is not zero. The AND operation can be generated with the TEST instruction listed in Table 6 if the original content of A must be preserved.

19.10 Conditional Branch Instructions

Table 7 gives a list of the most common branch instruction. Each mnemonic is constructed with the letter B (for branch) and an abbreviation of the condition name. When the opposite condition state is used, the letter N (for no) is inserted to define the o state. Thus, BC is branch on Carry and BNC is Branch on No Carry. If the stated condition is true, then program control is

transferred to the address specified by the instruction. If not, control continues with the instruction that follows. The conditional instructions can be associated also with the jump, skip, call, or return type of program control instructions.

The zero status bit is used for testing if the result of an ALU operation is equal to zero or not. The carry bit is used to check if there is a carry out of the most significant bit position of the ALU. It is also used in conjunction with the rotate instructions to check the bit shifted from the end position of a register into the carry position. The sign bit reflects the state of the most significant bit of the output from the ALU. $S = 0$ denotes a positive sign and $S = 1$ a negative sign. Therefore, a branch on

Table 7 : Conditional Branch Instructions

Minemonic	Branch Condition	Tested Condition
BZ	Branch if zero	$Z = 1$
BNZ	Branch if not zero	$Z = 0$
BC	Branch if carry	$C = 1$
BNC	Branch if not carry	$C = 0$
BP	Branch if plus	$S = 0$
BM	Branch if minus	$S = 1$
BV	Branch if overflow	$V = 1$
BNV	Branch if not overflow	$V = 0$
Unsigned Compare Conditions (A-B)		
BHI	Branch if higher	$A > B$
BHE	Branch if higher or equal	$A \geq B$
BLO	Branch if lower	$A < B$
BLOE	Branch if lower or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$

Signed Compare Condition (A-B)

BGT	Branch if greater than	$A > B$
BGE	Branch if greater or equal	$A \geq B$
BLT	Branch if less than	$A < B$
BLE	Branch if less or equal	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$

Plus checks for a sign bit of 0 and branch on minus checks for a sign bit of 1. It must be realized however, that these two conditional branch instructions can be used to check the value of the most significant bit whether it represents a sign or not. The overflow is used in conjunction with arithmetic operations done on signed numbers in 2's complement representation.

As stated previously, the compare instruction performs a subtraction of two operands, say A-B. The result of the operation is not transferred into a destination register, but the status bits are affected. The status register provides information about the relative magnitude of A and B. Some computers provide conditional branch instructions that can be applied right after the execution of a compare instruction. The specific conditions to be tested depend on whether the two numbers A and B are considered to be unsigned or signed numbers. Table 7 gives a list of such conditional branch instruction. Note that we use the words higher and lower to denote the relations between unsigned numbers, and greater and less than for signed numbers. The relative magnitude shown under the tested condition column in the table seems to be the same for unsigned and signed numbers. However, this is not case since each must be considered separately as explained in the following numerical example.

Consider an 8-bit ALU as shown in Fig. 1. The largest unsigned number than can be accommodated 8 bits is 255. The range of signed numbers is between + 127 and - 128. The subtraction of two numbers is the same whether they are unsigned or in signed 2's complement representation. Let $A = 11110000$ and $B = 00010100$. To perform A-B, the ALU takes the 2's complement of B and adds it to A.

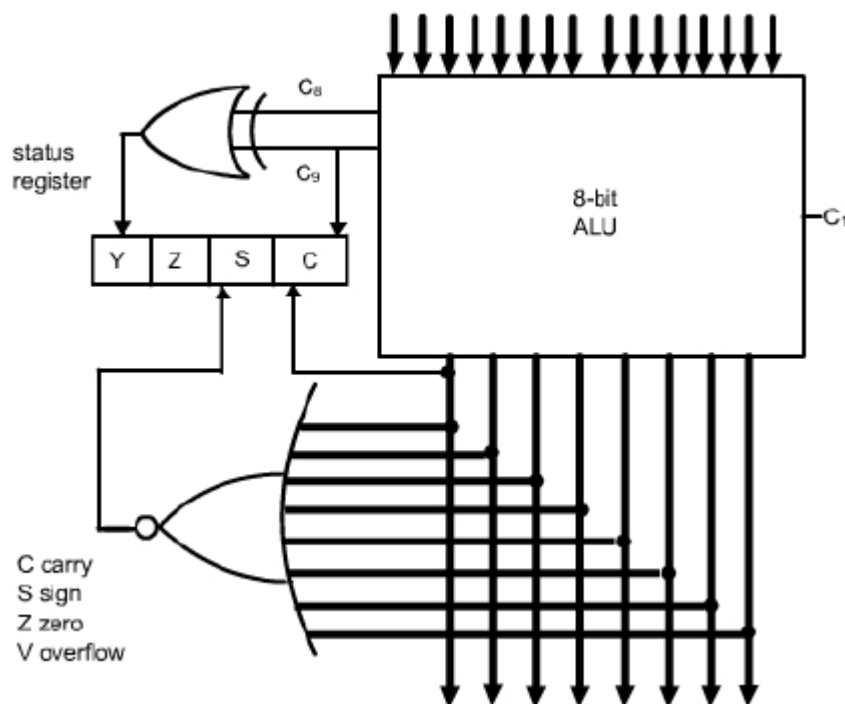


Fig.1 Setting bits in status register

A	:	11110000	
B+1	:	+ 11101100	
A = B	:	11011100	C = 1 S = 1 V = 0 Z = 0

The compare instruction updates the status bits as shown. C=1 because there is a carry out of the last stage. S = 1 because the left – most bit is 1. V = 0 because the last two carries are both equal to 1, and Z = 0 because the result is not equal to 0.

If we assume unsigned numbers, then the decimal equivalent of A is 240 and that of B is 20. The subtraction in decimal is $240 - 20 = 220$. The binary result 11011100 is indeed the equivalent of decimal 220. Since $240 > 20$, we have that $A > B$ and $A \neq B$. These two relations can also be derived from the fact that status bit C equal to 1 and Z is equal to 0. The instructions that will cause a branch after this comparison are BHI (branch if higher), BHE (branch if higher or equal), and BNE (branch if not equal).

If we assume signed numbers, then the decimal equivalent of A is – 16. This is because the sign of A is negative and 11110000 is the 2's complement of 00010000, which is the decimal equivalent of + 6. The decimal equivalent of B is + 20. The subtraction in decimal is $(-16) (+20) = - 36$. The binary result 11011100 (the 2's complement of 00100100) is indeed the equivalent of decimal 36. Since $(-16) (+20)$ we have the $A < B$ and $A \neq B$. These two relations can be derived also from the fact that status bits $S = 1$ (negative) $V = 0$ (no overflow) and $Z = 0$ (not zero) The instructions that will cause a branch after this comparison are BLT (branch if less than) BLE (branch if less or equal) and BNE (branch if not equal)

It should be noted that the instruction BNE and BNZ (branch if not zero) are identical. Similarly the two instructions BE (branch if equal) and BZ (branch if zero) are also identical. Each is repeated three times in Table 7 for the purpose of clarity and completeness.

It should be obvious from the example that the relative magnitude of two unsigned numbers can be determined (after a compare instruction) from the value of status bits C and Z. the relative magnitude of two signed numbers can be determined from the values of S, V and Z.

Some computers consider the C bit to be a borrow bit after a subtraction operation $A-B$. A borrow does not occur if $A > B$. but a bit must be borrowed from the next most significant position if $A < B$. The condition for borrow is the complement of the carry obtained when the subtraction is done by taking the 2's complement of B. For this reason, a processor that considers the C bit to be a borrow after a subtraction will complement the C bit after adding the 2's complement of the subtrahend and denote this bit a borrow.

19.11 Subroutine Call and Return

A subordinate is a self-contained sequence of instructions that performs a given computational task. During the execution of a program, a subordinate may be called to perform its function many times at various points in the main program. Each time a subroutine is called, a branch is executed to the beginning of the subroutine to start executing its set of instruction. After the subroutine has been executed, a branch is made back to the main program.

The instruction that transfers program control to a subroutine is known by different names. The most common names used are call subroutine, jump to subroutine, branch to subroutine, or branch and save address. A call subroutine instruction consists of an operation code together with an address that specifies the beginning of the subroutine. The instruction is executed by performing two operations (1) the address of the next instruction available in the

program counter (the return address) is stored in a temporary location so the subroutine knows where to return, and (2) control is transferred to the beginning of the subroutine. The last instruction of every subroutine, commonly called return from subroutine, transfers the return address from the temporary location into the program counter. This results in a transfer of program control of the instruction whose address was originally stored in the temporary location.

Different computers use a different temporary location for storing the return address. Some store the return address in the first memory location of the subroutine, some store it in a fix location in memory. Some store it in a processor register, and some store it in a memory stack. The most efficient way is to store the return address in a memory stack. The advantage of using a stack for the return address is that when a succession of subroutines is called, the sequential return addresses can be pushed into the stack. The return from subroutine instruction causes the stack to pop and the contents of the top of the stack transferred to the program counter. In this way, the return is always to the program which last called a subroutine. A subroutine call is implemented with the following micro-operation.

$SP \leftarrow SP + 1$	Increment stack Pointer
$M(SP) \leftarrow PC$	Push content of PC onto the stack
$PC \leftarrow \text{Effective addresses}$	Transfer control to the subrout.

If another subroutine is called by the current subroutine, the new return address is pushed into the stack, and so on. The instruction that returns from the last subroutine is implemented by the micro-operations.

$PC \leftarrow M(SP)$	Pop stack and transfer to PC
$SP \leftarrow SP - 1$	Decrement stack pointer.

By using a subroutine a stack, all return address are automatically stored by the hardware in one unit. The programmer does not have to be concerned or remember where the return address was stored.

A recursive subroutines is a subroutine that calls itself, if only one register or memory location is used to store the return address, and the recursive subroutine calls itself, it destroys the previous return address. This is undesirable because vital information is destroyed. This problem can be solved if different storage locations are employed for each use of the subroutine

while another higher level use is still active. When a stack is used, each return address can be pushed into the stack without destroying any previous values. This solves the problem of recursive subroutines because the next subroutine to exit is always the best subroutine that was called.

19.12 Program Interrupt

The concept of program interrupt used to handle a variety of problems which arise out of normal program sequence. Program interrupt refers to the transfers of program control from a currently running program to another service program as a result of an external or internal generated request. Control returns to the original program after the service program is executed.

The interrupt procedure is, in principle, quite similar to a subroutine call except for three variations. (1) The interrupt is usually initiated by a internal or external signal rather than from the execution of an instruction.

(2) The address of the interrupt service program is determined by the hardware rather than from the address field of an instruction. (3) An interrupt procedure usually stores all the information necessary to define the state of the CPU rather than storing only the program counter. These three procedural concepts are clarified further below.

After program has been interrupted and the service routine been executed, the CPU must return to exactly the same state that it was when the interrupt occurred. Only if this happens will the interrupted program be able to resume exactly as if nothing had happened. The state of the CPU at the end of the execute cycle (when the interrupt is recognized) is determined from:

1. The content of the program counter,
2. The content of all processor register,
3. The content of certain status conditions.

The collection of all status bit conditions in the CPU is sometimes called a program status word or PSW. The PSW is stored in a separate hardware register and contains the status information that characterizes the state of the CPU. Typically, it includes the status bit from the last ALU operation, it specifies the interrupts that are allowed to occur, and whether the CPU is operating in a supervisor or user mode. Many computers have a resident operating system that controls and supervises all other programs in the computer. When the CPU is executing a program which is part of the operating system, it is said to be in the supervisor or system mode.

Certain instructions are privileged and can be executed in this mode only. The CPU is normally in the user mode when executing user programs. The mode that the CPU is operating at any given time is determined from special status bits in the PSW.

Some computers store only the program counter when responding to an interrupt. Their service program must then include instructions to store status and register content before these resources are used. Only few computer store both program counter and all status and register, content in response to an interrupt. Most computers just store the program counter and the PSW. In some cases, there exist, the sets of processor registers within the computer, one for each CPU mode. In this way, when the program switches from the user to the supervisor mode (or vice versa) in response to an interrupt, it is not necessary to store the contents of processor registers as each modes uses its own set of registers.

The hardware procedure for processing an interrupt is very similar to the execution of a subroutine call instruction. The state of the CPU is pushed into a memory stack and the beginning address of the service routine is transferred to the program counter. The beginning address of the service routine is determined by the hardware rather than the address field of an instruction. Some computers assign one memory location where interrupts are always transferred. The service routine must then determine what caused the interrupt and processed to service it. Some computers assign a memory location for each possible interrupt. Sometimes, the hardware interrupt provides its own address that directs the CPU to the desired service routine. In any case, the CPU must possess some from of hardware procedure for selecting a branch address of servicing the interrupt.

The CPU does not respond to an interrupt until the end of an instruction execution. Just before going to the next fetch cycle control checks for any interrupt signals. If an interrupt is pending, control goes to hardware interrupt cycle. During this cycle, the contents of PC and PSW are pushed onto the stack. The branch address for the particular interrupt is then transferred to PC and a new PSW is loaded into the status register. The service program can now be executed starting from the branch address and having a CPU mode as specified in the new PSW.

The last instruction in the service program is a return from interrupt instruction. When this instruction is executed, the stack is popped to retrieve the old PSW and the return address. The PSW is transferred to the status register and the return address to the program counter. Thus, the CPU state is restored and the original program can continue executing.

19.13 Summary

Data Transfer Instructions

These instructions are used to transfer the data from the source operand to the destination operand.

Arithmetic Instructions

These instructions are used to perform arithmetic operations like addition, subtraction, multiplication, division, etc.

Logic Instruction: These instructions are used to perform Logic operations like AND, OR, NOT and EX-OR.

19.14 Model Questions

1. Explain any three data transfer instructions in a computer.
2. Explain any three branch instructions in a computer.

LESSON - 20

CENTRAL PROCESSING UNIT (CPU)

Structure

- 20.0 Objectives**
- 20.1 Introduction**
- 20.2 CPU**
- 20.3 Summary**
- 20.4 Model Questions**

20.0 Objectives

To introduce :

- The general architecture of a CPU.

20.1 Introduction

In this chapter we focus Central Processing Unit. CPU is considered as the brain of the computer. CPU performs all types of data processing operations.

20.2 CPU

Actually the term “Central Processing Unit” is a carry over from vacuum tube days. At that time, all arithmetic and control circuits were housed in a centrally located cabinet, cables between CPU and other units ran under the floor. The cables were independent, using the private telephone the system. Independent cables allow simultaneous data transfer. IC computers use the single-bus party line, or time sharing, to connect data lines time sharing reduces complexity, but only two units can communicate at any given time. Computers with a common data line have single bus architecture. The word “single” refers only to data transmission; address and control buses are also required for single bus architecture.

Processing data requires communication CPU and RAM. Information is stored in RAM but processed in the CPU, and therefore instructions and data must be transferred from RAM to the CPU. After being processed, data move in the reverse direction from CPU to RAM along the

bidirectional data bus. During processing control signals generated in the CPU enable the RAM and select the read mode. Similarly, storing results generated in the CPU requires enabling the RAM in the write mode. During read, data enter the CPU through the data register, and during write, data leave the CPU through the same data register. The data register is thus a buffer between data bus and CPU.

Figure –1 shows the decoding of RAM address A7H. If the RAM is in the read mode, the 02H, is transferred along the data bus into the data register. Similarly, if the RAM is in the write mode, then 02H is transferred from the data register along the data bus to RAM address A7H. The information, 02H may be data or an instruction.

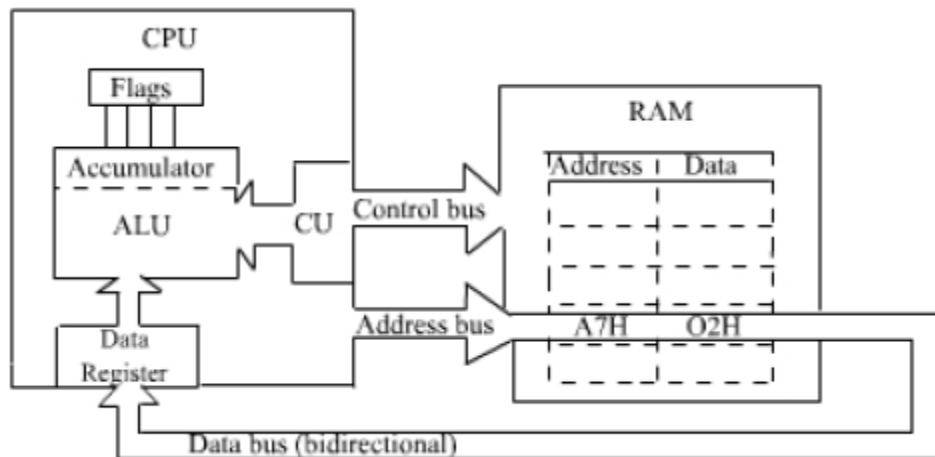


Fig. 1: CPU-RAM Interaction

Figure – 1 also shows flags condition code register connected to the accumulator. Each condition register is a flip-flop which monitors a different accumulators state. One flip-flop connected to the accumulator MSB. When this flip-flop is set the signed number is negative, conversely, if it is reset, the signal number is positive. Another flag tests the C_0 bit for overflow. If the C_0 flag is set, operations are discontinued.

The remaining flags tests other accumulator conditions. Each flag status has a different effect depending on the operations example, the MSB sign flag is important in arithmetic operations. However, when the accumulator word is an alphabetic or special character, the MSB does not indicate sign.

A more detailed picture of CPU architecture is shown in Fig.2 Two more registers, a program counter and an address register, are included. A program counter initiates the entire sequence of operation and is reset prior to use. Thus the program counter at 00_H when processing begins. This count is transferred into the address register, and the address register output is the address bus. Therefore 00_H is the first decoded address when RAM is accessed. The first user instruction is in location, 00_H and this word is transferred from RAM to data register along the data bus. Next, the data register transfers this word into the instruction decoder part of the control unit, which interprets the word. Then the control unit generates control signals for performing the instruction, such as addition, subtraction, or data transfer. The control signals are transmitted on the control bus, and information is transferred on the data bus.

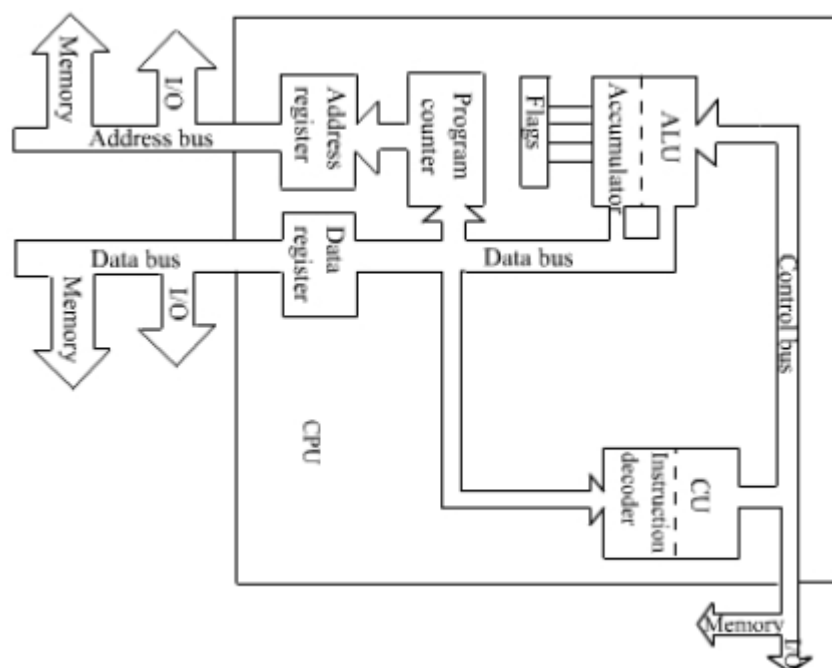


Figure 2 : Typical CPU architecture

While an instruction is decoded, the program counter is incremented to 01_H . Typically, location 01_H contains data pertaining to the instruction in 00_H . When the address bus is again enabled, the data in location 01_H are transferred from RAM to data register. Next, the appropriate control signals transfer the data register contents into the accumulator. When the program counter is stepped to 02_H , the next instruction is obtained. Processing continues, one location at a time, until the desired results are obtained.

Operation of the program counter, address bus, and data register in conjunction with the ALU and control unit is not a new concept. It was originally used in vacuum tube computers, and the basic procedure is still the same. User instructions and data are processed by repeating similar steps until results are obtained. Each step involves an instruction and data; first the instruction is obtained, and then the accompanying data is processed. This procedure is called fetch-execute. Several events occur during each fetch and execute. During fetch, an instruction is transferred from RAM to CPU, the instruction is then interpreted, and necessary control signals are generated. The program counter is also stepped during fetch. The fetch operation is performed in one machine cycle. Machine cycle time is constant for any given computer.

When fetch is completed, execute begins. During execute the data is obtained and processed. While fetch time always requires one machine cycle, execute time depends on instruction type. The time to complete a fetch execute sequence is called an instruction cycle. Instruction cycles vary in duration. But always require an integer multiple of the basic machine cycle. Machine cycle period is determined by chip type and clock frequency. Typical clock frequencies for byte-organized MOS CPUs are of the order of microseconds, e.g. 2-MHz clock results in 0.5 μ s machine cycle.

20.3 Summary

CPU is considered as the brain of the computer.

CPU performs all types of data processing operations.

It stores data, intermediate results, and instructions (program).

It controls the operation of all parts of the computer.

20.4 Model Questions

1. Describe the general architecture of a CPU.

LESSON - 21

CONTROL LOGIC DESIGN

STRUCTURE

- 21.0 Objectives**
- 21.1 Introduction**
- 21.2 Control Organization**
- 21.3 Sequence Register and Decoder Method (H/W Control)**
- 21.4 PLA Control**
- 21.5 Microprogram Control**
- 21.6 Summary**
- 21.7 Model Question**

21.0 Objectives

To introduce :

- H/W control
- PLA control
- Micro - program control

21.1 Introduction

The process of logic design is a complex undertaking. The most challenging and creative part of the design is the establishment of design objectives and the formulation of algorithms and procedures for achieving the stated objectives. This task requires a considerable amount of experience and ingenuity on the part of the designer. An algorithm is a procedure for obtaining a solution to a problem. A design algorithm is a procedure for implementing the problem with a given piece of equipment. The development of a design algorithm cannot start until the designer is certain of two things. First, the problems at hand must be thoroughly understood, Second, an initial configuration of equipment must be assumed for implementing the procedure. Starting from the problem statement and equipment availability, a solution is then found and an algorithm formed. The algorithm is stated by a finite number of well-defined procedural steps.

The binary information found in a digital system is stored in processor or memory registers, and it can be either data or control information. Data are discrete elements of information that are manipulated by micro-operations. Control information provides command signals for specifying the sequence of micro-operations. The logic design of a digital system is a process for deriving the digital circuits that provided control that provide control signals.

The timing for all registers in a synchronous digital system is controlled by a master-clock generator. The clock pulses are applied to all flip-flops and registers in the system, including the flip-flops and registers in the control unit. The continuous clock pulses do not change the state of a register unless the register is enabled by a control signal. The binary variables that control the selection variables and enable inputs or registers and generated in the control unit. The outputs of the control unit select and enable the data-processor part of the system and also determine the next state of the control unit itself.

The relationship between the control and the data processor in a digital system is shown in fig. 1. The data processor part may be a general-purpose processor unit, or it may consist of individual registers and associated digital functions. The control initiates all micro-operations in the data processor. The control logic that generates the signals for sequential circuit whose internal states dictate the control functions for the system. At any given time, the state of the sequential control initiates a prescribed set of micro-operations. Depending on status conditions or other inputs, the sequential control goes to the next state to initiate other micro-operations. Thus, the digital circuit that acts as the control logic provides a time sequence of signals for initiating the micro-operation in the data-processor part of the system.

At any given time, the state of the sequential control initiates a prescribed set of micro-operations. Depending on status conditions or other inputs, the sequential control goes to the next state to initiate other micro-operations. Thus, the digital circuit that acts as the control logic provides a time sequence of signals for initiating the micro-operation in the data-processor part of the system.

The design of a digital system that requires a control sequence starts with the assumption of the availability of timing variables. We designate each timing variable in the sequence by a state and then form a state diagram or an equivalent representation for the transition between states. Concurrent with the development of the control sequence, we develop a list of micro-operations to be initiated for each control state. If the system is too complicated for a state

diagram, it may be convenient to specify the system entirely in the register-transfer method by means of control functions and micro-operation statements.

The control sequence and register-transfer relationships may be derived directly from the word specification of the problem. However, it is sometimes convenient to use an intermediate representation to describe the needed sequence of operations for the system. Two representations which are helpful in the design of systems that need a control are timing diagrams and flowcharts.

A timing diagram clarifies the timing sequence and other relationships among the various control signals in the system. In a clocked sequential circuit, the clock pulses synchronize all operations, including signal transitions in control variables; in an asynchronous system, a signal transition in one control variable may cause a change of another control variable. A timing diagram is very useful in asynchronous control because it provides a pictorial representation of required changes and transitions of all control variables.

A flowchart is a convenient way to specify the sequence of procedural steps and decision paths for an algorithm. A flowchart for a design algorithm would normally use the variable names of registers defined in the initial equipment configuration. It translates an algorithm from its word statement to an information-flow diagram that enumerates the sequence of register-transfer operations together with the conditions necessary for their execution.

A flowchart is a diagram that consists of blocks connected by directed lines. Within the blocks, we specify the procedural steps for implementing the algorithm. The directed lines between blocks designate the path to be taken from one procedural step to the next. Two major types of blocks are used: A rectangular block designates a function block within which the micro-operations are listed. A diamond-shaped block is a decision block within which is listed a given status condition. A decision block has two or more alternate paths, and the path that is taken depends on the value of the status condition specified within the block.

A flowchart is very similar to a state diagram. Each function block in the flowchart is equivalent to a state in the diagram. The decision block in the flowchart is equivalent to the binary information written along the directed lines that connect two states in a state diagram. As a consequence, it is sometimes convenient to express an algorithm by means of a flowchart from which the control state diagram may be readily derived.

In this chapter, we first present four possible configurations for a control unit. The various configurations are presented in block diagram form to emphasize the differences in organization. We then demonstrate the various procedures available for control logic design by going through specific examples.

The design of control logic cannot be separated from the algorithmic development necessary for solving a design problem. Moreover, the control logic is directly related to the data-processor part of the system that it controls. As a consequence, the example presented in this chapter starts with the development of an algorithm for implementing the given problem. The data-processing part of the system is then derived from the stated algorithm. Only after this is done can we proceed to show the design of the control that sequences the data processor according to the step specified by the algorithm.

21.2 Control Organization

Once a control sequence has been established, the sequential system that implements the control operations must be designed. Since the control is a sequential circuit it can be designed by a sequential logic procedure. However, in most cases this method is impractical because of the large number of states that the control circuit may have. Design methods that use state and excitation tables can be used in theory, but in practice they are cumbersome and difficult to manage. Moreover the control circuit obtained by this method usually requires an excessive number of flip-flops and gates, which implies the use of SSI circuits. This type of implementation is inefficient, with respect to the number of IC packages used and the number of wires that must be interconnected. One major goal of control logic design should be the development of a circuit that implements the desired control sequence in a logical and straightforward manner.

The attempt to minimize the number of circuits would tend to produce an irregular network which would make it difficult for anyone but the designer to recognize the sequence of events that the control undergoes. As a consequence, it may be difficult to service and maintain the equipment when it is in operation.

Because of the reasons cited above, experienced logic designers use specialized methods for control logic design which may be considered an extension of the classical sequential-logic method combined with the register-transfer method. In this section, we consider three methods of control organization.

1. Sequence register and decoder method.
2. PLA control
3. Micro-program control.

The first method result in a circuit that must use SSI and MSI circuits for the implementation. The various circuits are interconnected by wires to form the control network. A control unit implemented with SSI and MSI devices is said to be a hardwired control. If any alterations or modifications are needed, the circuits must be rewired to fulfill the new requirements. This is in contrast to the PLA or micro-program control which uses as LSI device such as a programmable logic array or a read-only memory. Any alterations or modifications in a micro-program control can be easily achieved without wiring changes by removing the ROM from its socket and inserting another ROM programmed to fulfill the new specifications.

21.3 Sequence Register and Decoder Method (H/W Control)

This method uses a register to sequence the control states. The register is decoded to provide one output for each state. For n flip-flops in the sequence register, the circuit will have 2^n states and the decoder will have 2^n outputs. For example, 4-bit register can be in any one of 16 states. A 4x16 decoder will have 16 outputs, one for each state of the register. Both the sequence register and decoder are MSI devices.

Fig. 2 shows the configuration of a four state sequential control logic. The sequence register has two flip-flops and the decoder establishes separate outputs for each state in the register,. The transition to the next state in the sequence register is a function of the present state and the external input conditions.

Since the outputs of the decoder are available anyway, it is convenient to use them as present-state variables rather than use the direct flip-flop outputs. Other outputs which are a function of the present state and external inputs may initiate micro-operations in addition to the decoder outputs.

21.4 PLA Control

The PLA control is essentially similar to the sequence register and decoder method except that all combinational circuits are implemented withy a PLA, including the decoder and the decision logic. By using a PLA for the combinational circuit, it is possible to reduce the number of lcs and the number of interconnection wires.

Fig. 3 shows the configuration of a PLA controller. An external sequence register establishes the present state of the control circuit. The PLA outputs determine which micro-operations should be initiated, depending on external input conditions and the present state of the sequence register. At the same time, other PLA outputs determine the next state of the sequence register.

The sequence register is external to the PLA if the unit implements only combinational circuits. However, some PLAs are available which include not only gates, but also flip-flops within the unit. This type of PLA can implement a sequential circuit by specifying the links that must be connected to the flip-flops in the same manner that the gate links are specified.

21.5 Microprogram Control

The purpose of the control unit is to initiate a series of sequential steps of micro-operations. During any given time, certain operations are to be initiated while all other remain idle. Thus, the control variables at any given time can be represented by a string of 1's and 0's called a control word. As such, control words can be programmed to initiate the various components in the system in an organized manner.

A control unit whose control variables are stored in a memory is called a micro-programmed control unit. Each control word of memory is called a micro-instruction, and a sequence of micro-instructions is called a micro-program. Since alteration of the micro-program is seldom needed, the control memory can be a ROM. The use of a micro-program involves placing all control variables in words of the ROM for use by the control unit through successive read operations. The content of the word in the ROM at a given address specifies the micro-operations for the system.

A more advanced development known as dynamic micro programming permits a micro-program to be loaded initially from the computer console or from an auxiliary memory such as a magnetic disk. Control units that use dynamic micro programming employ a writable control memory (WCM). This type of memory can be used for writing (to change the micro-program) but is used mostly for reading. A ROM, PLA or WCM, when used in a control unit, is referred to as a control memory.

Figure 4 illustrates the general configuration of the micro-program control unit. The control memory is assumed to be a ROM, within which all control information is permanently stored. The control memory address register specifies the control word read from control memory. It must be realized that a ROM operates as a combinational circuit, with the address value as the

input and the corresponding word as the output. The content of the specified word remains on the output wires as long as the address value remains in the address register. No read signal is needed as in a random-access memory. The word out of the ROM should be transferred to a buffer register if the U address register changes while the ROM word is still in use. If the change in address and ROM word can occur simultaneously, no buffer register is needed.

The word read from control memory represents a micro-instruction. The microinstruction specifies one or more micro-operations for the components of the system. Once these operations are executed, the control unit must determine its next address. The location of the next micro-instruction may be the next one in sequence, or it may be located somewhere else in the control memory. For this reason, it is necessary to use some bits of the micro-instruction to control the generation of the address for the next micro-instruction. The next address may also be a function of external input conditions. While the micro-operations are being executed, the next address is computed in the next-address generator circuit and then transferred (with the next clock pulse) into the control address register to read the next micro-instruction. The detailed construction of the next address generator depends on the particular application.

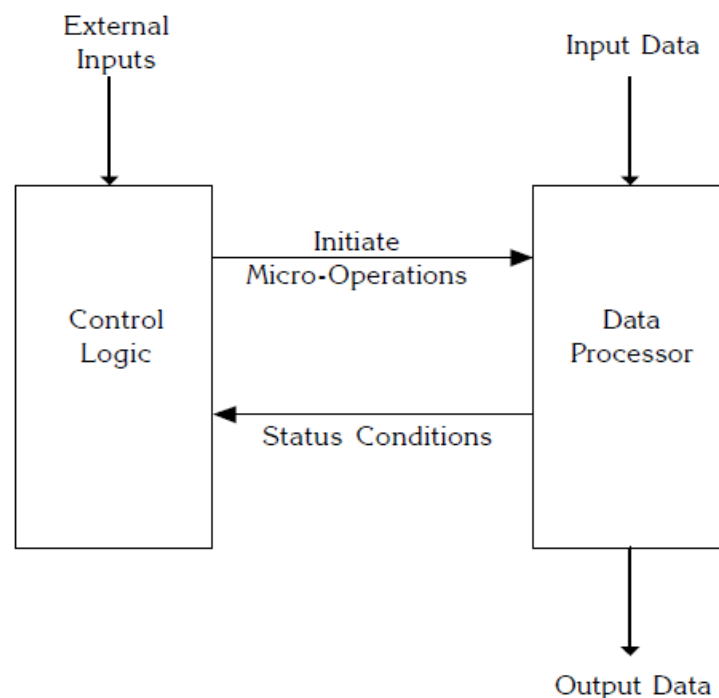


Fig. 1 Control and Data-Processor Interaction

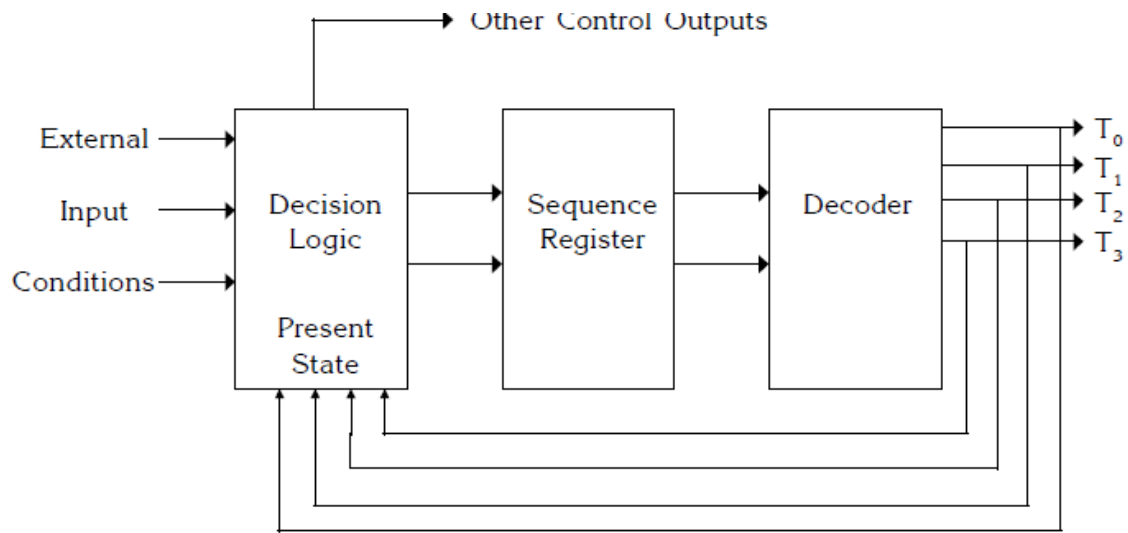


Fig. 2 Control Logic with Sequence Register and Decoder

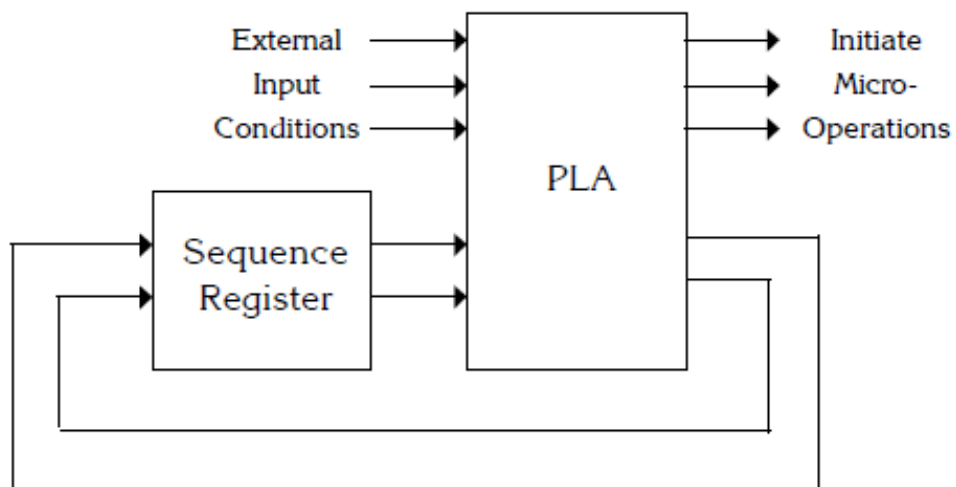


Fig. 3 PLA Control Logic

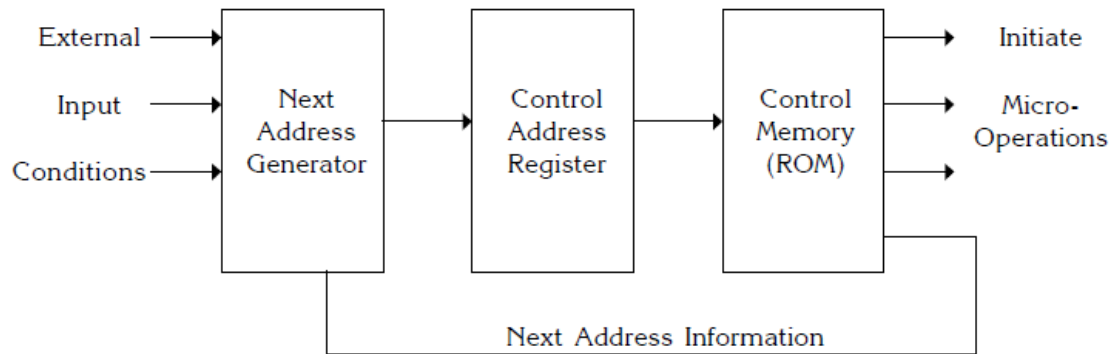


Fig. 4 Microprogram Control Logic

21.6 Summary

Control unit generates timing and control signals for the operations of the computer.

The control unit communicates with ALU and main memory.

It also controls the transmission between processor, memory and the various peripherals.

It also instructs the ALU which operation has to be performed on data.

Microprogrammed Control Unit-A sequence of micro operations is carried out by executing a program consisting of micro-instructions. In this organization any modifications or changes can be done by updating the micro program in the control memory by the programmer.

21.7 Model Question

1. Explain PLA control with neat diagram
2. Discuss about Micro Program Control

LESSON - 22

ARITHMETIC LOGIC UNIT (ALU)

Structure

- 22.0 Objectives**
- 22.1 Introduction**
- 22.2 Design of Arithmetic Circuit**
- 22.3 Effect of Output Carry**
- 22.4 Design of other Arithmetic Circuit**
- 22.5 Design of Logic Circuit**
- 22.6 Design of a Shifter**
- 22.7 Summary**
- 22.8 Model Questions**

22.0 Objectives

To study :

- The design of an arithmetic circuit.
- The design of logic shifter.

22.1 Introduction

The design of typical ALU will be carried out in three stages. First, the design of the arithmetic section will be undertaken. Second, the design of the logic section will be considered. Finally, the arithmetic section will be modified so that it can perform both arithmetic and logic operation. In this chapter, the design of Arithmetic section is explained.

22.2 Design of Arithmetic Circuit

The basic component of the arithmetic section of an ALU is a parallel adder. A parallel adder is considered with a number of full-adder circuits connected in cascade. By controlling the data inputs to the parallel adder, it is possible to obtain different types of arithmetic operations. Figure, 1 demonstrates the arithmetic operations obtained when one set of inputs to a parallel

adder is controlled externally. The number of bits in the parallel adder may be of any value. The input carry C_{in} goes to the full-adder circuit in the least significant bit position. The output carry C_{out} comes from the full-adder circuit in the most significant bit position.

The arithmetic addition is achieved when one set of inputs receives a binary is maintained at 0. This is show in Fig. 1(a). By making $C_{in} = 1$ as in Fig. (b) it is possible to add 1 to the sum in F. Now consider the effect of complementing all the bits of input B. With $C_{in} = 0$, the output produces $F = A + B$ which is the sum of A plus the 1's complement of B. Adding 1 to this sum by making $C_{in} = 1$ we obtain $F = A + B + 1$ which produces the sum of A plus the 2's complement of B. This operation similar to subtraction operation if the output carry is discarded. If we force all 0's into the B terminals, we obtain $F = A + 0$ which transfers input A into output F. Adding 1 through C_{in} as in Fig 1(f) we obtain $F = A + 1$ which is the increment operation.

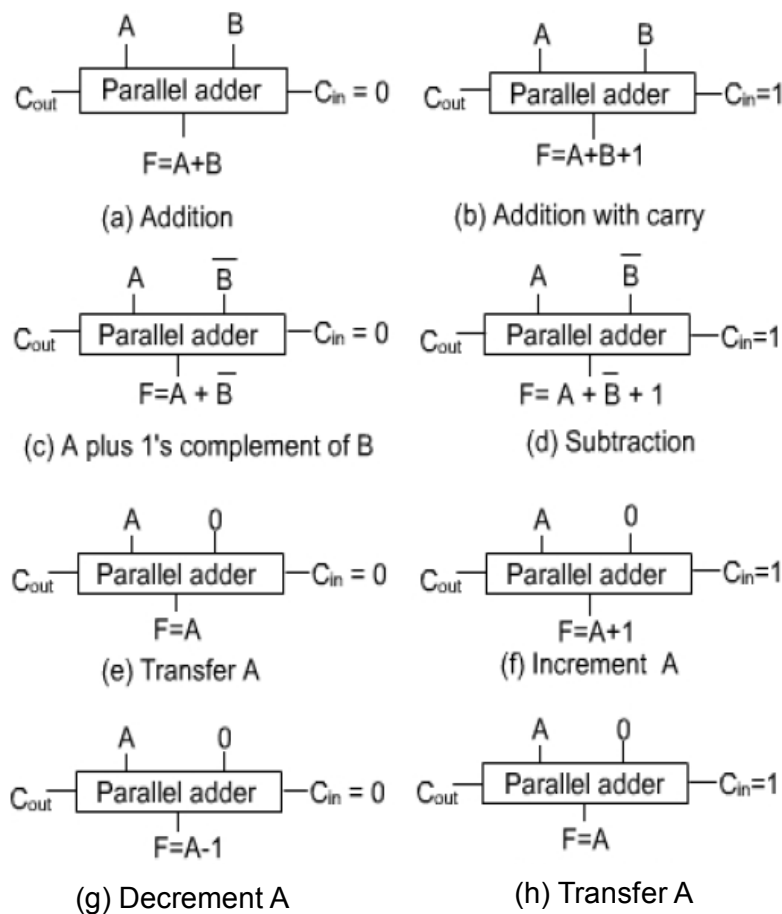


Fig. 1 Operations obtained by controlling one set of inputs to a parallel adder

The condition illustrated in Fig 1 (g) inserts all 1's into the B terminals. This produces the decrement operation $F = A - 1$. To show that this condition is indeed decrement operation. Consider a parallel adder with n full-adder circuits. When the output carry is 1, it represents the number 2^n because 2^n in binary consist of a 1 followed by n 0's. Subtracting 1 from $2^n - 1$ which in binary is a number of n 1's. Adding $2^n - 1$ to A , we obtain $F = A + 2^n - 1 = 2^n + A - 1$. If the output carry 2^n is removed, we obtain $F = A - 1$.

The condition illustrated in Fig 1 (g) inserts all 1's into the B terminals. This produces the decrement operation $F = A - 1$. To show that this condition is indeed decrement operation. Consider a parallel adder with n full-adder circuits. When the output carry is 1, it represents the number 2^n because 2^n in binary consist of a 1 followed by n 0's. Subtracting 1 from $2^n - 1$ which in binary is a number of n 1's. Adding $2^n - 1$ to A , we obtain $F = A + 2^n - 1 = 2^n + A - 1$. If the output carry 2^n is removed, we obtain $F = A - 1$.

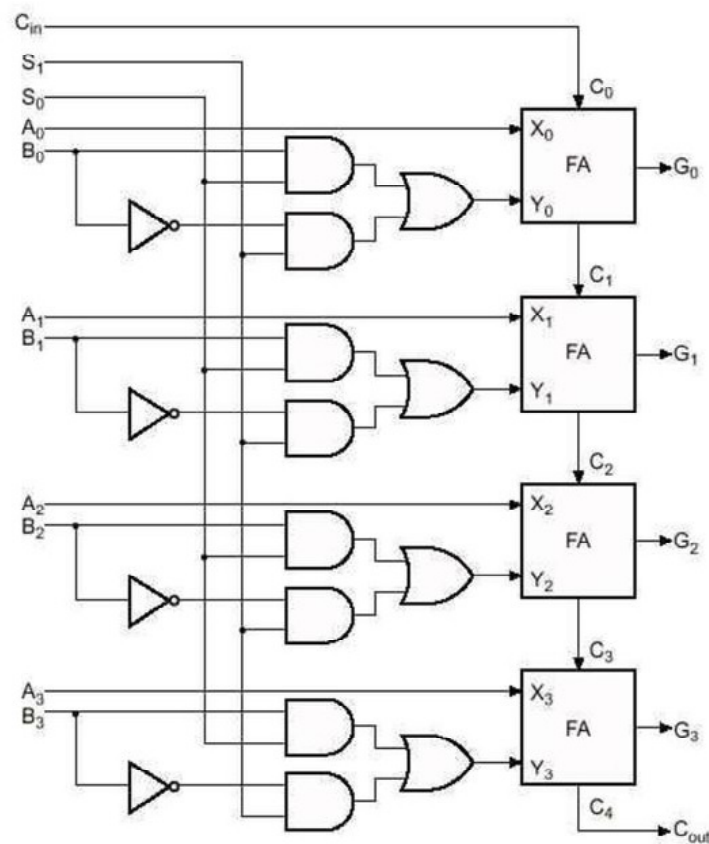


Fig. 2. Logic diagram of arithmetic circuit

The arithmetic operations implemented in the arithmetic circuit are listed in Table-1. The variables of the Y inputs to the full-adder circuits are a function of selection variable S_1 and S_0 . Adding the value of Y in each to the value of A plus the C in valve gives the arithmetic operation in each entry. The eight operations listed in the table follow directly from the function diagrams illustrated in Fig.1

Table-1. Function table for the arithmetic circuit of Fig. 2

Function Select			Y equals	Output equals	Function
S_1	S_0	C_{in}			
0	0	0	0	$F=A$	Transfer A
0	0	1	0	$F = A + 1$	Increment A
0	1	1	B	$F = A + B+1$	Add B to A
0	1	1	B	$F = A + B + 1$	Add B to A plus 1
1	0	0	B	$f = A + B$	Add 1's complement of B to A
1	0	1	B	$F = A + B + 1$	Add 2's complement of B to A
1	1	0	All 1's	$F = A-1$	Decrement A
1	1	1	All 1's	$F = A$	Transfer A

This example demonstrates the feasibility of constructing an arithmetic circuit by means of a parallel adder. The combinational circuit that must be inserted in each stage between the external inputs A_i and B_i and the inputs of the parallel adder X_i and Y_i is a function of the arithmetic operations that are to be implemented. The arithmetic circuit of Fig. 2 needs a combinational circuit in each stage specified by the Boolean functions :

$$X_i = A_i$$

$$Y_i = B_i S_0 + B_i S_1 \quad i = 1, 2, \dots, n$$

Where n is the number of bits in the arithmetic circuit. In each stage i , we use the same common selection variables S_1 and S_0 . The combinational circuit will be different if the circuit generates different arithmetic operations.

22.3 Effect of Output Carry

The output carry of an arithmetic circuit or ALU has special significance, especially after a subtraction operation. To investigate the effect of the output carry, we expand the arithmetic circuit of Fig. 3 n bits so that $C_{out} = 1$ when the output of the circuit is equal to greater than 2^n . Table 2 lists the conditions for having an output carry in the circuit. The function $F = A$ will always have the output carry equal to 0. The same applies to the increment operation $F = A + 1$ except when it goes from an all-1's condition to an all-0's condition, at which time it produces an output carry of 1. An output carry of 1 after an addition operation denotes an overflow condition. It indicates that the sum is greater than or equal to 2^{n+1} and that the sum consists of $n+1$ bits.

Table 2. Effect of output carry in the arithmetic circuit of Fig. 2

Function Select			Arithmetic function	$C_{out} = 1$ if	Comments
S_1	S_0	C_{in}			
0	0	0	$F = A$		C_{out} is always 0
0	0	1	$F = A + 1$	$A = 2^n - 1$	$C_{out} = 1$ and $F = 0$ if $A = 2^n - 1$
0	1	0	$F = A + B$	$(A + B) > 2^n$	Overflow occurs if $C_{out} = 1$
0	1	1	$F = A + B + 1$	$(A + B) > (2^n - 1)$	Overflow occurs if $C_{out} = 1$
1	0	0	$F = A - B - 1$	$A > B$	If $C_{out} = 0$, the $A \nless B$ and $F = 1$'s complement of $(B - A)$

1	0	1	$F = A - N$	$A \oplus B$	If $C_{out} = 0$, then $A < B$ and $F = 2$'s complement of $(B - A)$
1	1	0	$F = A - 1$	$A \oplus 0$	$C_{out} = 1$, except when $A = 0$
1	1	1	$F = A$		C_{out} is always 1

The operation $F = A + B$ adds the 1's complement of B to A . The complement of B can be expressed arithmetically as $n - 1 - B$. the arithmetic result in the output will be :

$$F = A + 2n - 1 - B = 2n + A - B - 1$$

If $A > B$, then $(A - B) > 0$ and $F > (2n - 1)$ so that $C_{out} = 1$ Removing the output carry $2n$ from this result gives:

$$F = A - B - 1$$

Which is subtraction with borrow. Note that if $A < B$, then $(A - B) < 0$

and $F < (2n - 1)$ so that $C_{out} = 0$. For this condition it is more convenient to express the arithmetic result as :

$$F = (2n - 1) - (B - A)$$

which is the 1's complement of $B - A$

The condition for output carry when $F = A + B + 1$ can be derived in a similar manner. $B + 1$ is the symbol for the 2's complement of B . Arithmetically, this is an operation that produces a number equal to $2n - B$. The result of the operation can be expressed as :

$$F = A + 2n - B = 2n + A - B$$

If $A > B$, the $(A - B) > 0$ and $F > 2n$, so that $C_{out} = 1$. Removing the output carry $2n$, we obtain
: $F = A - B$

Which is subtraction operation. If, however, $A < B$, then $(A - B) < 0$

and $F < 2n$, so that $C_{out} = 0$, The arithmetic result for this condition can be expressed as :

$$F = 2n - (B - A)$$

Which is the 2's complement of $B-A$. Thus, the output of the arithmetic subtraction is correct as long as $A > B$. The output should be $B - A$ if $B > A$, but the circuit generates the 2's complement of this number.

The decrement operation is obtained from $F = A + (2^n - 1) = 2^n + A - 1$. The output carry is always 1 except when $A = 0$. Subtracting 1 from 0 gives -1 , and -1 in 2's complement is $2^n - 1$, which is a number with all 1's. The last entry in Table-2 generates $F = (2^n - 1) + A + 1 = 2^n + A$. This operation transfers A into F and gives an output carry of 1.

22.4 Design of Other Arithmetic Circuits

The design of any arithmetic circuit that generates a set of basic operations can be undertaken by following the procedure outlined in the previous example.

22.5 Design Logic Circuit

The logic microoperations manipulate the bits of the operands separately and treat each bit as binary variable. 16 logic operations than can be performed with two binary variables. The 16 logic operations can be generated in one circuit and selected by means of four selection lines. Since all logic operations can be obtained by means of AND, OR, and NOT (complement) operations, it may be more convenient to employ a logic circuit with just these operations. For three operations, we need two selection variables. But two selection lines can select among four logic operations, so we choose also the exclusive – OR (XOR) function for the logic circuit to be designed.

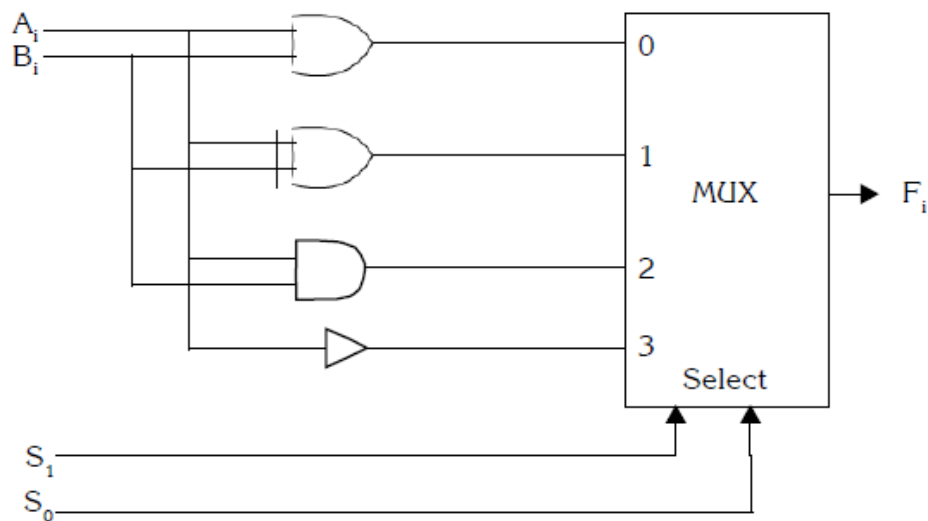


Figure 3 : Design of Logic Circuit

22.6 Design of a Shifter

A logic shifter may transfer the information directly without a shift or may shift the information to the right or left. Precision is sometimes made for no transfer from the ALU to the output bus. The shifter provides the shift micro operations commonly not available in an ALU.

An obvious circuit for a shifter is a bidirectional shift register with parallel load. The information from the ALU can be transferred to the register in parallel and then shifted to the right or left. In this configuration, a clock pulse is needed for the transfer to the shift register, and another pulse is needed for the shift. These two pulses are in addition to the pulse required to transfer the information from the shift register to a destination register.

The transfer from a source register to a destination register can be done with the clock pulse if the shifter is implemented with a combinational circuit. In a combinational logic shifter, the signals from the ALU to the output bus propagate through gates without the need for a clock pulse. Hence, the only clock pulse needed is the processor system is for loading the data from the output bus into the destination register.

A combinational logic shifter can be constructed with multiplexes as shown in (Fig. 22). The two selection variables, H_1 and H_0 applied to all four multiplexers select the type of operation in the shifter. With $H_1H_0 = 00$, no shift is executed and the signals from F go directly to the S lines. The next two selection variables cause a shift right operation and shift left operation. When $H_1H_0 = 11$, the multiplexers select the inputs attached to 0 and as a consequence the S outputs are also equal to 0, blocking the transfer of information from the ALU to the outputs bus. Table 22.1 summarizes the operation of the shifter.

The diagram of the Fig. 5 show only four stages of the shifter. The shifter, of course, must consist of n stages in a system with n parallel lines, Inputs I_R and I_L serve as a serial inputs for the last and first stages during a shift-right or shift-left, respectively. Another selection variable may be employed to specify what goes into I_R or I_L during the shift. For example, a third selection variable, H_2 , when in one state can select a 0 for the serial input during the shift. When H_2 is in the other state, the information can be circulated around together with the value of the carry status bit. In this way, a carry produced during an addition operation can be shifted to the right and into the most significant bit position of a register.

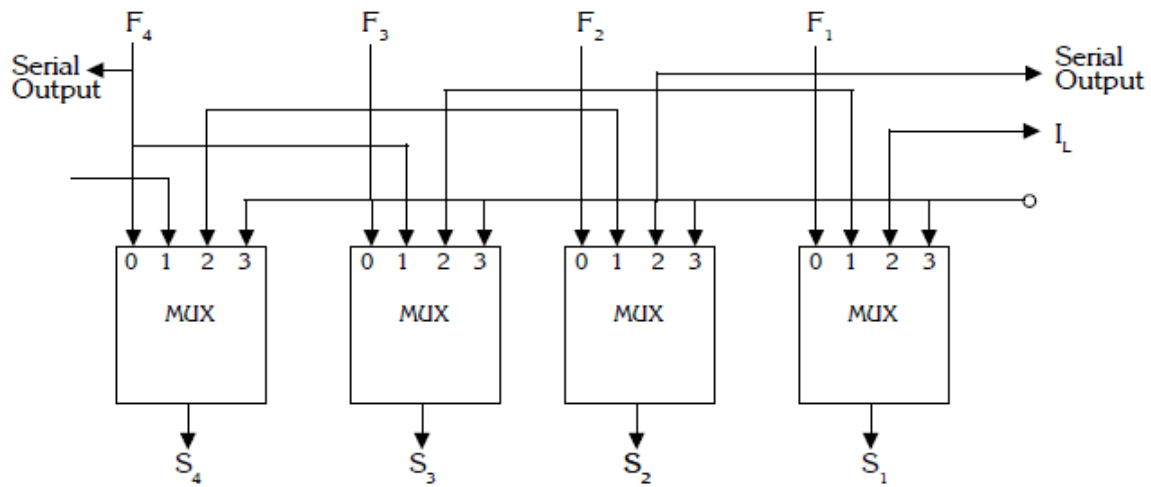


Fig. 5: Four Stage Shifter

H_1	H_0	Operation	Function
0	0	$S \leftarrow F$	Transfer F to S (No shift)
0	1	$S \leftarrow \text{SHR } F$	Shift-Right F into S
1	0	$S \leftarrow \text{SHL } F$	Shift-Left F into S
1	1	$S \leftarrow 0$	Transfer 0's into S

Table 1: 4-bit Combinational Logic Shifter

22.7 Summary

An arithmetic logic unit (ALU) is a digital circuit used to perform arithmetic and logic operations.

It represents the fundamental building block of the central processing unit (CPU) of a computer.

22.8 Model Questions

1. Describe with circuit an arithmetic circuit which provides the following functions
 - (i) Addition
 - (ii) Subtraction
 - (iii) Decrement
2. Design a logic circuit using multiplexers that can be used for shift left or shift right operations.

LESSON - 23

INPUT-OUTPUT ORGANIZATION

Structure

- 23.0 Objectives**
- 23.1 Introduction**
- 23.2 Peripheral Devices**
- 23.3 I/O Interface**
- 23.4 I/O Bus and Interface Module**
- 23.5 Summary**
- 23.6 Model Questions**

23.0 Objectives

To introduce

- I/O Devices and I/O interface.

23.1 Introduction

In this chapter we focus I/O devices and interface. The input-output subsystem of a computer provides an efficient mode of communication between the central system and the outside environment. In computer system, there are special hardware components between the CPU and peripherals to control or manage the input-output transfers.

23.2 Peripheral Devices

The input-output subsystem of a computer provides an efficient mode of communication between the central system and the outside environment. Programs and data must be entered into computer memory for processing and results obtained from computations must be recorded or displayed for the user. A computer serves no useful purpose without the ability to receive information from an outside source and to transmit results in a meaningful form. The simplest and cheapest way to communicate with a computer is by means of typewriter keyboard and printer. However, this is a very slow process and wastes computer time. A central processor is an extremely fast device capable of performing operations. It is an extremely fast device capable

of performing operations at very high speed. When input and output information are transferred to the processor via a slow terminal, the processor will be idle most of the time while waiting for the information to arrive. To use a computer efficiently a large amount of programs and data must be prepared in advance and entered into a storage medium such as magnetic tapes or disks. The information in the disk is then transferred into computer memory at a rapid rate. Results of programs should also be transferred into a high-speed storage, such as disks, from which they can be transferred later into printer to provide a printed output of results.

Devices that are under the direct control of the processor are said to be connected on line. These devices are designed to read information into or out of the memory unit upon command from the CPU and are considered to be part of the computer system. A device is off-line when it is operated independently of the computer. Off-line equipment devices are useful for preparing programs and data. For example, a key-punch is an off-line device where the operator prepares his program and data on punched cards. A line printer may be operated off-line to print results of programs from a magnetic tape. Input or output devices attached to the computers on-line or available off-line are called peripherals.

Among the most common peripherals are card readers and punches, paper-tape readers and punches, keyboards, printers, plotters, and display devices. Peripherals that are electromechanical and electromagnetic devices are of some complexity.

Magnetic tapes are used mostly for storing files of data, for example, a company's payroll record. Access is sequential and consists of records which can be accessed one after another as the tape moves along a stationary read-write mechanism. It is one of the cheapest and slowest methods for storage and has the advantage that tapes can be removed when not in use. Magnetic disks and drums have high-speed rotational surfaces coated with magnetic material. Access is achieved by moving a read/write mechanism to a track in the magnetized surface. Disks and drums are used mostly for bulk storage of programs and data.

Program and data are often prepared on punched cards for input to a computer. Of the different card that are in use, the Hollerith card is the most common. A punched card consists of 80 columns of 12 rows each. Each column represents an alphanumeric character of 12 bits by punching holes in the appropriate rows. A hole is sensed as a 1 and the absence of a hole as a 0. Punched cards offer a great convenience because they carry a complete record in a single separable card. The deletion of unwanted cards or the insertion of new cards in a card deck is

a very convenient for correcting errors in a program or for using different data for the same program. The 12 bit character code employed in punched cards is inefficient with respect to the number of bit used. Most computers convert the card into an internal 16-bit code to conserve bits of storage.

A card reader is an electromechanical input device that senses holes in cards and produces electrical signals. The sensing is done by brush contacts which develop signals as they detect holes in the card that slides over them. The cards can be moved past the brushes to sense one column at a time or one row at a time. Cards can be read in the manner at speeds of a few hundred per minute. Higher reading speeds are attainable by using photo electric sensing cells. Holes in the cards allow light to pass through to energize the photo cell that, in response to the light, produces electrical signals. Cards can be read in this manner at speeds of 1000 to 2000 per minute.

A card punch is an electromechanical output device that punches holes in cards. The holes in each columns are specified by the alphanumeric binary presented to a buffer register. Cards are seldom used for output since people prefer to obtain results in printed form. They are sometimes used to punch a binary program after it is translated by a compiler. The binary program can then be used again without the need of repeated compilation.

A paper – tape reader punch provide a relatively inexpensive method of input and output for a computer. The paper tape commonly used is a one-inch wide strip of paper with eight hold positions along the tape width. In addition, a single strip of small holes along the paper provides sprockets that are used to hold the paper properly in place and to sense when the tape is in a reading position. Binary information is punched in the eight information channels in each row with a hole representing a 1 and n hole a 0. Each row of eight bits may represent the code of one alphanumeric character or part of a binary program. Paper tapes are also available with five, six or seven hold positions per row. Often a paper-tape reader and punch are mounted on a teletypewriter. As the operator types characters on the key-board, the unit can be made to punch the character on paper tape. When the computer prints results on the typewriter, the paper punch may be turned on to punch the characters being printed. In this way, a permanent record is produced and the paper tape can be used off-line to type a duplicate record.

A teletypewriter is basically an electric typewriter, which, in addition, has a capability of sending keyboard information and receiving printed information remotely via electric wires. The information is transmitted by a series of pulses representing the binary code of the character

whose key is struck or the character that is printed. A teletypewriter is sometimes called a teleprinter or Teletype. The keyboard is used as an input terminal and the printer as an output terminal. The transfer rate of teletypewriters is very slow, ranging from 10 to 100 characters per second. They are sometimes used in very small computers as a primary input and output device. In many computers, they are used as an operator's console or as remote terminals for timesharing systems.

A interactive display device is similar to a teletypewriter except that characters are displayed on a cathode – ray tube instead of being printed.

These devices commonly offer editing capabilities and a local storage. The text entered from the keyboard is stored in the local memory for later transmission to the computer. A characteristic feature of display devices is a cursor which marks the position in the screen where the next character will be inserted. The cursor can be moved to any position in the screen, to a single character, the beginning of a word, or to any line. Edit keys add or delete information based on the cursor position. The display terminal can operate in a Teletype mode where each character entered on the screen through the keyboard is simultaneously transmitted to the computer. In the block mode, the edited text is first stored off-line in the local memory and when ready, it is transferred to the computer as a block of information.

Teletypewriters and display devices can print or display one character at a time across a line. For faster printing, a high – speed line printer is commonly employed. This type of printer can print entire lines of up to 120 characters at a rate of up to 1000 lines per minute. All characters for a line are transferred to the printer at electronic speed and stored in a buffer memory until a complete line is assembled. The characters for the line stored in the buffer memory are then printed with one mechanical operation. The paper moves at a uniform speed so that when the next set of characters is assembled, it is printed in the next line. Mechanical printers are limited in their speed by the mechanical motion of the parts. When multiple copies are not required, an electrostatic printer can be used to print lines at a speed of thousands of lines per minute. Electrostatic printers require special paper with electrical conductive material. The characters are produced by charging the special paper with small dot patterns by means of electrodes.

Other input and output devices encountered in computer systems are digital incremental plotters, optical and magnetic character readers, analog to digital converters and various procedures in real, time such as machine tooling, assembly line procedures and chemical and industrial processes. For such applications, a method must be provided for sensing status

conditions in the process and sending control signals to the process being controlled.

The input-output organization of a computer is a function of the size of the computer and the devices connected to it. The difference between a small and large system is largely dependent on the amount of hardware the computer has available for communicating with peripheral units and the number of peripherals connected to the system. Since each peripheral behaves differently from any other, it would be prohibitive to dwell on the detailed interconnections needed between the computer and each peripheral.

23.3 (I/O) Interface

Input – output (I/O) interface provides a method for transferring binary information between internal storage, such as memory and CPU registers, and external I/O devices. Peripherals connected on – line to a computer need special communication links for interfacing them with the central processor. The purpose of the communication link is to resolve the differences that exists between the central computer and each peripheral. The major differences are :

1. Peripherals are electromechanical devices and their manner of operation is different from the operation of the CPU and memory which are electronic devices.
2. The data transfer rate of peripherals is much slower then the transfer rate in the central computer.
3. The operation of the peripherals must be synchronized with the operation of the CPU and memory unit.
4. Data formats in peripherals differ from the word format in the central processor.
5. The operation of each peripheral must be controlled so as not to disturb the operation of the central computer and other peripherals connected to the system.

To resolve these differences, computer System invariably include special hardware components between the central computer and peripherals to supervise all input and output transfers. Large computer include an I/O processor in the system to provide a pathway for the transfer of information passing between input and output devices and internal memory. The I/O processor is sometimes called a data channel controller, since it controls and regulates the flow of data to and from the internal and external parts of the computer. To keep the price low, small computers come without an I/O processor but instead, an interface module is provided for each peripheral that is attached to the computer.

23.4 I/O BUS and Interface Module

A typical communication link between the CPU and several peripherals is shown in Fig. 1. The CPU has an I/O control that supervise the flow of information in the I/O bus. The computer I/O control receives instructions from program memory and proceeds to execute them by communicating with the peripheral through its Each peripheral has associated with it an interface module. The interface logic decodes the commands received from the I/O bus interprets them for the peripheral and provides signals for the peripheral controller. It also synchronizes the data flow and supervises the transfer rate between peripheral and computer. Each peripheral, has its own controller that opeates and controls the particular electromechanical device. For example, a line- printer controller controls the paper motion, the print timing, and the selection of printing characters. A controller may be housed separately or may by physically integrated with the peripheral. Several peripherals of the same type may sometimes share a common controller.

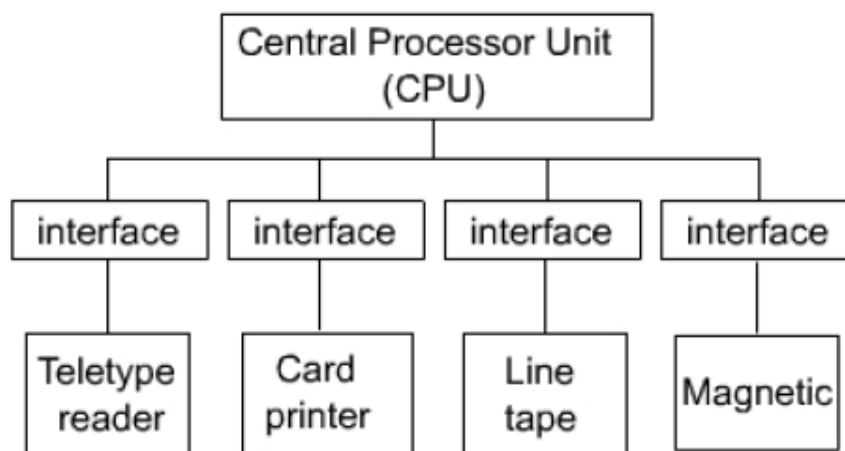


Figure – 1 : I/O Communication link

The I/O bus is a common bus attached to all peripheral interfaces. To communicate with a particular device, the CPU places a device address on a set of lines in the bus. Each interface attached to the common bus contains an address decoder which continuously the address lines when the interface detects its own address code, it activates the paths between the I/O bus lines and the peripheral attached to it. All peripherals whose code does not correspond to the address code in the bus are disabled by their interface.

At the same time that the address is made available in the address lines, the computer I/O control provides a function code on another set of lines in the bus. The selected interface decodes the function code and proceeds to execute it. The function code is often called a command and is in essence an instruction that is executed in the interface module.

To see how the computer generates commands for the peripheral, consider the I/O instruction format as shown in Fig-2. The instruction is read from memory during the fetch cycle and placed in an instruction register. The instruction has three fields. The operation code field disguises the instruction as an I/O type. The device address field can specify up to 16 peripherals

Op.code	Device address	Function Code
---------	----------------	---------------

(a) Instruction code format

1 1 1 1	1 1 0 1	0 1 0 1
---------	---------	---------

(b) A typical instruction code

Fig. 2 : Typical I/O instruction.

The function code specifies the command for the I/O interface. In this example, we assume that all I/O instructions have an operation code 1111.

The instruction specifies the peripheral with assigned address 1101 and the function code is a command for this peripheral.

The connection of the I/O bus to the computer is shown in Fig.3 consists of the address lines, the function code lines and bidirectional data lines for data transfer to and from the accumulator register and the interface. The sense lines detect the status of the peripheral as for example, the status of the flag that indicates the readiness of the peripheral for data transfer.

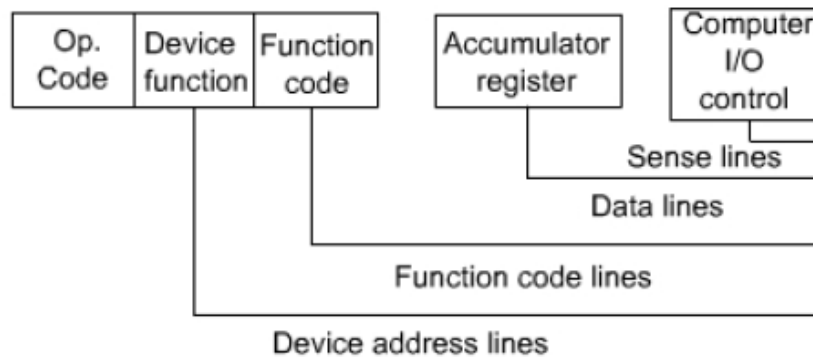


Fig-3 : Connection of I/O bus to computer

The connection of the I/O bus to an interface is shown in Fig. 4. The address bits are decoded through an AND gate and the output of the gate enables the control section of the interface. It is assumed that the assigned device address for this peripheral is 1101 so the output of the gate is 1 only when the address lines contain these bits; otherwise, the output of the gate is 0. The function code bits are decoded in the command decoder. The decoded function is then applied to the interface control to activate the specified command. The interface has a buffer register for holding the data being transferred through the I/O bus. A second register is used to transfer data to the output peripheral device. The status register stores the flags that notify the computer of important conditions in the interface. These include conditions that data has been received from the peripheral, that the interface is ready to receive data from the computer, as well as error indications such as parity error, or device power being off.

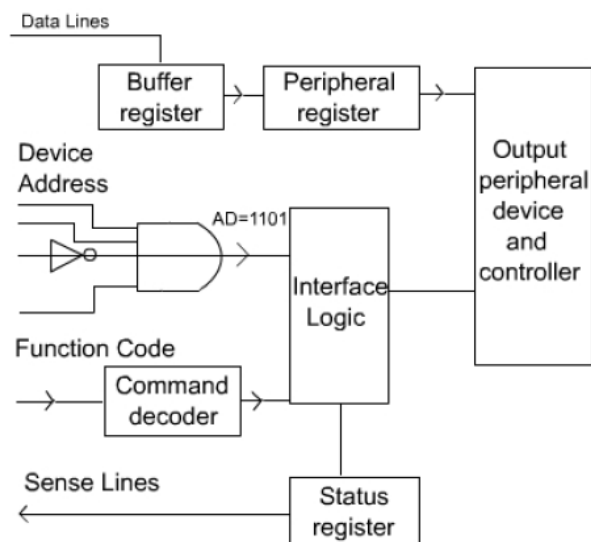


Fig. 4 Connection of I/O bus to one Interface

As interface is assigned the task of synchronizing the data flow between computer and peripheral. For data input synchronization the interface receives an item of data at the rate that the peripheral can provide and transfers in to the computer.

23.5 Summary

In computer system, there are special hardware components between the CPU and peripherals to control or manage the input-output transfers.

These components are called input-output interface units because they provide communication links between processor bus and peripherals.

They provide a method for transferring information between internal system and input-output devices.

23.6 Model Questions

1. Discuss the need and use of different input and output devices.

MCA
Model Question Paper
DIGITAL LOGIC FUNDAMENTALS
CORE PAPER - II

Time: 3 Hrs

Max Marks: 80

Section -A

Answer any 10 questions (10 X 2 = 20)

1. Convert Decimal 41 into binary number.
2. Define Truth Table.
3. Write the purpose of De multiplexer
4. What is encoder? Give its uses.
5. Define Microcomputer.
6. What are the addressing modes? List the types.
7. What is Program Counter?
8. Define Subroutine
9. What is an Interrupt? Mention its sources.
10. What is memory mapped I/O?
11. What are the different types of ROMs?
12. What is a bus? Define bus width.

Section- B

Answer any 5 Questions (5 X 6 = 30)

13. (i) Define binary logic.
(ii) How negative numbers are stored in a digital computer?

14. Discuss the following in detail :
 - (a) Shift-registers.
 - (b) Multiplexers.
15. Discuss the differences between combinational circuits and sequential circuits.
16. Write a short note on accumulator register
17. Explain the working of a T flip flop.
18. Simplify using K-map
$$Y = F(A,B,C,D) = (0,2,5,7,8,10,13,15)$$
19. Discuss in detail about Full Adder.

Section- C

Answer any 3 questions (3 X 10 = 30)

20. Explain Don't Care Condition and Don't Care's in a BCD system
21. Explain NAND as a Universal gate.
22. Explain Flip Flop and its types.
23. Explain Time Delay using register pairs with example.
24. Explain Bus Arbitration.